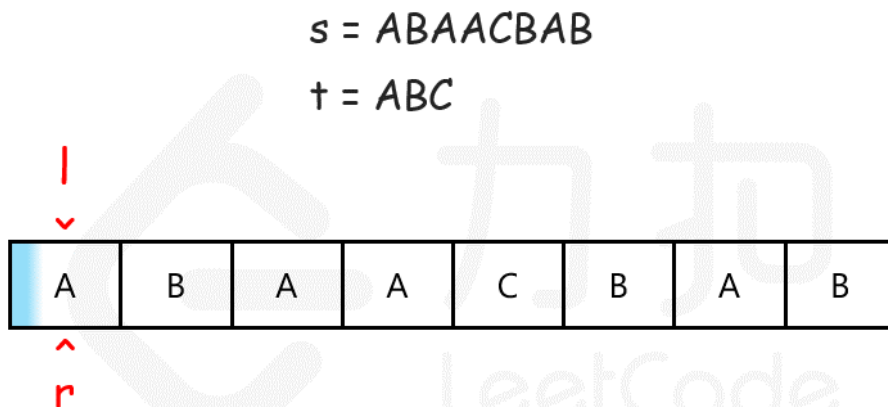


LeetCode 刷题记录

滑动窗口问题

核心思想: 我们可以用滑动窗口的思想解决这个问题, 在滑动窗口类型的问题中都会有两个指针。一个用于「延伸」现有窗口的 r 指针, 和一个用于「收缩」窗口的 l 指针。在任意时刻, 只有一个指针运动, 而另一个保持静止。我们在 s 上滑动窗口, 通过移动 r 指针不断扩张窗口。当窗口包含 t 全部所需的字符后, 如果能收缩, 我们就收缩窗口直到得到最小窗口。



窗口不包含 ABC , r 右移

```
// 基本框架
int left = 0, right = 0;
while (right < s.size()) {
    // 增大窗口
    window.add(s[right]);
    right++;
    while (window needs shrink) {
        // 缩小窗口
        window.remove(s[left]);
        left++;
    }
}
```

3. 无重复字符的最长子串 #todo

```
int lengthOfLongestSubstring(string s) {
    vector<int> hash(256, -1); // hash 记录每一个字符出现的位置
    int res = 0;
    int pre = -1; // pre 总是指向当前窗口左边界的前一个位置。 [pre+1, i]
    for(int i = 0; i < s.size(); i++)
    {
        pre = max(pre, hash[s[i]]); // 保证窗口左边界始终在「上一次重复字符位置」之后, 避免窗口内出现重复字符。
        hash[s[i]] = i;
    }
}
```

```

        res = max(res, i - pre);
    }
    return res;
}
// 1. pre 总是指向当前窗口左边界的前一个位置。
// 2. 窗口实际范围 = [pre+1, i]
// 3. 当遇到重复字符 s[i] 且它在窗口内时, 直接把 pre 移到 hash[s[i]], 窗口左边界同时右移。
// 4. 当前窗口长度 = i - pre → 用于更新 res。

// s = "abcabcbb"
// i=0: pre=-1 窗口: [a]
// i=1: pre=-1 窗口: [a b]
// i=2: pre=-1 窗口: [a b c]
// i=3: pre=0 窗口: [b c a] ← 'a' 重复, pre 移到 hash['a']=0
// i=4: pre=1 窗口: [c a b] ← 'b' 重复, pre 移到 hash['b']=1
// i=5: pre=2 窗口: [a b c] ← 'c' 重复, pre 移到 hash['c']=2
// i=6: pre=4 窗口: [c b] ← 'b' 重复, pre 移到 hash['b']=4
// i=7: pre=6 窗口: [b] ← 'b' 重复, pre 移到 hash['b']=6

```

30. 串联所有单词的子串

```

vector<int> findSubstring(string s, vector<string>& words) {
    // 遍历s中所有长度为 nxlen 的子串, 当剩余子串的长度小于 nxlen 时, 就不用再判断了。所以i从0开始, 到
    (int)s.size() - nxlen 结束就可以了, 注意这里一定要将 s.size() 先转为整型数, 再进行解法。一定要形成这
    样的习惯, 一旦 size() 后面要减去数字时, 先转为 int 型, 因为 size() 的返回值是无符号型, 一旦减去一个比自
    己大的数字, 则会出错。对于每个遍历到的长度为 nxlen 的子串, 需要验证其是否刚好由 words 中所有的单词构成,
    检查方法就是每次取长度为 len 的子串, 看其是否是 words 中的单词。为了方便比较, 建立另一个 HashMap, 当取出
    的单词不在 words 中, 直接 break 掉, 否则就将其在新的 HashMap 中的映射值加1, 还要检测若其映射值超过原
    HashMap 中的映射值, 也 break 掉, 因为就算当前单词在 words 中, 但若其出现的次数超过 words 中的次数, 还
    是不合题意的。在 for 循环外面, 若j正好等于n, 说明检测的n个长度为 len 的子串都是 words 中的单词, 并且刚好
    构成了 words, 则将当前位置i加入结果 res 即可
    if (s.empty() || words.empty()) return {};
    vector<int> res;
    int n = words.size(), len = words[0].size();
    unordered_map<string, int> wordCnt;
    for (auto &word : words) ++wordCnt[word];
    for (int i = 0; i <= (int)s.size() - n * len; ++i) {
        unordered_map<string, int> strCnt;
        int j = 0;
        for (j = 0; j < n; ++j) {
            string t = s.substr(i + j * len, len);
            if (!wordCnt.count(t)) break;
            ++strCnt[t];
            if (strCnt[t] > wordCnt[t]) break;
        }
        if (j == n) res.push_back(i);
    }
    return res;
}

```

53. 最大子数组和

```
int maxSubArray(vector<int>& nums) {
    if (nums.empty())
        return 0;
    int cur_sum = 0;
    int res = INT_MIN;
    for(int i = 0; i < nums.size(); i++)
    {
        cur_sum += nums[i];
        res = max(res, cur_sum);
        // 因为只有在 cur_sum 为正数时才有可能对后续的子数组和产生正面影响
        cur_sum = cur_sum > 0 ? cur_sum : 0;
    }
    return res;
}
```

76. 最小覆盖子串 # todo 20210419

```
string minWindow(string s, string t) {
    vector<int> m(128,0); // m 可以理解 需要多少个 如m[a] = -1,说明多了一个a, m[a] = 0,正好,
    m[a]=1说明缺一个a
    // 记录t中每个字符出现的次数
    for (char c : t)
        m[c]++;
    int count = 0;
    int left = 0;
    int min_len = INT_MAX; // 记录窗口的最小值,
    int min_left = -1; // 记录窗口的最小值对应的左边界
    for(int right = 0; right < s.size(); right++)
    {
        //减1后的映射值仍大于等于0, 说明当前遍历到的字母是t串中的字母
        if (--m[s[right]] >= 0)
            count ++;
        while (count == t.size()) // 形成窗口了, 并且当前窗口包含t中的所有字符
        {
            if (right - left + 1 < min_len)
            {
                min_len = right - left + 1;
                min_left = left;
            }
            // 开始收缩左边界
            if (++m[s[left]] > 0) // 不在t中的字符, ++ 之后不可能出现大于1的情况
            {
                count--;
            }
            ++left;
        }
    }
    return min_left == -1 ? "" : s.substr(min_left, min_len);
}
```

159. 最多有两个不同字符的最长子串

```
int lengthOfLongestSubstringTwoDistinct(string s) {
    // HashMap 记录每个字符的出现次数，然后如果 HashMap 中的映射数量超过两个的时候，这里需要删掉一个映射，比如此时 HashMap 中e有2个，c有1个，此时把b也存入了 HashMap，那么就有三对映射了，这时 left 是0，先从e开始，映射值减1，此时e还有1个，不删除，left 自增1。这时 HashMap 里还有三对映射，此时 left 是1，那么到c了，映射值减1，此时e映射为0，将e从 HashMap 中删除，left 自增1，然后更新结果为 i - left + 1
    // eceba
    int res = 0, left = 0;
    unordered_map<char, int> m;
    for (int i = 0; i < s.size(); ++i) {
        ++m[s[i]];
        while (m.size() > 2) {
            if (--m[s[left]] == 0) m.erase(s[left]);
            ++left;
        }
        res = max(res, i - left + 1);
    }
    return res;
}
```

340. 最多有K个不同字符的最长子串

```
int lengthOfLongestSubstringKDistinct(string s, int k) {
    int res = 0, left = 0;
    unordered_map<char, int> m;
    for (int i = 0; i < s.size(); ++i) {
        ++m[s[i]];
        while (m.size() > k) {
            if (--m[s[left]] == 0) m.erase(s[left]);
            ++left;
        }
        res = max(res, i - left + 1);
    }
    return res;
}
```

209. 长度最小的子数组

```
int minSubArrayLen(int target, vector<int>& nums) {
    int res = INT_MAX; // 初始化结果为最大值，表示尚未找到符合条件的子数组
    int left = 0; // 滑动窗口的左边界
    int cur_sum = 0; // 当前窗口内元素的和

    for (int right = 0; right < nums.size(); right++) {
        cur_sum += nums[right]; // 将当前元素添加到当前窗口内

        // 内部循环：收缩左边界以找到满足条件的子数组
        while (left <= right && cur_sum >= target) {
```



```

        res = min(res, right - left + 1); // 更新结果，记录当前满足条件的子数组长度
        cur_sum -= nums[left]; // 移除左边界元素，同时减去其值
        left++; // 收缩左边界
    }
}

// 如果结果仍然是初始值 INT_MAX，表示没有找到符合条件的子数组，返回0；否则，返回 res
return res == INT_MAX ? 0 : res;
}
=

```

713. 乘积小于 K 的子数组

```

int numSubarrayProductLessThanK(vector<int>& nums, int k) {
    int left = 0; // 初始化左指针
    int product = 1; // 初始化当前子数组的乘积
    int count = 0; // 初始化满足条件的子数组数量

    for (int right = 0; right < nums.size(); right++) {
        product *= nums[right]; // 更新当前子数组的乘积
        // 如果当前子数组的乘积大于等于k，需要收缩左边界
        while (product >= k && left <= right) {
            product /= nums[left]; // 去掉左边界元素，更新乘积
            left++; // 收缩左边界
        }
        // 如果当前子数组的乘积小于k，计算满足条件的子数组数量
        if (product < k) {
            count += (right - left + 1);
        }
    }
    return count; // 返回满足条件的子数组数量
}

int numSubarrayProductLessThanK(vector<int>& nums, int k) {
    if (nums.empty())
        return 0;
    int res = 0;
    int prod = 1;
    int left = 0;
    for (int i = 0; i < nums.size(); i++)
    {
        prod *= nums[i];
        // 形成窗口
        while (left <= i && prod >= k)
            prod /= nums[left++];
        res += i - left + 1;
    }
    return res;
}

```

239. 滑动窗口最大值

```
// 在滑动窗口内，你可以保证队列中的元素是按降序排列的，以便在 $O(1)$ 时间内找到窗口内的最大值
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    vector<int> result;
    deque<int> maxQueue;
    for (int i = 0; i < nums.size(); i++) {
        // 移除队列中小于当前元素的索引，保持降序
        while (!maxQueue.empty() && nums[maxQueue.back()] < nums[i]) {
            maxQueue.pop_back();
        }
        maxQueue.push_back(i); // 将当前元素的索引加入队列

        // 移除队列中小于等于当前元素的索引，保持降序
        if (maxQueue.front() <= i - k) // 检查队首元素是否过期，如果过期则弹出
            maxQueue.pop_front();
        // 当窗口满足k个元素时，记录窗口内的最大值
        if (i >= k - 1) {
            result.push_back(nums[maxQueue.front()]);
        }
    }
    return result;
}
```

346.滑动窗口的平均值 **todo** 得熟悉下队列的stl

```
class MovingAverage {
public:
    MovingAverage(int size) {
        this->size = size;
        sum = 0;
    }
    double next(int val) {
        if (q.size() >= size) {
            sum -= q.front(); q.pop();
        }
        q.push(val);
        sum += val;
        return sum / q.size();
    }

private:
    queue<int> q;
    int size;
    double sum;
};
```

424. 替换后的最长重复字符 **#todo**

见<https://www.cnblogs.com/grandyang/p/5999050.html>

```
/*
```

如果没有k的限制，让我们求把字符串变成只有一个字符重复的字符串需要的最小置换次数，那么就是字符串的总长度减去出现次数最多的字符的个数。如果加上k的限制，我们其实就是求满足（子字符串的长度减去出现次数最多的字符个数） $\leq k$ 的最大子字符串长度即可，搞清了这一点，我们也就应该知道怎么用滑动窗口来解了吧。我们用一个变量 `start` 记录滑动窗口左边界，初始化为0，然后遍历字符串，每次累加出现字符的个数，然后更新出现最多字符的个数，然后我们判断当前滑动窗口是否满足之前说的那个条件，如果不满足，我们就把滑动窗口左边界向右移动一个，并注意去掉的字符要在 `counts` 里减一，直到满足条件，我们更新结果 `res` 即可。需要注意的是，当滑动窗口的左边界向右移动了后，窗口内的相同字母的最大数貌似可能会改变啊，为啥这里不用更新 `maxCnt` 呢？这是个好问题，原因是此题让求的是最长的重复子串，`maxCnt` 相当于卡了一个窗口大小，我们并不希望窗口变小，虽然窗口在滑动，但是之前是出现过跟窗口大小相同的符合题意的子串，缩小窗口没有意义，并不会使结果 `res` 变大，所以我们才不更新 `maxCnt` 的

```
*/

// 解法一
int characterReplacement(string s, int k)
{
    int res = 0;
    int maxCnt = 0;
    left = 0;
    vector<int> m(128,0); // 用来记录窗口中每个字符出现的次数
    for (int i = 0; i < s.size(); i++)
    {
        maxCnt = max(maxCnt, ++m[s[i]]); // 记录当前窗口出现最多字符的个数
        // 判断当前窗口 left...i 是否满足条件
        if (i - left + 1 - maxCnt > k) // 不满足 从左开始收缩窗口
        {
            --m[s[left]];
            left++;
        }
        res = max(res, i - left + 1);
    }
    return res;
}
```

205. 同构字符串

```
bool isIsomorphic(std::string s, std::string t) {
    // int m1[256] = {-1}, m2[256] = {-1}, n = s.size();
    vector<int> m1(256,-1);
    vector<int> m2(256,-1);
    int n = s.size();
    for (int i = 0; i < n; ++i) {
        if (m1[s[i]] != m2[t[i]]) return false;
        m1[s[i]] = i;
        m2[t[i]] = i;
    }
    return true;
}
```

438. 找到字符串中所有字母异位词 todo

```
vector<int> findAnagrams(string s, string p)
```

```

{
    if (s.empty() || s.size() < p.size())
        return {};
    vector<int> res, m1(256, 0), m2(256, 0);
    for (int i = 0; i < p.size(); ++i)
    {
        ++m1[s[i]];
        ++m2[p[i]];
    }
    if (m1 == m2)
        res.push_back(0);
    // 在s上形成窗口 进行滑动 窗口大小为p.size()
    for (int i = p.size(); i < s.size(); ++i)
    {
        ++m1[s[i]];
        --m1[s[i - p.size()]];
        if (m1 == m2)
            res.push_back(i - p.size() + 1);
    }
    return res;
}

```

567. 字符串的排列 (和438 差不多) todo

解法一 其他解法见 [\[LeetCode\] Permutation in String 字符串中的全排列](#)

```

// 解法一
// 先来分别统计s1和s2中前n1个字符串中各个字符出现的次数，其中n1为字符串s1的长度，
// 这样如果二者字符出现次数的情况完全相同，说明s1和s2中前n1的字符互为全排列关系，那么符合题意了，
// 直接返回true。如果不是的话，那么我们遍历s2之后的字符，对于遍历到的字符，对应的次数加1，
// 由于窗口的大小限定为了n1，所以每在窗口右侧加一个新字符的同时就要在窗口左侧去掉一个字符，
// 每次都比较一下两个哈希表的情况，如果相等，说明存在
bool checkInclusion(string s1, string s2)
{
    if (s1.size() < s2.size())
        return false;
    vector<int> m1(128), m2(128);
    for (int i = 0; i < s1.size(); ++i)
    {
        ++m1[s1[i]];
        ++m2[s2[i]];
    }
    if (m1 == m2)
        return true;
    for (int i = s1.size(); i < s2.size(); i++)
    {
        ++m2[s2[i]];
        --m2[s2[i - s1.size()]];
        if (m1 == m2)
            return true;
    }
}

```

```
    return false;
}
```

1004. 最大连续1的个数 III

```
int longestOnes(vector<int>& nums, int k) {
    // 窗口内最多有 k 个 0。
    int res = 0, zeros = 0, left = 0;
    for (int right = 0; right < nums.size(); ++right) {
        if (nums[right] == 0) ++zeros;
        while (zeros > k) {
            if (nums[left++] == 0) --zeros;
        }
        res = max(res, right - left + 1);
    }
    return res;
}
```

1423. 可获得的最大点数

```
int maxScore(vector<int>& cardPoints, int k) {
    int cur_sum = 0;
    int res = INT_MAX;
    int left = 0;
    // 求剩下的数 窗口大小为 nums.size() - k 和最小
    for(int right = 0; right < cardPoints.size(); right++)
    {
        cur_sum += cardPoints[right];
        while (right - left + 1 == cardPoints.size() - k)
        {
            res = min(res, cur_sum);
            cur_sum -= cardPoints[left];
            left++;
        }
    }
    if (k == cardPoints.size())
        return accumulate(cardPoints.begin(), cardPoints.end(), 0);
    return accumulate(cardPoints.begin(), cardPoints.end(), 0) - res;
}

int maxScore(vector<int>& cardPoints, int k) {
    int n = cardPoints.size();
    // 滑动窗口大小为 n-k
    int windowSize = n - k;
    // 选前 n-k 个作为初始值
    int sum = accumulate(cardPoints.begin(), cardPoints.begin() + windowSize, 0);
    int minSum = sum;
    for (int i = windowSize; i < n; ++i) {
        // 滑动窗口每向右移动一格，增加从右侧进入窗口的元素值，并减少从左侧离开窗口的元素值
        sum += cardPoints[i] - cardPoints[i - windowSize];
        minSum = min(minSum, sum);
    }
}
```

```

    }
    return accumulate(cardPoints.begin(), cardPoints.end(), 0) - minSum;
}
// 作者：力扣官方题解
// 链接：https://leetcode.cn/problems/maximum-points-you-can-obtain-from-cards/solutions/514347/ke-huo-de-de-zui-da-dian-shu-by-leetcode-7je9/
// 来源：力扣（LeetCode）
// 著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。

```

双指针问题

todo: 11和42的区别

11. 盛最多水的容器 #todo 20210419

```

int maxArea(vector<int>& height) {
    if (height.empty())
        return 0;

    int res = 0;
    int left = 0;
    int right = height.size()-1;
    // 在每个状态下，无论长板或短板向中间收窄一格，都会导致水槽底边宽度 -1 • 变短
    // 若向内 移动短板，水槽的短板 min(h[left],h[right]) 可能变大，因此下个水槽的面积 可能增大。
    // 若向内 移动长板，水槽的短板 min(h[left],h[right]) 不变或变小，因此下个水槽的面积 一定变小。
    while(left < right)
    {
        res = max(res, min(height[right], height[left]) * (right - left));
        if (height[left] <= height[right])
            left++;
        else
            right--;
    }
    return res;
}

```

167. 两数之和 II - 输入有序数组

```

vector<int> twoSum(vector<int>& numbers, int target) {
    vector<int> res;
    if (numbers.empty())
        return res;
    int left = 0, right = numbers.size() - 1;
    int i = 0;

    while(left < right)
    {
        if (numbers[left] + numbers[right] == target)
            return {left+1, right+1};
        else if (numbers[left] + numbers[right] < target)

```

```

        left++;
    else if (numbers[left] + numbers[right] > target)
        right--;
    }
    return {};
}

```

15. 三数之和

```

vector<vector<int>> threeSum(vector<int>& nums) {
    vector<vector<int>> res;
    sort(nums.begin(), nums.end());
    if (nums.empty() || nums.back() < 0 || nums.front() > 0) return {};
    for (int i = 0; i < nums.size(); ++i)
    {

        if (nums[i] > 0)
            break;
        // 需要和上一次枚举的数不相同
        if (i > 0 && nums[i] == nums[i - 1])
            continue;
        int target = 0 - nums[i], left = i + 1, right = nums.size() - 1;
        while (left < right)
        {
            // 用两个指针分别指向 fix 数字之后开始的数组首尾两个数，如果两个数和正好为 target，则将这两个数和 fix 的数一起存入结果中。
            // 然后就是跳过重复数字的步骤了，两个指针都需要检测重复数字
            if (nums[left] + nums[right] == target)
            {
                res.push_back({nums[i], nums[left], nums[right]});
                while (left < right && nums[left] == nums[left + 1])
                    ++left;
                while (left < right && nums[right] == nums[right - 1])
                    --right;
                ++left;
                --right;
            }
            else if (nums[left] + nums[right] < target)
                ++left;
            else
                --right;
        }
    }
    return res;
}

```

16. 最接近的三数之和

```

int threeSumClosest(vector<int>& nums, int target)
{
    if (nums.size() < 3)

```

```

        return 0;
    int res = nums[0] + nums[1] + nums[2];
    sort(nums.begin(), nums.end());

    for(int start = 0; start < nums.size() - 2; start++)
    {
        if (start > 0 && nums[start] == nums[start-1] )
            continue;

        int left = start+1;
        int right = nums.size()-1;

        while(left < right)
        {
            int curSum = nums[start] + nums[left] + nums[right];
            if (curSum == target) // 如果当前和正好等于target,直接返回,
                return curSum;

            if (abs(target-curSum) < abs(target-res)) // 若不等于则进行范围缩小, 每一次都要记录
                res = curSum;

            if (curSum > target)
                --right;
            else
                ++left;
        }
    }
    return res;
}

```

一下

259. 3Sum Smaller 三数之和较小值

```

int threeSumSmaller(vector<int>& nums, int target) {
    // 双指针来做, 这里面有个 trick 就是当判断三个数之和小于目标值时, 此时结果应该加上 right-left, 因为
    // 数组排序了以后, 如果加上 num[right] 小于目标值的话, 那么加上一个更小的数必定也会小于目标值, 然后将左指针
    // 右移一位, 否则将右指针左移一位
    if (nums.size() < 3) return 0;
    int res = 0, n = nums.size();
    sort(nums.begin(), nums.end());
    for (int i = 0; i < n - 2; ++i) {
        int left = i + 1, right = n - 1;
        while (left < right) {
            if (nums[i] + nums[left] + nums[right] < target) {
                res += right - left;
                ++left;
            } else {
                --right;
            }
        }
    }
}

```



```

    }
    return res;
}

```

18.四数之和

```

vector<vector<int>> fourSum(vector<int>& nums, int target) {
    vector<vector<int>> result;
    sort(nums.begin(), nums.end());
    for (int k = 0; k < nums.size(); k++) {
        // 剪枝处理
        if (nums[k] > target && nums[k] >= 0) {
            break; // 这里使用break, 统一通过最后的return返回
        }
        // 对nums[k]去重
        if (k > 0 && nums[k] == nums[k - 1]) {
            continue;
        }
        for (int i = k + 1; i < nums.size(); i++) {
            // 2级剪枝处理
            if (nums[k] + nums[i] > target && nums[k] + nums[i] >= 0) {
                break;
            }

            // 对nums[i]去重
            if (i > k + 1 && nums[i] == nums[i - 1]) {
                continue;
            }
            int left = i + 1;
            int right = nums.size() - 1;
            while (right > left) {
                // nums[k] + nums[i] + nums[left] + nums[right] > target 会溢出
                if ((long) nums[k] + nums[i] + nums[left] + nums[right] > target) {
                    right--;
                }
                // nums[k] + nums[i] + nums[left] + nums[right] < target 会溢出
                else if ((long) nums[k] + nums[i] + nums[left] + nums[right] < target) {
                    left++;
                }
                else {
                    result.push_back(vector<int>{nums[k], nums[i], nums[left],
nums[right]});

                    // 对nums[left]和nums[right]去重
                    while (right > left && nums[right] == nums[right - 1]) right--;
                    while (right > left && nums[left] == nums[left + 1]) left++;
                    // 找到答案时, 双指针同时收缩
                    right--;
                    left++;
                }
            }
        }
    }
}

```

```

    }
    return result;
}

```

454.四数相加 II

```

int fourSumCount(vector<int>& nums1, vector<int>& nums2, vector<int>& nums3, vector<int>&
nums4) {
    unordered_map<int, int> hash;
    int res = 0;
    for(int i = 0; i < nums1.size(); i++)
    {
        for(int j = 0; j < nums2.size(); j++)
            hash[nums1[i] + nums2[j]] ++;
    }
    for(int i = 0; i < nums3.size(); i++)
    {
        for(int j = 0; j < nums4.size(); j++)
        {
            if (hash.find(0 - nums3[i] - nums4[j]) != hash.end())
                res += hash[0 - nums3[i] - nums4[j]];
        }
    }
    return res;
}

```

240. 搜索二维矩阵 II

```

bool searchMatrix(vector<vector<int>>& matrix, int target)
{
    if (matrix.empty())
        return false;
    int m = matrix.size();
    int n = matrix[0].size();

    int row = 0;
    int col = n - 1;
    // 从右上角或者左下角开始比较
    while (row < m && col >= 0)
    {
        if (target == matrix[row][col])
            return true;
        else if (matrix[row][col] < target)
            row++;
        else
            col--;
    }
    return false;
}

```

443. 压缩字符串

```
int compress(vector<char>& chars) {
    int n = chars.size();
    // 双指针，一个代表当前字符出现的第一个位置，一个表示修改后当前长度
    int left = 0, len = 0;
    for(int i = 0; i < n; i++) {
        // 遇到临界点（最后一个位置，或者两字符分界处）
        if(i == n - 1 || chars[i] != chars[i + 1]) {
            chars[len++] = chars[i];
            int nums = i - left + 1;
            if(nums > 1) {
                // 把数字加到末尾
                for(char num : to_string(nums)) {
                    chars[len++] = num;
                }
            }
            left = i + 1;
        }
    }
    return len;
}
```

524. 通过删除字母匹配到字典里最长单词 #todo

```
string findLongestWord(string s, vector<string>& dictionary) {
    string res = "";
    for(int i = 0; i < dictionary.size(); i++)
    {
        int count = 0;
        // 遍历每一个单词，用一个变量count来记录单词中的某个字母的位置，我们遍历给定字符串，
        // 如果遍历到单词中的某个字母来，i自增1，如果没有，就继续往下遍历。这样如果最后i和单词长度相等，说明单词中的所有字母都按顺序出现在了字符串s中
        string word_i = dictionary[i];
        for(int j = 0; js < s.size(); j++)
        {
            if (s[j] == word_i[count])
                count++;
        }

        if (count == word_i.size() && word_i.size() >= res.size())
        {
            if (word_i.size() > res.size() || word_i < res)
                res = word_i;
        }
    }
    return res;
}
```

581. 最短无序连续子数组 #todo 20210419 双指针

977. 有序数组的平方

```
vector<int> sortedSquares(vector<int>& nums) {  
    // 双指针来做，用两个变量分别指向开头和结尾，  
    // 然后比较，每次将绝对值较大的那个数的平方值先加入数组的末尾，然后依次往前更新，最后得到的就是所求的顺序  
    int left = 0, right = nums.size() - 1;  
    vector<int> res(nums.size());  
    for (int i = nums.size() - 1; i >= 0; --i)  
    {  
        if (abs(nums[left]) > abs(nums[right]))  
        {  
            res[i] = nums[left] * nums[left];  
            ++left;  
        } else {  
            res[i] = nums[right] * nums[right];  
            --right;  
        }  
    }  
    return res;  
}
```

背向双指针

5. 最长回文子串

125. 验证回文串

409. 最长回文串

双指针（2 Pointer）：

- 基础知识：常见双指针算法分为三类，同向（即两个指针都相同一个方向移动），背向（两个指针从相同或者相邻的位置出发，背向移动直到其中一根指针到达边界为止），相向（两个指针从两边出发一起向中间移动直到两个指针相遇）
- 背向双指针：(基本上全是回文串的题)
 - Leetcode 409. Longest Palindrome
 - Leetcode 125. Valid Palindrome
 - Leetcode 5. Longest Palindromic Substring
- 相向双指针：(以two sum为基础的一系列题)
 - Leetcode 1. Two Sum (这里使用的是先排序的双指针算法，不同于hashmap做法)
 - Leetcode 167. Two Sum II - Input array is sorted

- Leetcode 15. 3Sum
- Leetcode 16. 3Sum Closest
- Leetcode 18. 4Sum
- Leetcode 454. 4Sum II
- Leetcode 277. Find the Celebrity
- Leetcode 11. Container With Most Water
- 同向双指针：（个人觉得最难的一类题，可以参考下这里 [TimothyL: Leetcode 同向双指针/滑动窗口类代码模板](#)）
- - Leetcode 283. Move Zeroes
 - Leetcode 26. Remove Duplicate Numbers in Array
 - Leetcode 395. Longest Substring with At Least K Repeating Characters
 - Leetcode 340. Longest Substring with At Most K Distinct Characters
 - Leetcode 424. Longest Repeating Character Replacement
 - Leetcode 76. Minimum Window Substring
 - Leetcode 3. Longest Substring Without Repeating Characters
 - Leetcode 1004 Max Consecutive Ones III

单调栈系列问题

单调栈的两种写法 [LeetCode Monotone Stack Summary](#) [单调栈小结](#)

1. 单调栈里的元素具有单调性
 2. 元素加入栈前，会在栈顶端把破坏栈单调性的元素都删除
 3. 使用单调栈可以找到元素向左遍历第一个比他小的元素，也可以找到元素向左遍历第一个比他大的元素。
- 单调递增栈，利用波谷剔除栈中的波峰，留下波谷； 单调递增 结算波峰
 单调递减栈，利用波峰剔除栈中的波谷，留下波峰。 单调递减，结算波谷

口诀：

- 要找小 → 递增栈
- 要找大 → 递减栈

```
// 写法一
int trap(vector<int>& height)
{
    if (height.empty())
        return 0;

    * int res = 0;
    int i = 0;
    stack<int> monoStack;
    while( i < height.size())
    {
```

```

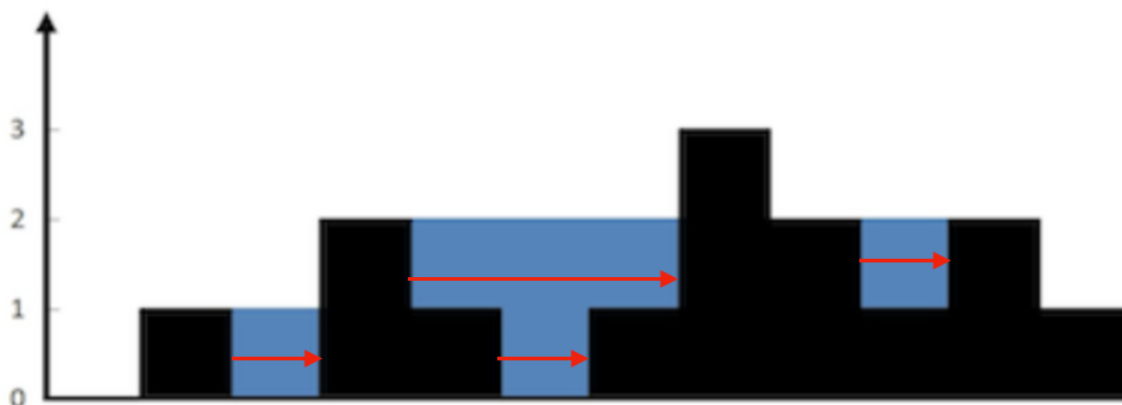
        if (monoStack.empty() || height[i] <= height[monoStack.top()])
        {
            monoStack.push(i++);
        }
        else
        {
            int tmp = monoStack.top();
            monoStack.pop();
            if (monoStack.empty())
                continue;

            int h = min(height[i], height[monoStack.top()]);
            res = res + (h - height[tmp]) * (i - monoStack.top() - 1);
        }
    }
    return res;
}
// 写法二
int largestRectangleArea(vector<int> height)
{
    stack<int> monoStack;
    int res = 0;
    height.push_back(0);
    for (int i = 0; i < height.size(); i++)
    {
        while(!monoStack.empty() && height[i] <= height[monoStack.top()]) // 但栈非空
            时, 且当前元素大于栈顶元素时, 进行弹出操作, 并且结算该弹出元素 弹出谁就结算谁
        {
            int h = height[monoStack.top()];
            monoStack.pop();
            int left = monoStack.empty() ? -1 : monoStack.top();
            // int left = monoStack.size() > 0 ? monoStack.top() : -1;
            res = max(res, h * (i - left - 1));
        }
        monoStack.push(i);
    }
    return res;
}

```

42. 接雨水

按照行计算



上面是由数组 `[0,1,0,2,1,0,1,3,2,1,2,1]` 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。感谢 **Marcos** 贡献此图。

//首先单调栈是按照行方向来计算雨水

```
int trap_1(vector<int> &height)
```

```
{
    if (height.empty())
        return 0;
    int res = 0;
    int i = 0;

    stack<int> monoStack;
    // height.push_back(0);
    for (int i = 0; i < height.size(); i++)
    {
        while (!monoStack.empty() && height[i] > height[monoStack.top()]) // 但栈非空时，且
            // 当前元素大于栈顶元素时，进行弹出操作，并且结算该弹出元素
            // 栈顶元素的下一个元素则为左边界，当前遍历到的height[i]则为右边界
        {
            int tmp = monoStack.top();
            monoStack.pop();
            if (monoStack.empty())
                continue;

            int h = min(height[i], height[monoStack.top()]);
            res = res + (h - height[tmp]) * (i - monoStack.top() - 1);
        }
        monoStack.push(i);
    }
    return res;
}
```

```
int trap(vector<int> &height)
```

```
{
    if (height.empty())
        return 0;
}
```

```

int res = 0;
int i = 0;
stack<int> monoStack; // 因为要求一个数左边比他大和右边比他大,所以应该是一个单调递减的栈, 这个栈
// 需要保持严格单调递减
while (i < height.size())
{
    // 如果满足入栈条件,则直接入栈
    if (monoStack.empty() || height[i] < height[monoStack.top()])
    {
        monoStack.push(i++);
    }
    else // 如果不满足入栈条件,则弹出栈顶元素,这个时候可以结算当前元素,栈顶元素的下一个元素则为左边界,
    // 当前遍历到的height[i] 则为右边界
    {
        int tmp = monoStack.top();
        monoStack.pop();
        if (monoStack.empty())
            continue;
        int h = min(height[i], height[monoStack.top()]);
        res = res + (h - height[tmp]) * (i - monoStack.top() - 1);
    }
}
return res;
}

```

// 解法二: 还不太懂

```

int trap(vector<int>& height)
{
    if (height.empty())
        return 0;

    int n = height.size();
    int res = 0;
    int left_max = height[0];
    int right_max = height[n-1];

    int l = 1;
    int r = n - 2;

    // 哪边小 就先结算谁, 因为小的真实最大值已经确定了
    while(l <= r)
    {
        if (left_max <= right_max)
        {
            res += max(0, left_max - height[l]);
            left_max = max(left_max, height[l]);
            l++;
        }
        else
        {
            res += max(0, right_max - height[r]);
            right_max = max(right_max, height[r]);
            r--;
        }
    }
}

```



```

        right_max = max(height[r], right_max);
        r--;
    }
}
return res;
}

```

84. 柱状图中最大的矩形 TODO

```

int largestRectangleArea(vector<int> &heights)
{
    if (heights.empty())
        return 0;
    int res = 0;
    // 需要找到每根柱子的左右边界 所以要单调递增
    stack<int> s;
    heights.push_back(0); // 为了使得最后一块板子也被处理, 这里用了个小 trick, 在高度数组最后面加上一个0
    for(int i = 0; i < heights.size(); i++)
    {
        while(!s.empty() && heights[i] <= heights[s.top()])
        {
            int res_index = s.top(); s.pop();
            int left = s.empty() ? -1 : s.top();
            res = max(res, (i - left - 1) * heights[res_index]);
        }
        s.push(i);
    }
    return res;
}

```

85. 最大矩形

```

class Solution {

    int largestRectangleArea(vector<int> height)
    {
        stack<int> monoStack;
        int res = 0;
        height.push_back(0);
        for (int i = 0; i < height.size(); i++)
        {
            while(!monoStack.empty() && height[i] <= height[monoStack.top()]) // 但栈非空时, 且当前元素大于栈顶元素时, 进行弹出操作, 并且结算该弹出元素
            {
                int h = height[monoStack.top()];
                monoStack.pop();
                int left = monoStack.empty() ? -1 : monoStack.top();
                // int left = monoStack.size() > 0 ? monoStack.top() : -1;
                res = max(res, h * (i - left - 1));
            }
        }
    }
}

```

```

        }
        monoStack.push(i);
    }
    return res;
}
public:
    int maximalRectangle(vector<vector<char>>& matrix) {
        if (matrix.empty())
            return 0;

        vector<int> heights(matrix[0].size(), 0);
        int res = 0;
        int m = matrix.size();
        int n = heights.size();

        for(int i = 0; i < m; i++)
        {
            for(int j = 0; j < n; j++)
            {
                if (matrix[i][j] == '1')
                {
                    heights[j] += 1;
                }
                else
                {
                    heights[j] = 0;
                }
            }
            res = max(largestRectangleArea(heights), res);
        }
        return res;
    }
};

```

162. 寻找峰值 todo

```

int findPeakElement(vector<int>& nums) {
    if (nums.empty())
        return -1;
    stack<int> s;
    int res_index = 0;
    for(int i = 0; i < nums.size(); i++)
    {
        while(!s.empty() && nums[i] < nums[s.top()])
        {
            res_index = s.top();
            s.pop();
            return res_index;
        }
        s.push(i);
    }
}

```

```

// 当数组严格单调递增时
if (s.size() == nums.size())
    return s.top();
return res_index;
}

```

581. 最短无序连续子数组 #todo 20210419 双指针

// <https://leetcode.cn/problems/shortest-unsorted-continuous-subarray/solution/si-lu-qing-xi-ming-liao-kan-bu-dong-bu-cun-zai-de-/>

```

class Solution {
public:
    int findUnsortedSubarray(std::vector<int>& nums) {
        int n = nums.size();
        std::stack<int> st;
        int left = n, right = 0;

        // 找到未排序子数组的左边界
        for (int i = 0; i < n; i++) {
            while (!st.empty() && nums[i] < nums[st.top()]) {
                // 栈顶元素对应的数组元素是当前已经遍历的最大元素。
                // 如果遇到一个元素小于栈顶元素对应的数组元素，这意味着它是无序的，
                // 我们需要找到未排序子数组的左边界
                // 因为当前的left 只是num[i]可以往左插入的最左边位置，后面可能还有比nums[i]更小的数

                left = std::min(left, st.top());
                st.pop();
            }
            st.push(i);
        }

        // 清空栈
        while (!st.empty()) {
            st.pop();
        }

        // 找到未排序子数组的右边界
        for (int i = n - 1; i >= 0; i--) {
            while (!st.empty() && nums[i] > nums[st.top()]) {
                right = std::max(right, st.top());
                st.pop();
            }
            st.push(i);
        }

        if (right > left) {
            return right - left + 1;
        } else {
            return 0;
        }
    }
}

```

字

```
};
```

739. 每日温度

```
vector<int> dailyTemperatures(vector<int>& T)
{
    int n = T.size();
    vector<int> res(n);
    stack<int> s;
    for (int i = 0; i < n; ++i)
    {
        while (!s.empty() && T[i] > T[s.top()])
        {
            int pre_index = s.top();
            res[pre_index] = i - pre_index; // 弹出谁就结算谁
            s.pop();
        }
        s.push(i);
    }
    return res;
}
```

316. 去除重复字母 todo

[一招吃遍力扣四道题，妈妈再也不用担心我被套路啦~](#)

```
string removeDuplicateLetters(string s) {
    vector<int> num(256, 0); // 记录字母出现个数
    for(auto &ch : s)
    {
        num[ch]++; // 记录每个字母出现的次数;
    }
    string ans; // string本身具有栈的属性，所以直接用string作为栈
    for(auto &ch : s)
    {
        if(ans.find(ch) != -1) // 如果该字母已经在栈里面，则不让进栈，同时进不去的字母次数也要-1
        {
            --num[ch];
            continue;
        }
        // 如果栈内不为空，且即将进栈的元素小于当前栈顶的元素，同时这个元素也不是最后一个元素
        while(!ans.empty() && ans.back() > ch && num[ans.back()] > 0)
        {
            ans.pop_back();
        }
        --num[ch]; // 入栈后的元素个数-1
        ans.push_back(ch);
    }
    return ans;
}
```

321. 拼接最大数

```
class Solution {
public:
    vector<int> maxNumber(vector<int>& nums1, vector<int>& nums2, int k) {
        int n1 = nums1.size(), n2 = nums2.size();
        vector<int> res;
        // 假设有i个元素来自nums1, k-i个元素来自s2
        for (int i = max(0, k - n2); i <= min(k, n1); ++i) {
            res = max(res, mergeVector(maxVector(nums1, i), maxVector(nums2, k - i)));
        }
        return res;
    }
    vector<int> maxVector(vector<int>& nums, int k) {
        int drop = (int)nums.size() - k;
        vector<int> res;
        for (int num : nums) {
            while (drop > 0 && !res.empty() && res.back() < num) {
                res.pop_back();
                --drop;
            }
            res.push_back(num);
        }
        res.resize(k);
        return res;
    }
    vector<int> mergeVector(vector<int> nums1, vector<int> nums2) {
        vector<int> res;
        while (!nums1.empty() || !nums2.empty()) {
            vector<int> &tmp = (nums1 > nums2) ? nums1 : nums2;
            res.push_back(tmp[0]);
            tmp.erase(tmp.begin());
        }
        return res;
    }
};
```

402. 移掉 K 位数字

```
// https://www.cnblogs.com/grandyang/p/5883736.html
// https://leetcode.cn/problems/remove-k-digits/solution/yi-zhao-chi-bian-li-kou-si-dao-ti-ma-ma-zai-ye-b-5/
string removeKdigits(string num, int k) {
    string res;
    int keep = num.size() - k;

    for(int i = 0; i < num.size(); i++)
    {
        while(k && !res.empty() && num[i] < res.back())
        {
```

```

        res.pop_back();
        --k;
    }
    res.push_back(num[i]);
}
res.resize(keep);
while(!res.empty() && res[0] == '0')
    res.erase(res.begin());
return res.empty() ? "0" : res;
}

```

496. 下一个更大元素 I

```

vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
    vector<int> res;
    stack<int> st;
    unordered_map<int, int> m;
    for (int num : nums2) {
        while (!st.empty() && num > st.top()) {
            m[st.top()] = num; st.pop();
        }
        st.push(num);
    }
    for (int num : nums1) {
        res.push_back(m.count(num) ? m[num] : -1);
    }
    return res;
}

```

503. 下一个更大元素 II

```

vector<int> nextGreaterElements(vector<int>& nums) {
    int n = nums.size();
    vector<int> res(n, -1);
    stack<int> st;

    // 遍历两倍的数组，然后还是坐标i对n取余，取出数字
    for (int i = 0; i < 2 * n; ++i) {
        int num = nums[i % n];
        while (!st.empty() && nums[st.top()] < num) {
            res[st.top()] = num; st.pop();
        }
        // res 的长度必须是n，超过n的部分我们只是为了给之前栈中的数字找较大值，所以不能压入栈
        if (i < n) st.push(i);
    }
    return res;
}

```

768. 最多能完成排序的块 II

```
// <https://www.cnblogs.com/grandyang/p/8850299.html>
int maxChunksToSorted(vector<int>& arr) {
    // 维护一个单调递增的栈，遇到大于等于栈顶元素的数字就压入栈，
    // 当遇到小于栈顶元素的数字后，处理的方法很是巧妙啊：首先取出栈顶元素，这个是当前最大值，因为我们维护的就是单调递增栈啊，然后我们再进行循环，如果栈不为空，且新的栈顶元素大于当前数字，则移除栈顶元素。
    stack<int> st;
    for (int i = 0; i < arr.size(); ++i) {
        if (st.empty() || st.top() <= arr[i]) {
            st.push(arr[i]);
        } else {
            int curMax = st.top(); st.pop();
            while (!st.empty() && st.top() > arr[i]) st.pop();
            st.push(curMax);
        }
    }
    return st.size();
}
```

二分查找

<https://leetcode.cn/problems/find-first-and-last-position-of-element-in-sorted-array/solution/yi-wen-dai-ni-gao-ding-er-fen-cha-zhao-j-ymwl/>

35. 搜索插入位置

```
int searchInsert(vector<int>& nums, int target) {
    if (nums.empty())
        return -1;
    int left = 0, right = nums.size() - 1;
    while(left <= right) // 终止循环的时候 left == right+1
    {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target)
            return mid;
        if (nums[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    // 返回的是插入位置 跳出循环了 说明没有找到target，这个地方返回left或者right+1 都可以
    return left; // todo: 这个地方为什么是返回left
}
```

34. 在排序数组中查找元素的第一个和最后一个位置 重点看下边界问题

<https://claude.ai/public/artifacts/f5d5e286-53ff-451c-889b-00ddb0362d72>

```
// 实际上返回的还是 插入的位置(在没有找到target的情况下)
// 当 nums[m] == target 时，说明小于 target 的元素在区间 [left, m - 1] 中，因此采用 right = m - 1 来缩小小区间，从而使指针 right 向小于 target 的元素靠近
```

```

int lower_bound(vector<int>& nums, int target)
{
    int left = 0, right = nums.size() - 1 ;
    while(left <= right) // 终止条件是 left = right + 1
    {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) // 如果当前位置就是第一个target的时候 那么接下来的判断都会是
            left=mid+1,终止条件就是left来带right+1位置,所以没有问题, 如果当前位置 不是第一个target的时候,
            right 一直往左边缩, 知道right 来到 第一个位置的前一个, 那么这个时候 [left...right]区间的上的数都会小于
            target, 那么left 就会一直+1, 直到跳出循环
            right = mid - 1;
        // 当 nums[mid] < target 时 left 移动, 这意味着指针 left 在向大于等于 target 的元素靠近
        else if (nums[mid] < target)
            left = mid + 1;
        else if (nums[mid] > target)
            right = mid - 1;
    }
    return left;
}

vector<int> searchRange(vector<int>& nums, int target) {

    if (nums.empty())
        return {-1, -1};

    int left = lower_bound(nums, target);
    int right = lower_bound(nums, target+1);
    if (left == nums.size() || nums[left] != target)
        return {-1, -1};
    return vector<int> {left, right - 1};
}

```

50. Pow(x, n) todo

```

double myPow(double x, int n) {
    //迭代的解法, 让i初始化为n, 然后看i是否是2的倍数, 不是的话就让 res 乘以x。然后x乘以自己, i每次循环缩
    小一半, 直到为0停止循环。最后看n的正负, 如果为负, 返回其倒数,
    double res = 1.0;
    for (int i = n; i != 0; i /= 2)
    {
        if (i % 2 != 0) res *= x;
        x *= x;
    }
    return n < 0 ? 1 / res : res;
}

```

69. Sqrt(x) #todo

```

int mySqrt(int x) {
    // 特殊情况: 0 和 1 的平方根就是它本身
}

```



```

if (x < 2) return x;

// 初始化二分区间:
// sqrt(x) 在 [1, x/2] 之间 (因为 x>=2 时, sqrt(x) 一定 <= x/2)
int left = 1, right = x / 2, ans = 0;

while (left <= right) {
    int mid = left + (right - left) / 2; // 防止溢出写法
    if (mid <= x / mid) {
        // 说明 mid*mid <= x, mid 是一个可行解
        ans = mid;
        left = mid + 1; // 继续尝试往右边找更大的平方根
    } else {
        right = mid - 1; // 说明 mid*mid > x, mid 太大了
    }
}
// 最终 ans 保存的是最大的 mid, 使得 mid*mid <= x
return ans;
}

```

367. 有效的完全平方数 todo

```

bool isPerfectSquare(int num) {
    if (num < 2) return true; // 0 和 1 都是完全平方数
    long left = 2, right = num / 2; // 平方根一定在 [2, num/2] 之间
    while (left <= right) {
        long mid = left + (right - left) / 2;
        long sq = mid * mid;

        if (sq == num) {
            return true; // 找到刚好平方数
        } else if (mid * mid < num) {
            left = mid + 1; // 平方太小, 往右找
        } else if (mid * mid > num) {
            right = mid - 1; // 平方太大, 往左找
        }
    }
    return false; // 没找到就是 false
}

```

33. 搜索旋转排序数组 #todo

// 即便数组被旋转了, 对于任意中间点 mid, 将整个数组切成两半, 至少有一半是有序的。

```

int search(vector<int>& nums, int target)
{
    int left = 0, right = nums.size() - 1;
    while (left <= right)
    {
        int mid = left + (right - left);
    }
}

```

```

        if (nums[mid] == target)
            return mid;
        if (nums[left] <= nums[mid]) // [left, mid] 递增序列 用等号因为需要考虑left和mid相等
的情况, 此时 [left, mid] 只有一个元素。
        {
            if (nums[left] <= target && target < nums[mid]) // 加等号, 因为 left 可能是
target
                right = mid - 1; // 在左侧 [left, mid) 查找 因为到了这个地方 num[mid] 已经不可能
等于target了
            else
                left = mid + 1;
        }
        else // (mid, right] 连续递增
        {
            if (nums[mid] < target && target <= nums[right]) // 加等号, 因为 right 可能是
target
                left = mid + 1; // 在右侧 (mid, right] 查找 因为到了这个地方 num[mid] 已经不可能
等于target了
            else
                right = mid - 1;
        }
    }
    return -1;
}

```

81. 搜索旋转排序数组 II #todo

```

// 核心点: 旋转数组里至少有一边是有序的 (如果不考虑重复元素的话)。
// 难点: 重复元素会导致无法直接判断哪边有序, 所以加了 nums[left] == nums[mid] 的处理。
bool search(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while(left <= right)
    {
        int mid = left + (right - left) / 2;
        // 1. 如果中点刚好等于目标, 直接返回 true
        if (nums[mid] == target)
            return true;
        // 2. 如果左半部分是有序的
        else if (nums[left] < nums[mid])
        {
            // 判断 target 是否在 [nums[left], nums[mid]) 这个有序区间里
            if (nums[left] <= target && target < nums[mid])
                right = mid - 1; // 在左半区间, 缩小右边界
            else
                left = mid + 1; // 否则去右半区间
        }
        // 3. 如果右半部分是有序的
        else if (nums[left] > nums[mid])
        {
            // 判断 target 是否在 (nums[mid], nums[right]] 这个有序区间里
            if (nums[mid] < target && target <= nums[right])

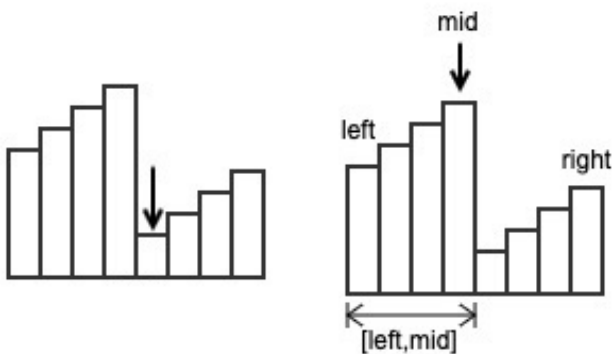
```

```

        left = mid + 1;    // 在右半区间，缩小左边界
    else
        right = mid - 1;   // 否则去左半区间
    }
    // 4. 处理 nums[left] == nums[mid] 的情况
    //     无法判断哪一边有序（因为有重复元素）
    //     例如 [1,0,1,1,1], mid = 2 时就会卡住
    //     这里我们只能安全地缩小搜索范围：left++
    else if (nums[left] == nums[mid])
    {
        left ++;
    }
}
// 如果循环结束还没找到，返回 false
return false;
}

```

153. 寻找旋转排序数组中的最小值



```

// https://imageslr.com/2020/03/06/leetcode-33.html
// 如果数组没有翻转，即 nums[left] <= nums[right]，则 nums[left] 就是最小值，直接返回
// 若 nums[left] <= nums[mid]，说明区间 [left,mid] 连续递增，则最小元素一定不在这个区间里，可以直接
// 排除。因此，令 left = mid+1，在 [mid+1,right] 继续查找
// 否则，说明区间 [left,mid] 不连续，则最小元素一定在这个区间里。因此，令 right = mid，在 [left,mid]
// 继续查找
// [left,right] 表示当前搜索的区间。注意 right 更新时会被设为 mid 而不是 mid-1，因为 mid 无法被排除。
// 这一点和「33 题 查找特定元素」是不同的
// 有序扔掉，无序保留
int findMin(vector<int> &nums)
{
    int left = 0;
    int right = nums.size() - 1;
    while(left <= right) // 实际上是不会跳出循环，当 left==right 时直接返回
    {
        if (nums[left] <= nums[right]) // = 是当区间只有一个元素的时候
            return nums[left];
        int mid = left + (right - left) / 2;
        if (nums[left] <= nums[mid]) // [left,mid] 连续递增，则在 [mid+1,right] 查找
        {
            left = mid + 1

```

```

    }
    else // [left,mid] 不连续, 在 [left,mid] 查找
    {
        right = mid
    }
}
return -1;
}

```

154. 寻找旋转排序数组中的最小值 II

```

int findMin(vector<int> &nums)
{
    int left = 0;
    int right = nums.size() - 1;
    while(left <= right)
    {
        // <= 换成 <, nums[left]==nums[right] 时无法判断 [left,right] 递增
        if (nums[left] < nums[right] || left == right) // 当区间中只有一个元素时, 直接返回
            return nums[left];

        int mid = left + (right - left) / 2;
        if (nums[left] < nums[mid]) // [left,mid] 连续递增, 则在 [mid+1,right] 查找
        {
            left = mid + 1;
        }
        else if (nums[left] == nums[mid])
        {
            left++;
        }
        else // [left,mid] 不连续, 在 [left,mid] 查找
        {
            right = mid;
        }
    }
    return -1;
}

```

278. 第一个错误的版本

```

int firstBadVersion(int n) {
    int left = 1, right = n;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (isBadVersion(mid)) right = mid;
        else left = mid + 1;
    }
    return left;
}

```

287. 寻找重复数 #TODO

```
//把数组看成链表：nums[i] 表示节点 i 指向的下一个节点。
// 重复数字导致环：相同的数字会让两个不同下标指向同一个节点 → 形成环。
// Floyd 判圈算法：
// 阶段一找到环内相遇点
// 阶段二从头和相遇点同时走，找到环入口 → 即重复数
int findDuplicate(vector<int>& nums) {
    // 寻找相遇点（环内）
    int slow = nums[0];
    int fast = nums[nums[0]]; // fast 先走一步，保证和 slow 不相等

    while (slow != fast) {
        slow = nums[slow]; // slow 每次走一步
        fast = nums[nums[fast]]; // fast 每次走两步
    }
    // 阶段二：找到环的入口（重复数）
    fast = 0; // fast 从头开始，注意这里是索引 0
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}

// 解法一：二分查找
// 给定一个包含 n + 1 个整数的数组 nums，其数字都在 [1, n] 范围内（包括 1 和 n），可知至少存在一个重复的整数。
int findDuplicate(vector<int>& nums)
{
    // 把1~n的数的数字从中间的数字mid分为两部分，1~mid，mid+1~n；
    // 如果1~mid中的数据数目超过mid，那么1~mid中间肯定存在重复数字，否则mid+1~n肯定存在重复数字
    int left = 1, right = nums.size();
    while (left < right){
        int mid = left + (right - left) / 2, cnt = 0;
        for (int num : nums) {
            if (num <= mid) ++cnt;
        }
        // 根据抽屉原理，小于等于 4 的个数如果严格大于 4 个
        // 此时重复元素一定出现在 [1, 4] 区间里
        if (cnt > mid) left = mid + 1;
        else right = mid; // 重复元素位于区间 [left, mid]
    }
    return left;
}
```

74. 搜索二维矩阵

```
bool searchMatrix(vector<vector<int>>& matrix, int target) {
    if (matrix.empty() || matrix[0].empty())
```

```

        return false;
int m = matrix.size();           // 矩阵行数
int n = matrix[0].size();       // 矩阵列数
// 2. 搜索起点选择：左下角
// 这里选择左下角 (row = m-1, col = 0) 作为起点
// 左下角的特点：
// - 上方的数比它小
// - 右方的数比它大
int left = m - 1; // 行索引
int right = 0;    // 列索引

while(left >= 0 && right < n)
{
    if (matrix[left][right] == target)
        return true; // 找到目标值，返回 true

    // 当前元素 < target → 目标值在右边
    if (matrix[left][right] < target)
        right++; // 列索引右移
    else
        left--; // 当前元素 > target → 目标值在上方，行索引上移
}
return false;
}

```

378. 有序矩阵中第K小的元素

1	3	5	7	9	11
2	4	6	8	10	12
3	5	7	9	11	13
4	6	8	10	12	14
5	7	9	11	13	15
6	8	10	12	14	16

```

class Solution {
public:
    // 计算矩阵中有多少个数字小于target
    int search_less_equal(vector<vector<int>>& matrix, int target) {
        int n = matrix.size();

```

```

int i = n - 1; // 从左下角开始
int j = 0;
int res = 0;    // 计数器

while (i >= 0 && j < n) {
    if (matrix[i][j] <= target) {
        res += i + 1; // 当前列的 i+1 个数 <= target
        ++j;          // 移动到下一列
    } else {
        --i;          // 上移到更小的元素
    }
}
return res;
}

int kthSmallest(vector<vector<int>>& matrix, int k) {
    int left = matrix[0][0];          // 最小值
    int right = matrix.back().back(); // 最大值

    while (left < right) {
        int mid = left + (right - left) / 2;
        int cnt = search_less_equal(matrix, mid);

        if (cnt < k)
            left = mid + 1; // 第 k 小在右边
        else
            right = mid;    // 第 k 小在左边或就是 mid
    }

    return left; // left == right, 就是第 k 小元素
}
};

```

540.有序数组中的单一元素 #todo

```

int singleNonDuplicate(vector<int>& nums) {
    // 通过数组的长度跟当前位置的关系，计算出右边和当前数字不同的数字的总个数，
    // 如果是偶数个，说明落单数左半边，反之则在右半边

```

// 由于给定数组有序 且 常规元素总是两两出现，因此如果不考虑“特殊”的单一元素的话，我们有结论：成对元素中的第一个所对应的下标必然是偶数，成对元素中的第二个所对应的下标必然是奇数。

// 然后再考虑存在单一元素的情况，假如单一元素所在的下标为x，那么下标 x 之前（左边）的位置仍满足上述结论，而下标 x 之后（右边）的位置由于 x 的插入，导致结论翻转

```

int left = 0, right = nums.size() - 1;
while (left < right) {
    int mid = left + (right - left) / 2;
    if (mid % 2 == 0) {
        if (mid + 1 < nums.size() && nums[mid] == nums[mid + 1])
            left = mid + 1;
        else

```

```

        right = mid;
    } else {
        if (mid - 1 >= 0 && nums[mid - 1] == nums[mid])
            left = mid + 1;
        else
            right = mid;
    }
}
return nums[right];
}

```

611. 有效三角形的个数

```

int triangleNumber(vector<int>& nums) {
    int res = 0, n = nums.size();
    // 三个数字中如果较小的两个数字之和大于第三个数字，那么任意两个数字之和都大于第三个数字，
    // 这很好证明，因为第三个数字是最大的，所以它加上任意一个数肯定大于另一个数
    // 先确定前两个数，将这两个数之和sum作为目标值，然后用二分查找法来快速确定第一个小于目标值的数
    sort(nums.begin(), nums.end());
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            int sum = nums[i] + nums[j], left = j + 1, right = n;
            while (left < right) {
                int mid = left + (right - left) / 2;
                if (nums[mid] < sum) left = mid + 1;
                else right = mid;
            }
            res += right - 1 - j;
        }
    }
    return res;
}

```

704. 二分查找

```

int search(vector<int> &nums, int target)
{
    if (nums.empty())
        return -1;
    int left = 0, right = nums.size() - 1;
    while(left <= right) // while(left <= right) 的终止条件是 left == right + 1, 写成区间的形
    式就是 [right + 1, right], 或者带个具体的数字进去 [3, 2], 可见这时候区间为空, 因为没有数字既大于等于 3
    又小于等于 2 的吧。所以这时候 while 循环终止是正确的, 直接返回 -1 即可。
    {
        int mid = left + (right - left) / 2;
        if (target == nums[mid])
            return mid;
        else if (target < nums[mid])
            right = mid - 1;
    }
}

```



```

        else
            left = mid + 1;
    }
    return -1;
}

```

排序

快排模板

912. 排序数组

```

[912. 排序数组](https://leetcode.cn/problems/sort-an-array/)
int partition(vector<int> &nums, int left, int right)
{
    // small指针: 指向下一个应该放置小于基准值元素的位置
    int small = left;
    for(int i = left; i < right; i++)
    {
        if (nums[i] < nums[right])
        {
            // 将当前元素与small位置的元素交换, 保证small左边的元素都小于基准值
            swap(nums[i], nums[small]);
            small++; // // small指针右移, 为下一个小元素预留位置
        }
    }
    swap(nums[small], nums[right]);
    return small;
}

void quickSort(vector<int> &nums, int left, int right)
{
    if (left < right)
    {
        int index = partition(nums, left, right);
        quickSort(nums, left, index - 1);
        quickSort(nums, index + 1, right);
    }
}

vector<int> sortArray(vector<int>& nums) {
    // write code here
    quickSort(nums, 0, nums.size()-1);
    return nums;
}

```

归并排序模板

```

class Solution {
public:

```

```

// 将数组不断拆分成左右两半，直到每个区间只有一个元素。
// 再将小区间两两合并，保证每次合并都是有序的。
//      [8, 4, 5, 7, 1, 3, 6, 2]
//      /           \
// [8, 4, 5, 7]      [1, 3, 6, 2]
//  /   \           /   \
// [8,4]  [5,7]      [1,3]  [6,2]
// /  \  /  \       /  \  /  \
// [8] [4] [5] [7]   [1] [3] [6] [2]
void merge(vector<int> &nums, int left, int mid, int right)
{
    vector<int> temp;

    int i = left, j = mid + 1;
    while(i <= mid && j <= right)
    {
        temp.push_back(nums[i] < nums[j] ? nums[i++] : nums[j++]);
    }
    while(i <= mid)
        temp.push_back(nums[i++]);
    while(j <= right)
        temp.push_back(nums[j++]);

    for(int i = 0; i < temp.size(); i++)
    {
        nums[i+left] = temp[i];
    }
}

void merge_sort(vector<int> &nums, int left, int right)
{
    if (left == right)
    {
        return;
    }
    int mid = left + (right - left) / 2;
    merge_sort(nums, left, mid);
    merge_sort(nums, mid+1, right);
    merge(nums, left, mid, right);
}

vector<int> sortArray(vector<int>& nums) {
    merge_sort(nums, 0, nums.size()-1);
    return nums;
}
};

```

堆排序模板

```

class Solution {
    void buildMaxHeap(vector<int>& nums) {

```

```

    int n = nums.size();
    // 从最后一个非叶子节点开始，向前遍历
    for (int i = (n - 1) / 2; i >= 0; --i) {
        maxHeapify(nums, i, n);
    }
}
// 通过不断比较父节点与其左右子节点的值，确保最大堆性质被维护，并在需要时交换元素位置以满足性质
void heapify(vector<int>& nums, int n, int i) {
    int largest = i;

    while (true) {
        int left = 2 * i + 1; // 左子节点的索引
        int right = 2 * i + 2; // 右子节点的索引

        // 如果左子节点存在且比当前节点大，则更新最大值索引
        if (left < n && nums[left] > nums[largest]) {
            largest = left;
        }

        // 如果右子节点存在且比当前节点大，则更新最大值索引
        if (right < n && nums[right] > nums[largest]) {
            largest = right;
        }

        // 如果最大值索引没有改变，说明当前节点满足最大堆性质，退出循环
        if (largest == i) {
            break;
        }

        // 否则，交换当前节点和最大值节点的值
        swap(nums[i], nums[largest]);
        i = largest; // 更新当前节点索引，继续向下迭代
    }
}

public:
    vector<int> sortArray(vector<int>& nums) {
        // heapSort 堆排序
        int n = nums.size();
        // 将数组整理成大根堆
        buildMaxHeap(nums);
        for (int i = n - 1; i >= 1; --i) {
            // 依次弹出最大元素，放到数组最后，当前排序对应数组大小 - 1
            swap(nums[0], nums[i]);
            --n;
            maxHeapify(nums, 0, n);
        }
        return nums;
    }
};

```

```

vector<int> sortArray(vector<int>& nums) {
    // bubbleSort
    int n = nums.size();
    for (int i = 0; i < n - 1; ++i) {
        bool flag = false;
        for (int j = 0; j < n - 1 - i; ++j) {
            if (nums[j] > nums[j + 1]) {
                swap(nums[j], nums[j + 1]);
                flag = true;
            }
        }
        if (flag == false) break; //无交换, 代表当前序列已经最优
    }
    return nums;
}

```

选择排序

```

class Solution {
public:
    vector<int> sortArray(vector<int>& nums) {
        // selectSort 选择排序
        int minIndex;
        int n = nums.size();
        for (int i = 0; i < n - 1; ++i) {
            minIndex = i;
            for (int j = i + 1; j < n; ++j) {
                if (nums[j] < nums[minIndex]) {
                    minIndex = j;
                }
            }
            swap(nums[i], nums[minIndex]);
        }
        return nums;
    }
};

```

插入排序

```

vector<int> sortArray(vector<int>& nums) {
    int n = nums.size();
    for(int i=1; i<n; i++){
        for(int j=i; j>0 && nums[j]<nums[j-1]; j--){
            swap(nums[j], nums[j-1]);
        }
    }
    return nums;
}

```

4. 寻找两个正序数组的中位数

```
class Solution
{
public:
    double findMedianSortedArrays(vector<int> &nums1, vector<int> &nums2)
    {
        // 使用一个小 trick, 分别找第 (m+n+1) / 2 个, 和 (m+n+2) / 2 个, 然后求其平均值即可, 这对奇
        // 偶数均适用
        int m = nums1.size(), n = nums2.size(), left = (m + n + 1) / 2, right = (m + n +
        2) / 2;
        return (findKth(nums1, 0, nums2, 0, left) + findKth(nums1, 0, nums2, 0, right)) /
        2.0;
    }

    // 两个有序数组中找到第k个元素
    // i和j分别来标记数组 nums1 和 nums2 的起始位置
    int findKth(vector<int> &nums1, int i, vector<int> &nums2, int j, int k)
    {
        // 某一个数组的起始位置大于等于其数组长度时, 说明其所有数字均已经被淘汰了, 相当于一个空数组了
        if (i >= nums1.size())
            return nums2[j + k - 1];
        if (j >= nums2.size())
            return nums1[i + k - 1];
        // k=1 的话, 只要比较 nums1 和 nums2 的起始位置i和j上的数字就可以了
        if (k == 1)
            return min(nums1[i], nums2[j]);
        int midVal1 = (i + k / 2 - 1 < nums1.size()) ? nums1[i + k / 2 - 1] : INT_MAX;
        int midVal2 = (j + k / 2 - 1 < nums2.size()) ? nums2[j + k / 2 - 1] : INT_MAX;
        // 如果第一个数组的第 k/2 个数字小的话, 那么说明要找的数字肯定不在 nums1 中的前 k/2 个数字, 可
        // 以将其淘汰, 将 nums1 的起始位置向后移动 k/2 个, 并且此时的k也自减去 k/2
        if (midVal1 < midVal2)
        {
            return findKth(nums1, i + k / 2, nums2, j, k - k / 2);
        }
        else
        {
            return findKth(nums1, i, nums2, j + k / 2, k - k / 2);
        }
    }
};
```

27. 移除元素

```
int removeElement(vector<int>& nums, int val)
{
    if (nums.empty())
        return 0;
    int len = 0;
    for(int i = 0; i < nums.size(); i++)
    {
```

```

        if (nums[i] != val)
        {
            nums[len] = nums[i];
            len++;
        }
    }
    return len;
}

```

75. 颜色分类

```

// 区间划分
// [0, small-1] → 0
// [small, index-1] → 1
// [index, large] → 未处理
// [large+1, n-1] → 2
// 遍历逻辑
// 如果是 0 → 交换到左边 (small 区), index++
// 如果是 1 → 正确位置, index++
// 如果是 2 → 交换到右边 (large 区), index 不++, 因为新交换进来的元素还未处理
void sortColors(vector<int>& nums) {
    if (nums.empty()) return;

    int small = 0;           // 指向“0区”下一个位置
    int large = nums.size() - 1; // 指向“2区”下一个位置
    int index = 0;           // 当前遍历元素位置

    while(index <= large) {
        if (nums[index] < 1) { // 当前元素是0
            swap(nums[index], nums[small]);
            index++;
            small++;
        }
        else if (nums[index] == 1) { // 当前元素是1
            index++;
        }
        else { // 当前元素是2
            swap(nums[index], nums[large]);
            large--;
            // 注意这里 index 不自增, 因为交换过来的元素还没处理
        }
    }
}

```

215. 数组中的第K个最大元素 todo

```

int findKthLargest(vector<int>& nums, int k) {
    // 第 k 大 = 前 k 大元素中最小的那个
    // 用一个大小为 k 的最小堆维护数组中当前遇到的 k 个最大元素, 堆顶就是第 k 大元素。
    // 1.初始化: 把数组前 k 个元素放入最小堆。
}

```

// 2.遍历剩余元素：如果当前元素比堆顶大，就替换堆顶（保证堆里始终是最大的 k 个元素）。

// 3.返回结果：遍历结束后，堆顶就是第 k 大的数。

```
priority_queue<int, vector<int>, greater<int>> min_heap;
```

```
for (int i = 0; i < k; i++)
{
    min_heap.push(nums[i]);
}
```

```
for (int i = k; i < nums.size(); i++)
{
    if (nums[i] > min_heap.top())
    {
        min_heap.pop();
        min_heap.push(nums[i]);
    }
}
return min_heap.top();
```

```
}
```

// 为什么要替换堆顶？

// 假设堆顶是 x，堆里元素是当前最大的 k 个数。

// 如果新来的元素 y <= x：

// y 比堆里最小的还小。

// 那么它连前 k 大都进不去 → 丢掉就行。

// 如果新来的元素 y > x：

// y 至少比堆里最小的一个大。

// 所以 y 有资格进入前 k 大。

// 那么就应该把最小的那个 (x) 踢掉，用 y 替换。

// 这样堆里继续保持“最大的 k 个数”。

```
class Solution {
public:
    int partition(vector<int> &nums, int left, int right)
    {
        int small = left; // 小于等于区域的下一个位置
        for(int i = left; i < right; i++)
        {
            if (nums[i] > nums[right]) // 从大到小
                swap(nums[i], nums[small++]);
        }
        swap(nums[small], nums[right]);
        return small;
    }

    int findKthLargest(vector<int>& nums, int k) {
        int left = 0, right = nums.size() - 1;
        while (true) {
            int pos = partition(nums, left, right);
            if (pos == k - 1) return nums[pos];
            if (pos > k - 1) right = pos - 1;
            else left = pos + 1;
        }
    }
};
```

```

    }

}

};

```

347. 前 K 个高频元素

```

vector<int> topKFrequent(vector<int>& nums, int k) {
    unordered_map<int, int> m;
    priority_queue<pair<int, int>> q;
    vector<int> res;
    // 先统计每个数字出现的次数
    for (auto a : nums)
        ++m[a];
    // 然后将次数 和数字 放到大根堆里面去, 然后从大根堆里面一次弹出k个数字
    for (auto it : m) q.push({it.second, it.first});
    for (int i = 0; i < k; ++i)
    {
        res.push_back(q.top().second);
        q.pop();
    }
    return res;
}

```

324. 摆动排序 II #todo

核心思想, 如果当前数小于num,当前数和小于区域的下一个数交换, 如果当前数大于num,当前数和大于区域的前一个数交换

```

class Solution {
public:
    int partition(vector<int> &nums, int left, int right)
    {
        int small = left; // small 指向小于区域的下一个元素 用最后一个元素作为枢轴元素
        for(int i = left; i < right; i++)
        {
            if (nums[i] < nums[right])
                swap(nums[i], nums[small++]);
        }
        swap(nums[small], nums[right]);
        return small;
    }

    int quickSelect(vector<int> &nums, int begin, int end, int k)
    {
        int index = partition(nums, begin, end);

        while(index != k)

```



```

{
    if (index > k)
    {
        end = index - 1;
        index = partition(nums, begin, end);
    }
    else
    {
        begin = index + 1;
        index = partition(nums, begin, end);
    }
}
return index;
}
void wiggleSort(vector<int>& nums)
{
    // 先找到中位数,

    int mid = nums[quickSelect(nums, 0, nums.size()-1, nums.size()/2)];
    auto midptr = nums.begin() + nums.size() / 2;
    int i = 0, j = 0, k = nums.size() - 1;
    // 3-way-partition
    // 然后根据中位数将原数组被分为3个部分, 左侧为小于中位数的数, 中间为中位数, 右侧为大于中位数的数
    while (j < k)
    {
        if (nums[j] > mid)
        {
            swap(nums[j], nums[k]);
            k--;
        }
        else if (nums[j] < mid)
        {
            swap(nums[j], nums[i]);
            i++;
            j++;
        }
        else
        {
            j++;
        }
    }

    if (nums.size() % 2)
        ++midptr;

    vector<int> tmp1(nums.begin(), midptr);
    vector<int> tmp2(midptr, nums.end());

    for (int i = 0; i < tmp1.size(); i++)
    {
        nums[2 * i] = tmp1[tmp1.size() - 1 - i];

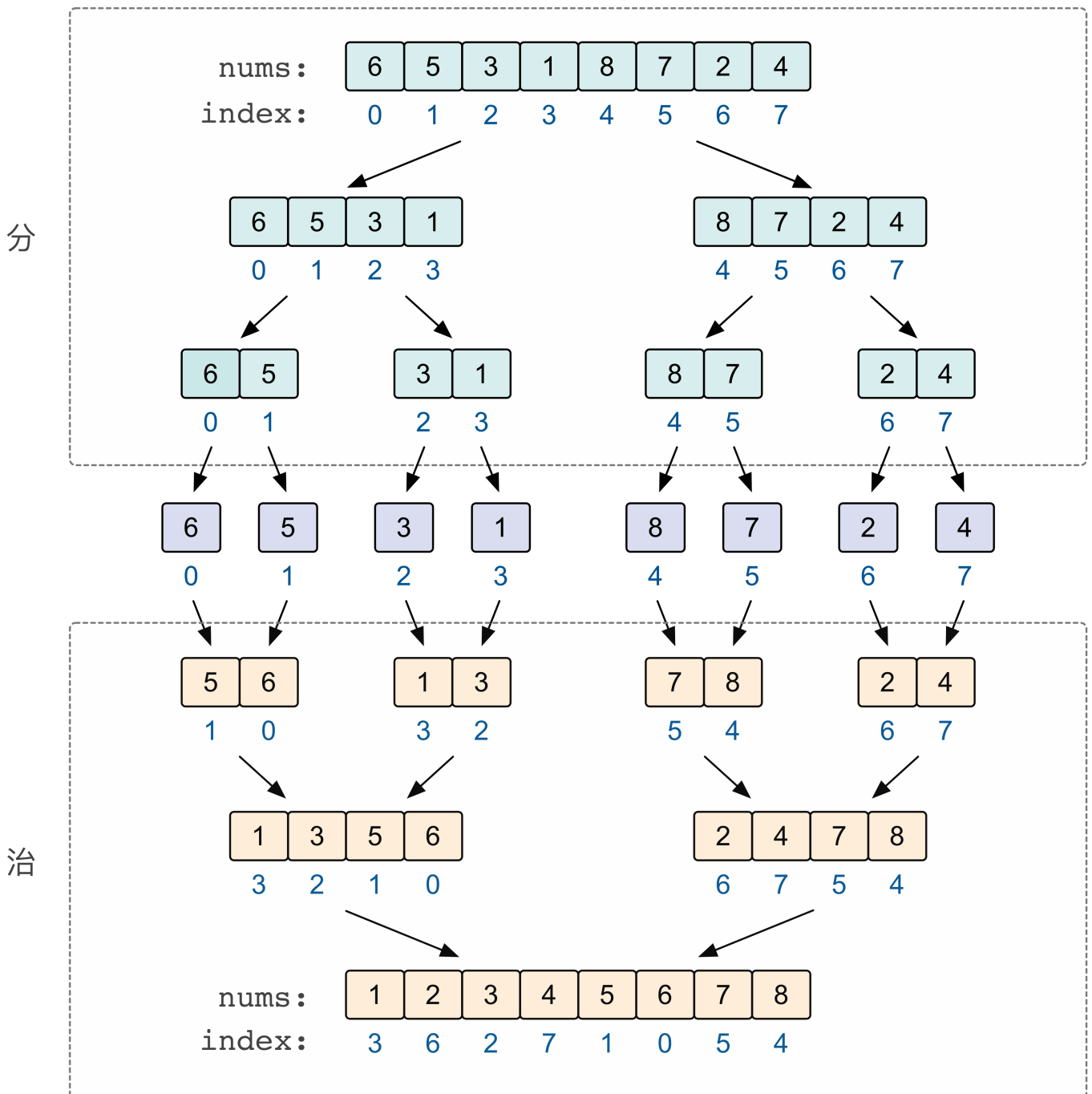
```

```

    }
    for (int i = 0; i < tmp2.size(); i++)
    {
        nums[2 * i + 1] = tmp2[tmp2.size() - 1 - i];
    }
}
};

```

315. 计算右侧小于当前元素的个数 todo



```

class Solution
{
public:

```

```

vector<int> index; //用于解决排序后原数组元素顺序变更问题
void merge(vector<int> &nums, int left, int mid, int right, vector<int> &res)
{
    vector<int> temp;
    vector<int> tempIndex;
    int i = left, j = mid + 1;
    while (i <= mid && j <= right)
    {
        if (nums[i] > nums[j]) //数组降序排列
        {
            res[index[i]] += right - j + 1; //用下标直接计算排序好的右半部分小于nums[i]元素
            tempIndex.push_back(index[i]); //记录元素位置的改变，与排序思想相同
            temp.push_back(nums[i++]);
        }
        else // nums[i]<=nums[j]，等号放在这里是更方便上面计算right-j+1这个数目
        {
            tempIndex.push_back(index[j]); //记录元素位置的改变，与排序思想相同
            temp.push_back(nums[j++]);
        }
    }
    while (i <= mid)
    {
        tempIndex.push_back(index[i]); //记录元素位置的改变，与排序思想相同
        temp.push_back(nums[i++]);
    }

    while (j <= right)
    {
        tempIndex.push_back(index[j]); //记录元素位置的改变，与排序思想相同
        temp.push_back(nums[j++]);
    }

    for (int i = 0; i < tempIndex.size(); ++i)
    {
        nums[left + i] = temp[i];
        index[left + i] = tempIndex[i];
    }
}

void mergesort(vector<int> &nums, int left, int right, vector<int> &res)
{
    if (left == right)
        return;
    int mid = (right - left) / 2 + left;
    mergesort(nums, left, mid, res);
    mergesort(nums, mid + 1, right, res);
    merge(nums, left, mid, right, res);
}

vector<int> countSmaller(vector<int> &nums)
{
    vector<int> res(nums.size(), 0);
    index.resize(nums.size());
}

```

的数目

```

        for (int i = 0; i < nums.size(); ++i)
            index[i] = i;
        mergesort(nums, 0, nums.size() - 1, res);
        return res;
    }
};

```

493. 翻转对

```

class Solution {
public:
    int merge(vector<int> &nums, int left, int mid, int right)
    {
        int i = left, j = mid + 1; // i指向左子数组, j指向右子数组
        vector<int> temp;           // 临时数组存储合并结果
        int count = 0;              // 统计逆序对数量

        // 第一阶段: 统计满足 nums[i] > 2 * nums[j] 的逆序对
        // 这里利用了两个子数组都已经有序的性质
        while(i <= mid && j <= right)
        {
            // 如果 nums[i] > 2 * nums[j], 说明找到了逆序对
            // 由于左子数组是有序的, 从i到mid的所有元素都满足条件
            if (nums[i] > 2LL * nums[j]) // 使用2LL避免整数溢出
            {
                count += mid - i + 1; // 一次性统计从i到mid的所有逆序对
                j++;                  // 移动右指针继续查找
            }
            else
            {
                i++;                  // 当前nums[i]不满足条件, 移动左指针
            }
        }

        // 第二阶段: 标准归并排序, 合并两个有序子数组
        i = left, j = mid + 1; // 重置指针

        // 比较两个子数组的元素, 将较小的放入临时数组
        while(i <= mid && j <= right)
        {
            temp.push_back(nums[i] < nums[j] ? nums[i++] : nums[j++]);
        }

        while(i <= mid)
            temp.push_back(nums[i++]);

        while(j <= right)
            temp.push_back(nums[j++]);

        for (int i = 0; i < temp.size(); i++)
            nums[i + left] = temp[i];
    }
};

```

```

        return count; // 返回跨越两个子数组的逆序对数量
    }

    int merge_sort(vector<int> &nums, int left, int right)
    {
        // 递归终止条件：只有一个元素或区间为空
        if (left == right)
            return 0;

        // 计算中点，避免整数溢出
        int mid = left + (right - left) / 2;

        // 递归处理左半部分，统计左半部分内部的逆序对
        int left_count = merge_sort(nums, left, mid);

        // 递归处理右半部分，统计右半部分内部的逆序对
        int right_count = merge_sort(nums, mid + 1, right);
        // 合并两部分，并统计跨越两部分的逆序对
        // 总的逆序对数 = 左半部分逆序对 + 右半部分逆序对 + 跨越两部分的逆序对
        return left_count + right_count + merge(nums, left, mid, right);
    }

    int reversePairs(vector<int>& nums) {
        // 边界情况处理：数组长度小于等于1时，不存在逆序对
        if (nums.size() <= 1)
            return 0;

        // 调用归并排序函数，统计整个数组的逆序对
        return merge_sort(nums, 0, nums.size() - 1);
    }
};

/*
算法思路总结：
1. 使用分治思想，将数组分为左右两部分
2. 递归统计左半部分和右半部分内部的逆序对
3. 在合并过程中统计跨越左右两部分的逆序对
4. 利用归并排序保持数组有序，为后续统计提供便利

时间复杂度：O(n log n)
空间复杂度：O(n)

关键优化点：
- 在统计阶段利用数组有序性质，一次性统计多个逆序对
- 使用2LL避免整数乘法溢出
- 分离统计和合并两个阶段，逻辑清晰
*/

```

LCR 170. 交易逆序对的总数

```

class Solution {

```

```

public:
    int merge(vector<int>& nums, int left, int mid, int right)
    {
        vector<int> temp;
        int i = left, j = mid + 1;
        int count = 0;           // 统计逆序对数量

        while(i <= mid && j <= right)
        {
            // 关键逻辑：当左子数组元素大于右子数组元素时
            if (nums[i] > nums[j])
            {
                // 由于左右子数组都是有序的，如果nums[i] > nums[j]
                // 那么从i到mid的所有元素都大于nums[j]，形成逆序对
                count += mid - i + 1; // 一次性统计多个逆序对
                temp.push_back(nums[j++]);
            }
            else
            {
                temp.push_back(nums[i++]);
            }
        }

        while(i <= mid)
            temp.push_back(nums[i++]);

        while(j <= right)
            temp.push_back(nums[j++]);

        // 将排序后的临时数组复制回原数组的对应位置
        // 这样确保了子数组在下次合并时是有序的
        for(int i = 0; i < temp.size(); i++)
        {
            nums[i + left] = temp[i];
        }

        return count; // 返回本次合并过程中统计到的逆序对数量
    }

    int merge_sort(vector<int>& nums, int left, int right)
    {
        // 递归终止条件：当区间只包含一个元素时，不存在逆序对
        if (left == right)
            return 0;
        int mid = left + (right - left) / 2;

        // 分治策略：
        // 1. 递归处理左半部分，统计左半部分内部的逆序对
        int left_count = merge_sort(nums, left, mid);

        // 2. 递归处理右半部分，统计右半部分内部的逆序对
        int right_count = merge_sort(nums, mid + 1, right);
    }

```

```

// 3. 合并两部分，统计跨越左右两部分的逆序对
int count = merge(nums, left, mid, right);

// 返回三部分逆序对的总和
return left_count + right_count + count;
}
int reversePairs(vector<int>& nums) {
    // 边界情况处理：空数组没有逆序对
    if (nums.empty())
        return 0;

    // 调用归并排序函数处理整个数组
    return merge_sort(nums, 0, nums.size() - 1);
}
};

```

/*

算法核心思想：

这是经典的逆序对问题，与之前的问题不同，这里统计的是满足 $i < j$ 且 $nums[i] > nums[j]$ 的数对。

关键创新点：

1. 将逆序对统计与归并排序过程完美结合
2. 在归并过程中，当发现 $nums[i] > nums[j]$ 时，利用数组有序性一次性统计从 i 到 mid 的所有逆序对
3. 通过分治将 $O(n^2)$ 的暴力解法优化为 $O(n \log n)$

算法流程：

1. 分治：将数组递归分为左右两部分
2. 征服：分别统计左右部分内部的逆序对
3. 合并：在合并有序数组的过程中统计跨越两部分的逆序对

时间复杂度： $O(n \log n)$ - 与归并排序相同

空间复杂度： $O(n)$ - 需要临时数组存储合并结果

应用场景：

- 数组排序程度的衡量
- 排序算法的交换次数计算
- 某些动态规划问题的子问题

*/

969. 煎饼排序 todo

```

vector<int> pancakeSort(vector<int>& arr) {
    vector<int> ret;
    for (int n = arr.size(); n > 1; n--) {
        int index = max_element(arr.begin(), arr.begin() + n) - arr.begin();
        if (index == n - 1) {
            continue;
        }
    }
}

```

```

    }
    reverse(arr.begin(), arr.begin() + index + 1);
    reverse(arr.begin(), arr.begin() + n);
    ret.push_back(index + 1);
    ret.push_back(n);
}
return ret;
}

```

// 每次先将数组中最大数字找出来，然后将最大数字翻转到首位置，然后翻转整个数组，这样最大数字就跑到最后去了

```

vector<int> pancakeSort(vector<int>& arr) {
    vector<int> res;
    for (int i = arr.size(), j; i > 0; --i) {
        for (j = 0; arr[j] != i; ++j);
        reverse(arr.begin(), arr.begin() + j + 1);
        res.push_back(j + 1);
        reverse(arr.begin(), arr.begin() + i);
        res.push_back(i);
    }
    return res;
}

```

链表

链表总结

1) i 从零开始 最终fast会停在第n个节点上，n从零开始 头结点开始

```

ListNode *fast = head;
for(int i = 0; i < n; i++)
{
    fast = fast->next;
}

```

2) 找链表的中间节点

```

ListNode *fast = head;
ListNode *slow = head;
// 如果链表个数为奇数，那么s直接走到了中间结点，如果是偶数则是中间结点的前一个
while(fast->next && fast->next->next)
{
    slow = slow->next;
    fast = fast->next->next;
}

ListNode *slow = head, *fast = head, *last = slow;
while (fast->next && fast->next->next) {
    last = slow;
}

```



```

    slow = slow->next;
    fast = fast->next->next;
}

```

3) 链表尾插法和头插

```

// 尾插法
cur->next = new ListNode(sum % 10);
cur = cur->next;

// 头插法
ListNode *temp = new ListNode(-1)
temp->next = fake_head->next;

// 把cur后面的一个节点temp摘下来，然后用头插法插入到pre后面
ListNode *temp = cur->next;
cur->next = temp->next;
temp->next = pre->next;
pre->next = temp;

```

4) 环形链表

```

if (head == nullptr || head->next == nullptr || head->next->next == nullptr)
    return false;

ListNode *slow = head->next;
ListNode *fast = head->next->next;

while(slow != fast)
{
    if (fast->next == nullptr || fast->next->next == nullptr)
        return false;
    slow = slow->next;
    fast = fast->next->next;
}

```

<https://www.nowcoder.com/practice/71cef9f8b5564579bf7ed93fbe0b2024?tpId=117&tqId=37729&companyId=665&rp=1&ru=%2Fcompany%2Fhome%2Fcode%2F665&qu=%2Fta%2Fjob-code-high%2Fquestion-ranking&tab=answerKey>

```

ListNode* deleteDuplicates(ListNode* head)
{
    // write code here
    if (!head || !head->next)
        return head;
    ListNode *dummy = new ListNode(-1), *pre = dummy;
    dummy->next = head;
    while (pre->next)

```

```

{
    ListNode *cur = pre->next;
    while (cur->next && cur->next->val == cur->val)
    {
        cur = cur->next;
    }
    if (cur != pre->next)
        pre->next = cur->next;
    else
        pre = pre->next;
}
return dummy->next;
}

```

普通链表

2. 两数相加

```

ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    ListNode *fake_head = new ListNode(-1), *cur = fake_head;
    int carry_num = 0;
    while(l1 || l2)
    {
        int val_1 = l1 ? l1->val : 0;
        int val_2 = l2 ? l2->val : 0;
        int sum = val_1 + val_2 + carry_num;
        carry_num = sum / 10;
        // 护了一个指针 cur, 每次 cur->next = new ListNode(...), 再把 cur 后移。新节点会接在链表尾部
        cur->next = new ListNode(sum % 10);
        cur = cur->next;
        if (l1) l1 = l1->next;
        if (l2) l2 = l2->next;
    }
    if (carry_num)
        cur->next = new ListNode(1);
    return fake_head->next;
}

```

445. 两数相加 II

```

ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
    stack<int> s1, s2;

    // 把两个链表的数字压栈
    while (l1) {
        s1.push(l1->val);
        l1 = l1->next;
    }
    while (l2) {
        s2.push(l2->val);
    }
}

```

```

        l2 = l2->next;
    }

    int carry = 0;
    ListNode *fake_head = new ListNode(-1);
    fake_head->next = nullptr;

    // 逐位相加
    while (!s1.empty() || !s2.empty() || carry) {
        int sum = carry;
        if (!s1.empty()) { sum += s1.top(); s1.pop(); }
        if (!s2.empty()) { sum += s2.top(); s2.pop(); }
        carry = sum / 10;
        // 头插法: 新节点放到前面
        ListNode *cur = new ListNode(sum % 10);
        cur->next = fake_head->next;
        fake_head->next = cur;
    }
    return fake_head->next;
}

```

24. 两两交换链表中的节点 #todo

```

ListNode* swapPairs(ListNode* head) {
    if (head == nullptr)
        return head;

    ListNode *fake_head = new ListNode(-1);
    ListNode *cur = fake_head;
    fake_head->next = head;

    while (cur->next && cur->next->next)
    {
        // 挨个摘下节点 然后使用头插法
        ListNode *temp = cur->next->next;
        cur->next->next = temp->next;
        temp->next = cur->next;
        cur->next = temp;
        cur = temp->next;
    }
    return fake_head->next;
}

```

82. 删除排序链表中的重复元素 II todo

```

ListNode* deleteDuplicates(ListNode* head) {
    // 虚拟头节点, 方便处理头部重复元素
    ListNode* fake_head = new ListNode(-1);
    fake_head->next = head;
    ListNode* prev = fake_head; // 指向最后一个不重复的节点
    ListNode* cur = head;      // 当前遍历节点

```

```

while (cur) {
    // 如果当前节点和下一个节点值相同，说明有重复
    if (cur->next && cur->val == cur->next->val) {
        int dup_val = cur->val;
        // 跳过所有重复节点
        while (cur && cur->val == dup_val) {
            cur = cur->next;
        }
        // 上一个不重复节点指向下一个不重复节点
        prev->next = cur;
    } else {
        // 当前节点不重复，prev 后移
        prev = cur;
        cur = cur->next;
    }
}
return fake_head->next;
}

```

83. 删除排序链表中的重复元素 todo

```

// 链表有序，所以重复节点一定是连续的，可以用 单指针遍历 + 判断下一个节点值 来做
ListNode* deleteDuplicates(ListNode* head) {
    ListNode* cur = head;
    // 遍历链表
    while (cur && cur->next) {
        if (cur->val == cur->next->val) {
            // 删除重复节点
            ListNode* temp = cur->next;
            cur->next = cur->next->next;
            delete temp; // 可选，释放内存
        } else {
            // 不重复，指针后移
            cur = cur->next;
        }
    }
    return head;
}

```

237. 删除链表中的节点

```

void deleteNode(ListNode* node) {
    node->val = node->next->val;
    ListNode *temp = node->next;
    node->next = node->next->next;
    delete temp;
}

```

138. 复制带随机指针的链表 todo

```

Node* copyRandomList(Node* head)
{
    if (head == nullptr)
        return head;
    Node *res = new Node(head->val, nullptr, nullptr);
    Node *node = res;
    Node *cur = head->next;
    unordered_map<Node *, Node *> m;
    m[head] = node;
    // 第一遍遍历生成所有新节点时同时建立一个原节点和新节点的 HashMap
    while(cur)
    {
        Node *temp = new Node(cur->val, nullptr, nullptr);
        node->next = temp;
        m[cur] = temp;
        node = node->next;
        cur = cur->next;
    }
    // 第二遍给随机指针赋值时
    node = res; cur = head;
    while(cur)
    {
        node->random = m[cur->random];
        cur = cur->next;
        node = node->next;
    }
    return res;
}

```

K路归并

[21. 合并两个有序链表](#)

```

ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
    // 迭代 + 尾插法
    if (list1 == nullptr && list2 == nullptr)
        return nullptr;
    ListNode *fake_head = new ListNode(-1);
    ListNode *cur = fake_head;
    while(list1 && list2){
        if (list1->val < list2->val) {
            cur->next = list1;
            list1 = list1->next;
        }
        else {
            cur->next = list2;
            list2 = list2->next;
        }
        cur = cur->next;
    }
    cur->next = list1 ? list1 : list2;
}

```

```

return fake_head->next;
}

```

23. 合并K个升序链表 #todo 最小堆的做法 需要熟悉STL

```

class Solution {
public:
    // 利用了最小堆这种数据结构，首先把k个链表的首元素都加入最小堆中，它们会自动排好序。然后每次取出最小的那个元素加入最终结果的链表中，然后把取出元素的下一个元素再加入堆中，下次仍从堆中取出最小的元素做相同的操作，以此类推，直到堆中没有元素了，此时k个链表也合并为了一个链表，返回首节点即可
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        // 自定义比较函数（用于小顶堆）
        // 如果 a->val > b->val, 则返回 true, 表示 a 的优先级比 b 低
        // priority_queue 默认是大顶堆，加这个 cmp 后变成小顶堆
        auto cmp = [] (ListNode *a, ListNode *b) {
            return a->val > b->val;
        };
        priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> min_heap(cmp);
        // 初始化：将每个链表的头节点加入堆中（如果不为空）
        for (auto node : lists) {
            if (node)
                min_heap.push(node);
        }
        ListNode *fake_head = new ListNode(-1);
        ListNode *cur = fake_head;

        // 当堆不为空时，持续取出当前最小的节点
        while (!min_heap.empty()) {
            // 取出堆顶（当前所有节点中值最小的）
            auto t = min_heap.top();
            min_heap.pop();
            // 将该最小节点接到结果链表尾部
            cur->next = t;
            cur = cur->next;
            if (t->next)
                min_heap.push(t->next);
        }

        return fake_head->next;
    }
};

```

快慢指针

剑指 Offer 22. 链表中倒数第k个节点

```

ListNode* getKthFromEnd(ListNode* head, int k)
{
    if (head == nullptr || k < 0)
        return head;
}

```

```

ListNode *fast = head;
ListNode *slow = head;

for(int i = 0; i < k; i++)
{
    fast = fast->next;
}
while (fast)
{
    fast = fast->next;
    slow = slow->next;
}
return slow;
}

```

19. 删除链表的倒数第 N 个结点 # todo 注意细节

```

ListNode* removeNthFromEnd(ListNode* head, int n) {
    if (head == nullptr)
        return nullptr;
    ListNode *fast = head, *slow = head;
    // 让 fast 先走 n 步, 这样 fast 和 slow 之间相隔 n 个节点
    for (int i = 0; i < n; i++) {
        fast = fast->next;
    }
    // 如果 fast 走到头, 说明要删除的是第一个节点
    if (fast == nullptr)
        return head->next;
    // fast 和 slow 同时前进, 直到 fast 到达最后一个节点
    // 此时 slow 停在待删除节点的前一个位置
    while (fast->next) {
        fast = fast->next;
        slow = slow->next;
    }
    slow->next = slow->next->next;
    return head;
}

```

61. 旋转链表

```

ListNode* rotateRight(ListNode* head, int k) {
    if (head == nullptr) return nullptr;
    // 1. 统计链表长度
    int count = 0;
    ListNode *cur = head;
    while (cur) {
        cur = cur->next;
        count++;
    }
    // 2. 取模, 避免无效的整圈旋转
    int last_n = k % count;
}

```

```

    if (last_n == 0) return head; // 旋转整圈等于不变
    // 3. 快慢指针, 让 fast 先走 last_n 步
    ListNode *fast = head, *slow = head;
    for (int i = 0; i < last_n; i++) {
        fast = fast->next;
    }
    // 4. 同时移动 fast 和 slow, 直到 fast 到达链表末尾
    // 此时 slow 正好停在“新头节点的前一个节点”
    while (fast->next) {
        fast = fast->next;
        slow = slow->next;
    }
    // 5. 断开并重连
    fast->next = head; // 尾部连到原头部, 形成环
    ListNode *new_head = slow->next; // 新头节点
    slow->next = nullptr; // 断开环
    return new_head;
}

```

141. 环形链表

假设链表中的非环部分的长度为 $L1$, 环的长度为 $L2$, 环的入口到相遇点的距离为 D , 相遇点到环的入口的距离为 R 。快指针每次走两步, 慢指针每次走一步。在相遇时, 快指针走过的距离是慢指针的两倍, 因此可以得到以下关系

$2(L1 + D) = L1 + D + n(L2 + D)$ 进一步整理

$L1 + D = n(L2 + D)$

这意味着在相遇点时, 如果从链表头部开始走 $L1$ 的距离, 慢指针也刚好会到达环的入口, 而从相遇点继续走 $n(L2 + D)$ 的距离也会到达环的入口。

```

bool hasCycle(ListNode *head) {
    if (head == nullptr || head->next == nullptr) {
        return false;
    }
    ListNode *slow = head;
    ListNode *fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next; // 慢指针走一步
        fast = fast->next->next; // 快指针走两步

        if (slow == fast) { // 相遇说明有环
            return true;
        }
    }
    return false; // fast 到了链表末尾 -> 没环
}

```

142. 环形链表 II

```

ListNode *detectCycle(ListNode *head) {

```



```

if (head == nullptr || head->next == nullptr) {
    return nullptr;
}
ListNode *slow = head;
ListNode *fast = head;
// 第一步: 判断是否有环
while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;

    if (slow == fast) {
        // 第二步: 找到环的入口
        ListNode *ptr1 = head;
        ListNode *ptr2 = slow; // 相遇点
        while (ptr1 != ptr2) {
            ptr1 = ptr1->next;
            ptr2 = ptr2->next;
        }
        return ptr1; // 或者 return ptr2;
    }
}
return nullptr; // 无环
}

```

143. 重排链表 #TODO

```

/*
解法一：
1. 使用快慢指针来找到链表的中点，并将链表从中点处断开，形成两个独立的链表。
2. 将第二个链翻转。
3. 将第二个链表的元素间隔地插入第一个链表中。
*/
void reorderList(ListNode* head) {
    if (!head || !head->next) return;

    // 1. 找到中点
    ListNode *slow = head, *fast = head;
    while (fast->next && fast->next->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // 2. 反转后半部分
    ListNode *prev = nullptr, *cur = slow->next;
    slow->next = nullptr; // 切断前半部分和后半部分
    while (cur) {
        ListNode *tmp = cur->next;
        cur->next = prev;
        prev = cur;
        cur = tmp;
    }
}

```

```

// 3. 合并两个链表
ListNode *l1 = head, *l2 = prev;
while (l1 && l2) {
    ListNode *n1 = l1->next;
    ListNode *n2 = l2->next;

    l1->next = l2;
    if (n1 == nullptr) break; // 避免尾部成环
    l2->next = n1;
    l1 = n1;
    l2 = n2;
}
}

```

/**

* 解法二:

* 其实可以借助栈的后进先出的特性来做，如果我们按顺序将所有的结点压入栈，那么出栈的时候就可以倒序了，实际上就相当于翻转了链表。由于只需将后半段链表翻转，那么我们就要控制出栈结点的个数，还好栈可以直接得到结点的个数，我们减1除以2，就是要出栈结点的个数了。

*/

```

void reorderList(ListNode *head)
{
    if (!head || !head->next || !head->next->next)
        return;
    stack<ListNode*> st;
    ListNode *cur = head;
    while (cur)
    {
        st.push(cur);
        cur = cur->next;
    }
    int cnt = ((int)st.size() - 1) / 2;
    cur = head;
    while (cnt > 0)
    {
        auto t = st.top();
        st.pop();
        ListNode *next = cur->next;
        cur->next = t;
        t->next = next;
        cur = next;
        cnt--;
    }
    st.top()->next = NULL;
}

```

160. 相交链表 # todo 双指针 虚假的easy

// 让两条链表分别从各自的开头开始往后遍历，当其中一条遍历到末尾时，跳到另一个条链表的开头继续遍历

// 两条链表分别从各自的开头开始往后遍历，当其中一条遍历到末尾时，跳到另一个链表的开头继续遍历。两个指针最终会相等，而且只有两种情况，一种情况是在交点处相遇，另一种情况是在各自的末尾的空节点处相等。为什么一定会相等呢，因为两个指针走过的路程相同，是两个链表的长度之和，所以一定会相等、

// 创建两个指针 ptrA 和 ptrB，分别初始化为链表 A 的头节点和链表 B 的头节点。

// 同时遍历链表 A 和链表 B，如果某个指针到达链表末尾，则将其重新指向另一个链表的头节点。

// 当两个指针相遇时，它们在相交点相遇，如果没有相交点，它们最终都会指向nullptr。

// 这个代码的核心思想是让两个指针同时遍历链表A和链表B，如果没有相交点，它们最终都会指向nullptr。

// 如果有相交点，它们会在相交点相遇

```
ListNode *getIntersectionNode(ListNode *headA, ListNode *headB)
{
    ListNode* ptrA = headA;
    ListNode* ptrB = headB;
    while (ptrA != ptrB) {
        // 如果ptrA到达链表末尾，将其重新指向链表B的头节点
        if (ptrA == nullptr) {
            ptrA = headB;
        } else {
            ptrA = ptrA->next;
        }
        // 如果ptrB到达链表末尾，将其重新指向链表A的头节点
        if (ptrB == nullptr) {
            ptrB = headA;
        } else {
            ptrB = ptrB->next;
        }
    }
    return ptrA; // 返回相交的起始节点或nullptr
}
```

234. 回文链表

```
bool isPalindrome(ListNode* head) {
    if (!head || !head->next) return true;

    // 1. 找中点
    ListNode *slow = head, *fast = head;
    while(fast->next && fast->next->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // 2. 反转后半部分
    ListNode *cur = slow->next, *pre = nullptr;
    slow->next = nullptr; // 切断前半部分
    while(cur) {
        ListNode *tmp = cur->next;
        cur->next = pre;
        pre = cur;
        cur = tmp;
    }
}
```

```

// 3. 比较前后半部分
ListNode *p1 = head, *p2 = pre;
while(p2) {
    if(p1->val != p2->val) return false;
    p1 = p1->next;
    p2 = p2->next;
}
return true;
}

```

链表排序

86. 分隔链表 # todo

```

ListNode* partition(ListNode* head, int x) {
    if (!head) return nullptr;
    // small_head 用来存放小于 x 的节点
    // large_head 用来存放大于等于 x 的节点
    ListNode* small_head = new ListNode(-1);
    ListNode* large_head = new ListNode(-1);

    ListNode* small = small_head; // small 链表的尾指针
    ListNode* large = large_head; // large 链表的尾指针

    // 遍历原链表
    while (head) {
        if (head->val < x) {
            small->next = head; // 挂到 small 链表
            small = small->next; // 更新 small 尾指针
        } else {
            large->next = head; // 挂到 large 链表
            large = large->next; // 更新 large 尾指针
        }
        head = head->next; // 遍历下一个节点
    }

    // 拼接两个链表: small 的尾巴指向 large_head->next
    small->next = large_head->next;
    // large 的尾巴必须断开, 否则可能出现环
    large->next = nullptr;
    return small_head->next;
}

```

147. 对链表进行插入排序 todo

```

ListNode* insertionSortList(ListNode* head) {
    if (!head || !head->next) {
        return head; // 链表为空或只有一个节点, 直接返回
    }
    ListNode* dummy = new ListNode(0);
    dummy->next = head;
    // lastSorted 指向当前已排序链表的最后一个节点

```

```

// cur 指向待插入的节点
ListNode* lastSorted = head;
ListNode* cur = head->next;
while (cur) {
    if (cur->val >= lastSorted->val) {
        // 当前节点已经在正确位置, 无需移动
        lastSorted = cur;
    } else {
        // 需要将 cur 插入到前面合适位置
        ListNode* prev = dummy;
        // 从头开始找插入位置
        while (prev->next->val < cur->val) {
            prev = prev->next;
        }
        // 把 cur 插入到 prev 和 prev->next 之间
        lastSorted->next = cur->next;
        cur->next = prev->next;
        prev->next = cur;
    }
    cur = lastSorted->next; // 更新 cur (注意: cur 是否更新取决于是否插入)
}
return dummy->next;
}

```

148. 排序链表 对链表使用归并的方式排序

```

ListNode *merge_list(ListNode *l1, ListNode *l2)
{
    ListNode *fake_head = new ListNode(-1);
    ListNode *cur = fake_head;
    while(l1 && l2) {
        if (l1->val < l2->val) {
            cur->next = l1;
            l1 = l1->next;
        }
        else {
            cur->next = l2;
            l2 = l2->next;
        }
        cur = cur->next;
    }
    cur->next = l1 ? l1 : l2;
    return fake_head->next;
}

ListNode* sortList(ListNode* head) {
    if (head == nullptr || head->next == nullptr)
        return head;
    ListNode *slow = head;
    ListNode *fast = head;
    while(fast->next && fast->next->next) {
        slow = slow->next;
    }
}

```

```

        fast = fast->next->next;
    }
    // 分割链表
    ListNode* mid = slow->next;
    slow->next = nullptr;
    // 递归排序两部分并合并
    ListNode* left = sortList(head);
    ListNode* right = sortList(mid);
    return merge_list(left, right);
}

```

148. 排序链表 对链表使用快排

```

class Solution {
public:
    ListNode* quickSortList(ListNode* head) {
        // 基准情况：如果链表为空或只有一个节点，返回链表本身
        if (!head || !head->next) {
            return head;
        }

        // 选择基准节点
        ListNode* pivot = head;

        // 分割链表为小于基准值、等于基准值和大于基准值的三部分
        ListNode* less_head = nullptr;
        ListNode* less_tail = nullptr;
        ListNode* equal_head = nullptr;
        ListNode* equal_tail = nullptr;
        ListNode* greater_head = nullptr;
        ListNode* greater_tail = nullptr;

        ListNode* current = head;

        while (current) {
            if (current->val < pivot->val) {
                // 当前节点小于基准值，放入小于基准值的链表中
                if (less_head == nullptr) {
                    less_head = current;
                    less_tail = current;
                } else {
                    less_tail->next = current;
                    less_tail = current;
                }
            } else if (current->val == pivot->val) {
                // 当前节点等于基准值，放入等于基准值的链表中
                if (equal_head == nullptr) {
                    equal_head = current;
                    equal_tail = current;
                } else {
                    equal_tail->next = current;

```

```

        equal_tail = current;
    }
} else {
    // 当前节点大于基准值，放入大于基准值的链表中
    if (greater_head == nullptr) {
        greater_head = current;
        greater_tail = current;
    } else {
        greater_tail->next = current;
        greater_tail = current;
    }
}
current = current->next;
}

// 递归对小于和大于基准值的两部分进行快速排序
less_tail = equal_tail = greater_tail = nullptr; // 清空尾节点
less_head = quickSortList(less_head);
greater_head = quickSortList(greater_head);

// 拼接排好序的链表
if (less_tail) {
    less_tail->next = equal_head;
    equal_tail = (equal_tail) ? equal_tail : less_tail;
}
equal_tail->next = greater_head;

return (less_head) ? less_head : equal_head;
}

ListNode* sortList(ListNode* head) {
    return quickSortList(head);
}
};

```

708.排序的循环链表

```

class Solution {
public:
    Node* insert(Node* head, int insertVal) {
        if (!head) {
            // 如果链表为空，创建新节点并返回
            head = new Node(insertVal, NULL);
            head->next = head; // 新节点指向自身，形成循环
            return head;
        }
        Node *pre = head; // 前一个节点指针
        Node *cur = pre->next; // 当前节点指针
        while (cur != head) {
            // 找到插入位置

```

```

        if (pre->val <= insertVal && cur->val >= insertVal) break;
        // 考虑插入值小于最小节点值或大于最大节点值的情况
        if (pre->val > cur->val && (pre->val <= insertVal || cur->val >= insertVal))
break;

        pre = cur; // 移动指针
        cur = cur->next;
    }
    // 插入新节点
    pre->next = new Node(insertVal, cur);
    return head;
}
};

```

链表翻转

25. K 个一组翻转链表 #todo

```

ListNode* reverseKGroup(ListNode* head, int k) {
    if (!head || k <= 1) return head;

    // 创建虚拟头节点，方便处理头部翻转
    ListNode* dummy = new ListNode(-1);
    dummy->next = head;

    ListNode* pre = dummy; // pre 指向每组翻转的前一个节点
    ListNode* cur = dummy; // cur 用于统计链表长度

    // 统计链表长度
    int n = 0;
    while (cur->next) {
        ++n;
        cur = cur->next;
    }

    // 逐组翻转
    while (n >= k) {
        cur = pre->next; // cur 指向本组的第一个节点
        // 从第二个节点开始，逐个摘下来放到 pre 后面（头插法）
        for (int i = 1; i < k; i++) {
            ListNode* temp = cur->next;
            cur->next = temp->next;
            temp->next = pre->next;
            pre->next = temp;
        }
        // 更新 pre 指针，指向这一组的尾节点
        pre = cur;
        n -= k;
    }
    return dummy->next;
}

```


92. 反转链表 II

```
ListNode* reverseBetween(ListNode* head, int m, int n) {
    if (!head || m == n) return head; // 无需翻转

    // 创建虚拟头节点，方便统一处理边界
    ListNode* dummy = new ListNode(-1);
    dummy->next = head;

    // 1. 找到第 m-1 个节点 pre
    ListNode* pre = dummy;
    for (int i = 0; i < m - 1; ++i) {
        pre = pre->next;
    }

    // 2. cur 指向第 m 个节点（即翻转区间的第一个节点）
    ListNode* cur = pre->next;

    // 3. 头插法：逐个把 cur->next 插到 pre 后面
    for (int i = m; i < n; ++i) {
        ListNode* temp = cur->next; // 待插入节点
        cur->next = temp->next;      // 从原链表摘下
        temp->next = pre->next;      // 插入到 pre 后面
        pre->next = temp;            // 更新 pre->next
    }
    return dummy->next;
}
```

206. 反转链表

```
// 非递归
ListNode* reverseList(ListNode* head) {
    if (head == nullptr || head->next == nullptr)
        return head;
    ListNode *cur = head, *pre = nullptr;
    while(cur)
    {
        ListNode *temp = cur->next;
        cur->next = pre;
        pre = cur;
        cur = temp;
    }
    return pre;
}

// 递归
ListNode* reverseList(ListNode* head)
{
    if (!head || !head->next) return head;
    ListNode *newHead = reverseList(head->next);
    head->next->next = head;
    head->next = NULL;
}
```

```

    return newHead;
}

```

328. 奇偶链表 todo

```

// 将链表按奇偶索引重新排序
// 奇数位置的节点在前，偶数位置的节点在后
ListNode* oddEvenList(ListNode* head) {
    // 边界情况：空链表或单节点链表，直接返回
    if (!head || !head->next) return head;

    ListNode* odd = head; // odd 指向奇数链表的头（即 head）
    ListNode* even = head->next; // even 指向偶数链表的头（即 head->next）
    ListNode* evenHead = even; // 保存偶数链表的头，最后需要拼接

    // 迭代处理：奇偶链表分别向前推进
    while (even && even->next) {
        odd->next = even->next; // 奇数节点指向下一个奇数节点
        odd = odd->next;       // odd 指针前进
        even->next = odd->next; // 偶数节点指向下一个偶数节点
        even = even->next;     // even 指针前进
    }
    // 拼接：奇数链表的尾部接上偶数链表的头
    odd->next = evenHead;
    return head;
}

```

动态规划

Leetcode 题解 - 动态规划

1. 坐标型动态规划

状态: $f(x)$ 表示从起点走到坐标 x , $f[x][y]$ 表示我从起点走到坐标 x, y ; 方程: 研究走到 x, y 这个点之前的一步; 初始化: 起点; 答案: 终点

62. 不同路径 #todo 空间优化

```

// 空间优化: https://leetcode-cn.com/problems/unique-paths/solution/san-chong-shi-xian-xiang-xi-tu-jie-62-bu-4jz1/
int uniquePaths(int m, int n) {
    // 观察依赖关系:
    // dp[i][j] 只依赖 上一行的同一列 (dp[i-1][j]) 和 当前行的左边 (dp[i][j-1])。
    // 因此，不需要整个 m x n 的二维数组:
    // 用一维数组 dp[j] 保存当前行的路径数。
    // dp[j] 更新时: dp[j] = dp[j] + dp[j-1]:
    // dp[j] (原值) 代表上一行的 dp[i-1][j]
    // dp[j-1] 代表当前行左边的 dp[i][j-1]
    vector<int> dp(n, 1);
    for(int i = 1; i < m; i++)
    {

```

```

        for(int j = 1; j < n; j++)
        {
            dp[j] = dp[j] + dp[j-1];
        }
    }
    return dp[n-1];
}

int uniquePaths(int m, int n)
{
    // dp[i][j] 表示从[0][0]--->[i][j] 有多少种走法
    // 第0行和第0列 在边界上所以只有一种方法
    vector<vector<int>> dp(m, vector<int>(n, 1));
    for(int i = 1; i < m; i++)
    {
        for(int j = 1; j < n; j++)
        {
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
        }
    }
    return dp[m-1][n-1];
}

```

63. 不同路径 II # todo 空间优化

```

// https://leetcode-cn.com/problems/unique-paths-ii/solution/si-chong-shi-xian-xiang-xi-tu-jie-63-bu-0qyz7/
int uniquePathsWithObstacles(vector<vector<int>>& obstacleGrid) {
    int m = obstacleGrid.size();
    int n = obstacleGrid[0].size();

    // 边界情况：起点就是障碍物
    if (obstacleGrid[0][0] == 1) return 0;

    // dp[j] 表示当前行第 j 列的路径数
    vector<int> dp(n, 0);
    dp[0] = 1; // 起点路径数 = 1

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (obstacleGrid[i][j] == 1) {
                dp[j] = 0; // 遇到障碍物，无法到达
            } else if (j > 0) {
                dp[j] += dp[j - 1];
                // 状态转移：当前格子 = 来自上方(dp[j]) + 来自左方(dp[j-1])
                // 注意：这里的 dp[j] 已经是上一行的值
            }
        }
    }
    return dp[n - 1];
}

```

```

int uniquePathsWithObstacles(vector<vector<int>> &obstacleGrid)
{
    int m = obstacleGrid.size();
    int n = obstacleGrid[0].size();
    if (obstacleGrid.empty() || obstacleGrid[0].empty() || obstacleGrid[0][0] == 1)
    {
        return 0;
    }
    vector<vector<int>> dp(m, vector<int>(n, 0));
    // 先初始化边界
    for (int i = 0; i < m; i++)
    {
        if (obstacleGrid[i][0] != 1)
            dp[i][0] = 1;
        else
            break;
    }
    for (int j = 0; j < n; j++)
    {
        if (obstacleGrid[0][j] != 1)
            dp[0][j] = 1;
        else
            break;
    }
    // dp[i][j] 表示从[0][0]--->[i][j] 有多少种走法
    for (int i = 1; i < m; i++)
    {
        for (int j = 1; j < n; j++)
        {
            if (obstacleGrid[i][j] == 1)
                dp[i][j] = 0;
            else
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    }
    return dp[m - 1][n - 1];
}

```

64. 最小路径和 # todo 空间优化

```

int minPathSum(vector<vector<int>>& grid) {
    int m = grid.size();
    int n = grid[0].size();
    // 初始化第一列
    for (int i = 1; i < m; i++) {
        grid[i][0] += grid[i-1][0];
    }
    // 初始化第一行
    for (int j = 1; j < n; j++) {
        grid[0][j] += grid[0][j-1];
    }
}

```

```

    }
    // 状态转移：直接在 grid 上累加
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            grid[i][j] += min(grid[i-1][j], grid[i][j-1]);
        }
    }
    return grid[m-1][n-1];
}

int minPathSum(vector<vector<int>>& grid) {
    int m = grid.size();
    int n = grid[0].size();
    // dp[i][j] 表示从[0][0]-->[i][j]的最短路径和
    vector<int> dp(n, 0);
    dp[0] = grid[0][0];
    // 先遍历第一行
    for (int j = 1; j < n; j++) {
        dp[j] = dp[j - 1] + grid[0][j];
    }

    // 从第二行开始遍历
    for (int i = 1; i < m; i++) {
        dp[0] += grid[i][0];
        for (int j = 1; j < n; j++) {
            dp[j] = min(dp[j], dp[j - 1]) + grid[i][j];
        }
    }
    return dp[n - 1];
}

int minPathSum(vector<vector<int>>& grid) {
    int m = grid.size();
    int n = grid[0].size();
    // dp[i][j] 表示从 (0,0) 到 (i,j) 的最小路径和
    vector<vector<int>> dp(grid); // 初始化为 grid 的副本，避免覆盖原数据
    // 初始化第一列：只能从上往下走，所以每一格只能累加上方的路径和
    for (int i = 1; i < m; i++) {
        dp[i][0] += dp[i-1][0];
    }
    // 初始化第一行：只能从左往右走，所以每一格只能累加左边的路径和
    for (int j = 1; j < n; j++) {
        dp[0][j] += dp[0][j-1];
    }
    // 状态转移：到达 (i,j) 的最小路径和 = min(来自上方, 来自左方) + 当前格子值
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j];
        }
    }
    return dp[m-1][n-1];
}

```

// 两个变量prev1和prev2来追踪前两个阶段的方法数，初始时prev1表示到达第1阶的方法数，prev2表示到达第2阶的方法数。然后，我们从第3阶开始，计算当前阶段的方法数，然后更新prev1和prev2为前两个阶段的值。最终，prev2中存储的就是到达第n阶楼梯的方法数，而prev1中存储的是到达第n-1阶楼梯的方法数

```
int climbStairs(int n) {
    if (n <= 2) {
        return n;
    }
    int prev1 = 1; // prev1表示到达第1阶的方法数
    int prev2 = 2; // prev2表示到达第2阶的方法数
    for (int i = 3; i <= n; i++) {
        int current = prev1 + prev2;
        prev1 = prev2;
        prev2 = current;
    }
    return prev2;
}

int climbStairs(int n)
{
    if (n <= 1)
        return 1;
    vector<int> dp(n, 0);
    dp[0] = 1;
    dp[1] = 2;
    for (int i = 2; i < n; i++)
    {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n - 1];
}
```

118. 杨辉三角

```
vector<vector<int>> generate(int numRows) {
    vector<vector<int>> result;
    if (numRows <= 0) {
        return result;
    }
    result.push_back({1}); // 第一行的特殊情况
    for (int i = 1; i < numRows; i++) {
        vector<int> newRow(i + 1, 0); // 创建一个新行，初始化为0
        newRow[0] = 1; // 每行的第一个元素为1
        newRow[i] = 1; // 每行的最后一个元素为1
        for (int j = 1; j < i; j++) {
            newRow[j] = result[i - 1][j - 1] + result[i - 1][j];
        }
        result.push_back(newRow);
    }
    return result;
}

vector<vector<int>> generate(int numRows)
```

```

{
    vector<vector<int>> res(numRows, vector<int>());
    for(int i = 0; i < numRows; i++)
    {
        res[i].resize(i+1, 1);
        for(int j = 1; j < i; j++)
        {
            res[i][j] = res[i-1][j-1] + res[i-1][j];
        }
    }
    return res;
}

```

120. 三角形最小路径和 #todo 2023 1027

```

int minimumTotal(vector<vector<int>>& triangle) {
    int n = triangle.size();
    vector<int> dp = triangle.back(); // 初始化为最后一行
    for (int i = n - 2; i >= 0; i--) {
        for (int j = 0; j <= i; j++) {
            dp[j] = triangle[i][j] + min(dp[j], dp[j+1]);
        }
    }
    return dp[0];
}

int minimumTotal(vector<vector<int>>& triangle) {
    int n = triangle.size();
    // 从倒数第二层开始往上更新
    for (int i = n - 2; i >= 0; i--) {
        for (int j = 0; j < triangle[i].size(); j++) {
            triangle[i][j] += min(triangle[i+1][j], triangle[i+1][j+1]);
        }
    }
    return triangle[0][0];
}

```

2. 单序列动态规划

状态: $f[i]$ 表示前 i 个位置/数字/字符, 第 i 个; 方程: $f[i] = f[f[j]]$, j 是 i 之前的一个位置; 初始化: $f[0]$; 答案: $f[n-1]$; 小技巧: 一般有 N 个数字/字符, 就开 $N+1$ 个位置的数组, 第 0 个位置单独留出来作初始化。(跟坐标相关的动态规划除外)

32. 最长有效括号

```

int longestValidParentheses(string s)
{
    int n = s.length();
    vector<int> dp(n, 0); // dp[i] 表示以 s[i] 结尾的最长有效字符串长度
    int res = 0;
    for(int i = 1; i < n; i++)
    {

```

```

        if (s[i] == '(') // 如果遇到左括号，说明以当前字符结尾不可能形成有效括号字符串，所以dp[i] =
0;

        dp[i] = 0;

        if (s[i] == ')') // 当前字符为右括号时，那么找到前一个字符位置形成的最长有效括号字符串的长
度，在这个长度之前的字符串如果是左括号，那么可以形成有效括号字符串
        // 即 dp[i] = dp[i-1] + 2，需要注意的是 需要加上 前一个字符位置形成的有效括号字符串；
        {
            int pre = i - dp[i-1] - 1;
            if (pre >= 0 && s[pre] == '(')
            {
                dp[i] = dp[i-1] + 2 + (pre > 0 ? dp[pre-1] : 0);
            }
        }
        res = max(res, dp[i]);
    }
    return res;
}

```

55. 跳跃游戏 todo 贪心

```

bool canJump(vector<int>& nums)
{
    // dp[i] 表示达到i位置时剩余的跳力，若到达某个位置时跳力为负了，说明无法到达该位置
    // 所以当前位置的剩余跳力（dp 值）和当前位置新的跳力中的较大那个数决定了当前能到的最远距离，而下一个位
置的剩余跳力（dp 值）就等于当前的这个较大值减去1
    vector<int> dp(nums.size(), 0);
    for (int i = 1; i < nums.size(); ++i)
    {
        dp[i] = max(dp[i - 1], nums[i - 1]) - 1;
        if (dp[i] < 0)
            return false;
    }
    return true;
}

bool canJump(vector<int>& nums) {
    int n = nums.size(), reach = 0;
    for (int i = 0; i < n; ++i) {
        if (i > reach || reach >= n - 1) break;
        reach = max(reach, i + nums[i]);
    }
    return reach >= n - 1;
}

```

91. 解码方法 #todo 20210415

```

int numDecodings(string s) {
    int n = s.size();
    if (n == 0 || s[0] == '0') return 0;

```



```

vector<int> dp(n + 1, 0);
dp[0] = 1;           // 空字符串有 1 种解码方式
dp[1] = 1;           // 第一个字符不是 '0', 就有 1 种解码方式

for (int i = 2; i <= n; i++) {
    int oneDigit = s[i - 1] - '0';           // 单个数字
    int twoDigits = (s[i - 2] - '0') * 10 + oneDigit; // 两位数字

    // 如果单个数字合法 (1~9), 加上 dp[i-1]
    if (oneDigit >= 1 && oneDigit <= 9)
        dp[i] += dp[i - 1];

    // 如果两位数字合法 (10~26), 加上 dp[i-2]
    if (twoDigits >= 10 && twoDigits <= 26)
        dp[i] += dp[i - 2];
}
return dp[n];
}

```

132. 分割回文串 II

```

// 解法1:
int minCut(string s)
{
    if (s.empty())
        return 0; // 空字符串不需要分割
    int n = s.size();
    vector<vector<bool>> p(n, vector<bool>(n, false)); // p[i][j] 表示子串 s[i..j] 是否是回文
    vector<int> dp(n, INT_MAX); // dp[i] 表示子串 s[0..i] 的最少分割次数
    for (int j = 0; j < n; j++) // 枚举右端点 j
    {
        for (int i = 0; i <= j; i++) // 枚举左端点 i, 判断 s[i..j] 是否是回文
        {
            // 回文判断: 1. 首尾字符相等 s[i] == s[j]
            // 2. 子串长度 <=2 或者中间子串 s[i+1..j-1] 是回文
            p[i][j] = (s[i] == s[j]) && (j - i < 2 || p[i + 1][j - 1]);
            if (p[i][j]) // 如果 s[i..j] 是回文
            {
                if (i == 0)
                    dp[j] = 0; // 整段回文, 从开头到 j 不需要切割
                else
                    // dp[i-1] 表示 s[0..i-1] 的最少分割次数
                    // 加上当前回文 s[i..j] 之后, 可能需要一次分割
                    dp[j] = min(dp[j], dp[i - 1] + 1);
            }
        }
    }
    return dp[n - 1]; // 返回整个字符串 s[0..n-1] 的最少分割次数
}

```

```

// 解法2:
int minCut_2(string s)
{
    int n = s.size();
    if (n <= 0)
        return 0;

    // dp[i]表示s[i...n-1]的最小分割次数
    vector<int> dp(n + 1, 0);
    dp[n] = -1;
    vector<vector<bool>> p(n, vector<bool>(n, false));

    for (int i = n - 1; i >= 0; i--)
    {
        dp[i] = INT_MAX;
        for (int j = i; j < n; j++)
        {
            if (s[i] == s[j] && (j - i < 2 || p[i + 1][j - 1])) // 判断s[i...j]是不是回文子串
            {
                p[i][j] = true;
                dp[i] = min(dp[j + 1] + 1, dp[i]);
            }
        }
    }
    return dp[0];
}

```

139. 单词拆分 #TODO 20210415

```

bool wordBreak(string s, vector<string>& wordDict) {
    if (wordDict.size() == 0)
        return false; // 字典为空, 无法拆分
    int n = s.size();
    // dp[i] 表示子串 s[0..i-1] 是否可以拆分成字典中的单词
    vector<int> dp(n+1, false);
    dp[0] = true; // 空字符串可以拆分
    for(int i = 1; i <= n; i++) // 枚举右端点 i
    {
        for(int j = 0; j < i; j++) // 枚举分割点 j, 判断 s[0..i-1] 是否可以拆分
        {
            // 条件: 1. s[0..j-1] 可以拆分 (dp[j] == true)
            // 2. s[j..i-1] 在字典中
            if (dp[j] && count(wordDict.begin(), wordDict.end(), s.substr(j, i - j)) != 0)
            {
                dp[i] = true; // s[0..i-1] 可以拆分
                break; // 找到一种拆分即可, 不需要继续枚举 j
            }
        }
    }
    return dp[n]; // 整个字符串是否可以拆分
}

```

279. 完全平方数

```
int numSquares(int n)
{
    if (n <= 0)
        return 0;
    // dp[i] 表示数字 i 最少由几个完全平方数组成
    vector<int> dp(n + 1, INT_MAX);
    dp[0] = 0; // 0 可以表示为 0 个平方数
    for (int i = 1; i <= n; i++) // 枚举从 1 到 n 的每个数字 i
    {
        for (int j = 1; j * j <= i; j++) // 枚举所有小于等于 i 的平方数 j*j
        {
            // 状态转移: 用平方数 j*j 加上剩余数字 i - j*j 的最少平方数个数
            dp[i] = min(dp[i], dp[i - j * j] + 1);
        }
    }
    return dp[n]; // 返回数字 n 的最少平方数个数
}
```

300. 最长递增子序列

```
int lengthOfLIS(vector<int>& nums)
{
    if (nums.empty())
        return 0;
    // dp[i] 表示考虑前 i 个元素, 以 nums[i] 结尾的最长上升子序列长度
    vector<int> dp(nums.size(), 0);
    int res = INT_MIN; // 最终结果, 整个数组的最长上升子序列长度
    // 枚举每个元素作为子序列的结尾
    for(int i = 0; i < nums.size(); i++)
    {
        dp[i] = 1; // 至少包含自己, 长度为 1
        // 枚举 i 前面的所有元素, 寻找比 nums[i] 小的元素
        for(int j = 0; j < i; j++)
        {
            if(nums[j] < nums[i])
            {
                // 如果 nums[i] 可以接在 nums[j] 后面, 更新 dp[i]
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
        // 更新全局最长子序列长度
        res = max(res, dp[i]);
    }
    return res; // 返回整个数组的最长上升子序列长度
}
```

1626. 无矛盾的最佳球队

```

int bestTeamScore(vector<int>& scores, vector<int>& ages) {
    int n = scores.size();
    vector<pair<int,int>> people(n);
    for (int i = 0; i < n; i++) {
        people[i] = {ages[i], scores[i]};
    }
    // 按年龄升序；如果年龄一样，按分数升序
    sort(people.begin(), people.end(), [](const pair<int,int>& a, const pair<int,int>& b){
        if (a.first != b.first) return a.first < b.first;
        return a.second < b.second;
    });

    vector<int> dp(n, 0);
    int answer = 0;

    for (int i = 0; i < n; i++) {
        int age_i = people[i].first;
        int score_i = people[i].second;
        dp[i] = score_i; // 至少可以自己做团队
        for (int j = 0; j < i; j++) {
            int age_j = people[j].first;
            int score_j = people[j].second;
            // 因为排序保证 age_j ≤ age_i,
            // 我们只需要确保 score_j ≤ score_i 才不会冲突
            if (score_j ≤ score_i) {
                dp[i] = max(dp[i], dp[j] + score_i);
            }
        }
        answer = max(answer, dp[i]);
    }
    return answer;
}

```

两个长度相同的数组score和weight

```

#include <bits/stdc++.h>
using namespace std;

```

/*

题目：

- 给两个数组 score 和 weight，长度相同
- 选择若干元素，要求选出的元素在 score 和 weight 两个维度都严格递增
(即如果 $score[i] > score[j]$ ，那么必须有 $weight[i] > weight[j]$)
- 目标：最大化所选元素的 score 之和

思路：

- 这是一个二维 LIS (Longest Increasing Subsequence) 问题的“最大和”变体
- 我们需要在原顺序上挑选子序列（不能排序打乱）
- 用 DP 求解：
 $dp[i]$ = 以第 i 个元素结尾的合法子序列的最大 score 和

```

    转移:  $dp[i] = score[i] + \max(dp[j])$ , 其中  $j < i$  且  $score[j] < score[i]$  且  $weight[j] < weight[i]$ 
- 答案 =  $\max(dp[i])$ 
*/

class Solution {
public:
    int maxIncreasingScore(vector<int>& score, vector<int>& weight) {
        int n = score.size();
        vector<int> dp(n, 0); //  $dp[i]$  = 以  $i$  结尾的最大 score 和
        int ans = 0; // 最终答案

        for (int i = 0; i < n; i++) {
            dp[i] = score[i]; // 至少可以选自己
            // 枚举前面所有元素, 尝试接在它们后面
            for (int j = 0; j < i; j++) {
                // 必须严格递增:  $score[j] < score[i]$  且  $weight[j] < weight[i]$ 
                if (score[j] < score[i] && weight[j] < weight[i]) {
                    dp[i] = max(dp[i], dp[j] + score[i]);
                }
            }
            ans = max(ans, dp[i]); // 更新全局最大值
        }
        return ans;
    }
};

int main() {
    Solution sol;

    // 示例
    vector<int> score = {6, 7, 8, 3, 5};
    vector<int> weight = {4, 5, 3, 1, 2};

    cout << sol.maxIncreasingScore(score, weight) << endl;
    // 输出 13, 对应选择 (6,4) 和 (7,5)

    return 0;
}

```

343. 整数拆分

```

int integerBreak(int n) {
    if (n <= 2)
        return 1; // n=2 时只能拆分为 1+1, 乘积为 1

    //  $dp[i]$  表示整数  $i$  拆分后的最大乘积
    vector<int> dp(n + 1, 0);

    // 从 3 开始计算每个数字的最大乘积

```

```

for (int i = 3; i <= n; i++)
{
    // 枚举拆分点 j, 表示将 i 拆分为 j + (i-j)
    for (int j = 1; j < i; j++)
    {
        // 有两种情况: 1. 拆分为两个数字: j * (i-j)
        // 2. 拆分为 j + 其他数字的最优拆分: j * dp[i-j]
        // 取二者的最大值, 再与 dp[i] 比较, 更新 dp[i]
        dp[i] = max(dp[i], max(j * dp[i - j], j * (i - j)));
    }
}
return dp[n]; // 返回 n 拆分后的最大乘积
}

```

674. 最长连续递增序列

```

class Solution {
public:
    // 使用一个计数器, 如果遇到大的数字, 计数器自增1;
    // 如果是一个小的数字, 则计数器重置为1。用一个变量 cur 来表示前一个数字, 初始化为整型最大值,
    // 当前遍历到的数字 num 就和 cur 比较就行了, 每次用 cnt 来更新结果 res
    int findLengthOfLCIS(vector<int>& nums) {
        int res = 0, cnt = 0, cur = INT_MAX;
        for (int num : nums) {
            if (num > cur) ++cnt;
            else cnt = 1;
            res = max(res, cnt);
            cur = num;
        }
        return res;
    }
    int findLengthOfLCIS1(vector<int>& nums) {
        if (nums.size() == 0)
            return 0;
        int res = 1;
        // dp[i]: 以下标i为结尾的连续递增的子序列长度为dp[i]
        vector<int> dp(nums.size(), 1);
        for (int i = 1; i < nums.size(); i++) {
            if (nums[i] > nums[i - 1]) // 连续记录
                dp[i] = dp[i - 1] + 1;
            else
                dp[i] = 1;
            res = max(res, dp[i]);
        }
        return res;
    }
};

```

312. 戳气球

```

int maxCoins(vector<int>& nums)

```

```

{
    int n = nums.size();
    nums.insert(nums.begin(), 1);
    nums.push_back(1);
    // dp, 其中 dp[i][j] 表示打爆区间 [i, j] 中的所有气球能得到的最多金币
    // 用k来遍历吧, k在区间 [i, j] 中, 假如第k个气球最后被打爆, 那么此时区间 [i, j] 被分成了三部分, [i, k-1], [k], 和 [k+1, j],
    // 只要之前更新过了 [i, k-1] 和 [k+1, j] 这两个子区间的 dp 值, 可以直接用 dp[i][k-1] 和 dp[k+1][j],
    // dp[i][k-1] 的意义是什么呢, 是打爆区间 [i, k-1] 内所有的气球后的最大得分, 此时第 k-1 个气球已经不能用了,
    // 同理, 第 k+1 个气球也不能用了, 相当于区间 [i, j] 中除了第k个气球, 其他的已经爆了, 那么周围的气球只能是第 i-1 个, 和第 j+1 个了, 所以得分应为 nums[i-1] * nums[k] * nums[j+1], 分析到这里, 状态转移方程应该已经跃然纸上了吧, 如下所示:
    //dp[i][j] = max(dp[i][j], nums[i - 1] * nums[k] * nums[j + 1] + dp[i][k - 1] + dp[k + 1][j])
    // ( i ≤ k ≤ j )
    vector<vector<int>> dp(n + 2, vector<int>(n + 2, 0));
    for (int len = 1; len <= n; ++len) {
        for (int i = 1; i <= n - len + 1; ++i) {
            int j = i + len - 1;
            for (int k = i; k <= j; ++k) {
                dp[i][j] = max(dp[i][j], nums[i - 1] * nums[k] * nums[j + 1] + dp[i][k - 1] + dp[k + 1][j]);
            }
        }
    }
    return dp[1][n];
}

```

3.双序列动态规划

状态: $f[i][j]$ 表示第一个sequence的前*i*个数字/字符, 配上第二个sequence的前*j*个;

方程: $f[i][j]$ = 研究第*i*个和第*j*个的匹配关系;

初始化: $f[i][0]$ 和 $f[0][i]$;

答案: $f[n][m]$, 其中 $n = s1.length()$; $m = s2.length()$;

10. 正则表达式匹配

```

bool isMatch(string s, string p) {
    int m = s.length();
    int n = p.length();

    // dp[i][j] 表示 s[0..i-1] 与 p[0..j-1] 是否匹配
    vector<vector<bool>> dp(m + 1, vector<bool>(n + 1, false));

    dp[0][0] = true; // 空字符串与空模式匹配

    // s 不为空, p 为空 → 不匹配
    for(int i = 1; i <= m; i++)
        dp[i][0] = false;
}

```

```

// s 为空, p 不为空
for(int j = 1; j <= n; j++)
    // p[j-1] 是 '*' 且前面模式能匹配空字符串 (dp[0][j-2])
    dp[0][j] = j > 1 && p[j-1] == '*' && dp[0][j-2];

// 枚举字符串 s 的每个字符
for(int i = 1; i <= m; i++) {
    // 枚举模式 p 的每个字符
    for(int j = 1; j <= n; j++) {
        if (p[j-1] != '*') {
            // 当前字符不是 '*', 匹配条件: 1. 前面部分匹配 dp[i-1][j-1]
            // 2. 当前字符相等或者模式是 '.'
            dp[i][j] = dp[i-1][j-1] && (s[i-1] == p[j-1] || p[j-1] == '.');
        }
        else {
            // 当前字符是 '*', 匹配条件: 1. '*' 表示出现 0 次: dp[i][j-2]
            // 2. '*' 表示出现 >=1 次, 且前一个字符匹配: dp[i-1][j] && (s[i-1] == p[j-2] ||
p[j-2] == '.')
            dp[i][j] = dp[i][j-2] || ((s[i-1] == p[j-2] || p[j-2] == '.') && dp[i-1]
[j]);
        }
    }
}
return dp[m][n]; // 整个字符串是否与整个模式匹配
}

```

44. 通配符匹配

```

bool isMatch(string s, string p)
{
    int m = s.size();
    int n = p.size();

    // dp[i][j] 表示s[0...i-1] 和 p[0...j-1]是否能够匹配
    // vector<vector<bool>> dp(m+1, vector<bool>(n+1, false));
    vector<vector<bool>> dp(m + 1, vector<bool>(n + 1, false));
    dp[0][0] = true; // 空串能匹配

    for (int j = 1; j <= n; j++)
    {
        if (p[j - 1] == '*')
            dp[0][j] = dp[0][j - 1];
    }
    // * 可以匹配空串和任意字符串
    // 如果我们不使用这个星号, 那么就会从 dp[i][j-1] 转移而来; 如果我们使用这个星号, 那么就会从 dp[i-
1][j]dp[i-1][j] 转移而来。

    // dp[i-1][j] 表示 s[0...i-2] 和p[0...j-1]匹配成功, 因为星号可以匹配任意字符串, 再加一个任意
字符也没问题
    // dp[i][j-1] 表示 s[0...i-1] 和p[0...j-2]匹配成功, 星号可以匹配空串

```



```

for (int i = 1; i <= m; i++)
{
    for (int j = 1; j <= n; j++)
    {
        if (p[j - 1] == '*')
            dp[i][j] = dp[i - 1][j] || dp[i][j - 1];
        else
        {
            dp[i][j] = dp[i - 1][j - 1] && (s[i - 1] == p[j - 1] || p[j - 1] == '?');
        }
    }
}
return dp[m][n];
}

```

72. 编辑距离 #todo

```

int minDistance(string word1, string word2) {
    int m = word1.size();
    int n = word2.size();
    // dp[i][j] 表示将 word1[0..i-1] 转换成 word2[0..j-1] 的最少操作次数
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
    // 初始化边界情况
    for (int i = 0; i <= m; i++)
        dp[i][0] = i; // word2 为空, 需要删除 i 个字符
    for (int j = 0; j <= n; j++)
        dp[0][j] = j; // word1 为空, 需要插入 j 个字符

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1[i - 1] == word2[j - 1]) // 当前字符相同, 无需操作, 继承 dp[i-1][j-1]

                dp[i][j] = dp[i - 1][j - 1];
            else
                // 当前字符不同, 三种操作取最小 +1:
                // 1. 替换: dp[i-1][j-1] + 1
                // 2. 删除 word1[i-1]: dp[i-1][j] + 1
                // 3. 插入 word2[j-1]: dp[i][j-1] + 1
                dp[i][j] = min(dp[i - 1][j - 1], min(dp[i - 1][j], dp[i][j - 1])) +
                    1;
        }
    }
    return dp[m][n]; // 返回将 word1 转换为 word2 的最少操作数
}

```

583. 两个字符串的删除操作

```

int minDistance(string word1, string word2) {
    int m = word1.size();
    int n = word2.size();
    // dp[i][j] 表示将 word1[0..i-1] 转换为 word2[0..j-1] 的最少操作次数

```

```

vector<vector<int>>> dp(m + 1, vector<int>(n + 1, 0));
// 初始化边界情况
for (int i = 0; i <= m; i++)
    dp[i][0] = i; // word2 为空, 需要删除 i 个字符
for (int j = 0; j <= n; j++)
    dp[0][j] = j; // word1 为空, 需要插入 j 个字符

// 填充 dp 表
for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (word1[i - 1] == word2[j - 1])
            // 当前字符相同, 无需操作, 继承 dp[i-1][j-1]
            dp[i][j] = dp[i - 1][j - 1];
        else
            // 当前字符不同, 允许操作为: 1. 删除 word1[i-1]: dp[i-1][j] + 1
            // 2. 插入 word2[j-1]: dp[i][j-1] + 1
            // 不考虑替换操作, 所以只取删除和插入的最小值 +1
            dp[i][j] = min(dp[i - 1][j], dp[i][j - 1]) + 1;
    }
}
return dp[m][n]; // 返回将 word1 转换为 word2 的最少操作数
}

```

115. 不同的子序列

```

int numDistinct(string s, string t) {
    int m = s.size();
    int n = t.size();

    // dp[i][j] 表示 s[0..i-1] 中 t[0..j-1] 的不同子序列个数
    vector<vector<unsigned long long>> dp(m + 1, vector<unsigned long long>(n + 1, 0));

    // 边界情况: 空字符串 t 的子序列个数为 1 (什么都不选)
    for (int j = 0; j <= n; j++) {
        dp[j][0] = 1;
    }

    // 填充 dp 表
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s[i - 1] != t[j - 1]) // 当前字符不匹配: 只能跳过 s[i-1]
                dp[i][j] = dp[i - 1][j];
            else
                // 当前字符匹配: 1. 使用 s[i-1] 匹配 t[j-1] → dp[i-1][j-1]
                // 2. 不使用 s[i-1] → dp[i-1][j] 总数为两种情况之和
                dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
        }
    }

    return dp[m][n]; // 返回 s 中 t 的不同子序列个数
}

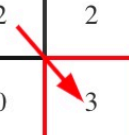
```

392. 判断子序列

t: "ahbgdc"

s: "abc"

	a	h	b	g	d	c
a	0	0	0	0	0	0
b	0	1	1	1	1	1
c	0	0	0	2	2	2
	0	0	0	0	0	3



D

公众号:代码随想录

```
bool isSubsequence(string s, string t) {
    int m = s.size();
    int n = t.size();
    // dp[i][j] 表示 s[0..i-1] 和 t[0..j-1] 的最长公共子序列长度
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    // 填充 dp 表
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s[i - 1] == t[j - 1])
                // 当前字符相同: 把当前字符加入最长公共子序列
                // 长度 = 前一段的最长公共子序列长度 + 1
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                // 当前字符不同: 忽略 t[j-1], 保持 s[0..i-1] 与 t[0..j-2]
                // 的最长公共子序列长度
                dp[i][j] = dp[i][j - 1];
        }
    }
    // 如果 s 的长度等于 dp[m][n], 说明 s 完全匹配 t 中的某个子序列
    return dp[m][n] == m;
}
```

712. 两个字符串的最小ASCII删除和

```
int minimumDeleteSum(string s1, string s2) {
    int m = s1.size();
    int n = s2.size();
    // dp[i][j] 表示 s1[0..i-1] 和 s2[0..j-1]
    // 变成相同字符串需要删除字符的最小 ASCII 和
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
```

```

// 边界情况: s2 为空时, 需要删除 s1 的所有字符
for (int i = 1; i <= m; i++)
    dp[i][0] = dp[i - 1][0] + s1[i - 1];

// 边界情况: s1 为空时, 需要删除 s2 的所有字符
for (int j = 1; j <= n; j++)
    dp[0][j] = dp[0][j - 1] + s2[j - 1];

for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1[i - 1] == s2[j - 1]) // 当前字符相同, 不需要删除, 继承 dp[i-1][j-1]
            dp[i][j] = dp[i - 1][j - 1];
        else
            // 当前字符不同: 删除 s1[i-1] 或删除 s2[j-1], 取最小值
            dp[i][j] = min(dp[i - 1][j] + s1[i - 1], dp[i][j - 1] + s2[j - 1]);
    }
}
return dp[m][n];
}

```

97. 交错字符串

```

bool isInterleave(string s1, string s2, string s3) {
    int m = s1.size();
    int n = s2.size();
    int k = s3.size();

    // 如果长度不匹配, s3 不可能是交错组成
    if (m + n != k)
        return false;

    // dp[i][j] 表示 s1[0..i-1] 和 s2[0..j-1] 能否交错组成 s3[0..i+j-1]
    vector<vector<bool>> dp(m + 1, vector<bool>(n + 1, false));

    dp[0][0] = true; // 空字符串交错组成空字符串

    // 初始化边界情况: s2 为空, 只能由 s1 构成 s3
    for (int i = 1; i <= m; i++)
        dp[i][0] = dp[i - 1][0] && (s1[i - 1] == s3[i - 1]);

    // 初始化边界情况: s1 为空, 只能由 s2 构成 s3
    for (int j = 1; j <= n; j++)
        dp[0][j] = dp[0][j - 1] && (s2[j - 1] == s3[j - 1]);

    // 填充 dp 表
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            // 两种情况只要有一种满足即可:
            // 1. s1[i-1] 匹配 s3[i+j-1] 且 dp[i-1][j] 为 true
            // 2. s2[j-1] 匹配 s3[i+j-1] 且 dp[i][j-1] 为 true
            if ((s1[i - 1] == s3[i + j - 1] && dp[i - 1][j]) ||

```

```

        (s2[j - 1] == s3[i + j - 1] && dp[i][j - 1]))
        dp[i][j] = true;
    }
}
return dp[m][n];
}

```

718. 最长重复子数组 todo 滚动数组

```

// 利用滚动数组优化成一维
int findLength(vector<int>& nums1, vector<int>& nums2) {
    // dp[j] 表示以 nums1[i-1] 和 nums2[j-1] 结尾的最长公共子数组长度
    // 由于只依赖上一行的 dp[j-1]，可以用一维数组滚动优化
    vector<int> dp(nums2.size() + 1, 0);

    int res = 0; // 记录最长公共子数组长度

    for (int i = 1; i <= nums1.size(); i++) {
        // 注意：从后往前遍历 nums2，保证 dp[j-1] 是上一行的值
        for (int j = nums2.size(); j > 0; j--) {
            if (nums1[i - 1] == nums2[j - 1]) {
                // 当前元素相等，可以延长公共子数组长度
                dp[j] = dp[j - 1] + 1;
            } else {
                // 当前元素不等，公共子数组断开
                dp[j] = 0;
            }
            // 更新全局最大长度
            res = max(res, dp[j]);
        }
    }
    return res; // 返回最长公共子数组长度
}

// 二维数组
int findLength(vector<int>& nums1, vector<int>& nums2) {
    int m = nums1.size();
    int n = nums2.size();
    // dp[i][j] 表示以 nums1[i-1] 和 nums2[j-1] 结尾的最长公共子数组长度
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
    int res = 0; // 记录最长公共子数组长度
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (nums1[i - 1] == nums2[j - 1])
                // 当前元素相等，可以延长前面的公共子数组长度
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                // 当前元素不等，连续公共子数组断开
                dp[i][j] = 0;
            res = max(res, dp[i][j]);
        }
    }
}

```

```

        return res; // 返回最长公共子数组长度
    }

```

1035. 不相交的线

// 直线不能相交，这就是说明在字符串A中 找到一个与字符串B相同的子序列，且这个子序列不能改变相对顺序，只要相对顺序不改变，链接相同数字的直线就不会相交

```

int maxUncrossedLines(vector<int>& nums1, vector<int>& nums2) {
    vector<vector<int>> dp(nums1.size() + 1, vector<int>(nums2.size() + 1, 0));
    for (int i = 1; i <= nums1.size(); i++) {
        for (int j = 1; j <= nums2.size(); j++) {
            if (nums1[i - 1] == nums2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[nums1.size()][nums2.size()];
}

```

1143. 最长公共子序列 todo 滚动数组

		0	1	2	3	4	5
	dp	a	b	c	b	a	
0	0	0	0	0	0	0	0
1 a	0	1	1	1	1	1	1
2 b	0	1	2	2	2	2	2
3 c	0	1					
4 b							
5 c							
6 b							
7 a							

```

// 空间优化
int longestCommonSubsequence(string text1, string text2) {
    if (text1.empty() || text2.empty())
        return 0;
    int n = text1.size();
    int m = text2.size();
    vector<int> dp(m + 1, 0);
    for (int i = 1; i <= n; i++) {
        int upLeft = dp[0]; // 每行开始的时候需要更新下upLeft，这里其实每次都是0
        for (int j = 1; j <= m; j++) {
            int tmp = dp[j]; // 记录未被覆盖之前的dp[j]，它会在计算 j+1的时候作为upLeft用到
            if (text1[i - 1] == text2[j - 1])
                dp[j] = upLeft + 1;
        }
    }
}

```

```

        else
            dp[j] = max(dp[j - 1], dp[j]);
        upLeft = tmp; // 更新upLeft
    }
}
return dp[m];
}

int longestCommonSubsequence(string word1, string word2) {
    int m = word1.size();
    int n = word2.size();
    if (m == 0 && n == 0)
        return 0; // 两个空字符串的最长公共子序列长度为0
    // dp[i][j] 表示 word1[0..i-1] 和 word2[0..j-1] 的最长公共子序列长度
    // 初始化 dp 长度为 m+1 和 n+1, 方便处理第0行和第0列 (空串情况)
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
    // 填充 dp 表
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1[i - 1] == word2[j - 1])
                // 当前字符相同, 可以延长前一段最长公共子序列长度
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                // 当前字符不同, 最长公共子序列长度取舍前一个字符的最大值
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    return dp[m][n];
}

```

4.划分型动态规划

状态: $f[i]$ 表示前 i 个元素的最大值; 方程: $f[i] = \text{前} i \text{ 个元素里面选一个区间的最大值}$; 初始化: $f[0]$; 答案: $f[n - 1]$

[188. Best Time to Buy and Sell Stock IV](#)

5.背包型动态规划

特点: 1). 用值作为DP维度, 2). DP过程就是填写矩阵, 3). 可以滚动数组优化 状态: $f[i][S]$ 前 i 个物品, 取出一些能否组成和为 S ; 方程: $f[i][S] = f[i-1][S-a[i]] \text{ or } f[i-1][S]$; 初始化: $f[i][0]=\text{true}$; $f[0][1...\text{target}]=\text{false}$; 答案: 检查所有 $f[n][j]$

有些代码细节 需要注意, 初始化dp的时候, 可以用 `nums.size()` 也可用 `nums.size() + 1`

[416. 分割等和子集](#) # todo 空间优化

```

// 解法一: 需要空间优化
bool canPartition(vector<int>& nums)
{
    int sum = 0;

```

```

for(int i = 0; i < nums.size(); i++)
{
    sum += nums[i];
}
int target = sum / 2;
if (sum % 2 == 1)
    return false;

int n = nums.size();
// 状态定义: dp[i][j]表示从数组的 [0, i] 这个子区间内挑选一些正整数, 每个数只能用一次, 使得这些数的和恰好等于 j。
// 状态转移方程: 很多时候, 状态转移方程思考的角度是「分类讨论」, 对于「0-1 背包问题」而言就是「当前考虑到的数字选与不选」。
// 不选择 nums[i], 如果在 [0, i - 1] 这个子区间内已经有一部分元素, 使得它们的和为 j, 那么 dp[i][j] = true;
// 选择 nums[i], 如果在 [0, i - 1] 这个子区间内就得找到一部分元素, 使得它们的和为 j - nums[i]。
vector<vector<bool>> dp(n, vector<bool>(target + 1, false));

// base case: 背包容量为0, 无论有多少个元素, 都可以选取, 因此为true
for (int i = 0; i < n; i++)
    dp[i][0] = true;

// 动态规划过程
for (int i = 1; i < n; i++) {
    for (int j = 0; j <= target; j++) {
        if (j - nums[i] >= 0) {
            // 如果当前背包容量j大于等于当前数字nums[i]
            // 可以选择装入或者不装入背包, 两者只要有一个为true, dp[i][j]就为true
            dp[i][j] = dp[i - 1][j] || dp[i - 1][j - nums[i]];
        } else {
            // 背包容量不足, 不能装入第 i 个物品, 保持上一行的状态
            dp[i][j] = dp[i - 1][j];
        }
    }
}
return dp[n - 1][target];
}

bool canPartition(vector<int> &nums)
{
    int sum = 0;
    for (int i = 0; i < nums.size(); i++)
    {
        sum += nums[i];
    }
    if (sum % 2 == 1)
        return false;
    int targetSum = sum / 2;
    // dp[i] 表示原数组是否可以取出若干个数字, 其和为i
    vector<bool> dp(targetSum + 1, false);
    dp[0] = true;
    for (int i = 1; i < nums.size(); i++)

```



```

{
    for (int j = targetSum; j > 0; j--)
    {
        if (j >= nums[i])
        {
            dp[j] = dp[j] || dp[j - nums[i]]; // 两种情况 分别是使用当前数字nums[i] 和不使用当前数字nums[i]
        }
    }
}
return dp[targetSum];
}

```

```

bool canPartition(vector<int>& nums) {
    int sum = accumulate(nums.begin(), nums.end(), 0);
    if (sum % 2 == 1)
        return false;
    int target = sum / 2;
    int n = nums.size();
    // dp[i][j]表示从数组nums[0...i]中选数放进容量为j的背包的最大价值
    vector<vector<int>> dp(n, vector<int>(target + 1, 0));

    // 初始化第一行
    for (int j = 0; j <= target; j++) {
        dp[0][j] = (nums[0] <= j) ? nums[0] : 0;
    }

    // 初始化第一列
    for (int i = 1; i < n; i++) {
        dp[i][0] = 0;
    }

    for(int i = 1; i < n; i++) {
        for(int j = 1; j <= target; j++) {
            if (j - nums[i] < 0)
                dp[i][j] = dp[i-1][j];
            else
                dp[i][j] = max(dp[i-1][j], dp[i-1][j - nums[i]] + nums[i]);
        }
    }
    return dp[n-1][target] == target;
}

```

```

/* 计算 nums 中有几个子集的和为 sum */
bool canPartition(vector<int>& nums) {
    int sum = 0;
    for(int i = 0; i < nums.size(); i++)
    {
        sum += nums[i];
    }
    int target = sum / 2;
    if (sum % 2 == 1)

```

```

        return false;

//dp[i]表示 背包总容量是i, 最大可以凑成i的子集总和为dp[i]。
vector<int> dp(target + 1 , 0);

for(int i = 0; i < nums.size(); i++)
{
    for(int j = target; j >= nums[i]; j--)
    {
        dp[j] = max(dp[j], dp[j-nums[i]] + nums[i]);
    }
}
return dp[target] == target;
}

```

698. 划分为k个相等的子集 [todo](#)

```

class Solution {
public:
    bool canPartitionKSubsets(vector<int>& nums, int k) {
        int n = nums.size();
        int total = accumulate(nums.begin(), nums.end(), 0);
        if (total % k != 0) return false; // 总和不能整除 k, 直接 false

        int target = total / k;

        // 排序 (从大到小), 有利于剪枝
        sort(nums.rbegin(), nums.rend());

        // 如果最大数比 target 还大, 不可能划分
        if (nums[0] > target) return false;

        vector<int> bucket(k, 0); // 每个桶的当前和
        return backtrack(nums, 0, bucket, target);
    }

    bool backtrack(vector<int>& nums, int index, vector<int>& bucket, int target) {
        if (index == nums.size()) {
            // 如果所有数都放完了, 检查每个桶是否都达到 target
            for (int b : bucket) {
                if (b != target) return false;
            }
            return true;
        }

        int num = nums[index];

        for (int i = 0; i < bucket.size(); i++) {
            // 如果放进去不超过目标

```

```

        if (bucket[i] + num <= target) {
            bucket[i] += num; // 尝试放进第 i 个桶
            if (backtrack(nums, index + 1, bucket, target)) return true;
            bucket[i] -= num; // 回溯

            // 剪枝: 如果当前桶是空的, 说明失败后其他空桶也会失败
            if (bucket[i] == 0) break;
        }
    }

    return false;
}

};

```

474. 一和零

```

int findMaxForm(vector<string> &strs, int m, int n)
{
    //dp[i][j]表示有i个0和j个1时能组成的最多字符串的个数
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
    for (string str : strs)
    {
        int zeros = 0, ones = 0;
        for (char c : str)
            (c == '0') ? ++zeros : ++ones;
        // // 遍历背包容量且从后向前遍历!
        for (int i = m; i >= zeros; --i)
        {
            for (int j = n; j >= ones; --j)
            {
                dp[i][j] = max(dp[i][j], dp[i - zeros][j - ones] + 1);
            }
        }
    }
    return dp[m][n];
}

```

494. 目标和

```

class Solution {
public:

    int target_sum(vector<int> &nums, int sum)
    {
        int n = nums.size();

        // dp[i][j] 表示 数组中nums[0...i]数字 放进背包容量为j的其价值等于j的组合数;
        vector<vector<int>> dp(n, vector<int>(sum+1, 0));

        dp[0][0] = 1;
        if(nums[0] == 0) dp[0][0] += 1;
    }
};

```

```

        for(int j = 1; j <= sum; j++) {
            if(j == nums[0]) dp[0][j] = 1;
        }

        for (int i = 1; i < n; i++)
        {
            for(int j = 0; j <= sum; j++)
            {
                if (j - nums[i] < 0)
                    dp[i][j] = dp[i-1][j];
                else
                    dp[i][j] = dp[i-1][j] + dp[i-1][j-nums[i]];
            }
        }
        return dp[n-1][sum];
    }

    int findTargetSumWays(vector<int>& nums, int target) {
        int sum = accumulate(nums.begin(), nums.end(), 0);
        if ((target + sum) / 2 < 0)
            return 0;
        return sum < target || (target + sum) % 2 == 1 ? 0 : target_sum(nums, (target +
sum) / 2);
    }
};

```

1049. 最后一块石头的重量 II

```

int lastStoneWeightII(vector<int>& stones) {
    int sum = accumulate(stones.begin(), stones.end(), 0);
    int target = sum / 2; // 目标值为总重量的一半
    int n = stones.size();

    // dp[i][j] 表示前 i 个石头中挑选一些，使得它们的总重量不超过 j 时的最大总重量
    vector<vector<int>> dp(n + 1, vector<int>(target + 1, 0));

    // 动态规划过程
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= target; j++) {
            if (j >= stones[i - 1]) {
                // 如果当前石头的重量不超过 j，则可以选择装入或者不装入背包
                dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - stones[i - 1]] + stones[i -
1]);
            } else {
                // 当前石头的重量超过了 j，无法装入背包
                dp[i][j] = dp[i - 1][j];
            }
        }
    }
}

```

```

// 返回总重量减去装入背包的最大重量的两倍，即石头的最小可能重量
return sum - 2 * dp[n][target];
}

int lastStoneWeightII(vector<int>& stones) {
    int sum = 0;
    for (int i = 0; i < stones.size(); i++)
        sum += stones[i];
    vector<int> dp(sum+1, 0);

    int target = sum / 2;
    for (int i = 0; i < stones.size(); i++) { // 遍历物品
        for (int j = target; j >= stones[i]; j--) { // 遍历背包
            dp[j] = max(dp[j], dp[j - stones[i]] + stones[i]);
        }
    }
    return sum - dp[target] - dp[target];
}

```

279. 完全平方数

```

int numSquares(int n)
{
    if (n <= 0)
        return 0;

    // dp[i] 表示数字i最少有几个平方数的和
    vector<int> dp(n+1, INT_MAX);
    dp[0] = 0;
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j * j <= i; j++)
        {
            dp[i] = min(dp[i], dp[i-j*j]+1); //这里有两种选择，第j个要不要
        }
    }

    return dp[n];
}

```

518. 零钱兑换 II #todo 空间优化 377

```

int change(int amount, vector<int>& coins) {
    vector<int> dp(amount + 1, 0);
    dp[0] = 1;
    for (int i = 0; i < coins.size(); i++) { // 遍历物品
        for (int j = coins[i]; j <= amount; j++) { // 遍历背包
            dp[j] += dp[j - coins[i]];
        }
    }
}

```

```

    return dp[amount];
}

int change(int amount, vector<int> &coins)
{
    //dp[i][j] 表示用前i个硬币组成钱数为j的不同组合方法
    vector<vector<int>> dp(coins.size() + 1, vector<int>(amount + 1, 0));
    dp[0][0] = 1;
    // 采用的方法是一个硬币一个硬币的增加，每增加一个硬币，都从1遍历到 amount，对于遍历到的当前钱数j，组成方法就是不加上当前硬币的拼法 dp[i-1][j]，还要加上去掉当前硬币值的钱数的组成方法
    for (int i = 1; i <= coins.size(); ++i)
    {
        for (int j = 0; j <= amount; ++j)
        {
            if(j >= coins[i - 1])
                dp[i][j] = dp[i - 1][j] + dp[i][j - coins[i - 1]]; // 第i个硬币有 使用和不使用两种情况
            else
                dp[i][j] = dp[i - 1][j];
        }
    }
    return dp[coins.size()][amount];
}

```

322. 零钱兑换

```

int coinChange(vector<int> &coins, int amount)
{
    // int dp[amount+1] = {amount+1};
    // dp[i] 表示钱数为i时的最小硬币数的找零，注意由于数组是从0开始的，所以要多申请一位，数组大小为 amount+1，这样最终结果就可以保存在 dp[amount] 中了
    //vector<int> dp(amount + 1, amount + 1);
    vector<int> dp(amount + 1, INT_MAX-1); //用这种初始化的方式要好理解点
    int size = coins.size();
    dp[0] = 0;
    for (int i = 1; i <= amount; i++)
    {
        for (int j = 0; j < size; j++)
        {
            if (coins[j] <= i)
            {
                dp[i] = min(dp[i], dp[i - coins[j]] + 1); //这里有两种选择，第j个硬币要不要
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}

```

6. 区间型动态规划

特点: 1). 求一段区间的解max/min/count; 2). 转移方程通过区间更新; 3). 从大到小的更新; 这种题目共性就是区间最后求[0, n-1]这样一个区间逆向思维分析, 从大到小就能迎刃而解

区间 DP 是在一个区间上进行的一系列动态规划, 在一个线性的数据上对区间进行状态转移, dp[i][j]表示i到j的区间。dp[i][j]可以由子区间的状态转移而来, 关键是 dp[i][j]表示什么, 然后找 dp[i][j]和子区间的关系

5. 最长回文子串

```
string longestPalindrome(string s)
{
    if (s.empty())
        return "";
    // dp[i][j] 表示 s[i...j]上是否为回文子串
    vector<vector<bool>> dp(s.size(), vector<bool>(s.size(), false));
    int len = 0;
    int left = 0;

    for(int i = 0; i < s.size(); i++)
    {
        for(int j = 0; j <= i; j++)
        {
            // 字符 s[i] 和 s[j] 是否相等以及内部子串 s[j+1...i-1] 是否为回文子串
            // i - j < 2 处理长度为 1 和 2 的子串的基本情况。
            dp[j][i] = s[i] == s[j] && (i - j < 2 || dp[j+1][i-1]);
            if (dp[j][i] && i - j + 1 > len)
            {
                len = i - j + 1;
                left = j;
            }
        }
    }
    return s.substr(left, len);
}
```

131. 分割回文串

```
bool isPalindrome(const string& s, int start, int end)
{
    while(start <= end) {
        if(s[start++] != s[end--])
            return false;
    }
    return true;
}

void dfs(string &s, vector<string> &temp, int start, vector<vector<string>> &res)
{
    if(start == s.size())
    {
        res.push_back(temp);
        return;
    }
}
```

```

for(int i = start; i < s.size(); i++)
{
    if (isPalindrome(s, start, i))
    {
        temp.push_back(s.substr(start, i - start + 1)); // index
        dfs(s, temp, i+1, res);
        temp.pop_back();
    }
}
}

vector<vector<string>> partition(string s)
{
    vector<vector<string>> > res;
    if(s.empty()) return res;

    vector<string> temp;
    dfs(s, temp, 0, res);

    return res;
}

```

132. 分割回文串 II

```

class Solution {
public:
    int minCut(string s) {
        if (s.empty())
            return 0;
        int n = s.size();
        // dp[i][j] 表示 s[i...j] 是否是回文子串
        vector<vector<bool>> dp(n, vector<bool>(n, false));
        // p[i] 表示 s[0...i] 上的最小分割次数
        vector<int> p(n, n);
        // 遍历字符串s的所有子串
        for(int i = 0; i < n; i++) {
            for(int j = 0; j <= i; j++) {
                // 判断s[j...i]是否是回文子串
                // 这里 i - j < 2 要放在前面 需要提前检查 子串长度为1、2的情况
                dp[j][i] = (s[j] == s[i]) && (i - j < 2 || dp[j+1][i-1]);

                // 如果s[j...i]是回文子串, 更新最小分割次数p[i]
                if (dp[j][i]) {
                    p[i] = (j == 0) ? 0 : min(p[i], p[j-1] + 1);
                }
            }
        }
        return p[n-1];
    }
};

```



```

// 解法2:
int minCut_2(string s)
{
    int n = s.size();
    if (n <= 0)
        return 0;

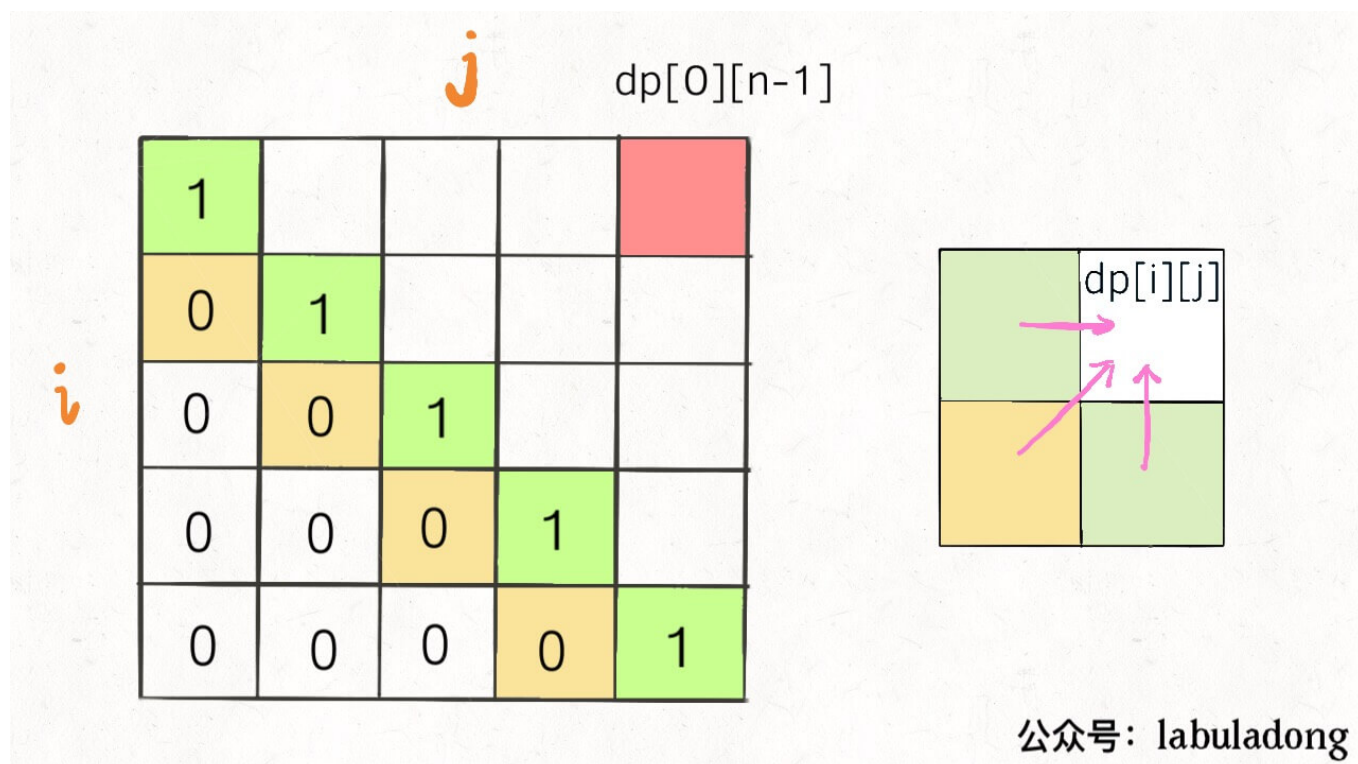
    // dp[i]表示s[i...n-1]的最小分割次数
    vector<int> dp(n + 1, 0);
    dp[n] = -1;
    vector<vector<bool>> p(n, vector<bool>(n, false));

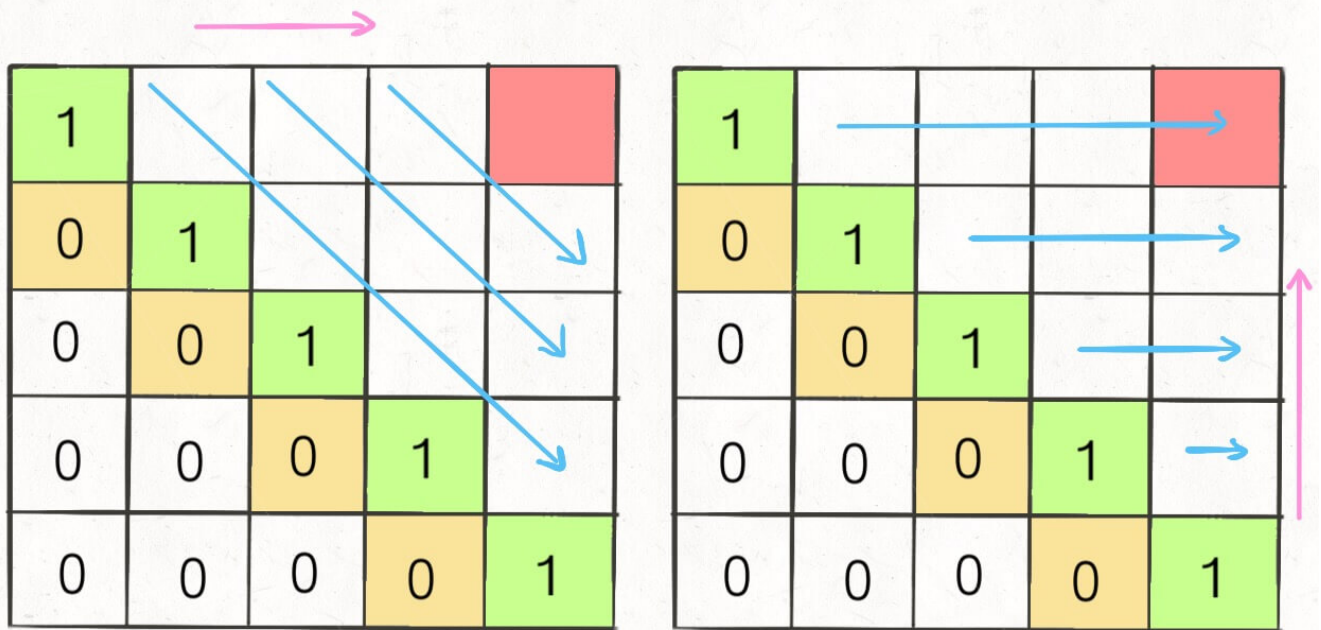
    for (int i = n - 1; i >= 0; i--)
    {
        dp[i] = INT_MAX;
        for (int j = i; j < n; j++)
        {
            if (s[i] == s[j] && (j - i < 2 || p[i + 1][j - 1])) // 判断s[i...j]是不是回
            {
                p[i][j] = true;
                dp[i] = min(dp[j + 1] + 1, dp[i]);
            }
        }
    }
    return dp[0];
};

```

文字串

516. 最长回文子序列 后面两题 逆向遍历 为什么??? 重点是画图 空间优化





公众号: labuladong

```
int longestPalindromeSubseq(string s)
{
    int n = s.size();
    // dp[i][j]表示[i,j]区间内的字符串的最长回文子序列
    vector<vector<int>> dp(n, vector<int>(n));
```

//如果 $s[i] == s[j]$, 那么 i 和 j 就可以增加2个回文串的长度, 我们知道中间 $dp[i + 1][j - 1]$ 的值, 那么其加上2就是 $dp[i][j]$ 的值。如果 $s[i] != s[j]$, 那么我们可以去掉 i 或 j 其中的一个字符, 然后比较两种情况下所剩的字符串谁 dp 值大, 就赋给 $dp[i][j]$

```
for (int i = 0; i < n; i++)
{
    dp[i][i] = 1;
    for (int j = i-1; j >= 0; j--)
    {
        if (s[i] == s[j])
        {
            dp[j][i] = dp[j+1][i-1] + 2;
        }
        else
        {
            dp[j][i] = max(dp[j][i-1], dp[j+1][i]);
        }
    }
}
return dp[0][n-1];
}
```

```
int longestPalindromeSubseq(string s) {
    int n = s.length();
```

```

int[] dp = new int[n];
for (int i = 0; i < n; ++i)
    dp[i] = 1;
for (int i = n - 1; i >= 0; --i) {
    int pre = 0;
    for (int j = i + 1; j < n; ++j) {
        int temp = dp[j];
        if (s.charAt(i) == s.charAt(j)) dp[j] = pre + 2;
        else dp[j] = Math.max(dp[j], dp[j - 1]);
        pre = temp;
    }
}
return dp[n - 1];
}

```

647. 回文子串 todo: 空间优化

```

int countSubstrings(string s)
{
    int n = s.size();
    if (n <= 0)
        return 0;
    int res = 0;
    // dp[i][j] 表示区间s[i...j]上是否为回文子串
    vector<vector<bool>> dp(n, vector<bool>(n, false));

    for (int i = 0; i < n; i++)
    {
        // dp[i][i] = true;
        for (int j = 0; j <= i; j++)
        {
            dp[j][i] = s[i] == s[j] && (i - j < 2 || dp[j + 1][i - 1]);
            if (dp[j][i])
                res++;
        }
    }
    return res;
}

// 空间优化
int countSubstrings(string s) {
    int n = s.size();
    if (n <= 0)
        return 0;
    int res = 0;
    // dp[i][j] 表示区间s[i...j]上是否为回文子串
    vector<bool> dp(n, false);

    for (int i = 0; i < n; i++)
    {

```

```

        for (int j = 0; j <= i; j++)
        {
            dp[j] = s[i] == s[j] && (i - j < 2 || dp[j + 1]);
            if (dp[j])
            {
                res++;
            }
        }
    }
    return res;
}

```

1312. 让字符串成为回文串的最少插入次数

```

class Solution {
public:
    int minInsertions(string s) {
        int n = s.size();
        vector<vector<int>> dp(n, vector<int>(n, 0));

        for (int len = 2; len <= n; ++len) {
            for (int i = 0; i + len - 1 < n; ++i) {
                int j = i + len - 1;
                if (s[i] == s[j]) {
                    dp[i][j] = dp[i+1][j-1];
                } else {
                    dp[i][j] = min(dp[i+1][j], dp[i][j-1]) + 1;
                }
            }
        }

        return dp[0][n-1];
    }
};

```

```

int minInsertions(string s) {
    int n = s.size();
    // dp[i][j]表示[i,j]区间内的字符串的最长回文子序列
    vector<vector<int>> dp(n, vector<int>(n));

```

//如果s[i]==s[j]，那么i和j就可以增加2个回文串的长度，我们知道中间dp[i + 1][j - 1]的值，那么其加上2就是dp[i][j]的值。如果s[i] != s[j]，那么我们可以去掉i或j其中的一个字符，然后比较两种情况下所剩的字符串谁dp值大，就赋给dp[i][j]

```

    for (int i = 0; i < n; i++)
    {
        dp[i][i] = 1;
        for (int j = i-1; j >= 0; j--)
        {

```

```

        if (s[i] == s[j])
        {
            dp[j][i] = dp[j+1][i-1] + 2;
        }
        else
        {
            dp[j][i] = max(dp[j][i-1], dp[j+1][i]);
        }
    }
}
return s.size() - dp[0][n-1];
}

```

7. 博弈型动态规划状态

定义一个人的状态; 方程: 考虑两个人的状态做状态更新; 初始化: 暂无; 答案: 先思考最小状态, 再思考大的状态 -> 往小的递推, 适合记忆化搜索 动态规划, 循环(从小到大递推), 记忆化搜索(从大到小搜索, 画搜索树); 什么时候 用记忆化搜索: 1). 状态转移特别麻烦, 不是顺序性, 2). 初始化状态不是很容易找到; 题目类型: 1). 博弈类问题, 2). 区间类问题; 适合解决题目: 1). 状态特别复杂, 2). 不好初始化

[486. Predict the Winner](#)

```

class Solution
{
    // 作为先发者 在i...j范围上先发获得的收益
    int f(vector<int> &nums, int i, int j)
    {
        if (i == j) // 如果只有一个数并且又是先发者, 则直接拿走该数
            return nums[i];

        else
            return max(s(nums, i + 1, j) + nums[i], s(nums, i, j - 1) + nums[j]);
    }
    // / 作为后发者 在i...j范围上后发获得的收益
    int s(vector<int> &nums, int i, int j)
    {
        if (i == j)
        {
            return 0;
        }
        else // 对方也是绝顶聪明, 作为后发者, 此时只能选先发者拿完之后 剩下最小的
            return min(f(nums, i + 1, j), f(nums, i, j - 1));
    }
public:
    bool PredictTheWinner(vector<int> &nums)
    {

        if (nums.empty())
            return false;
        int sum = 0;
    }
}

```

```

        for (int i = 0; i < nums.size(); i++)
            sum += nums[i];

        int res = f(nums, 0, nums.size() - 1);
        return sum - res > res ? false : true;
    }
};

```

8.打家劫舍系列

198. 打家劫舍

```

// 本质相当于在一列数组中取出一个或多个不相邻数，使其和最大
// dp, 其中 dp[i] 表示 [0, i] 区间可以抢夺的最大值
// 对当前i来说，有抢和不抢两种互斥的选择，不抢即为 dp[i-1]（等价于去掉 nums[i] 只抢 [0, i-1] 区间最大值），
// 抢即为 dp[i-2] + nums[i]（等价于去掉 nums[i-1]）
int rob(vector<int>& nums) {
    if (nums.size() <= 1)
        return nums.empty() ? 0 : nums[0];
    // dp, 其中 dp[i] 表示 [0, i] 区间可以抢夺的最大值
    vector<int> dp(nums.size(), 0);

    dp[0] = nums[0];
    dp[1] = max(nums[0], nums[1]);
    // 对当前i来说，有抢和不抢两种互斥的选择，不抢即为 dp[i-1]（等价于去掉 nums[i] 只抢 [0, i-1] 区间最大值），
    // 抢即为 dp[i-2] + nums[i]（等价于去掉 nums[i-1]）
    for(int i = 2; i < nums.size(); i++)
    {
        dp[i] = max(nums[i] + dp[i-2], dp[i-1]);
    }
    return dp[nums.size()-1];
}

```

// 使用两个变量 rob 和 notRob，其中 rob 表示抢当前的房子，notRob 表示不抢当前的房子，那么在遍历的过程中，先用两个变量 preRob 和 preNotRob 来分别记录更新之前的值，由于 rob 是要抢当前的房子，那么前一个房子一定不能抢，所以使用 preNotRob 加上当前的数字赋给 rob，然后 notRob 表示不能抢当前的房子，那么之前的房子就可以抢也可以不抢，所以将 preRob 和 preNotRob 中的较大值赋给 notRob

```

int rob(vector<int>& nums) {
    int rob = 0, notRob = 0, n = nums.size();
    for (int i = 0; i < n; ++i) {
        int preRob = rob, preNotRob = notRob;
        rob = preNotRob + nums[i];
        notRob = max(preRob, preNotRob);
    }
    return max(rob, notRob);
}

```

213. 打家劫舍 II

// 现在房子排成了一个圆圈，则如果抢了第一家，就不能抢最后一家，因为首尾相连了，所以第一家 and 最后一家只能抢其中的一家，或者都不抢，
// 那这里变通一下，如果把第一家 and 最后一家分别去掉，各算一遍能抢的最大值，然后比较两个值取其中较大的一个即为所求

```
int rob(vector<int> &nums, int left, int right) {
    if (right - left <= 1) return nums[left];
    vector<int> dp(right, 0);
    dp[left] = nums[left];
    dp[left + 1] = max(nums[left], nums[left + 1]);
    for (int i = left + 2; i < right; ++i) {
        dp[i] = max(nums[i] + dp[i - 2], dp[i - 1]);
    }
    return dp.back();
}

int rob(vector<int> &nums, int left, int right) {
    int rob = 0, notRob = 0;
    for (int i = left; i < right; ++i) {
        int preRob = rob, preNotRob = notRob;
        rob = preNotRob + nums[i];
        notRob = max(preRob, preNotRob);
    }
    return max(rob, notRob);
}

int rob(vector<int>& nums) {
    if (nums.size() <= 1) return nums.empty() ? 0 : nums[0];
    return max(rob(nums, 0, nums.size() - 1), rob(nums, 1, nums.size()));
}
```

337. 打家劫舍 III

```
// https://www.cnblogs.com/grandyang/p/5275096.html
int dfs(TreeNode *root, unordered_map<TreeNode*, int> &m)
{
    if (!root) return 0;
    if (m.count(root)) return m[root];
    int val = 0;
    if (root->left) {
        val += dfs(root->left->left, m) + dfs(root->left->right, m);
    }
    if (root->right) {
        val += dfs(root->right->left, m) + dfs(root->right->right, m);
    }
    val = max(val + root->val, dfs(root->left, m) + dfs(root->right, m));
    m[root] = val;
    return val;
}

int rob(TreeNode* root) {
    unordered_map<TreeNode*, int> m;
    return dfs(root, m);
}
```

```
}
```

9.股票系列

[121. 买卖股票的最佳时机](#)

// 遍历一次数组，用一个变量记录遍历过程中的最小值，然后每次计算当前值和这个最小值之间的差值最为利润，然后每次选较大的利润来更新

```
int maxProfit(vector<int>& prices)
{
    int res = INT_MIN;
    int cur_min = INT_MAX;
    for(int i = 0; i < prices.size(); i++)
    {
        cur_min = min(cur_min, prices[i]);
        res = max(prices[i]-cur_min, res);
    }
    return res;
}
```

[122. 买卖股票的最佳时机 II](#)

```
int maxProfit(vector<int>& prices)
{
    // 从第二天开始，如果当前价格比之前价格高，则把差值加入利润中
    int res = 0, n = prices.size();
    for (int i = 0; i < n - 1; ++i) {
        if (prices[i] < prices[i + 1]) {
            res += prices[i + 1] - prices[i];
        }
    }
    return res;
}
```

[123. 买卖股票的最佳时机 III](#)

```
int maxProfit(vector<int> &prices) {
    if (prices.empty()) return 0;
    int n = prices.size(), g[n][3] = {0}, l[n][3] = {0};
    for (int i = 1; i < prices.size(); ++i) {
        int diff = prices[i] - prices[i - 1];
        for (int j = 1; j <= 2; ++j) {
            l[i][j] = max(g[i - 1][j - 1] + max(diff, 0), l[i - 1][j] + diff);
            g[i][j] = max(l[i][j], g[i - 1][j]);
        }
    }
    return g[n - 1][2];
}
```

[309. 最佳买卖股票时机含冷冻期](#)


```
int maxProfit(vector<int>& prices) {
    int buy = INT_MIN, pre_buy = 0, sell = 0, pre_sell = 0;
    for (int price : prices) {
        pre_buy = buy;
        buy = max(pre_sell - price, pre_buy);
        pre_sell = sell;
        sell = max(pre_buy + price, pre_sell);
    }
    return sell;
}
```

10.其他

221. 最大正方形 todo

```
int maximalSquare(vector<vector<char>>& matrix)
{
    if (matrix.empty() || matrix[0].empty())
        return 0;
    int m = matrix.size(), n = matrix[0].size(), res = 0;
    // dp[i][j] 表示到达 (i, j) 位置所能组成的最大正方形的边长
    // p[i][j], 由于该点是正方形的右下角, 所以该点的右边, 下边, 右下边都不用考虑, 关心的就是左边, 上边, 和左上边。这三个位置的dp值 suppose 都应该算好的, 还有就是要知道一点, 只有当前 (i, j) 位置为1, dp[i][j] 才有可能大于0, 否则 dp[i][j] 一定为0。当 (i, j) 位置为1, 此时要看 dp[i-1][j-1], dp[i][j-1], 和 dp[i-1][j] 这三个位置, 我们找其中最小的值, 并加上1, 就是 dp[i][j] 的当前值了
    vector<vector<int>> dp(m, vector<int>(n, 0));
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            if (i == 0 || j == 0) dp[i][j] = matrix[i][j] - '0';
            else if (matrix[i][j] == '1') {
                dp[i][j] = min(dp[i - 1][j - 1], min(dp[i][j - 1], dp[i - 1][j])) + 1;
            }
            res = max(res, dp[i][j]);
        }
    }
    return res * res;
}
```

1277. 统计全为 1 的正方形子矩阵

```
int countSquares(vector<vector<int>>& matrix) {
    int m = matrix.size(), n = matrix[0].size(), res = 0;
    // 定义一个二维 dp 数组, 其中 dp[i][j] 表示以 (i, j) 为右下顶点的最大的正方形子数组的边长,
    // 同时正好也是以 (i, j) 为右下顶点的正方形子数组的个数
    // dp 数组可以直接就初始化为 matrix, 因为每个为1的点, 正好也是一个边长为1的正方形子数组, 满足 dp 的定义
    vector<vector<int>> dp = matrix;
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            if (dp[i][j] > 0 && i > 0 && j > 0) {
```

```

        dp[i][j] = min({dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]}) + 1;
    }
    res += dp[i][j];
}
}
return res;
}

```

887. 鸡蛋掉落

```

int superEggDrop(int K, int N) {
    // DP, 其中 dp[i][j] 表示有i个鸡蛋, j层楼要测需要的最小操作数
    // 鸡蛋碎掉: 接下来就要用 i-1 个鸡蛋来测 k-1 层, 所以需要 dp[i-1][k-1] 次操作。
    // 鸡蛋没碎: 接下来还可以用i个鸡蛋来测 j-k 层, 所以需要 dp[i][j-k] 次操作。
    // 面对最坏的情况, 所以在第j层扔, 需要 max(dp[i-1][k-1], dp[i][j-k])+1 步, 状态转移方程为:
    vector<vector<int>> dp(K + 1, vector<int>(N + 1));
    for (int j = 1; j <= N; ++j) dp[1][j] = j;
    for (int i = 2; i <= K; ++i) {
        for (int j = 1; j <= N; ++j) {
            dp[i][j] = j;
            int left = 1, right = j;
            while (left < right) {
                int mid = left + (right - left) / 2;
                if (dp[i - 1][mid - 1] < dp[i][j - mid]) left = mid + 1;
                else right = mid;
            }
            dp[i][j] = min(dp[i][j], max(dp[i - 1][right - 1], dp[i][j - right]) + 1);
        }
    }
    return dp[K][N];
}

```

<https://leetcode.cn/problems/pile-box-lcci/solution/ti-mu-zong-jie-zui-chang-shang-sheng-zi-7jfd3/>

分治

395. Longest Substring with At Least K Repeating Characters

深度优先搜索DFS

和BFS一样一般需要一个set来记录访问过的节点, 避免重复访问造成死循环; Word XXX 系列面试中非常常见, 例如word break, word ladder, word pattern, word search。

- Leetcode 341 Flatten Nested List Iterator (339 364)
- Leetcode 394 Decode String
- Leetcode 291 Word Pattern II (I为简单的Hashmap题) 收费
- Leetcode 856 Score of Parentheses
- Leetcode 1087 Brace Expansion
- Leetcode 399 Evaluate Division

- Leetcode 1274 Number of Ships in a Rectangle
- Leetcode 1376 Time Needed to Inform All Employees
- Leetcode 694 Number of Distinct Islands

经典DFS

```
int dirs[8][2] = {1,1,1,0,1,-1,0,1,0,-1,-1,1,-1,0,-1,-1};

void dfs(const vector<vector<int> >& nums, vector<vector<bool> >& visit, int i, int j,
int& value)
{
    // 先写终止条件
    if ( nums[i][j] == 0 || visit[i][j])
        return;
    visit[i][j] = true;
    value += nums[i][j];
    for(int k = 0; k < 8; k++)
    {
        int x = i + dirs[k][0];
        int y = j + dirs[k][1];
        if (x < 0 || x > nums.size() || y < 0 || y > nums[0].size() || visit[x][y])
            continue;
        dfs(nums, visit, x, y, value);
    }
}
```

36. 有效的数独

```
bool isValidSudoku(vector<vector<char>>& board)
{
    //那么根据数独矩阵的定义，在遍历每个数字的时候，就看看包含当前位置的行和列以及 3x3 小方阵中是否已经出现该数字，
    // 这里需要三个 boolean 型矩阵，大小跟原数组相同，
    // 分别记录各行，各列，各小方阵是否出现某个数字，其中行和列标志下标很好对应，就是小方阵的下标需要稍稍转换一下，
    vector<vector<bool>> rowFlag(9, vector<bool>(9));
    vector<vector<bool>> colFlag(9, vector<bool>(9));
    vector<vector<bool>> cellFlag(9, vector<bool>(9));
    for (int i = 0; i < 9; ++i)
    {
        for (int j = 0; j < 9; ++j)
        {
            if (board[i][j] == '.')
                continue;
            int c = board[i][j] - '1';
            if (rowFlag[i][c] || colFlag[c][j] || cellFlag[3 * (i / 3) + j / 3][c])
                return false;
            rowFlag[i][c] = true;
            colFlag[c][j] = true;
```

```

        cellFlag[3 * (i / 3) + j / 3][c] = true;
    }
}
return true;
}

```

37. 解数独

```

class Solution {
    bool isValid(vector<vector<char>>& board, int row, int col, char c){
        for(int i = 0; i < 9; i++) {
            if(board[i][col] != '.' && board[i][col] == c) return false; //check row
            if(board[row][i] != '.' && board[row][i] == c) return false; //check column
            if(board[3 * (row / 3) + i / 3][ 3 * (col / 3) + i % 3] != '.' &&
                board[3 * (row / 3) + i / 3][3 * (col / 3) + i % 3] == c) return false;
        }
        //check 3*3 block
        return true;
    }

    bool dfs(vector<vector<char>>& board)
    {
        for(int i = 0; i < board.size(); i++)
        {
            for(int j = 0; j < board[0].size(); j++)
            {
                if(board[i][j] == '.')
                {
                    for(char c = '1'; c <= '9'; c++)
                    {
                        if (isValid(board, i, j, c))
                        {
                            // 暴力递归
                            board[i][j] = c;
                            if (dfs(board))
                                return true;
                            else
                                board[i][j] = '.';
                        }
                    }
                    return false;
                }
            }
        }
        return true;
    }

public:
    void solveSudoku(vector<vector<char>>& board) {
        dfs(board);
    }
};

```

51. N 皇后

```
class Solution {
public:
    vector<vector<string>> solveNQueens(int n) {
        vector<vector<string>> res;
        vector<string> queens(n, string(n, '.'));
        helper(0, queens, res);
        return res;
    }
    void helper(int curRow, vector<string>& queens, vector<vector<string>>& res) {
        int n = queens.size();
        if (curRow == n) {
            res.push_back(queens);
            return;
        }
        for (int i = 0; i < n; ++i) {
            if (isValid(queens, curRow, i)) {
                queens[curRow][i] = 'Q';
                helper(curRow + 1, queens, res);
                queens[curRow][i] = '.';
            }
        }
    }
    bool isValid(vector<string>& queens, int row, int col) {
        for (int i = 0; i < row; ++i) {
            if (queens[i][col] == 'Q') return false;
        }
        for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; --i, --j) {
            if (queens[i][j] == 'Q') return false;
        }
        for (int i = row - 1, j = col + 1; i >= 0 && j < queens.size(); --i, ++j) {
            if (queens[i][j] == 'Q') return false;
        }
        return true;
    }
};
```

52. N皇后 II

```
class Solution {
    bool isValid(vector<int> &record, int i, int j)
    {
        for(int k = 0; k < i; k++) {
            if (j == record[k] || abs(k - i) == abs(record[k] - j))
                return false;
        }
        return true;
    }
    int dfs(int i, vector<int> &record, int n)
    {

```

```

        if (i == n) {
            return 1;
        }
        int res = 0;
        for(int j = 0; j < n; j++){
            if (isValid(record, i, j)) {
                record[i] = j;
                res += dfs(i+1, record, n);
            }
        }
        return res;
    }
public:
    int totalNQueens(int n)
    {
        vector<int> record(n, 0);
        if (n < 1)
            return 0;
        return dfs(0, record, n);
    }
};

```

79. 单词搜索

```

class Solution {
public:
    // 深度优先搜索函数
    bool dfs(vector<vector<char>>& board, string &word, int i, int j, int step) {
        // 判断是否越界或者当前位置的字符不匹配
        if (i >= board.size() || j >= board[0].size() || i < 0 || j < 0 || step >=
word.size() || word[step] != board[i][j])
            return false;

        // 如果已经匹配到最后一个字符，则说明找到了完整的单词
        if (step == word.size() - 1 && word[step] == board[i][j])
            return true;

        // 保存当前字符，并将其标记为已访问
        char temp = board[i][j];
        board[i][j] = '0';

        // 在上、下、左、右四个方向进行深度优先搜索
        bool res = dfs(board, word, i + 1, j, step + 1) ||
                    dfs(board, word, i - 1, j, step + 1) ||
                    dfs(board, word, i, j + 1, step + 1) ||
                    dfs(board, word, i, j - 1, step + 1);

        // 恢复当前位置的字符，并返回搜索结果
        board[i][j] = temp;
        return res;
    }
}

```

```

bool exist(vector<vector<char>>& board, string word) {
    // 首先检查输入矩阵是否为空
    if (board.size() == 0)
        return false;

    // 遍历整个矩阵
    for(int i = 0; i < board.size(); i++) {
        for(int j = 0; j < board[0].size(); j++) {
            // 对于每一个起始位置，调用深度优先搜索函数
            if (dfs(board, word, i, j, 0))
                return true;
        }
    }
    // 如果遍历完整个矩阵都没有找到符合条件的单词，则返回 false
    return false;
}
};

```

130. 被围绕的区域

```

class Solution {
public:
    void solve(vector<vector<char>> &board)
    {

        if (board.empty() || board[0].empty())
            return;

        int m = board.size();
        int n = board[0].size();
        for (int i = 0; i < m; ++i)
        {
            for (int j = 0; j < n; ++j)
            {
                // 从出边界出发，将边界上为o的并且连在一起的都变成$
                if ((i == 0 || i == m - 1 || j == 0 || j == n - 1) && board[i][j] == 'O')
                    solveDFS(board, i, j, m, n);
            }
        }
        // 先将没有变成$也就是不和边界上的o连在一起的改为x,
        for (int i = 0; i < board.size(); ++i) {
            for (int j = 0; j < board[i].size(); ++j) {
                if (board[i][j] == 'O') board[i][j] = 'X';
                if (board[i][j] == '$') board[i][j] = 'O';
            }
        }
    }

    void solveDFS(vector<vector<char>> &board, int i, int j, int m, int n) {

        if (i >= m || i < 0 || j >= n || j < 0 || board[i][j] != 'O') return;
    }
}

```

```

        board[i][j] = '$';
        solveDFS(board, i - 1, j, m, n);
        solveDFS(board, i, j + 1, m, n);
        solveDFS(board, i + 1, j, m, n);
        solveDFS(board, i, j - 1, m, n);
    }
};

```

200. 岛屿数量

```

class Solution
{
    void dfs(vector<vector<char>> &grid, int i, int j, int m, int n)
    {
        if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] != '1')
            return;
        grid[i][j] = '2';
        dfs(grid, i + 1, j, m, n);
        dfs(grid, i - 1, j, m, n);
        dfs(grid, i, j + 1, m, n);
        dfs(grid, i, j - 1, m, n);
    }

public:
    int numIslands(vector<vector<char>> &grid)
    {
        if (grid.empty())
            return 0;
        int m = grid.size();
        int n = grid[0].size();
        int res = 0;
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (grid[i][j] == '1')
                {
                    res++;
                    dfs(grid, i, j, m, n);
                }
            }
        }
        return res;
    }
};

```

695. 岛屿的最大面积


```

int dfs(vector<vector<int>>& grid, int i, int j)
{
    if (i >= grid.size() || i < 0 || j >= grid[0].size() || j < 0 || grid[i][j] != 1)
        return 0;
    int res = 0;
    if (grid[i][j] == 1)
    {
        grid[i][j] = 2;
        res = 1 + dfs(grid, i + 1, j )
            + dfs(grid, i - 1, j)
            + dfs(grid, i, j + 1)
            + dfs(grid, i, j - 1);
        // grid[i][j] = 1;
    }
    return res;
}

int maxAreaOfIsland(vector<vector<int>>& grid) {
    int res = 0;
    for(int i = 0; i < grid.size(); i++)
    {
        for(int j = 0; j < grid[0].size(); j++)
        {
            if (grid[i][j] != 1)
                continue;
            res = max(res, dfs(grid, i, j));
        }
    }
    return res;
}

```

827. 最大人工岛

```

int dirs[4][2] = {0, 1, 1, 0, 0, -1, -1, 0};
int dfs(vector<vector<int>>& grid, vector<vector<bool>>& visited, int i, int j)
{
    int m = grid.size();
    int n = grid[0].size();
    if (i < 0 || i >= m || j < 0 || j >= n || visited[i][j] == true || grid[i][j] != 1 )
        return 0;
    visited[i][j]=true;
    int curSize = 1;

    for(int k = 0; k < 4; k++)
    {
        int x = i + dirs[k][0];
        int y = j + dirs[k][1];
        curSize += dfs(grid, visited, x, y);
    }
    return curSize;
}

```

```

        // return 1 + dfs(grid, visited, i+1,j) + dfs(grid, visited, i-1,j) + dfs(grid,
visited, i,j+1) + dfs(grid, visited, i,j-1);
    }
    int largestIsland(vector<vector<int>>& grid)
    {
        if (grid.empty())
            return 0;
        int m = grid.size();
        int n = grid[0].size();
        if(grid==vector<vector<int>>(m,vector<int>(n,1))) return m*n;
        vector<vector<bool>> visited(m, vector<bool>(n, false));
        vector<vector<bool>> temp(m, vector<bool>(n, false));
        int res = 0;
        int maxSize = INT_MIN;
        for(int i = 0; i < m; i++)
        {
            for(int j = 0; j < n; j++)
            {
                if (grid[i][j] == 0)
                {
                    grid[i][j] = 1;
                    res = dfs(grid, visited, i, j);
                    visited=temp;
                    maxSize = max(res, maxSize);
                    grid[i][j]=0;
                }
            }
        }
        return maxSize;
    }
}

```

212. 单词搜索 II

341. 扁平化嵌套列表迭代器

```

class NestedIterator {
private:
    stack<NestedInteger> s;
public:
    NestedIterator(vector<NestedInteger> &nestedList) {
        for (int i = nestedList.size() - 1; i >= 0; --i) {
            s.push(nestedList[i]);
        }
    }
    int next() {
        NestedInteger t = s.top(); s.pop();
        return t.getInteger();
    }

    bool hasNext() {
        while (!s.empty()) {

```

```

        NestedInteger t = s.top();
        if (t.isInteger()) return true;
        s.pop();
        for (int i = t.getList().size() - 1; i >= 0; --i) {
            s.push(t.getList()[i]);
        }
    }
    return false;
}
};

```

399. 除法求值

547. 省份数量

```

class Solution {
public:
    // 深度优先搜索来标记与当前城市直接或间接相连的城市，然后遍历每个城市，如果某个城市未被访问过，则说明
    // 它属于一个新的省份。通过深度优先搜索标记所有与该城市直接或间接相连的城市，并增加省份数量。最后返回省份数量。
    // 深度优先搜索，标记与当前城市直接或间接相连的城市
    void dfs(vector<vector<int>>& isConnected, vector<bool>& visited, int index) {
        visited[index] = true;

        // 遍历与当前城市直接或间接相连的城市
        for (int i = 0; i < isConnected.size(); i++) {
            // 如果两个城市直接相连或者已经访问过，则跳过
            if (isConnected[index][i] == 0 || visited[i]) {
                continue;
            }
            // 对于未访问过的城市，继续深度优先搜索
            dfs(isConnected, visited, i);
        }
    }

    // 计算省份数量
    int findCircleNum(vector<vector<int>>& isConnected) {
        int provinces = 0; // 省份数量
        int n = isConnected.size(); // 城市数量
        vector<bool> visited(n, false); // 标记城市是否被访问过
        // 遍历每个城市
        for (int i = 0; i < n; i++) {
            // 如果当前城市未被访问过，说明它属于一个新的省份
            if (!visited[i]) {
                dfs(isConnected, visited, i); // 对当前城市进行深度优先搜索，标记与之相连的城市
                provinces++; // 增加省份数量
            }
        }
        return provinces;
    }
}

```

```
};
```

785.判断二分图

```
class Solution {
public:
    // 染色法，大体上的思路是要将相连的两个顶点染成不同的颜色，
    // 一旦在染的过程中发现有两连的两个顶点已经被染成相同的颜色，说明不是二分图
    // 使用两种颜色，分别用1和 -1 来表示，初始时每个顶点用0表示未染色，然后遍历每一个顶点，如果该顶点未被
    // 访问过，则调用递归函数，
    // 如果返回 false，那么说明不是二分图，则直接返回 false
    bool dfs(vector<vector<int>> &graph, int color, int cur, vector<int> &colors)
    {
        // 如果当前顶点已经染色，如果该顶点的颜色和将要染的颜色相同，则返回 true，否则返回 false
        if (colors[cur] != 0)
            return colors[cur] == color;
        colors[cur] = color; // 给当前节点染色

        // 如果该顶点未被访问过，则调用递归函数
        for(int i = 0; i < graph[cur].size(); i++)
        {
            if (!dfs(graph, -1 * color, graph[cur][i], colors))
                return false;
        }
        return true;
    }

    // 遍历整个顶点，如果未被染色，则先染色为1，然后使用 BFS 进行遍历，将当前顶点放入队列 queue 中，然后
    // while 循环 queue 不为空，取出队首元素，遍历其所有相邻的顶点，如果相邻顶点未被染色，则染成和当前顶点相反的颜色，然后把相邻顶点加入 queue 中，否则如果当前顶点和相邻顶点颜色相同，直接返回 false，循环退出后返回
    // true
    bool bfs(vector<vector<int>>& graph)
    {
        vector<int> colors(graph.size());
        for(int i = 0; i < graph.size(); i++)
        {
            if (colors[i] != 0)
                continue;

            colors[i] = 1;
            queue<int> q;
            q.push(i);
            while(!q.empty())
            {
                int t = q.front(); q.pop();
                for(int i = 0; i < graph[t].size(); i++)
                {
                    if (colors[graph[t][i]] == colors[t])
                        return false;
                    if (colors[graph[t][i]] == 0)
                    {

```

```

        colors[graph[t][i]] = -1 * colors[t];
        q.push(graph[t][i]);
    }
}
}
}
return true;
}

bool isBipartite(vector<vector<int>>& graph) {
    return bfs(graph);
}
};

```

797. 所有可能的路径

```

class Solution {
public:
    // 深度优先搜索函数 邻接表
    // 深度优先搜索 (DFS) 算法，从节点 0 开始进行遍历，每次遍历都将当前节点加入路径，并继续递归遍历该节点的
    // 邻居节点。如果当前节点是终点（目标节点），则将当前路径加入结果集
    void dfs(vector<vector<int>>& graph, vector<vector<int>>& res, vector<int> path, int
cur) {
        path.push_back(cur); // 将当前节点加入路径

        // 如果当前节点是终点（即目标节点），将当前路径加入结果集
        if (cur == graph.size() - 1) {
            res.push_back(path);
            return;
        }

        // 遍历当前节点能够到达的所有节点，进行深度优先搜索
        for (int i = 0; i < graph[cur].size(); i++) {
            dfs(graph, res, path, graph[cur][i]);
        }
    }

    vector<vector<int>> allPathsSourceTarget(vector<vector<int>>& graph) {
        vector<vector<int>> res; // 存储结果的二维向量
        vector<int> path; // 存储当前路径的向量
        dfs(graph, res, path, 0); // 从节点 0 开始进行深度优先搜索
        return res; // 返回结果集
    }
};

```

856. 括号的分数

```

class Solution {
public:
    int dfs(const string& s, int& index) {
        int score = 0;

```

```

while (index < s.size()) {
    if (s[index] == '(') {
        index++; // 跳过左括号
        int innerScore = dfs(s, index);
        score += (innerScore == 0 ? 1 : 2 * innerScore);
    } else if (s[index] == ')') {
        index++; // 跳过右括号
        return score;
    }
}
return score;
}

int scoreOfParentheses(string s) {
    int index = 0;
    return dfs(s, index);
}
};

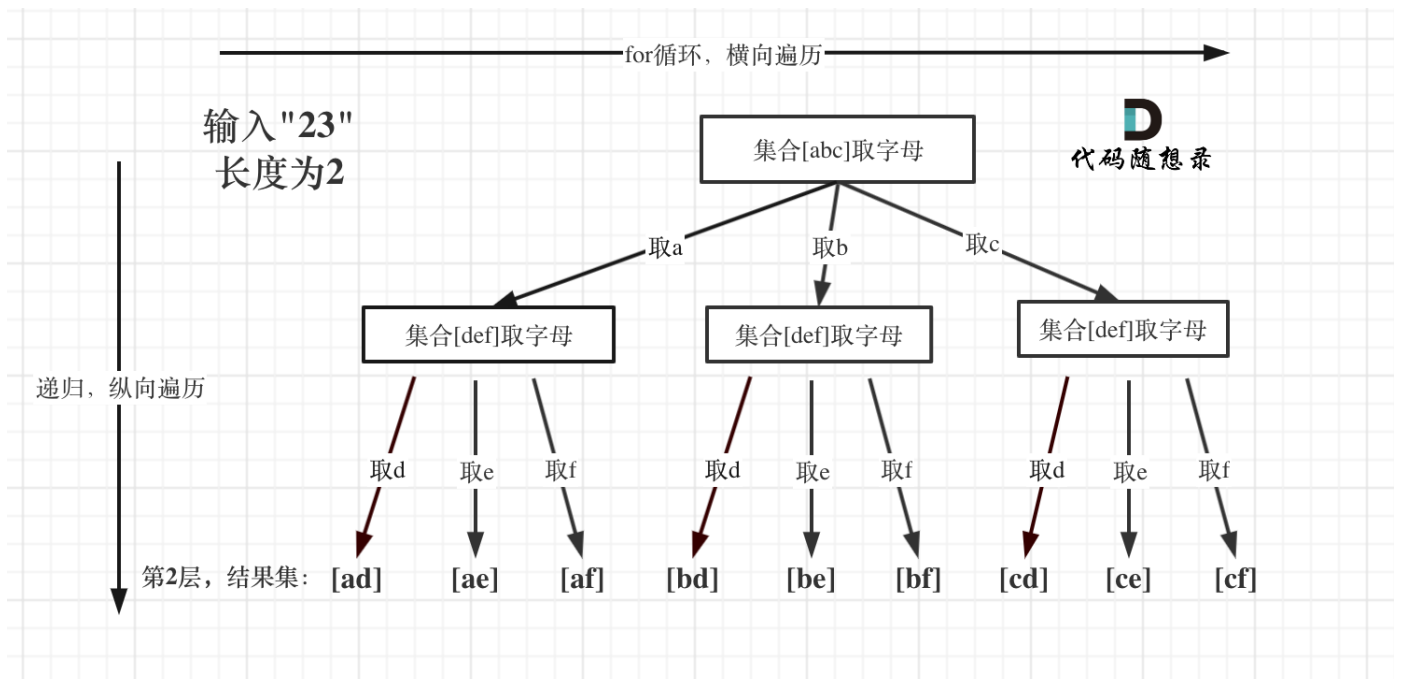
```

1376. 通知所有员工所需的时间

回溯

其实与图类DFS方法一致，但是排列组合的特征更明显

17. 电话号码的字母组合



```

//Input: digits = "23"
//Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]
vector<string> dict = {"abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
void dfs(vector<string> &res, string temp, string digits, int index)
{
    if (index == digits.size())

```

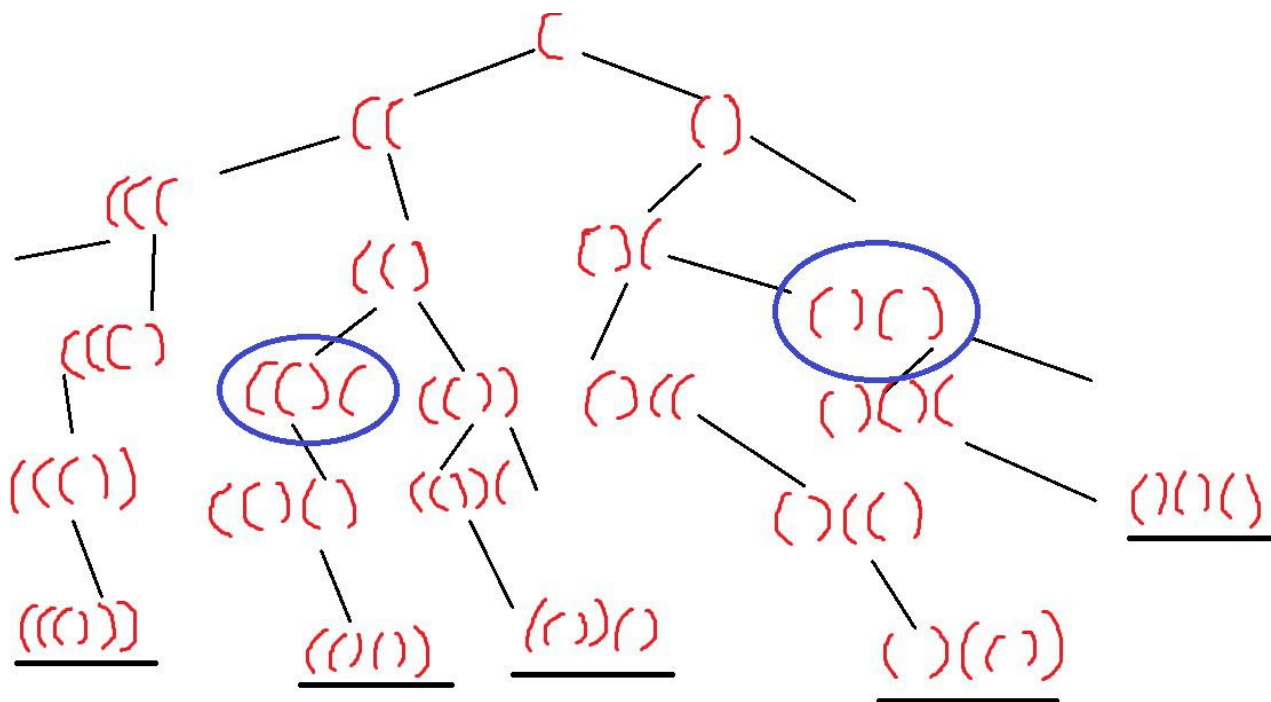
```

    {
        res.push_back(temp);
        return ;
    }
    int digit = digits[index] - '0' - 2;
    string letters = dict[digit];
    for(int i = 0; i < letters.size(); i++)
    {
        temp.push_back(letters[i]);
        dfs(res, temp, digits, index+1);
        temp.pop_back();
    }
}

vector<string> letterCombinations(string digits) {
    vector<string> res;
    string temp;
    if (digits.empty())
        return res;
    dfs(res, temp, digits, 0);
    return res;
}

```

22. 括号生成 todo



```
void dfs(vector<string> &res, string temp, int left, int right, int n)
{
    // 当 right > left 即右括号比左括号多的时候，后续无论插入左括号还是右括号都不是合法的
    if (right > left || left > n || right > n)
        return ;
    if (left == n && right == n)
        res.push back(temp);
}
```

```

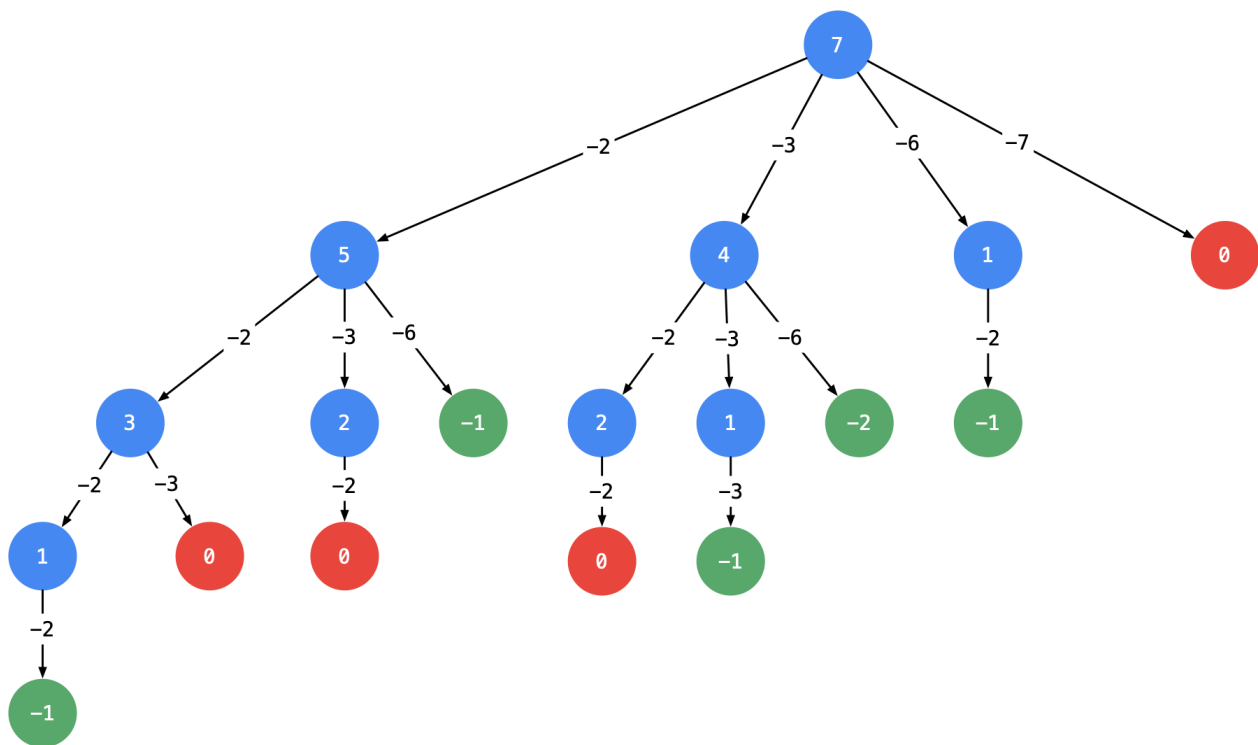
    dfs(res, temp + '(', left + 1, right, n);
    dfs(res, temp + ')', left, right + 1, n);
}

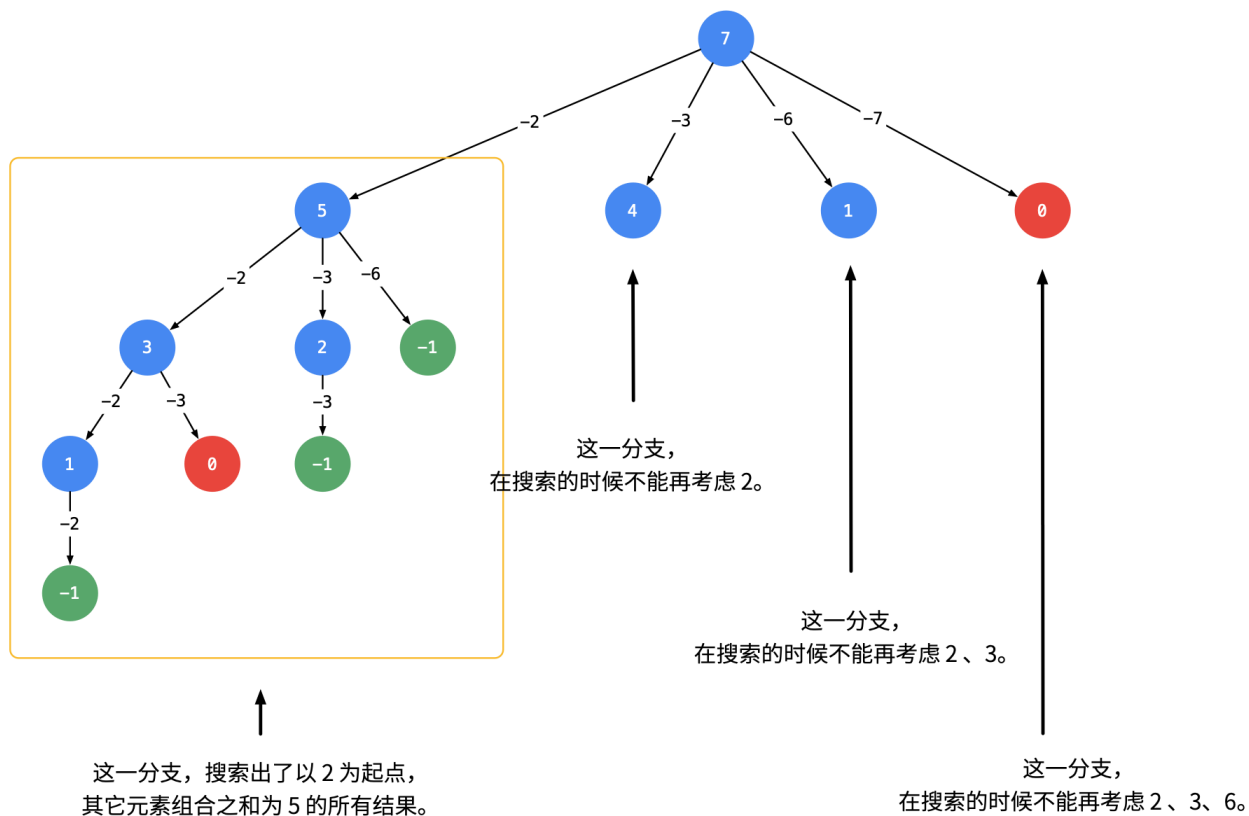
vector<string> generateParenthesis(int n) {
    vector<string> res;
    string temp = "";
    dfs(res, temp, 0, 0, n);
    return res;
}

```

301. 删除无效的括号

39. 组合总和





```
class Solution
{
public:
    void backtrack(vector<int>& candidates, int target, vector<int> &temp,
vector<vector<int>> &res, int start)
    {
        if (target < 0)
            return;
        if (target == 0) // 满足条件了 直接返回
        {
            res.push_back(temp);
            return ;
        }
        //遇到这一类相同元素不计算顺序的问题，我们在搜索的时候就需要 按某种顺序搜索。具体的做法是：每一次
        //搜索的时候设置 下一轮搜索的起点 begin，请看下图。
        // 对比上面两张图
        for(int i = start; i < candidates.size(); i++)
        {
            if(target - candidates[i] < 0)
                break;
            temp.push_back(candidates[i]);
            backtrack(candidates, target - candidates[i], temp, res, i); // i表示每个数字可
            //以用多次
            temp.pop_back();
        }
    }
public:
    vector<vector<int>> combinationSum(vector<int>& candidates, int target)
```

```

{
    sort(candidates.begin(), candidates.end()); // 减枝
    vector<vector<int>> res;
    vector<int> tmp; // 用来存放每一次满足条件的结果
    backtrack(candidates, target, tmp, res, 0);
    return res;
}
};

```

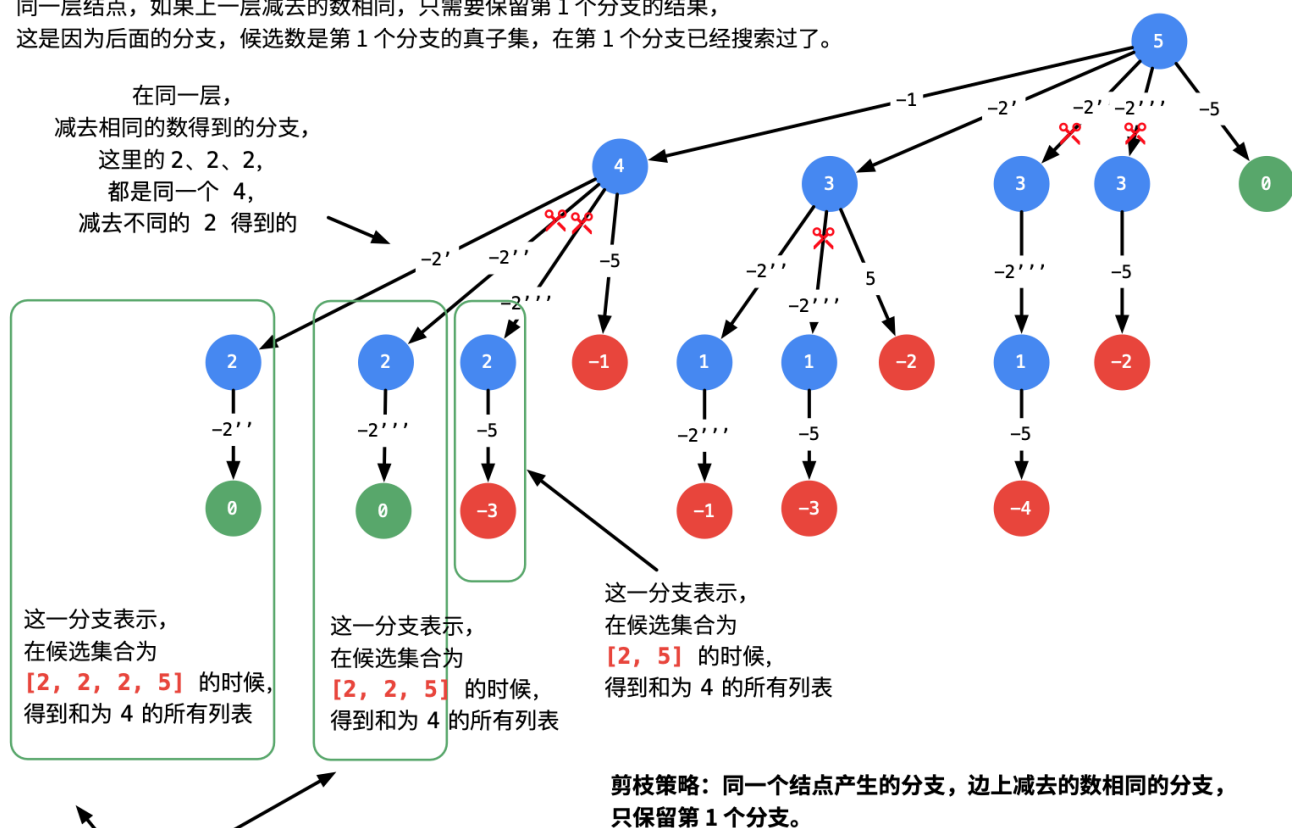
40. 组合总和 II

示例 2: 输入: candidates = [2, 5, 2, 1, 2], target = 5

题目中的关键信息: 每一个数只能用一次。因此 **深层结点可以考虑的分支数会越来越少**。

同一层结点, 如果上一层减去的数相同, 只需要保留第 1 个分支的结果,

这是因为后面的分支, 候选数是第 1 个分支的真子集, 在第 1 个分支已经搜索过了。



和一样 (都是 4), 但是第 2 个和第 3 个分支的候选集合是第 1 个分支的
真子集, 搜索到的结果, 一定是第 1 个分支搜索到的结果的子集。
因此对于输入数组进行排序, 是剪枝的前提。

```

class Solution {
public:
    void dfs(vector<vector<int>> &res, vector<int> path, vector<int>& candidates, int index,
            int target)
    {
        if (target < 0) {
            return;
        }
    }
}

```

```

        if (target == 0){
            res.push_back(path);
            return;
        }
        // for循环剪枝
        for(int i = index; i < candidates.size() && target - candidates[i] >= 0; i++)
        {
            if (i > index && candidates[i] == candidates[i - 1] )
                continue;
            path.push_back(candidates[i]);
            target -= candidates[i];
            dfs(res, path, candidates, i+1, target);
            target += candidates[i];
            path.pop_back();
        }

    }
    vector<vector<int>> combinationSum2(vector<int>& candidates, int target) {
        sort(candidates.begin(), candidates.end()); // 减枝
        vector<vector<int>> res;
        vector<int> path; // 用来存放每一次满足条件的结果
        dfs(res, path, candidates, 0, target);
        return res;
    }
};

```

77. 组合

```

void dfs(vector<vector<int>> &res, vector<int> path, int index, int n, int k)
{
    if (path.size() == k)
    {
        res.push_back(path);
        return;
    }
    // todo: 这个地方的n 还是有优化空间的
    for(int i = index; i <= n; i++)
    {
        path.push_back(i);
        dfs(res, path, i+1, n, k);
        path.pop_back();
    }
}

vector<vector<int>> combine(int n, int k) {
    vector<vector<int>> res;
    vector<int> path;
    dfs(res, path, 1, n, k);
    return res;
}

```

216. 组合总和 III

```

void dfs(vector<vector<int>> &res, vector<int> path, int index, int n, int k, int cur_sum)
{
    if (cur_sum > n)
        return;
    if (path.size() == k && cur_sum == n)
    {
        res.push_back(path);
        return;
    }
    // todo: 这个地方的n 还是有优化空间的
    for(int i = index; i <= 9; i++)
    {
        path.push_back(i);
        cur_sum += i;
        dfs(res, path, i+1, n, k, cur_sum);
        path.pop_back();
        cur_sum -= i;
    }
}

vector<vector<int>> combinationSum3(int k, int n) {
    vector<vector<int>> res;
    vector<int> path;
    dfs(res, path, 1, n, k, 0);
    return res;
}

```

31. 下一个排列 #todo



// 给定若干个数字，将其组合为一个整数。如何将这些数字重新排列，以得到下一个更大的整数。如 123 下一个更大的数为 132

// 从后往前找到第一个【相邻升序对】，即 $A[cur] < A[cur+1]$ 。此时 $A[cur+1, end)$ 为降序。

// 在区间 $[cur+1, end)$ 中，从后往前找到第一个大于 $A[cur]$ 的元素 $A[j]$

// 交换 $A[cur]$ 和 $A[j]$ ，此时 $A[cur+1, end)$ 一定还是降序，因为 $A[j]$ 是从右侧起第一个大于 $A[i]$ 的值

// 反转 $A[cur+1, end)$ ，变成升序

```

void nextPermutation(vector<int>& nums) {
    int cur = 0;
    // // 从后往前找到第一个相邻升序对

```

```

for(cur = nums.size()-2; cur >= 0; cur--)
{
    if (nums[cur] < nums[cur+1])
        break;
}

if (cur == -1) // 无相邻升序对, 必定为非递减序列
    reverse(nums.begin(), nums.end());
else
{
    // 从后往前[i+1,end)找第一个大于a[i+1]的值
    for(int j = nums.size() - 1; j >= cur+1; j--)
    {
        if (nums[cur] < nums[j])
        {
            swap(nums[cur], nums[j]); // 交换二者
            reverse(nums.begin()+cur+1, nums.end()); // 反转[i+1,end), 变成升序
            break;
        }
    }
}
}
}

```

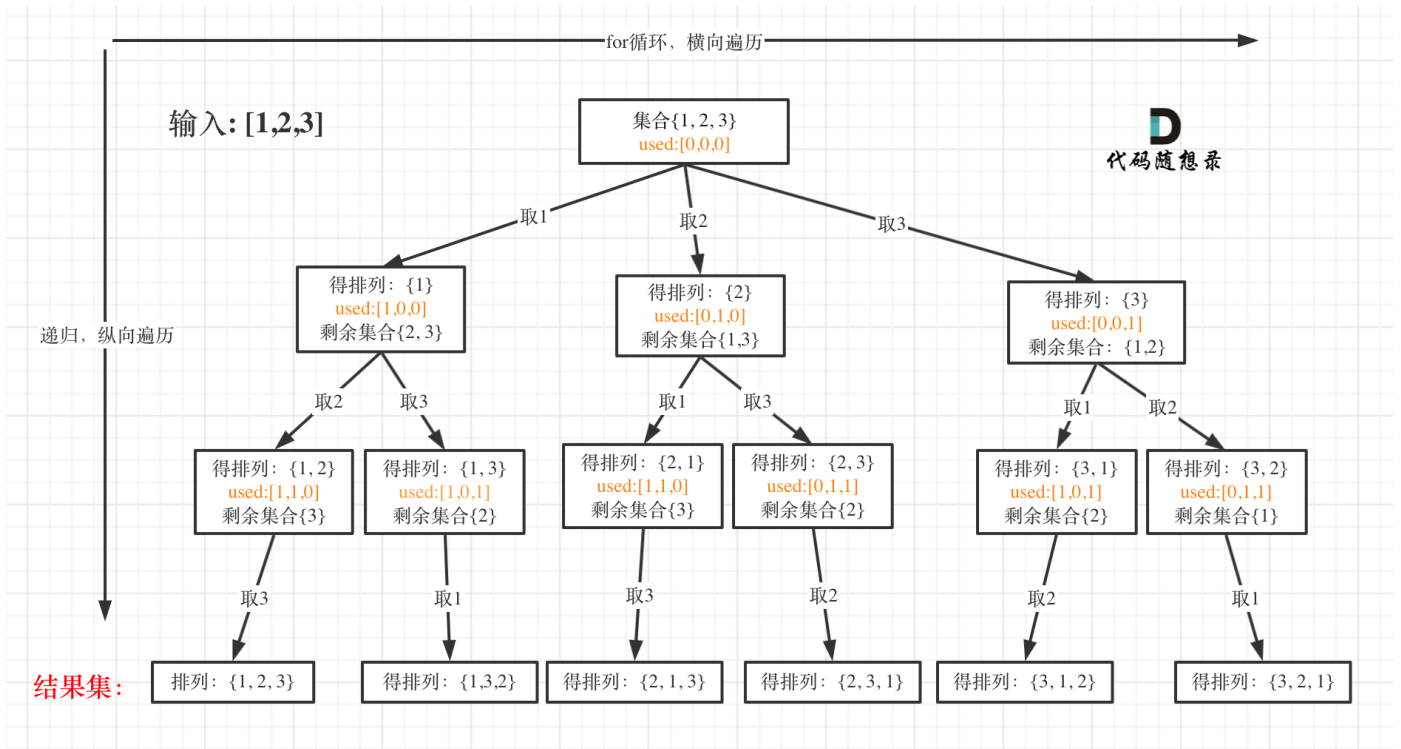
556. 下一个更大元素 III

```

int nextGreaterElement(int n) {
    string str = to_string(n);
    for(int i = str.size()-2; i >= 0; i--){
        //从后往前找到第一个不满足递减的数
        if(str[i] < str[i+1]){
            //将该数后面的数字排序以保证更小
            sort(str.begin() + i + 1, str.end());
            //找到第一个大于该数的数字下标
            auto it = upper_bound(str.begin()+i+1, str.end(), str[i]);
            //交换两数
            swap(str[i], str[distance(str.begin(), it)]);
            long long ans = stoll(str);
            return ans > INT_MAX ? -1 : ans;
        }
    }
    return -1;
}

```

46. 全排列



```

class Solution
{
public:
    vector<vector<int>> res;
    vector<int> temp;

    void dfs(vector<int> &nums, vector<bool> &uesd)
    {
        if (temp.size() == nums.size())
        {
            res.push_back(temp);
            return;
        }
        for (int i = 0; i < nums.size(); i++)
        {
            if (!uesd[i])
            {
                temp.push_back(nums[i]);
                uesd[i] = true;
                dfs(nums, uesd);
                uesd[i] = false;
                temp.pop_back();
            }
        }
    }

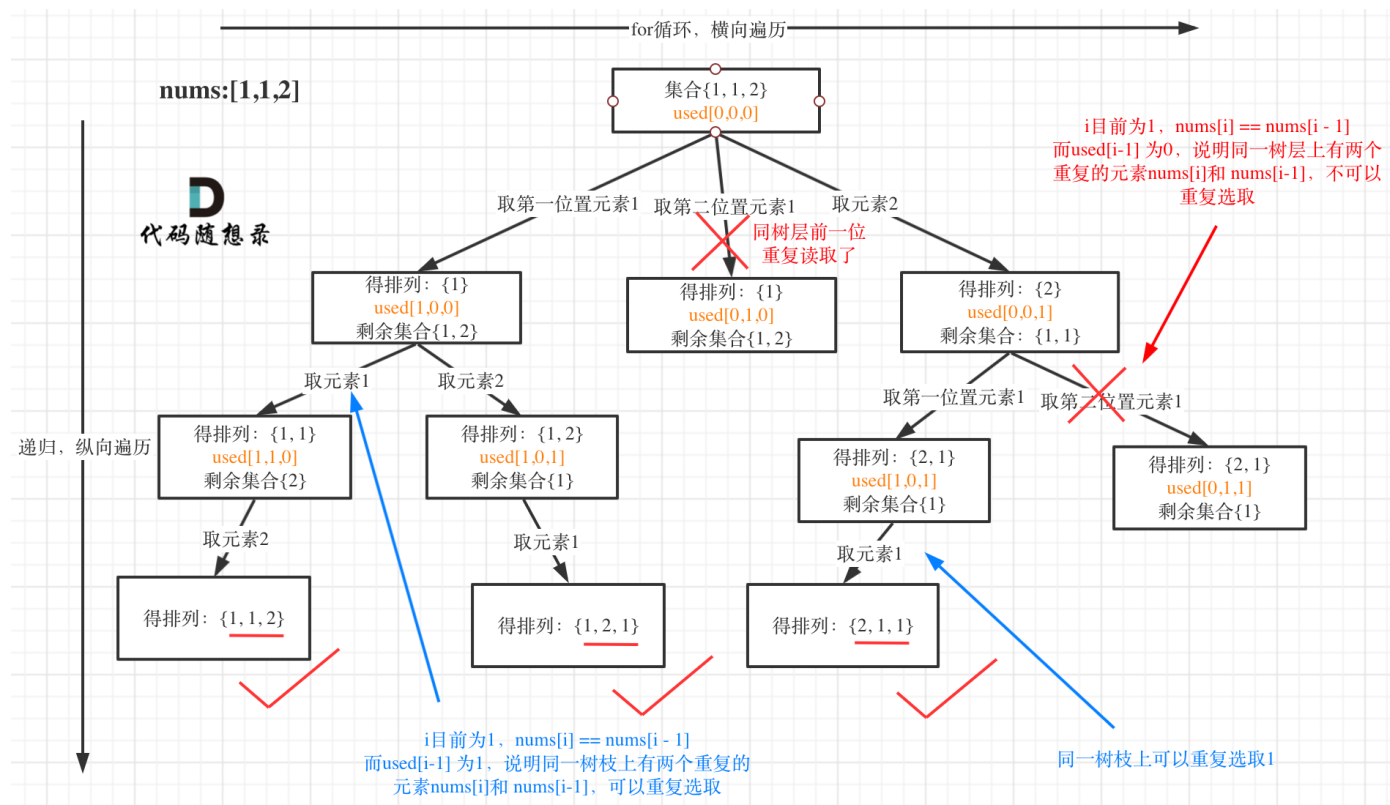
    vector<vector<int>> permute(vector<int> &nums)
    {
        vector<bool> uesd(nums.size());
        dfs(nums, uesd);
        return res;
    }
}
  
```

```

}
};

```

47. 全排列 II



```

class Solution
{
public:
    void dfs(vector<vector<int>> &res, vector<int> &temp, vector<int> &nums, vector<bool>
&uesd, int start)
    {

        if (temp.size() == nums.size())
        {
            res.push_back(temp);
            return;
        }

        for(int i = 0; i < nums.size(); i++)
        {
            if (!uesd[i])
            {
                if (i > 0 && nums[i] == nums[i-1] && uesd[i-1])
                    continue;
                uesd[i] = true;
                temp.push_back(nums[i]);
                dfs(res, temp, nums, uesd, i+1);
                uesd[i] = false;
                temp.pop_back();
            }
        }
    }
}

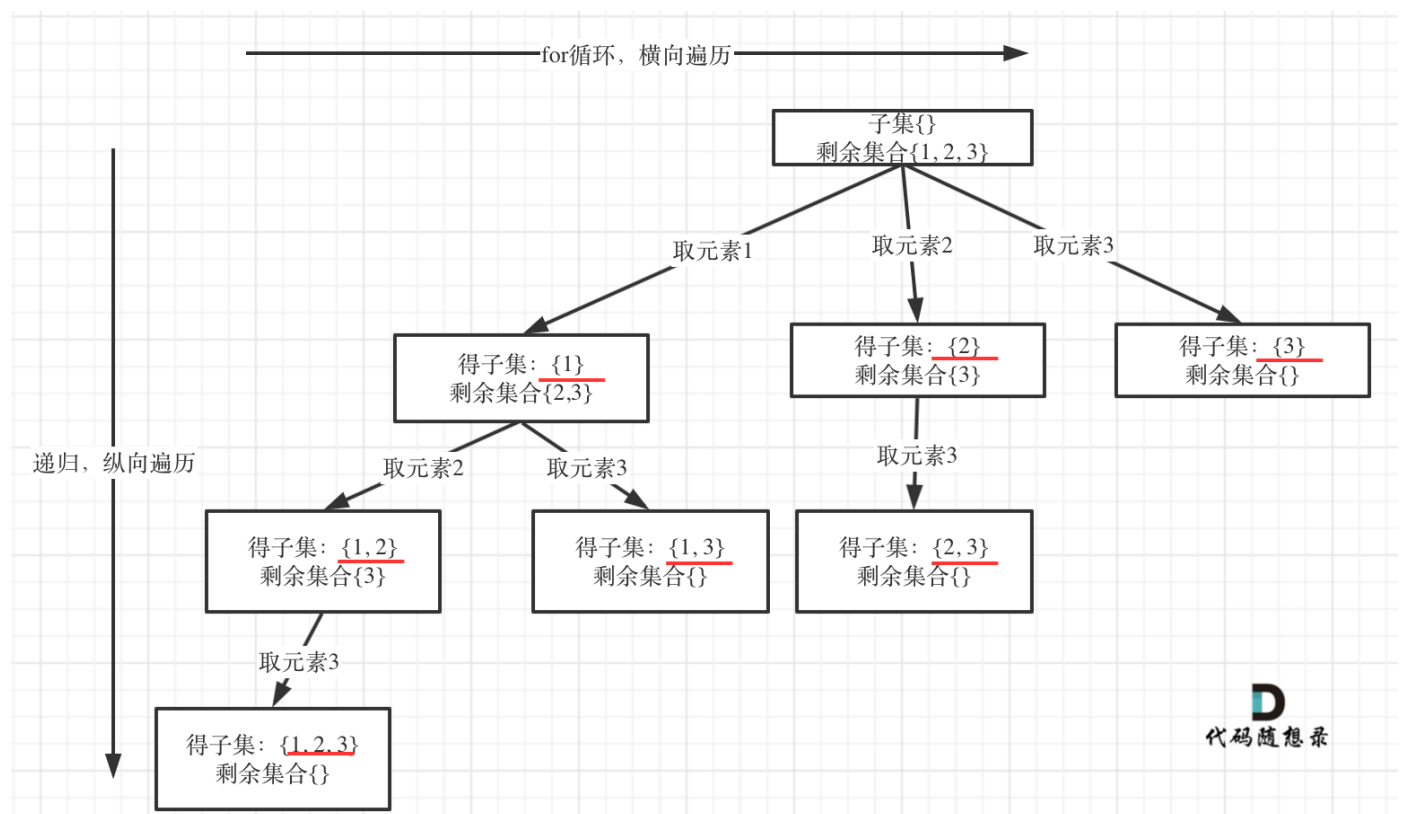
```

```

    }
}
public:
    vector<vector<int>> permuteUnique(vector<int>& nums)
    {
        vector<vector<int>> res;
        vector<int> temp;
        vector<bool> used(nums.size());
        sort(nums.begin(), nums.end());
        dfs(res, temp, nums, used, 0);
        return res;
    }
};

```

78. 子集 todo: 20211202 还不会



```

//每次看看有几个数能选, 然后选一个
//用 for 枚举出当前可选的数, 比如选第一个数: 有 1、2、3 可选。
//选了 1, 选第二个数, 有 2、3 可选; 如果选 2, 选第二个数, 只有 3 可选, 如下图
class Solution
{
public:
    void dfs(vector<vector<int>> &res, vector<int> &temp, vector<int> &nums, int index)
    {
        res.push_back(temp);
        for(int i = index; i < nums.size(); i++)
        {

```



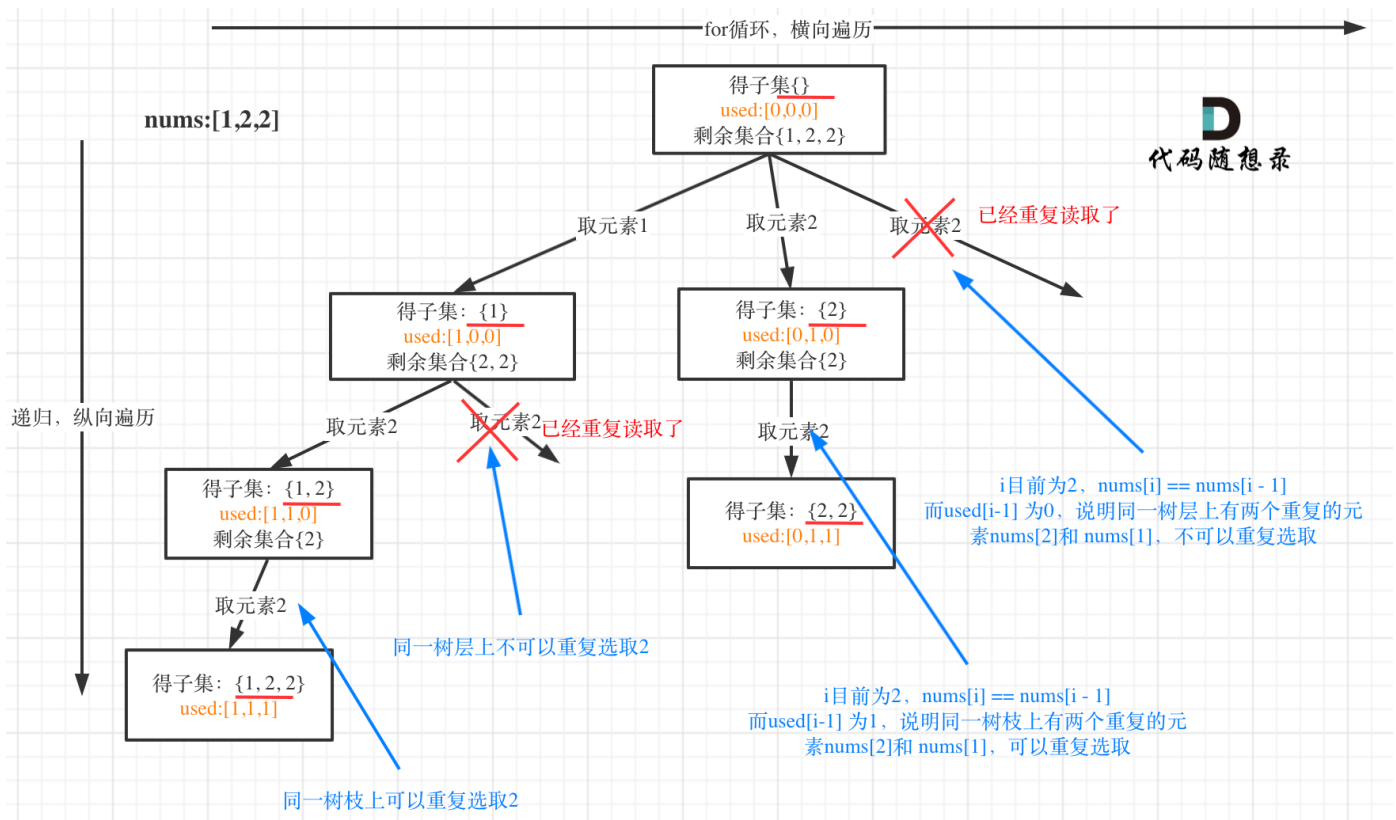
```

        temp.push_back(nums[i]);
        dfs(res, temp, nums, i+1);
        temp.pop_back();
    }
}

vector<vector<int>> subsets(vector<int>& nums)
{
    vector<vector<int>> res;
    vector<int> temp;
    dfs(res, temp, nums, 0);
    // res.push_back({});
    return res;
}
};

```

90. 子集 II



```

class Solution
{
    void dfs(vector<int>& nums, int index, vector<int>& temp, vector<vector<int>>& res)
    {
        res.push_back(temp);
        for(int i = index; i < nums.size(); i++)
        {
            // // 而我们要对同一树层使用过的元素进行跳过
            if (i > index && nums[i] == nums[i-1])
                continue;
            temp.push_back(nums[i]);
            dfs(nums, i+1, temp, res);
        }
    }
}

```

```

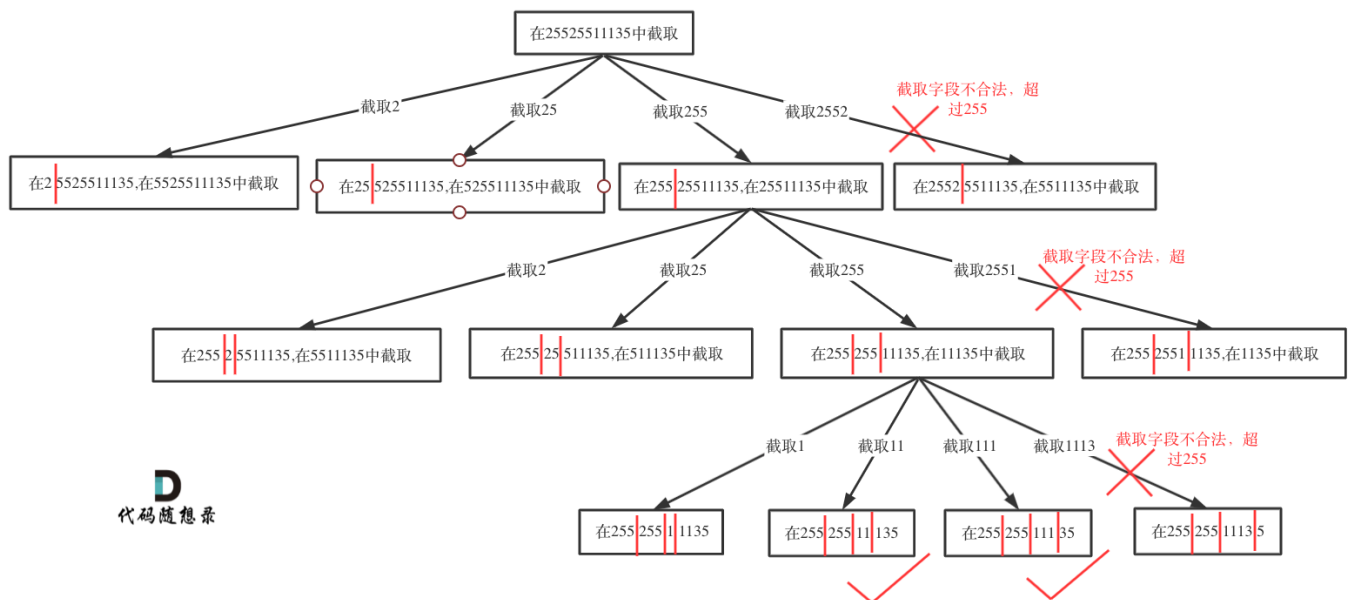
        temp.pop_back();

    }
}
public:
    vector<vector<int>> subsetsWithDup(vector<int>& nums)
    {
        vector<int> temp;
        vector<vector<int>> res;
        sort(nums.begin(), nums.end());
        dfs(nums, 0, temp, res);
        return res;
    }
};

```

91. 解码方法

93. 复原IP地址



D
代码随想录

```

class Solution {
public:
    // 判断字符串s在左闭又闭区间[start, end]所组成的数字是否合法
    bool isValid(const string& s, int start, int end) {
        if (start > end) {
            return false;
        }
        if (s[start] == '0' && start != end) { // 0开头的数字不合法
            return false;
        }
        int num = 0;
        for (int i = start; i <= end; i++) {
            if (s[i] > '9' || s[i] < '0') { // 遇到非数字字符不合法
                return false;
            }
        }
    }
};

```

```

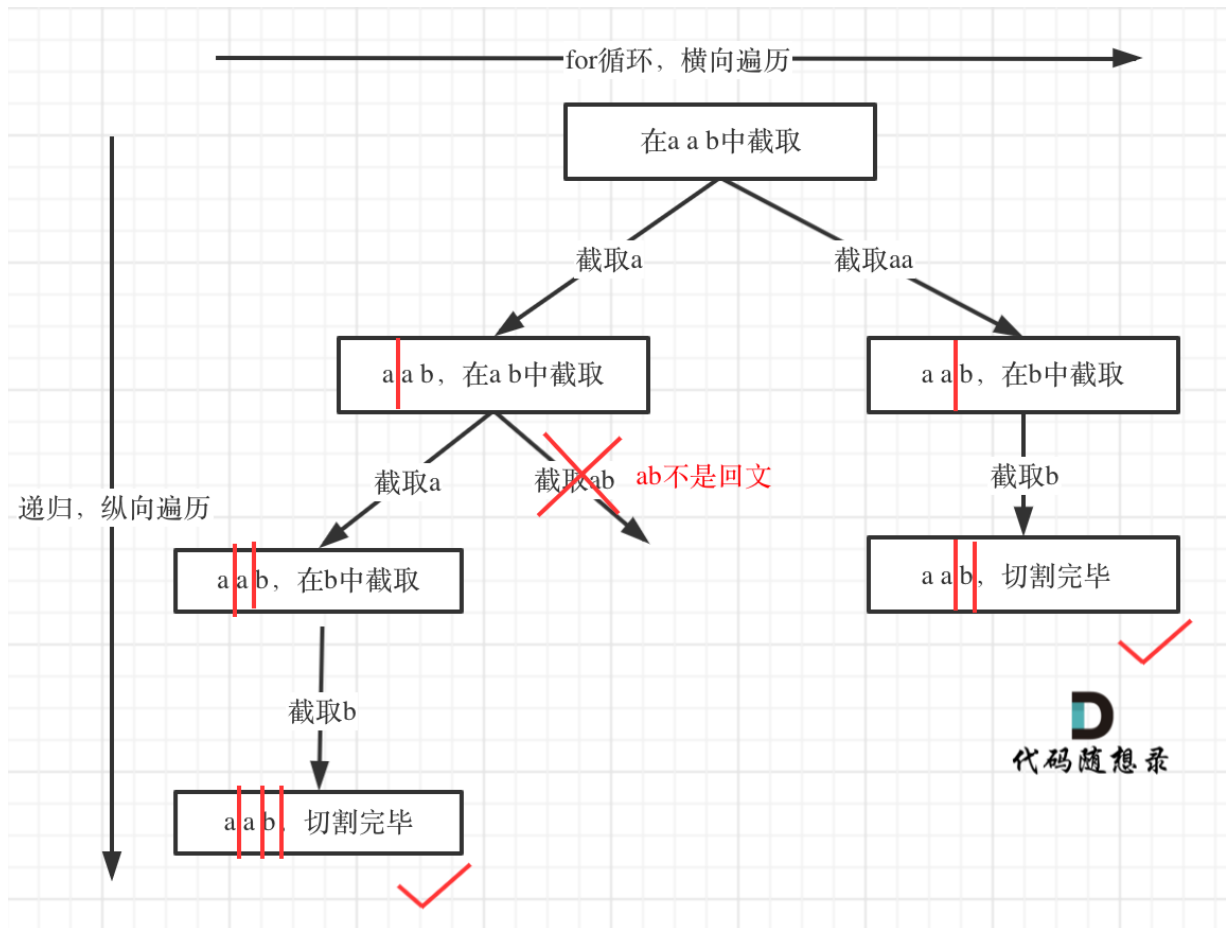
        num = num * 10 + (s[i] - '0');
        if (num > 255) { // 如果大于255了不合法
            return false;
        }
    }
    return true;
}

// index: 搜索的起始位置, pointNum:添加逗点的数量
void dfs(vector<string> &res, string& s, int index, int pointNum) {
    if (pointNum == 3) { // 逗点数量为3时, 分隔结束
        // 判断第四段子字符串是否合法, 如果合法就放进result中
        if (isValid(s, index, s.size() - 1)) {
            res.push_back(s);
        }
        return;
    }
    for (int i = index; i < s.size(); i++) {
        if (isValid(s, index, i)) { // 判断 [index,i] 这个区间的子串是否合法
            s.insert(s.begin() + i + 1, '.'); // 在i的后面插入一个逗点
            pointNum++;
            dfs(res, s, i + 2, pointNum); // 插入逗点之后下一个子串的起始位置为i+2
            pointNum--; // 回溯
            s.erase(s.begin() + i + 1); // 回溯删掉逗点
        }
    }
}

vector<string> restoreIpAddresses(string s) {
    vector<string> res;
    if (s.size() < 4 || s.size() > 12) return res; // 算是剪枝了
    dfs(res, s, 0, 0);
    return res;
}
};

```

131. 分割回文串



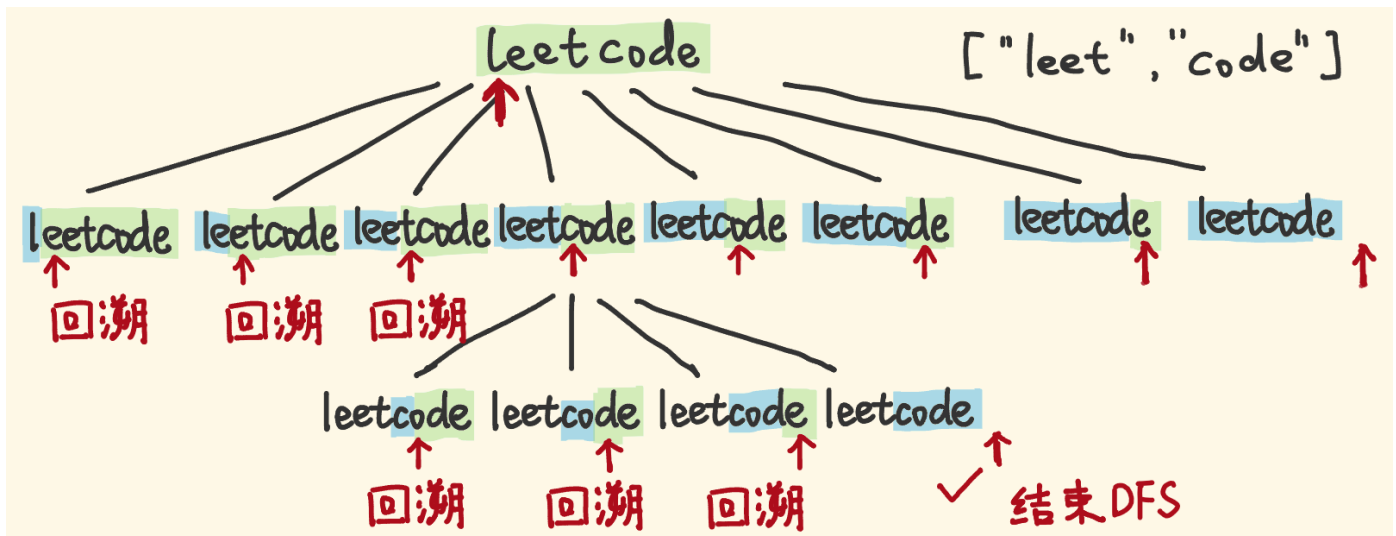
```
class Solution
{
    bool isPalindrome(const string& s, int start, int end)
    {
        while(start <= end) {
            if(s[start++] != s[end--])
                return false;
        }
        return true;
    }
}

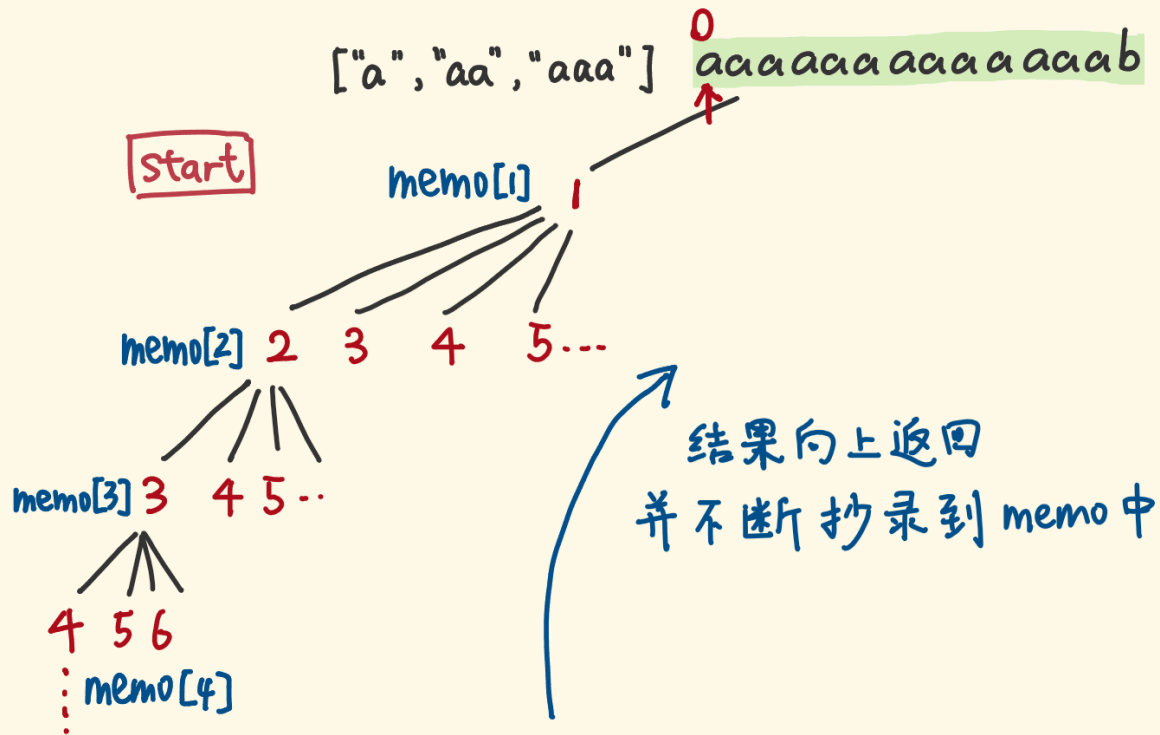
// dfs含义是
void dfs(string &s, int index, vector<string> &temp, vector<vector<string>> &res)
{
    if (index == s.size())
    {
        res.push_back(temp);
        return;
    }
    for(int i = index; i < s.size(); i++) // a, aa, aab for循环横向遍历
    {
        if (isPalindrome(s, index, i)) // 这个地方可以用动态规划去优化
        {
            temp.push_back(s.substr(index, i - index + 1)); // 获取[index,i]在s中的子串
            dfs(s, i + 1, temp, res);
            temp.pop_back();
        }
    }
}
```

```

    }
}

public:
    vector<vector<string>> partition(string s)
    {
        vector<vector<string>> > res;
        if(s.empty()) return res;
        vector<string> temp;
        dfs(s, 0, temp, res);
        return res;
    }
};
```





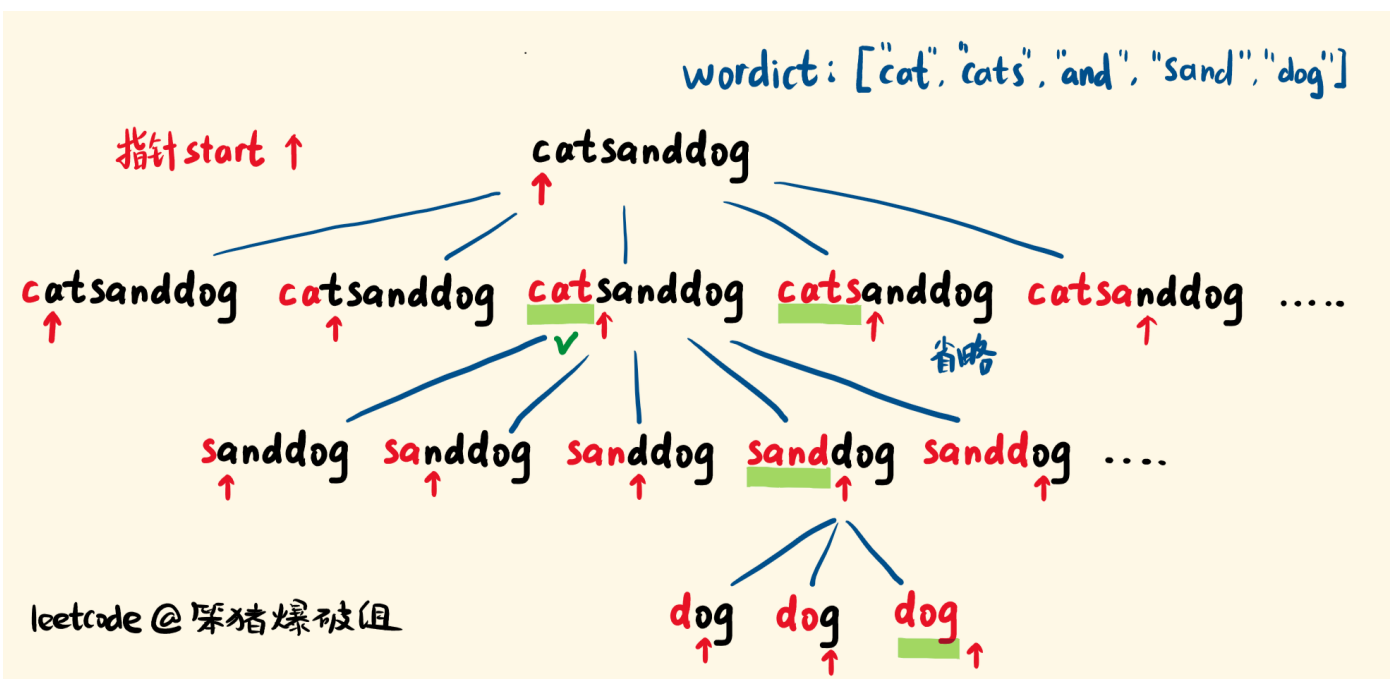
```
class Solution
{
public:
    /**
     * 解法一：
     * memo[i] 定义为范围为 [i, n] 的子字符串是否可以拆分，初始化为 -1，表示没有计算过，如果可以拆分，
     * 则赋值为1，反之为0
     */
    bool dfs(string &s, vector<string>& wordDict, int index, vector<int> &memo)
    {
        if (index == s.size()) // 指针越界，s都划分成单词了才走到这步的，没有剩余子串了，返回真，结束递归
            return true;
        if (memo[index] != -1)
            return memo[index];

        for(int i = index; i < s.size(); i++)
        {
            // 先判断字典中是否能找到子串 [index...i] 然后递归调用[i+1...s.size()]
            if (find(wordDict.begin(), wordDict.end(), s.substr(index, i - index + 1)) != wordDict.end() && dfs(s, wordDict, i + 1, memo))
            {
                memo[index] = true;
                return true;
            }
        }
        memo[index] = false;
        return false;
    }
}
```

```
bool wordBreak(string s, vector<string>& wordDict)
{
    vector<int> memo(s.size(), -1);
    return dfs(s, wordDict, 0, memo);
}

/**
 * 解法二：
 * 其中 dp[i] 表示范围 [0, i) 内的子串是否可以拆分，注意这里 dp 数组的长度比s串的长度大1，是因为我们要 handle 空串的情况，我们初始化 dp[0] 为 true，然后开始遍历
 */
bool wordBreak(string s, vector<string> &wordDict)
{
    unordered_set<string> wordSet(wordDict.begin(), wordDict.end());
    vector<bool> dp(s.size() + 1);
    dp[0] = true;
    for (int i = 0; i < dp.size(); ++i)
    {
        for (int j = 0; j < i; ++j)
        {
            if (dp[j] && wordSet.count(s.substr(j, i - j)))
            {
                dp[i] = true;
                break;
            }
        }
    }
    return dp.back();
}

};
```



// 解法一 暴力递归超时

```
class Solution {
public:
    vector<string> wordBreak(string s, vector<string>& wordDict) {
        vector<string> result; // 存储结果
        vector<string> path;   // 存储当前拼接的路径
        dfs(s, wordDict, result, path, 0); // 调用深度优先搜索函数
        return result; // 返回所有可能的句子
    }

    // 深度优先搜索函数
    void dfs(string& s, vector<string>& wordDict, vector<string>& result, vector<string>& path, int index) {
        if (index == s.size()) { // 如果已经处理完整个字符串, 将当前路径加入结果中
            result.push_back(joinWords(path));
            return;
        }

        for (int i = index; i < s.size(); i++) { // 从当前位置开始尝试拆分
            string word = s.substr(index, i - index + 1); // 从index到i位置的子串
            if (find(wordDict.begin(), wordDict.end(), word) != wordDict.end()) {
                path.push_back(word); // 将合法单词添加到路径
                dfs(s, wordDict, result, path, i + 1); // 递归处理余下部分
                path.pop_back(); // 回溯, 删除最后一个单词
            }
        }
    }

    // 辅助函数, 将单词列表连接成句子
    string joinWords(vector<string>& words) {
        string result = words[0];
        for (int i = 1; i < words.size(); i++) {
            result += " " + words[i];
        }
        return result;
    }
};
```

// 解法二: dp+回溯

```
class Solution {
public:
    vector<string> wordBreak(string s, vector<string>& wordDict) {
        int n = s.size();
        vector<bool> dp(n + 1, false); // dp[i] 表示 s[0:i-1] 是否可以被拆分成词典中的单词
        dp[0] = true;

        for (int i = 1; i <= n; i++) {
            for (string word : wordDict) {
                int len = word.size();
                if (i >= len && dp[i - len] && s.substr(i - len, len) == word) {
                    dp[i] = true;
                }
            }
        }
    }
};
```



```

        }
    }
}

vector<string> result;
if (!dp[n]) {
    return result; // 无法拆分成词典中的单词, 返回空结果
}

vector<string> path;
backtrack(s, wordDict, dp, result, path, 0);

return result;
}

void backtrack(string& s, vector<string>& wordDict, vector<bool>& dp, vector<string>&
result, vector<string>& path, int index) {
    if (index == s.size()) {
        result.push_back(joinWords(path));
        return;
    }

    for (int i = index; i < s.size(); i++) {
        string word = s.substr(index, i - index + 1);
        if (dp[i + 1] && find(wordDict.begin(), wordDict.end(), word) !=
wordDict.end()) {
            path.push_back(word);
            backtrack(s, wordDict, dp, result, path, i + 1);
            path.pop_back();
        }
    }
}

string joinWords(vector<string>& words) {
    string result = words[0];
    for (int i = 1; i < words.size(); i++) {
        result += " " + words[i];
    }
    return result;
}
};

```

[332. 重新安排行程](#)

[473. 火柴拼正方形](#)

```

class Solution {
public:
    bool makesquare(vector<int>& matchsticks) {
        if (matchsticks.size() < 4) {
            return false; // 如果火柴棒数量小于4, 无法构成正方形
        }
    }
};

```

```

    }

    int totalLength = 0;
    for (int length : matchsticks) {
        totalLength += length;
    }

    if (totalLength % 4 != 0) {
        return false; // 总长度不是4的倍数，无法构成正方形
    }

    vector<int> sides(4, 0); // 用于表示正方形的四边

    return backtrack(matchsticks, sides, 0, totalLength / 4);
}

bool backtrack(vector<int>& matchsticks, vector<int>& sides, int index, int target) {
    if (index == matchsticks.size()) {
        return sides[0] == sides[1] && sides[1] == sides[2] && sides[2] == sides[3];
    }
    // 所有火柴棒都使用完且四边长度相等

    int length = matchsticks[index];
    for (int i = 0; i < 4; i++) {
        if (sides[i] + length <= target) {
            sides[i] += length;
            if (backtrack(matchsticks, sides, index + 1, target)) {
                return true;
            }
            sides[i] -= length;
        }

        if (sides[i] == 0) {
            break; // 如果这一边为0，将会导致无意义的重复，所以跳过
        }
    }

    return false;
}
};

```

491. 递增子序列

```

void dfs(vector<int>& nums, vector<vector<int>>& result, vector<int>& path, int
startIndex) {
    if (path.size() >= 2) {
        result.push_back(path); // 如果当前递增子序列长度大于等于2，加入结果中
        // 注意这里不要加return，要取树上的节点
        // return;
    }
}

```

```

unordered_set<int> used; // 用于去重

for (int i = startIndex; i < nums.size(); i++) {
    if (!path.empty() && nums[i] < path.back()) {
        continue; // 如果当前元素小于当前递增子序列的最后一个元素，跳过
    }

    if (used.count(nums[i]) > 0) {
        continue; // 如果当前元素已经使用过，跳过
    }

    used.insert(nums[i]);
    path.push_back(nums[i]); // 将当前元素加入递增子序列
    dfs(nums, result, path, i + 1); // 递归处理下一位置
    path.pop_back(); // 回溯，删除最后一个元素
}
}

vector<vector<int>> findSubsequences(vector<int>& nums) {
    vector<vector<int>> result; // 用于存储结果
    vector<int> path; // 用于存储当前递增子序列
    dfs(nums, result, path, 0);
    return result;
}

```

526. 优美的排列

```

int countArrangement(int N) {
    int res = 0;
    vector<int> visited(N + 1, 0);
    helper(N, visited, 1, res);
    return res;
}

void helper(int N, vector<int>& visited, int pos, int& res) {
    if (pos > N) {
        ++res;
        return;
    }
    for (int i = 1; i <= N; ++i) {
        if (visited[i] == 0 && (i % pos == 0 || pos % i == 0)) {
            visited[i] = 1;
            helper(N, visited, pos + 1, res);
            visited[i] = 0;
        }
    }
}

```

679. 24 点游戏

```

bool dfs(vector<double>& nums){

```

```

if(nums.size()==1)return abs(nums[0]-24)<=1e-6; //如果只有一个数, 判断跟24的差值

for(int i=0;i<nums.size();i++){
    for(int j=0;j<nums.size();j++){
        if(i==j)continue; //不能重复取数
        double a=nums[i]; //第一步, 取出两个数
        double b=nums[j];
        vector<double>shengyu;
        for(int k=0;k<nums.size();k++){
            if(k==i||k==j)continue;
            shengyu.push_back(nums[k]); //第二步, 存储下剩余的元素, 等待第一步取出的两个数在四则
            //运算之后的结果存入当前容器, 再次进行dfs运算
        }
        double sum=a+b; //四则运算
        double sub=a-b;
        double mul=a*b;
        double div=a/b;
        double yunsuan[4]={sum,sub,mul,div};
        for(int w=0;w<4;w++){
            shengyu.push_back(yunsuan[w]);
            if(dfs(shengyu))return true;
            shengyu.pop_back(); //回溯
        }
    }
}
return false;
}

bool judgePoint24(vector<int>& cards) {
    vector<double> nums; //转化为double, 为了便于除法运算 (会产生小数)
    for(int card:cards){nums.push_back(double(card));}
    return dfs(nums);
}

```

698. 划分为k个相等的子集

```

bool canPartitionKSubsets(vector<int>& nums, int k) {
    int sum = accumulate(nums.begin(), nums.end(), 0);
    if (sum % k != 0) return false;
    vector<bool> visited(nums.size());
    return helper(nums, k, sum / k, 0, 0, visited);
}

bool helper(vector<int>& nums, int k, int target, int start, int curSum, vector<bool>& visited) {
    if (k == 1) return true;
    if (curSum == target) return helper(nums, k - 1, target, 0, 0, visited);
    for (int i = start; i < nums.size(); ++i) {
        if (visited[i]) continue;
        visited[i] = true;
        if (helper(nums, k, target, i + 1, curSum + nums[i], visited)) return true;
        visited[i] = false;
    }
    return false;
}

```

```

    }
    return false;
}

```

记忆化搜索

算是动态规划的一种，递归每次返回时同时记录下已访问过的节点特征，避免重复访问同一个节点，可以有效的把指数级别的DFS时间复杂度降为多项式级别；注意这一类的DFS必须在最后有返回值，不可以用排列组合类型的DFS方法写；for循环的dp题目都可以用记忆化搜索的方式写，但是不是所有的记忆化搜索题目都可以用for循环的dp方式写。

[72. 编辑距离](#)

```

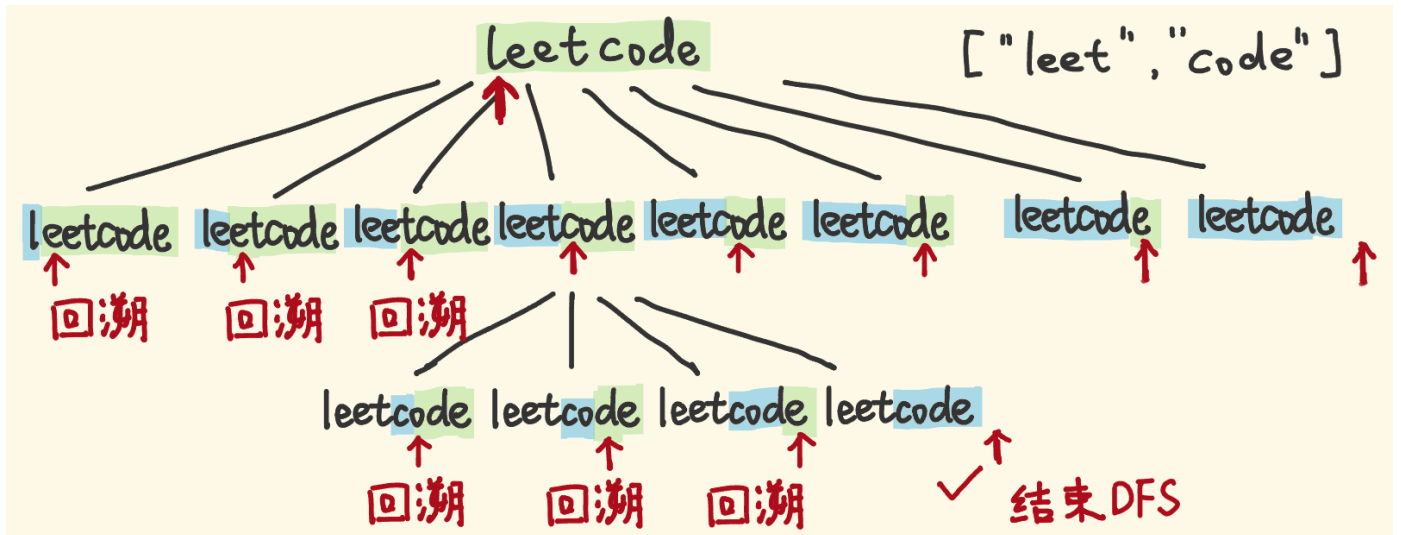
// https://www.cnblogs.com/grandyang/p/4344107.html
int dfs(string& word1, int i, string& word2, int j, vector<vector<int>>& dp) {
    if (i == word1.size()) return (int)word2.size() - j;
    if (j == word2.size()) return (int)word1.size() - i;
    if (dp[i][j] > 0) return dp[i][j];
    int res = 0;
    if (word1[i] == word2[j]) {
        return dfs(word1, i + 1, word2, j + 1, dp);
    } else {
        int insertCnt = dfs(word1, i, word2, j + 1, dp);
        int deleteCnt = dfs(word1, i + 1, word2, j, dp);
        int replaceCnt = dfs(word1, i + 1, word2, j + 1, dp);
        res = min(insertCnt, min(deleteCnt, replaceCnt)) + 1;
    }
    return dp[i][j] = res;
}

int minDistance(string word1, string word2)
{
    int m = word1.size(), n = word2.size();
    vector<vector<int>> dp(m, vector<int>(n));
    return dfs(word1, 0, word2, 0, dp);
}

```

[97. 交错字符串](#)

[139. 单词拆分](#) 回溯+记忆化搜索



// 解法一：暴力递归超时了

```
bool dfs(string &s, unordered_set<string> &wordSet, int i, vector<int> &memo)
{
    if (i >= s.size())
        return true;
    for(int j = i+1; j <= s.size(); j++)
    {
        if (wordSet.count(s.substr(i, j - i)) && dfs(s, wordSet, j, memo))
            return true;
    }
    return false;
}

bool wordBreak(string s, vector<string> &wordDict)
{
    unordered_set<string> wordSet(wordDict.begin(), wordDict.end());
    vector<int> memo(s.size(), -1);
    return dfs(s, wordSet, 0, memo);
}
```

// 解法一：增加记忆化搜索

```
bool wordBreak(string s, vector<string> &wordDict)
{
    unordered_set<string> wordSet(wordDict.begin(), wordDict.end());
    vector<int> memo(s.size(), -1);
    return dfs(s, wordSet, 0, memo);
}

// dfs 表示字符串s[start,n] 是否可分
bool dfs(string s, unordered_set<string> &wordDict, int index, vector<int> &dp)
{
    if (index >= s.size())
        return true;
    if (dp[index] != -1)
        return dp[index];

    for(int i = index + 1; i <= s.size(); i++)
    {

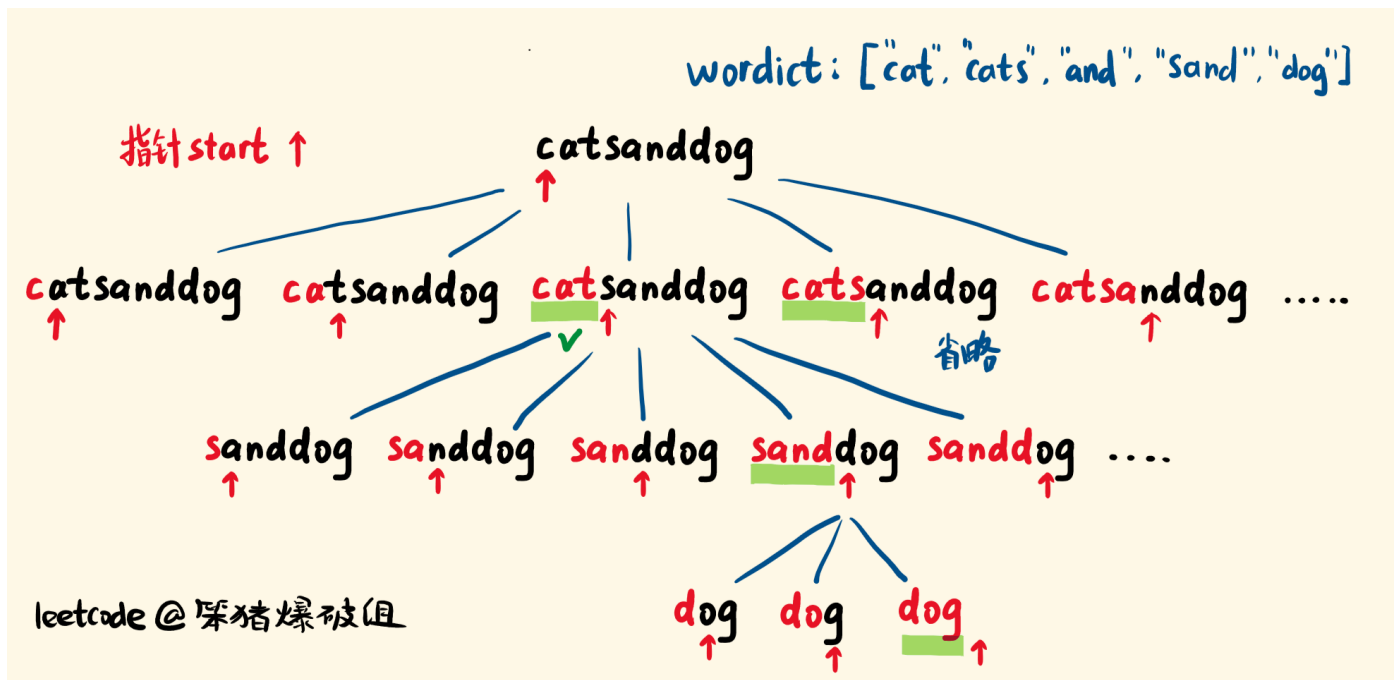
```

```

        if (wordDict.count(s.substr(index, i - index)) && dfs(s, wordDict, i, dp))
        {
            dp[index] = true;
            return true;
        }
    }
    dp[index] = false;
    return false;
}

```

140. 单词拆分 II 注意和139的区别 todo 好像还不怎么会



```

// 解法一 暴力递归超时了 想改成不带返回值得形式
vector<string> dfs(string s, unordered_set<string> &wordSet, int start)
{
    if (start == s.size())
    {
        return {" "};
    }
    vector<string> res;
    for (int i = start; i < s.size(); ++i)
    {
        if (wordSet.count(s.substr(start, i - start + 1)) > 0)
        {
            vector<string> tmp = dfs(s, wordSet, i + 1);
            for (auto str : tmp)
            {
                res.push_back(s.substr(start, i - start + 1) + (str.empty() ? " " : " ") +
str);
            }
        }
    }
    return res;
}

```

```

}

vector<string> wordBreak(string s, vector<string> &wordDict)
{
    unordered_set<string> wordSet(wordDict.begin(), wordDict.end());
    return dfs(s, wordSet, 0);
}

// 解法二：增加记忆化搜索
vector<string> dfs(string s, unordered_set<string> &wordSet, int start, unordered_map<int,
vector<string>> &memo)
{
    if (memo.count(start) > 0)
    {
        return memo[start];
    }
    if (start == s.size())
    {
        return {" "};
    }
    vector<string> res;
    for (int i = start; i < s.size(); ++i)
    {
        if (wordSet.count(s.substr(start, i - start + 1)) > 0)
        {
            vector<string> tmp = dfs(s, wordSet, i+1, memo);
            for (auto str : tmp)
            {
                res.push_back(s.substr(start, i - start + 1) + (str.empty() ? " " : " ") +
str);
            }
        }
    }
    memo[start] = res;
    return res;
}

vector<string> wordBreak(string s, vector<string> &wordDict)
{
    unordered_set<string> wordSet(wordDict.begin(), wordDict.end());
    unordered_map<int, vector<string>> memo;
    return dfs(s, wordSet, 0, memo);
}

```

329. 矩阵中的最长递增路径 记忆化搜索

```

int dirs[4][2] = {0, 1, 1, 0, 0, -1, -1, 0};

// 表示从 (i, j) 出发的最长路径长度
int dfs(vector<vector<int>>& matrix, vector<vector<int>>& dp, int i, int j)
{

```



```

    if (dp[i][j] > 0)
        return dp[i][j];
    int res = 1;
    for(int k = 0; k < 4; k++)
    {
        int x = i + dirs[k][0];
        int y = j + dirs[k][1];
        if (x >= 0 && x < matrix.size() && y >= 0 && y < matrix[0].size() && matrix[x][y]
> matrix[i][j])
        {
            int dist = 1 + dfs(matrix, dp, x, y);
            res = max(dist, res);
        }
    }
    dp[i][j] = res;
    return res;
}

int longestIncreasingPath(vector<vector<int>>& matrix)
{
    if (matrix.empty())
        return 0;
    // dp[i][j]表示数组中以(i,j)为起点的最长递增路径的长度，初始将dp数组都赋为0，当我们用递归调用时，遇到某个位置(x, y)，如果dp[x][y]不为0的话，我们直接返回dp[x][y]即可，不需要重复计算。
    vector<vector<int>> dp( matrix.size(), vector<int>(matrix[0].size(), 0) );
    int longest = INT_MIN;
    for(int i = 0; i < matrix.size(); i++)
    {
        for(int j = 0; j < matrix[0].size(); j++)
        {
            int len = dfs(matrix, dp, i, j);
            longest = max(longest, len);
        }
    }
    return longest;
}

```

[377. 组合总和 IV](#) #todo 优化成动态规划 20211207

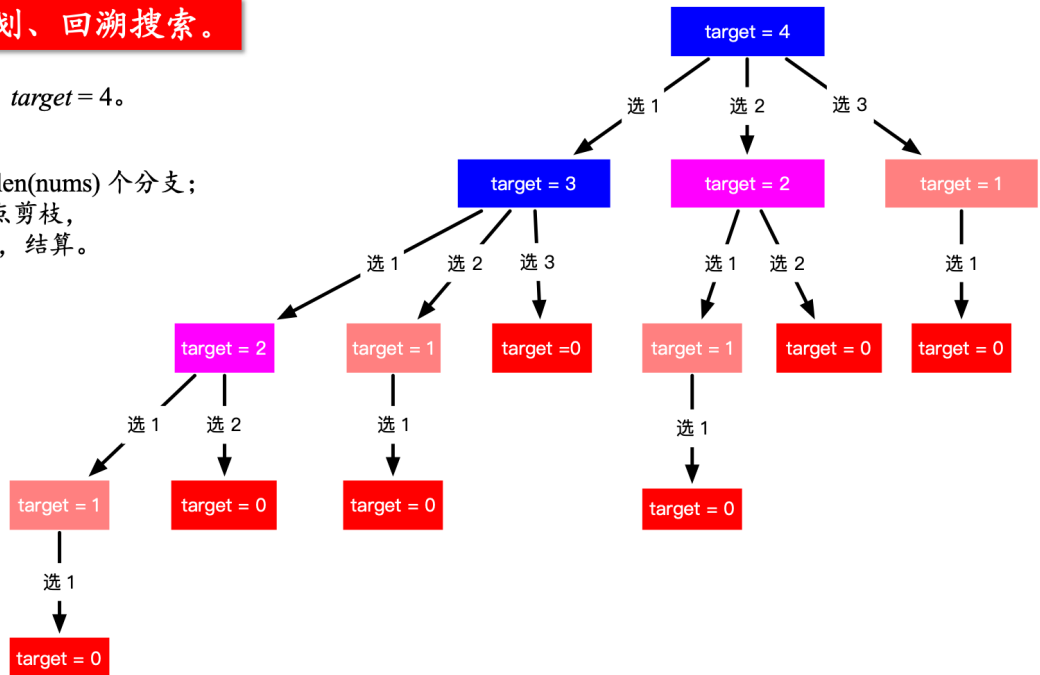
LeetCode 第 377 题：“组合总和 IV” 题解配图 (1)

思路：动态规划、回溯搜索。

示例：nums = [1, 2, 3], target = 4。

该树的特点：

- 1、每个结点最多只有 len(nums) 个分支；
- 2、到 target ≤ 0 的结点剪枝，
当 target = 0 的时候，结算。



// 记忆化搜索

```
int dfs(vector<int> &nums, int target, unordered_map<int, int> &memo)
{
    if(memo.count(target)) return memo[target];
    if (target < 0) return 0;
    if (target == 0) return 1;

    int res = 0;
    for(int i = 0; i < nums.size(); i++)
    {
        res += dfs(nums, target - nums[i], memo);
    }
    memo[target] = res;
    return res;
}

int combinationSum4(vector<int>& nums, int target)
{
    unordered_map<int, int> memo;
    return dfs(nums, target, memo);
}
```

// 优化成动态规划

```
int combinationSum4(vector<int>& nums, int target)
{
    vector<int> dp(target + 1);
    dp[0] = 1;
    for (int i = 1; i <= target; ++i) {
        for (auto a : nums) {
            if (i >= a) dp[i] += dp[i - a];
        }
    }
}
```

```

    }
    return dp.back();
}

```

403. 青蛙过河

472. 连接词

576. 出界的路径数 记忆化搜索

```

int dirs[4][2] = {0, 1, 1, 0, 0, -1, -1, 0};
int dfs(vector<vector<vector<uint>>> &dp, int x, int y, int step, int m, int n)
{
    if (x < 0 || y < 0 || x >= m || y >= n)    // 一旦超出边界直接返回1
        return 1;
    if (x-step >= 0 && x + step < m && y - step >= 0 && y + step < n)    // 不管从哪个方向走
        step步之后 都到不了边界外
        return 0;
    if(step <= 0)    // 如果没得走了
        return 0;

    if (dp[step][x][y] > 0)
        return dp[step][x][y];
    int count = 0;
    for(int k = 0; k < 4; k++)
    {
        int i = x + dirs[k][0];
        int j = y + dirs[k][1];
        count += dfs(dp, i, j, step - 1, m, n);
        count %= 1000000007;
    }

    dp[step][x][y] = count;
    return count;
}

int findPaths(int m, int n, int N, int i, int j)
{
    // dp[k][i][j]表示总共走k步, 从(i,j)位置走出边界的总路径数
    vector<vector<vector<uint>>> dp(N+1, vector<vector<uint>>(m, vector<uint>(n, 0)));
    int count = dfs(dp, i, j, N, m, n) % 1000000007;
    return count;
}

```

688. “马”在棋盘上的概率 记忆化搜索

```

vector<vector<int>> dirs{{-1,-2},{-2,-1},{-2,1},{-1,2},{1,2},{2,1},{2,-1},{1,-2}};
double dfs(vector<vector<vector<double>>> &dp, int i, int j, int k, int N)
{
    if (i < 0 || i >= N || j < 0 || j >= N)
        return 0.0;
    if (k == 0)

```

```

        return 1.0;

    if (dp[k][i][j] != 0.0)
        return dp[k][i][j];
    double count = 0.0;

    for (auto dir : dirs)
    {
        int x = i + dir[0];
        int y = j + dir[1];
        // if (x < 0 || x >= N || y < 0 || y >= N)
        //     continue;
        count += dfs(dp, x, y, k-1, N);
    }
    dp[k][i][j] = count;
    return count;
}

double knightProbability(int N, int K, int r, int c)
{
    // dp[k][i][j]表示总共走k步，从(i,j)位置没有走出边界的总路径数
    if (K == 0)
        return 1;
    vector<vector<vector<double>>> dp(K+1, vector<vector<double>>(N, vector<double>(N,
0.0)));
    double total_step = dfs(dp, r, c, K, N);
    return dp[K][r][c] / pow(8, K);
}

```

[1235. 规划兼职工作](#)

[1335. 工作计划的最低难度](#)

- Leetcode 1216 Valid Palindrome III （收费题）

宽度优先搜索（BFS）

基础知识：

常见的BFS用来解决什么问题？（1）简单图（有向无向皆可）的最短路径长度，注意是长度而不是具体的路径（2）拓扑排序（3）遍历一个图（或者树）BFS基本模板（需要记录层数或者不需要记录层数）

多数情况下时间复杂度空间复杂度都是 $O(N+M)$ ， N 为节点个数， M 为边的个数

基于图的BFS

（一般需要一个set来记录访问过的节点）

[126. 单词接龙 II](#)

```

vector<vector<string>> findLadders(string beginWord, string endWord, vector<string>
&wordList)

```

```

{
    vector<vector<string>> res;
    unordered_set<string> dict(wordList.begin(), wordList.end());
    vector<string> p{beginWord};
    queue<vector<string>> paths;
    paths.push(p);
    int level = 1, minLevel = INT_MAX;
    unordered_set<string> words;
    while (!paths.empty())
    {
        auto t = paths.front();
        paths.pop();
        if (t.size() > level)
        {
            for (string w : words)
                dict.erase(w);
            words.clear();
            level = t.size();
            if (level > minLevel)
                break;
        }
        string last = t.back();
        for (int i = 0; i < last.size(); ++i)
        {
            string newLast = last;
            for (char ch = 'a'; ch <= 'z'; ++ch)
            {
                newLast[i] = ch;
                if (!dict.count(newLast))
                    continue;
                words.insert(newLast);
                vector<string> nextPath = t;
                nextPath.push_back(newLast);
                if (newLast == endWord)
                {
                    res.push_back(nextPath);
                    minLevel = level;
                }
                else
                    paths.push(nextPath);
            }
        }
    }
    return res;
}

```

127. 单词接龙

```

int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
    unordered_set<string> wordSet(wordList.begin(), wordList.end());
    if (!wordSet.count(endWord))

```

```

        return 0;
    queue<string> q;
    q.push(beginWord);
    int res = 0;
    while (!q.empty())
    {
        int n = q.size();
        // 层次遍历，一层一层的
        for (int k = 0; k < n; k++)
        {
            string word = q.front();
            q.pop();
            if (word == endWord)
                return res + 1;

            // 开始遍历每一个字母，
            for (int i = 0; i < word.size(); ++i)
            {
                string newWord = word;
                for (char ch = 'a'; ch <= 'z'; ++ch)
                {
                    newWord[i] = ch;
                    if (wordSet.count(newWord) && newWord != word)
                    {
                        q.push(newWord);
                        wordSet.erase(newWord);
                    }
                }
            }
            ++res;
        }
    }
    return 0;
}

```

130. 被围绕的区域

```

void solve(vector<vector<char>>& board)
{
    if (board.empty() || board[0].empty()) return;
    int m = board.size(), n = board[0].size();
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            if (i != 0 && i != m - 1 && j != 0 && j != n - 1) continue;
            if (board[i][j] != 'O') continue;
            board[i][j] = '$';
            queue<pair<int,int>> q;
            q.push({i,j});
            int dirs[4][2] = {0, 1, 1, 0, 0, -1, -1, 0};
            while (!q.empty()) {
                auto t = q.front();

```

```

        q.pop();
        for(int i = 0; i < 4; i++)
        {
            int x = t.first + dirs[i][0];
            int y = t.second + dirs[i][1];
            if (x >= 0 && x < m && y >= 0 && y < n && board[x][y] == 'O')
            {
                board[x][y] = '$';
                q.push({x,y});
            }
        }
    }
}

for (int i = 0; i < m; ++i) {
    for (int j = 0; j < n; ++j) {
        if (board[i][j] == 'O') board[i][j] = 'X';
        if (board[i][j] == '$') board[i][j] = 'O';
    }
}
}

```

133. 克隆图 TODO

```

Node *cloneGraph(Node *node)
{
    // BFS 来遍历图，使用队列 queue 进行辅助，还是需要一个 HashMap 来建立原图结点和克隆结点之间的映射。
    // 先克隆当前结点，然后建立映射，并加入 queue 中，进行 while 循环。在循环中，取出队首结点，遍历其所有 neighbor 结点，
    // 若不在 HashMap 中，我们根据 neighbor 结点值克隆一个新 neighbor 结点，建立映射，并且排入 queue 中。
    // 然后将 neighbor 结点在 HashMap 中的映射结点加入到克隆结点的 neighbors 数组中即可
    if (!node)
        return NULL;
    unordered_map<Node *, Node *> m;
    queue<Node *> q;
    q.push(node);
    Node *clone = new Node(node->val);
    m[node] = clone;
    while (!q.empty())
    {
        Node *t = q.front();
        q.pop();
        for (Node *neighbor : t->neighbors)
        {
            if (!m.count(neighbor))
            {
                m[neighbor] = new Node(neighbor->val);
                q.push(neighbor);
            }
        }
    }
}

```

```

        m[t]->neighbors.push_back(m[neighbor]);
    }
}
return clone;
}

```

200. 岛屿数量

//遍历到 '1' 的时候，且该位置没有被访问过，那么就调用一个 BFS 即可，借助队列 queue 来实现，现将当前位置加入队列，然后进行 while 循环，将队首元素提取出来，并遍历其周围四个位置，若没有越界的话，就将 visited 中该邻居位置标记为 true，并将其加入队列中等待下次遍历即可

```

int numIslands(vector<vector<char>> &grid)
{
    if (grid.empty() || grid[0].empty())
        return 0;
    int m = grid.size(), n = grid[0].size(), res = 0;
    int dirs[4][2] = {0, 1, 1, 0, 0, -1, -1, 0};
    for (int i = 0; i < m; ++i)
    {
        for (int j = 0; j < n; ++j)
        {
            if (grid[i][j] == '1')
            {
                ++res;
                grid[i][j] = '0';
                queue<pair<int, int>> queue;
                queue.push({i, j});
                while (!queue.empty())
                {
                    auto rc = queue.front();
                    queue.pop();
                    int row = rc.first, col = rc.second;
                    for (int i = 0; i < 4; i++)
                    {
                        int x = row + dirs[i][0];
                        int y = col + dirs[i][1];

                        if (x < 0 || x >= m || y < 0 || y >= n || grid[x][y] == '0')
                            continue;

                        queue.push({x, y});
                        grid[x][y] = '0';
                    }
                }
            }
        }
    }
    return res;
}

```

301. 删除无效的括号


```

class Solution {
public:
    vector<string> removeInvalidParentheses(string s)
    {
        //可以用 BFS 来解，我把给定字符串排入队中，然后取出检测其是否合法，若合法直接返回，不合法的话，对其进行遍历，对于遇到的左右括号的字符，去掉括号字符生成一个新的字符串，如果这个字符串之前没有遇到过，将其排入队中，用 HashSet 记录一个字符串是否出现过。对队列中的每个元素都进行相同的操作，直到队列为空还没找到合法的字符串的话，那就返回空集
        vector<string> res;
        unordered_set<string> visited{{s}};
        queue<string> q{{s}};
        bool found = false;
        while (!q.empty()) {
            string t = q.front(); q.pop();
            if (isValid(t)) {
                res.push_back(t);
                found = true;
            }
            if (found) continue;
            for (int i = 0; i < t.size(); ++i) {
                if (t[i] != '(' && t[i] != ')') continue;
                string str = t.substr(0, i) + t.substr(i + 1);
                if (!visited.count(str)) {
                    q.push(str);
                    visited.insert(str);
                }
            }
        }
        return res;
    }
    bool isValid(string t) {
        int cnt = 0;
        for (int i = 0; i < t.size(); ++i) {
            if (t[i] == '(') ++cnt;
            else if (t[i] == ')') && --cnt < 0) return false;
        }
        return cnt == 0;
    }
};

```

317. 建筑物的最短距离

```

int shortestDistance(vector<vector<int>>& grid) {
    //对于每一个建筑的位置都进行一次全图BFS遍历，每次都建立一个dist的距离场，由于我们BFS遍历需要标记应经访问过的位置，而我们并不想建立一个visit的二维矩阵，那么怎么办呢，这里用一个小trick，我们第一遍历的时候，都是找0的位置，遍历完后，我们将其赋为-1，这样下一轮遍历我们就找-1的位置，然后将其都赋为-2，以此类推直至遍历完所有的建筑物，然后在遍历的过程中更新dist和sum的值，注意我们的dist算是个局部变量，每次都初始化为grid，真正的距离场累加在sum中，由于建筑的位置在grid中是1，所以dist中初始化也是1，累加到sum中就需要减1，我们用sum中的值来更新结果res的值，最后根据res的值看是否要返回-1
    int res = INT_MAX, val = 0, m = grid.size(), n = grid[0].size();
    vector<vector<int>> sum = grid;

```

```

vector<vector<int>> dirs{{0,-1},{-1,0},{0,1},{1,0}};
for (int i = 0; i < grid.size(); ++i) {
    for (int j = 0; j < grid[i].size(); ++j) {
        if (grid[i][j] == 1) {
            res = INT_MAX;
            vector<vector<int>> dist = grid;
            queue<pair<int, int>> q;
            q.push({i, j});
            while (!q.empty()) {
                int a = q.front().first, b = q.front().second; q.pop();
                for (int k = 0; k < dirs.size(); ++k) {
                    int x = a + dirs[k][0], y = b + dirs[k][1];
                    if (x >= 0 && x < m && y >= 0 && y < n && grid[x][y] == val) {
                        --grid[x][y];
                        dist[x][y] = dist[a][b] + 1;
                        sum[x][y] += dist[x][y] - 1;
                        q.push({x, y});
                        res = min(res, sum[x][y]);
                    }
                }
            }
            --val;
        }
    }
}
return res == INT_MAX ? -1 : res;
}

```

542.01 矩阵

```

class Solution {
public:
    vector<vector<int>> updateMatrix(vector<vector<int>>& matrix) {
        // 遍历一次矩阵，将值为0的点都存入 queue，将值为1的点改为 INT_MAX
        // 从 queue 中取出一个数字，遍历其周围四个点，如果越界或者周围点的值小于等于当前值加1，则直接跳过。
        // 因为周围点的距离更小的话，就没有更新的必要，否则将周围点的值更新为当前值加1，然后把周围点的坐标加入 queue
        if(matrix.empty() || matrix[0].empty())
            return matrix;
        queue<pair<int,int>> q;
        int m = matrix.size();
        int n = matrix[0].size();

        for(int i = 0; i < m; i++)
        {
            for(int j = 0; j < n; j++)
            {
                if (matrix[i][j] == 0)
                {
                    q.push({i,j});

```

```

        }
        else
        {
            matrix[i][j] = INT_MAX;
        }
    }
}
int dirs[4][2] = {-1, 0, 1, 0, 0, -1, 0, 1};
while(!q.empty())    // 广度优先遍历
{
    int x = q.front().first, y = q.front().second;
    q.pop();
    for(int i = 0; i < 4; i++)    // 遍历4个方向
    {
        int nx = x + dirs[i][0];
        int ny = y + dirs[i][1];
        if(nx >= 0 && ny >= 0 && nx < m && ny < n && matrix[nx][ny] > matrix[x]
[Y])
        {
            matrix[nx][ny] = matrix[x][y] + 1;
            q.push({nx,ny});
        }
    }
}
return matrix;
}
};

```

934. 最短的桥

```

int shortestBridge(vector<vector<int>>& A) {
    //使用 BFS 来找出所有相邻的1，再加上后面的层序遍历的 BFS，总共需要两个 BFS，注意这里第一个 BFS 不
要是层序遍历的，而第二个 BFS 是必须层序遍历
    int res = 0, n = A.size();
    queue<int> q, que;
    vector<int> dirX{-1, 0, 1, 0}, dirY = {0, 1, 0, -1};
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (A[i][j] == 0) continue;
            A[i][j] = 2;
            que.push(i * n + j);
            break;
        }
        if (!que.empty()) break;
    }
    while (!que.empty()) {
        int t = que.front(); que.pop();
        q.push(t);
        for (int k = 0; k < 4; ++k) {
            int x = t / n + dirX[k], y = t % n + dirY[k];

```

```

        if (x < 0 || x >= n || y < 0 || y >= n || A[x][y] == 0 || A[x][y] == 2)
            continue;
        A[x][y] = 2;
        que.push(x * n + y);
    }
}
while (!q.empty()) {
    for (int i = q.size(); i > 0; --i) {
        int t = q.front(); q.pop();
        for (int k = 0; k < 4; ++k) {
            int x = t / n + dirX[k], y = t % n + dirY[k];
            if (x < 0 || x >= n || y < 0 || y >= n || A[x][y] == 2) continue;
            if (A[x][y] == 1) return res;
            A[x][y] = 2;
            q.push(x * n + y);
        }
    }
    ++res;
}
return res;
}

```

994. 腐烂的橘子 #todo

```

int orangesRotting(vector<vector<int>>& grid) {
    // 先遍历一遍整个二维数组，统计出所有新鲜橘子的个数，并把腐烂的橘子坐标放入一个队列 queue，
    // 之后进行 while 循环，循环条件是队列不会空，且 freshLeft 大于0，使用层序遍历的方法，用个
    for 循环在内部。
    // 每次取出队首元素，遍历其周围四个位置，越界或者不是新鲜橘子都跳过，否则将新鲜橘子标记为腐烂，加入
    队列中，并且 freshLeft 自减1。
    // 每层遍历完成之后，结果 res 自增1，最后返回的时候，若还有新鲜橘子，即 freshLeft 大于0时，返回
    -1，否则返回 res 即可
    int res = 0;
    int m = grid.size();
    int n = grid[0].size();
    int fresh_count = 0;

    queue<vector<int>> q;
    int dirs[4][2] = {0, 1, 0, -1, 1, 0, -1, 0};
    for(int i = 0; i < m; i++)
    {
        for(int j = 0; j < n; j++)
        {
            if (grid[i][j] == 1)
                fresh_count++;
            else if (grid[i][j] == 2)
                q.push({i,j});
        }
    }
    while(!q.empty() && fresh_count > 0)
    {

```

```

    int size_q = q.size();
    for(int i = 0; i < size_q; i++)
    {
        auto cur = q.front(); q.pop();
        for(int k = 0; k < 4; k++)
        {
            int x = cur[0] + dirs[k][0], y = cur[1] + dirs[k][1];
            if (x < 0 || x >= m || y < 0 || y >= n || grid[x][y] != 1) continue;
            grid[x][y] = 2;
            q.push({x, y});
            --fresh_count;
        }
    }
    res++;
}
return fresh_count > 0 ? -1 : res;
}
}

```

490. The Maze

323. Connected Component in Undirected Graph

[752. 打开转盘锁](#)

```

int openLock(vector<string>& deadends, string target) {
    if (target == "0000") return 0;
    unordered_set<string> deadlock(deadends.begin(), deadends.end());
    if (deadlock.count("0000")) return -1;
    int res = 0;
    unordered_set<string> visited{"0000"};
    queue<string> q{"0000"};
    while (!q.empty()) {
        ++res;
        for (int k = q.size(); k > 0; --k) {
            auto t = q.front(); q.pop();
            for (int i = 0; i < t.size(); ++i) {
                char c = t[i];
                string str1 = t.substr(0, i) + to_string(c == '9' ? 0 : c - '0' + 1) +
t.substr(i + 1);
                string str2 = t.substr(0, i) + to_string(c == '0' ? 9 : c - '0' - 1) +
t.substr(i + 1);
                if (str1 == target || str2 == target) return res;
                if (!visited.count(str1) && !deadlock.count(str1)) q.push(str1);
                if (!visited.count(str2) && !deadlock.count(str2)) q.push(str2);
                visited.insert(str1);
                visited.insert(str2);
            }
        }
    }
    return -1;
}

```

815. 公交线路

```
int numBusesToDestination(vector<vector<int>>& routes, int S, int T) {
    if (S == T) return 0;
    int res = 0;
    unordered_map<int, vector<int>> stop2bus;
    queue<int> q{{S}};
    unordered_set<int> visited;
    for (int i = 0; i < routes.size(); ++i) {
        for (int j : routes[i]) {
            stop2bus[j].push_back(i);
        }
    }
    while (!q.empty()) {
        ++res;
        for (int i = q.size(); i > 0; --i) {
            int t = q.front(); q.pop();
            for (int bus : stop2bus[t]) {
                if (visited.count(bus)) continue;
                visited.insert(bus);
                for (int stop : routes[bus]) {
                    if (stop == T) return res;
                    q.push(stop);
                }
            }
        }
    }
    return -1;
}
```

1091. 二进制矩阵中的最短路径

```
int shortestPathBinaryMatrix(vector<vector<int>>& grid) {
    if (grid[0][0] == 1) return -1;
    int res = 0, n = grid.size();
    set<vector<int>> visited;
    visited.insert({0, 0});
    queue<vector<int>> q;
    q.push({0, 0});
    vector<vector<int>> dirs{{-1, 0}, {-1, 1}, {0, 1}, {1, 1}, {1, 0}, {1, -1}, {0, -1},
{-1, -1}};
    while (!q.empty()) {
        ++res;
        for (int i = q.size(); i > 0; --i) {
            auto t = q.front(); q.pop();
            if (t[0] == n - 1 && t[1] == n - 1) return res;
            for (auto dir : dirs) {
```

```

        int x = t[0] + dir[0], y = t[1] + dir[1];
        if (x < 0 || x >= n || y < 0 || y >= n || grid[x][y] == 1 ||
visited.count({x, y})) continue;
        visited.insert({x, y});
        q.push({x, y});
    }
}
}
return -1;
}

```

1293. 网格中的最短路径

```

int shortestPath(vector<vector<int>>& grid, int k) {
    int res = 0, m = grid.size(), n = grid[0].size();
    vector<vector<int>> dirs{{-1, 0}, {0, 1}, {1, 0}, {0, -1}};
    vector<vector<int>> visited(m, vector<int>(n, -1)); // The number of obstacles that we
can still remove after walking through that cell
    visited[0][0] = k;
    queue<vector<int>> q;
    q.push({0, 0, k});
    while (!q.empty()) {
        for (int i = q.size(); i > 0; --i) {
            auto t = q.front(); q.pop();
            if (t[0] == m - 1 && t[1] == n - 1) return res;
            for (auto dir : dirs) {
                int x = t[0] + dir[0], y = t[1] + dir[1];
                if (x < 0 || x >= m || y < 0 || y >= n) continue;
                int newK = t[2] - grid[x][y];
                if (newK < 0 || newK <= visited[x][y]) continue;
                visited[x][y] = newK;
                q.push({x, y, newK});
            }
        }
        ++res;
    }
    return -1;
}

```

拓扑排序

207. 课程表

```

bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
    vector<vector<int>> graph(numCourses, vector<int>(0));
    vector<int> indegree(numCourses, 0); // 定点入度表
    // 定义二维数组 graph 来表示这个有向图，一维数组 in 来表示每个顶点的入度。
    // 开始先根据输入来建立这个有向图，并将入度数也初始化好。
    // 然后定义一个 queue 变量，将所有入度为0的点放入队列中，然后开始遍历队列，
    // 从 graph 里遍历其连接的点，每到达一个新节点，将其入度减一，如果此时该点入度为0，则放入队列末尾。
    // 直到遍历完队列中所有的值，若此时还有节点的入度不为0，则说明环存在，返回 false，反之则返回 true
}

```

```

for (int i = 0; i < prerequisites.size(); i++)
{
    graph[prerequisites[i][0]].push_back(prerequisites[i][1]);
    ++indegree[prerequisites[i][1]];
}
queue<int> q;
for (int i = 0; i < numCourses; i++)
{
    if (indegree[i] == 0)
    {
        q.push(i);
    }
}
int count = 0;
while (!q.empty())
{
    int temp = q.front();
    q.pop();
    count++;
    for (int i = 0; i < graph[temp].size(); i++)
    {
        if (--indegree[graph[temp][i]] == 0)
        {
            q.push(graph[temp][i]);
        }
    }
}
return count == numCourses;
}

```

210. 课程表 II

```

vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
    vector<int> res;
    vector<vector<int>> graph(numCourses, vector<int>(0));
    vector<int> in(numCourses, 0);
    for (auto &a : prerequisites) {
        graph[a[1]].push_back(a[0]);
        ++in[a[0]];
    }
    queue<int> q;
    for (int i = 0; i < numCourses; ++i) {
        if (in[i] == 0) q.push(i);
    }
    while (!q.empty()) {
        int t = q.front();
        res.push_back(t);
        q.pop();
        for (auto &a : graph[t]) {
            --in[a];
            if (in[a] == 0) q.push(a);
        }
    }
    return res;
}

```



```

    }
}
if (res.size() != numCourses) res.clear();
return res;
}

```

310. 最小高度树

```

vector<int> findMinHeightTrees(int n, vector<vector<int>>& edges)
{
    vector<int> res;
    if (n == 1)
    {
        res.push_back(0);
        return res;
    }
    vector<vector<int>> graph(n);
    vector<int> degree(n, 0);
    for (int i = 0; i < edges.size(); i++)
    {
        graph[edges[i][0]].push_back(edges[i][1]);
        graph[edges[i][1]].push_back(edges[i][0]);
        degree[edges[i][0]]++;
        degree[edges[i][1]]++;
    }
    int count = n;
    while (count > 2)
    {
        vector<int> records;
        for (int i = 0; i < n; i++) //记录所有度为1的结点
        {
            if (degree[i] == 1)
            {
                records.push_back(i);
                degree[i] = -1; // 相当于删掉度为1的结点
                count--;
            }
        }
        for (int i = 0; i < records.size(); i++) //删掉度为1的结点之后 重新调整每个结点的度
        {
            for(int j = 0; j < graph[records[i]].size(); j++)
            {
                degree[graph[records[i]][j]]--;
            }
        }
    }
    for (int i = 0; i < n; i++)
    {
        if (degree[i] == 1 || degree[i] == 0)
        {
            res.push_back(i);
        }
    }
}

```

```

    }
}
return res;
}

```

Leetcode 444 Sequence Reconstruction

Leetcode 269 Alien Dictionary

Leetcode 310 Minimum Height Trees

二叉树遍历相关（深度&层次遍历）

144. 二叉树的前序遍历

```

// 前序遍历递归版本
void preorderTraversal(TreeNode* root, vector<int> &res)
{
    if (root == nullptr)
        return;
    res.push_back(root->val);
    preorderTraversal(root->left, res);
    preorderTraversal(root->right, res);
}

// 前序遍历非递归版本
void preorderTraversal_2(TreeNode* root, vector<int> &res)
{
    if (root == nullptr)
        return;

    stack<TreeNode*> s;
    s.push(root);
    while(!s.empty())
    {
        root = s.top(); s.pop();
        res.push_back(root->val);

        if (root->right)
            s.push(root->right);
        if (root->left)
            s.push(root->left);
    }
}

```

94. 二叉树的中序遍历

```

// 二叉树中序非递归遍历
// 逮到一棵树，不管三七二十一，先把左边界全部压栈，依次弹出每一个节点的时候，让他的右节点也进行同样的操作
vector<int> inorderTraversal(TreeNode* root)

```

```

{
    vector<int> res;
    if (root == nullptr)
    {
        return {};
    }
    stack<TreeNode*> s;
    while(!s.empty() || root)
    {
        // 一路压左边界
        if (root != nullptr)
        {
            s.push(root);
            root = root->left;
        } else {
            root = s.top();
            res.push_back(root->val); // 弹出即打印
            s.pop();
            root = root->right;
        }
    }
    return res;
}

```

145. 二叉树的后序遍历

```

vector<int> postorderTraversal(TreeNode* root)
{
    vector<int> res;
    if (root == nullptr)
        return res;

    stack<TreeNode*> s1;
    stack<TreeNode*> s2;
    s1.push(root);
    while(!s1.empty())
    {
        root = s1.top();
        s1.pop();
        s2.push(root);

        if (root->left)
            s1.push(root->left);
        if (root->right)
            s1.push(root->right);
    }
    while(!s2.empty())
    {
        res.push_back(s2.top()->val);
        s2.pop();
    }
}

```

```
    return res;
}
```

114. 二叉树展开为链表 二叉树前序遍历非递归

```
void flatten(TreeNode* root)
{
    if(root == nullptr)
        return;
    stack<TreeNode*> s;
    vector<TreeNode*> node_list;
    s.push(root);
    while(!s.empty())
    {
        root = s.top();
        s.pop();
        node_list.push_back(root);
        // 因为需要先访问左子树，所以左子树后压栈
        if (root->right)
            s.push(root->right);
        if (root->left)
            s.push(root->left);
    }
    for(int i = 1; i < node_list.size(); i++)
    {
        auto prev = node_list[i - 1], curr = node_list[i];
        prev->left = nullptr;
        prev->right = curr;
    }
}
```

958. 二叉树的完全性检验 #TODO

```
bool isCompleteTree(TreeNode* root) {
    if (root == NULL)
        return true;

    queue<TreeNode*> q;
    q.push(root);
    // 1. 找到第一个空结点
    while (!q.empty())
    {
        TreeNode* front = q.front();
        q.pop();
        if (front == NULL)
            break;
        else
        {
            q.push(front->left);
            q.push(front->right);
        }
    }
}
```

```

    }
}
// 2. 检查队列中剩余结点是否有非空结点
while (!q.empty())
{
    TreeNode* front = q.front();
    q.pop();
    if (front)
        return false;
}
return true;
}

```

102. 二叉树的层序遍历

```

vector<vector<int>> levelOrder(TreeNode *root)
{
    vector<vector<int>> ret;
    if (root == NULL)
        return ret;

    queue<TreeNode *> q;
    q.push(root);
    while (!q.empty())
    {
        int size = q.size();
        vector<int> level;
        for (int i = 0; i < size; i++)
        {
            TreeNode *node = q.front();
            level.push_back(node->val);
            q.pop();
            if (node->left)
                q.push(node->left);
            if (node->right)
                q.push(node->right);
        }
        ret.push_back(level);
    }
    return ret;
}

```

103. 二叉树的锯齿形层序遍历

//由于每层的结点数是知道的，就是队列的元素个数，所以可以直接初始化数组的大小，使用一个变量 leftToRight 来标记顺序，初始时是 true，当此变量为 true 的时候，每次加入数组的位置就是i本身，若变量为 false 了，则加入到 size-1-i 位置上，这样就相当于翻转了数组

```

vector<vector<int>> zigzagLevelOrder(TreeNode *root)
{
    vector<vector<int>> res;

```

```

if (root == NULL)
    return res;
queue<TreeNode *> q;
q.push(root);
bool leftToRight = true;
while (!q.empty())
{
    int size = q.size();
    vector<int> oneLevel(size); // 这个地方注意 要给定数组大小
    for (int i = 0; i < size; ++i)
    {
        TreeNode *t = q.front();
        q.pop();
        int idx = leftToRight ? i : (size - 1 - i);
        oneLevel[idx] = t->val;
        if (t->left)
            q.push(t->left);
        if (t->right)
            q.push(t->right);
    }
    leftToRight = !leftToRight;
    res.push_back(oneLevel);
}
return res;
}

```

111. 二叉树的最小深度

```

int minDepth(TreeNode *root)
{
    if (!root)
        return 0;

    queue<TreeNode *> q;
    q.push(root);
    int res = 1;
    while (!q.empty())
    {
        int size = q.size();
        for (int i = 0; i < size; i++)
        {
            root = q.front();
            q.pop();
            if (!root->left && !root->right)
                return res;
            if (root->left)
                q.push(root->left);

            if (root->right)
                q.push(root->right);
        }
    }
}

```

```

        res++;
    }
    return -1;
}

```

116. 填充每个节点的下一个右侧节点指针

```

Node *connect(Node *root)
{
    if (!root) return NULL;
    queue<Node *> q;
    q.push(root);
    while (!q.empty())
    {
        int size = q.size();
        for (int i = 0; i < size; ++i)
        {
            Node *t = q.front();
            q.pop();
            if (i < size - 1)
                t->next = q.front();
            if (t->left) q.push(t->left);
            if (t->right) q.push(t->right);
        }
    }
    return root;
}

```

117. 填充每个节点的下一个右侧节点指针 II

```

Node* connect(Node* root) {
    if (!root) return NULL;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        int len = q.size();
        for (int i = 0; i < len; ++i) {
            Node *t = q.front(); q.pop();
            if (i < len - 1) t->next = q.front();
            if (t->left) q.push(t->left);
            if (t->right) q.push(t->right);
        }
    }
    return root;
}

```

199. 二叉树的右视图

```

vector<int> rightSideView(TreeNode* root)
{

```

```

vector<int> res;
if (root == NULL)
    return res;
queue<TreeNode *> q;
q.push(root);
while (!q.empty())
{
    int size = q.size();
    vector<int> level;
    for (int i = 0; i < size; i++)
    {
        TreeNode *node = q.front();
        level.push_back(node->val);
        q.pop();
        if (node->left)
            q.push(node->left);
        if (node->right)
            q.push(node->right);
    }
    res.push_back(level.back());
}
return res;
}

```

513. 找树左下角的值

```

int findBottomLeftValue(TreeNode* root) {
    int res = 0;
    queue<TreeNode *> q;
    q.push(root);
    // 使用个小trick, 使得其更加的简洁, 由于一般的层序是从左往右的, 那么如果我们反过来,
    // 每次从右往左遍历, 这样就不用检测每一层的起始位置了, 最后一个处理的结点一定是最后一层的最左结点, 我们直接返回其结点值即可
    while (!q.empty())
    {
        root = q.front(); q.pop();
        if (root->right) q.push(root->right);
        if (root->left) q.push(root->left);
    }
    return root->val;
}

```

662. 二叉树最大宽度 todo

```

int widthOfBinaryTree(TreeNode* root) {
    if (!root) return 0;
    int res = 0;
    queue<pair<TreeNode*,int>> q;
    q.push({root, 1});
    while (!q.empty()) {

```



```

    if (q.size() == 1) q.front().second = 1;
    int left = q.front().second, right = left, n = q.size();
    for (int i = 0; i < n; ++i) {
        auto t = q.front().first;
        right = q.front().second; q.pop();
        if (t->left) q.push({t->left, right * 2});
        if (t->right) q.push({t->right, right * 2 + 1});
    }
    res = max(res, right - left + 1);
}
return res;
}

```

Leetcode 297 Serialize and Deserialize Binary Tree （很好的BFS和双指针结合的题）

Leetcode 314 Binary Tree Vertical Order Traversal

二叉树的DFS

通常采用递归

- 基于树的DFS：需要记住递归写前序中序后序遍历二叉树的模板
- - Leetcode 543 Diameter of Binary Tree
 - Leetcode 226 Invert Binary Tree
 - Leetcode 101 Symmetric Tree
 - Leetcode 951 Flip Equivalent Binary Trees
 - Leetcode 124 Binary Tree Maximum Path Sum
 - Leetcode 236 Lowest Common Ancestor of a Binary Tree (相似题：235、1650)
 - Leetcode 105 Construct Binary Tree from Preorder and Inorder Traversal
 - Leetcode 104 Maximum Depth of Binary Tree
 - Leetcode 987 Vertical Order Traversal of a Binary Tree
 - Leetcode 1485 Clone Binary Tree With Random Pointer
 - Leetcode 572 Subtree of Another Tree
 - Leetcode 863 All Nodes Distance K in Binary Tree
 - Leetcode 1110 Delete Nodes And Return Forest

100. 相同的树

```

bool isSameTree(TreeNode *p, TreeNode *q)
{
    if (p == nullptr && q == nullptr)
        return true;
    if ((p == nullptr && q != nullptr) || (p != nullptr && q == nullptr))
        return false;
    if (p->val != q->val)
        return false;
    else
        return isSameTree(p->left, q->left) && isSameTree(p->right, q->right);
}

```

101. 对称二叉树

```

class Solution {
public:
    bool isSymmetricTree(TreeNode *root1, TreeNode *root2)
    {
        if (root1 == nullptr && root2 == nullptr)
            return true;
        if (root1 == nullptr || root2 == nullptr)
            return false;

        return root1->val == root2->val && isSymmetricTree(root1->left, root2->right) &&
            isSymmetricTree(root1->right, root2->left);
    }
    bool isSymmetric(TreeNode* root) {
        if (!root)
            return true;
        return isSymmetricTree(root->left, root->right);
    }
};

```

104. 二叉树的最大深度

```

int maxDepth(TreeNode* root)
{
    if (root == NULL)
        return 0;
    else return max(maxDepth(root->left), maxDepth(root->right)) + 1;
}

```

110. 平衡二叉树

```

int height(TreeNode *root)
{
    if(root == NULL)return 0;
    return max(height(root->left), height(root->right)) + 1;
}
bool isBalanced(TreeNode* root)
{
    if(root == NULL)
        return true;
    else return isBalanced(root->right) && isBalanced(root->left) && abs(height(root->left) - height(root->right)) <= 1;
}

```

235. 二叉搜索树的最近公共祖先 todo

```

TreeNode *lowestCommonAncestor(TreeNode *root, TreeNode *p, TreeNode *q)
{
    // 如果根节点的值 比pq都大,那么最近公共祖先只能在左子树上, 否则在右子树上,
    // 只有当root->val 在pq之间的话, root才是最近公共祖先
    while (true)
    {
        if (root->val > p->val && root->val > q->val)
            root = root->left;
        else if (root->val < p->val && root->val < q->val)
            root = root->right;
        else
            break;
    }
    return root;
}

```

// 如果根节点的值大于p和q之间的较大值, 说明p和q都在左子树中, 那么此时我们就进入根节点的左子节点继续递归, 如果根节点小于p和q之间的较小值, 说明p和q都在右子树中, 那么此时我们就进入根节点的右子节点继续递归, 如果都不是, 则说明当前根节点就是最小共同父节点, 直接返回即可 递归版本

```

TreeNode *lowestCommonAncestor_2(TreeNode *root, TreeNode *p, TreeNode *q)
{
    if (!root)
        return NULL;
    if (root->val > max(p->val, q->val))
        return lowestCommonAncestor(root->left, p, q);
    else if (root->val < min(p->val, q->val))
        return lowestCommonAncestor(root->right, p, q);
    else
        return root;
}

```

236. 二叉树的最近公共祖先 todo

```

TreeNode *lowestCommonAncestor(TreeNode *root, TreeNode *p, TreeNode *q)
{

```

// 看当前结点是否为空，若为空则直接返回空，若为p或q中的任意一个，也直接返回当前结点。否则的话就对其左右子结点分别调用递归函数，由于这道题限制了p和q一定都在二叉树中存在，那么如果当前结点不等于p或q，p和q要么分别位于左右子树中，要么同时位于左子树，或者同时位于右子树

```
if (root == nullptr)
    return nullptr;
if (root == p || root == q)
    return root;

TreeNode *left = lowestCommonAncestor(root->left, p, q);
TreeNode *right = lowestCommonAncestor(root->right, p, q);
if (left && right)
    return root;
else
    return left != nullptr ? left : right;
}
```

572. 另一个树的子树

```
class Solution {
    bool isSame(TreeNode* s, TreeNode* t)
    {
        if (s == nullptr && t == nullptr)
            return true;
        if (s == nullptr || t == nullptr)
            return false;

        if (s->val != t->val)
            return false;
        else
            return isSame(s->left, t->left) && isSame(s->right, t->right);
    }

public:
    bool isSubtree(TreeNode* s, TreeNode* t)
    {
        if (s == nullptr && t == nullptr)
            return true;
        if (s == nullptr || t == nullptr)
            return false;
        if (isSame(s, t))
            return true;
        return isSubtree(s->left, t) || isSubtree(s->right, t);
    }
};
```

112. 路径总和

```

bool hasPathSum(TreeNode *root, int sum)
{
    if (root == NULL)
        return false;
    if (root->left == NULL && root->right == NULL && sum == root->val) // 叶子节点
        return true;
    else
        return hasPathSum(root->right, sum - root->val) || hasPathSum(root->left, sum - root->val);
}

```

113. 路径总和 II

```

// **本质上还是回溯**
class Solution
{
public:
    void dfs(TreeNode *node, int sum, vector<int> &temp, vector<vector<int>> &res)
    {
        if (node == nullptr)
            return;
        temp.push_back(node->val);
        // 叶子节点
        if (sum == node->val && node->left == nullptr && node->right == nullptr)
        {
            res.push_back(temp);
        }
        dfs(node->left, sum - node->val, temp, res);
        dfs(node->right, sum - node->val, temp, res);
        temp.pop_back();
    }
    vector<vector<int>> pathSum(TreeNode *root, int sum)
    {
        vector<vector<int>> res;
        vector<int> temp;
        dfs(root, sum, temp, res);
        return res;
    }
};

```

437. 路径总和 III

```

// 本质上还是回溯
// 每一个节点都有记录了一条从根节点到当前节点到路径 path
// 用一个变量 curSum 记录路径节点总和, 然后看 curSum 和 sum 是否相等, 相等的话结果 res 加1,
// 不等的话继续查看子路径和有没有满足题意的, 做法就是每次去掉一个节点, 看路径和是否等于给定值
int pathSum(TreeNode* root, int sum)
{
    int res = 0;
    vector<TreeNode*> temp;

```

```

    dfs(root, sum, 0, temp, res);
    return res;
}

void dfs(TreeNode* node, int sum, int curSum, vector<TreeNode*>& temp, int& res) {
    if (!node) return;
    curSum += node->val;
    temp.push_back(node);
    if (curSum == sum) ++res;
    // 相当于先序遍历二叉树，对于每一个节点都有记录了一条从根节点到当前节点到路径，同时用一个变量 curSum
    // 记录路径节点总和，然后看 curSum 和 sum 是否相等，相等的话结果 res 加1，
    // 不等的话继续查看子路径和有没有满足题意的，做法就是每次去掉一个节点，看路径和是否等于给定值，
    // 注意最后必须留一个节点，不能全去掉了，因为如果全去掉了，路径之和为0，而如果给定值刚好为0的话就会有
    // 问题
    int t = curSum;
    for (int i = 0; i < temp.size() - 1; ++i) {
        t -= temp[i]->val;
        if (t == sum) ++res;
    }
    dfs(node->left, sum, curSum, temp, res);
    dfs(node->right, sum, curSum, temp, res);
    temp.pop_back();
}

```

124. 二叉树中的最大路径和 **todo: 还不是很懂**

```

class Solution
{
public:
    // 递归函数表示 从node出发能得到的最大路径和
    // 1) Max path sum lies only in the right half.
    // 2) Max path sum lies only in the left half.
    // 3) Max path passes from left to right half (or vice versa) through the root node.
    int dfs(TreeNode *node, int &res)
    {
        if (node == nullptr)
            return 0;

        // 分别求出左右子树上的最大路径和
        int left = max(dfs(node->left, res), 0);
        int right = max(dfs(node->right, res), 0);

        // todo: 这个地方不理解
        res = max(left + right + node->val, res);

        // 对当前node来说，它的最大路径和就是当前节点值加上 其左右子树上的最大值
        return max(left, right) + node->val;
    }

    int maxPathSum(TreeNode *root)
    {
        int res = INT_MIN;
    }
}

```

```

        dfs(root, res);
        return res;
    }
};

```

543. 二叉树的直径 # todo其实和124套路一样 虚假的easy

```

int dfs(TreeNode* node, int& res)
{
    if (!node) return 0;
    int left = dfs(node->left, res);
    int right = dfs(node->right, res);
    res = max(res, left + right);
    return max(left, right) + 1;
}

int diameterOfBinaryTree(TreeNode* root)
{
    int res = 0;
    dfs(root, res);
    return res;
}

```

1339. 分裂二叉树的最大乘积

```

typedef long long LL;
class Solution {
public:
    // 计算整棵树的总和
    LL dfs_sum(TreeNode* root) {
        if (root == NULL) {
            return 0;
        }
        return root->val + dfs_sum(root->left) + dfs_sum(root->right);
    }

    // 后序遍历的同时，求出子树和
    LL dfs_sub_sum(TreeNode* root, LL root_sum, LL& res) {
        if (root == NULL) {
            return 0;
        }
        LL left_sum = dfs_sub_sum(root->left, root_sum, res);
        LL right_sum = dfs_sub_sum(root->right, root_sum, res);
        LL sub_sum = root->val + left_sum + right_sum;
        // 对结果取最大
        res = max(res, (root_sum - sub_sum) * sub_sum);
        return sub_sum;
    }

    int maxProduct(TreeNode* root) {
        LL root_sum = dfs_sum(root);
        LL res = 0;
        dfs_sub_sum(root, root_sum, res);
    }
};

```

```

        return res % LL(1e9 + 7);
    }
};

```

129. 求根节点到叶节点数字之和

```

void dfs(TreeNode *root, int &sum, int cur_sum)
{
    if (root == nullptr)
        return;
    cur_sum = cur_sum * 10 + root->val;
    if (!root->left && !root->right)
    {
        sum += cur_sum;
    }
    dfs(root->left, sum, cur_sum);
    dfs(root->right, sum, cur_sum);
}
int sumNumbers(TreeNode* root)
{
    int res = 0;
    dfs(root, res, 0);
    return res;
}

```

226. 翻转二叉树

```

TreeNode* invertTree(TreeNode* root) {
    if (root == nullptr)
        return root;
    TreeNode *temp = root->left;
    root->left = root->right;
    root->right = temp;

    root->left = invertTree(root->left);
    root->right = invertTree(root->right);

    return root;
}

```

617. 合并二叉树


```

TreeNode* mergeTrees(TreeNode* t1, TreeNode* t2) {
    if (!t1) return t2;
    if (!t2) return t1;
    TreeNode *t = new TreeNode(t1->val + t2->val);
    t->left = mergeTrees(t1->left, t2->left);
    t->right = mergeTrees(t1->right, t2->right);
    return t;
}

```

剑指 Offer 33. 二叉搜索树的后序遍历序列

```

class Solution {
public:
    // 判断一个整数数组是否是二叉搜索树的后序遍历结果，可以利用二叉搜索树的性质。在后序遍历中，最后一个元素是根节点，而根据二叉搜索树的性质，左子树的值都小于根节点的值，右子树的值都大于根节点的值。
    // 因此，我们可以利用这个性质，递归地判断数组是否满足这个条件。具体步骤如下：
    // 找到根节点的值（数组的最后一个元素）。
    // 在数组中找到第一个大于根节点值的元素，该元素之前的部分是左子树的后序遍历结果，之后的部分是右子树的后序遍历结果。
    // 递归地判断左子树和右子树是否是二叉搜索树的后序遍历结果。
    bool dfs(vector<int>& postorder, int start, int end) {
        if (start >= end) {
            return true;
        }
        // 根节点的值数组最后一个元素
        int rootValue = postorder[end];

        // 在数组中找到第一个大于根节点值的元素
        int splitIndex = start;
        while (splitIndex < end && postorder[splitIndex] < rootValue) {
            splitIndex++;
        }

        // 检查右子树的元素是否都大于根节点值
        for (int i = splitIndex; i < end; i++) {
            if (postorder[i] < rootValue) {
                return false;
            }
        }
        // 递归判断左右子树
        return dfs(postorder, start, splitIndex - 1) &&
            dfs(postorder, splitIndex, end - 1);
    }
    bool verifyTreeOrder(vector<int>& postorder) {
        return dfs(postorder, 0, postorder.size() - 1);
    }
};

```

二叉搜索树 (BST)

BST特征：中序遍历为单调递增的二叉树，换句话说，根节点的值比左子树任意节点值都大，比右子树任意节点值都小，增删查改均为 $O(h)$ 复杂度， h 为树的高度；注意不是所有的BST题目都需要递归，有的题目只需要while循环即可

Leetcode 270 Closest Binary Search Tree Value

Leetcode 333 Largest BST Subtree (与98类似)

Leetcode 285 Inorder Successor in BST (I, II)

[98. 验证二叉搜索树](#)

```
bool isValidBST(TreeNode* root) {
    if (root == nullptr)
        return true;
    stack<TreeNode*> s;
    TreeNode *pre = nullptr;
    TreeNode *cur = root;

    while(!s.empty() || cur) {
        if (cur) {
            s.push(cur);
            cur = cur->left;
        } else {
            cur = s.top(); s.pop();
            if (pre && pre->val >= cur->val)
                return false;
            pre = cur;
            cur = cur->right;
        }
    }
    return true;
}
```

[99. 恢复二叉搜索树](#) todo

```
class Solution {
public:
    // 恢复二叉搜索树的思路一般是通过中序遍历，找到那两个被错误交换的节点，然后交换它们的值。在中序遍历过程中，节点的值应该是递增的。如果存在两个节点被错误地交换，那么在中序遍历的序列中就会出现两次降序。
    // 具体步骤如下：
    // 中序遍历二叉搜索树，找到两次降序的节点，分别记为 first 和 second。
    // 交换 first 和 second 的值，恢复正确的二叉搜索树。

    void recoverTree(TreeNode* root) {
        TreeNode *first = nullptr, *second = nullptr; // 用于记录需要交换的两个节点
        TreeNode *prev = nullptr; // 用于记录中序遍历的前一个节点

        stack<TreeNode*> s;
```

```

while (root || !s.empty()) {
    while (root) {
        s.push(root);
        root = root->left;
    }

    root = s.top();
    s.pop();

    // 判断当前节点值与前一个节点值的关系
    if (!first && prev && prev->val >= root->val) {
        first = prev;
    }
    if (first && prev && prev->val >= root->val) {
        second = root;
    }
    // 更新前一个节点
    prev = root;
    root = root->right;
}

// 交换两个节点的值
if (first && second) {
    swap(first->val, second->val);
}
}

void recoverTree_v1(TreeNode* root) {
    TreeNode *first = nullptr, *second = nullptr; // 用于记录需要交换的两个节点
    TreeNode *prev = nullptr; // 用于记录中序遍历的前一个节点

    // 中序遍历
    inorderTraversal(root, first, second, prev);

    // 交换两个节点的值
    if (first && second) {
        swap(first->val, second->val);
    }
}

private:
void inorderTraversal(TreeNode* node, TreeNode*& first, TreeNode*& second, TreeNode*& prev) {
    if (node) {
        // 遍历左子树
        inorderTraversal(node->left, first, second, prev);

        // 判断当前节点值与前一个节点值的关系
        if (!first && prev && prev->val >= node->val) {
            first = prev;
        }
        if (first && prev && prev->val >= node->val) {

```

```

        second = node;
    }

    // 更新前一个节点
    prev = node;

    // 遍历右子树
    inorderTraversal(node->right, first, second, prev);
}
}
};

```

99_2找到搜索二叉树中两个错误的节点

```

class Solution {
public:
    /**
     *
     * @param root TreeNode类 the root
     * @return int整型vector
     */
    void inorderTraverse(TreeNode* &root, TreeNode* &pre, TreeNode* &first, TreeNode*
&second)
    {
        if (root == nullptr)
            return;

        stack<TreeNode*> s;

        while(!s.empty() || root)
        {
            if(root)
            {
                s.push(root);
                root = root->left;
            }
            else
            {
                root = s.top();
                s.pop();

                if (pre != nullptr && pre->val > root->val)
                {
                    first = first == nullptr ? pre : first;
                    second = root;
                }
                pre = root;
                root = root->right;
            }
        }
    }
}

```

```

vector<int> findError(TreeNode* root) {
    // write code here
    TreeNode* first=nullptr;
    TreeNode* second=nullptr;
    TreeNode* pre= nullptr;
    vector<int> res;
    inOrderTraverse(root, pre, first, second);
    res.push_back(first->val);
    res.push_back(second->val);
    sort(res.begin(), res.end());
    return res;
}

```

108. 将有序数组转换为二叉搜索树

```

TreeNode* dfs(vector<int>& nums, int left, int right)
{
    if (left > right) return NULL;
    int mid = left + (right - left) / 2;
    TreeNode *cur = new TreeNode(nums[mid]);
    cur->left = dfs(nums, left, mid - 1);
    cur->right = dfs(nums, mid + 1, right);
    return cur;
}
TreeNode* sortedArrayToBST(vector<int>& nums)
{
    return dfs(nums, 0, (int)nums.size() - 1);
}

```

109. 有序链表转换二叉搜索树 todo

```

class Solution {
public:
    TreeNode* sortedListToBST(ListNode* head) {
        if (head == nullptr)
            return nullptr;

        if (head->next == nullptr)
            return new TreeNode(head->val);

        // 快慢指针找链表中点
        ListNode *fast = head, *slow = head;
        ListNode *pre = nullptr; // 记录链表中点的前一个
        while(fast->next && fast->next->next)
        {
            pre = slow;
            fast = fast->next->next;
        }
    }
}

```

```

        slow = slow->next;
    }
    // 断开链表
    if (pre)
        pre->next = nullptr;
    else
        head = nullptr;
    // 中点作为根节点
    TreeNode *root = new TreeNode(slow->val);

    // 递归处理左右两个子链表
    root->left = sortedListToBST(head);
    root->right = sortedListToBST(slow->next);
    return root;
}

```

[173. 二叉搜索树迭代器](#)

[230. 二叉搜索树中第K小的元素](#)

```

int kthSmallest(TreeNode* root, int k)
{
    if (!root || k < 0)
        return 0;

    stack<TreeNode*> s;
    TreeNode *cur = root;
    while (!s.empty() || cur)
    {
        if (cur)
        {
            s.push(cur);
            cur = cur->left;
        }
        else
        {
            TreeNode *temp = s.top();
            s.pop();
            k--;
            if (k == 0)
                return temp->val;
            cur = temp->right;
        }
    }
    return 0;
}

```

285.二叉搜索树中的中序后继 #todo

```
TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {  
    // 用迭代的中序遍历方法，然后用一个 bool 型的变量b，初始化为 false，  
    // 进行中序遍历，对于遍历到的节点，首先看如果此时b已经为 true，说明之前遍历到了p，那么此时返回当前节点，  
    // 如果b仍为 false，看遍历到的节点和p是否相同，如果相同，此时将b赋为 true，那么下一个遍历到的节点就能返回了  
    stack<TreeNode *> s;  
    bool is_node_p = false;  
    while(!s.empty() || root)  
    {  
        if (root)  
        {  
            s.push(root);  
            root = root->left;  
        }  
        else  
        {  
            root = s.top(); s.pop();  
            if (is_node_p)  
                return root;  
            if (root == p)  
                is_node_p = true;  
            root = root->right;  
        }  
    }  
    return nullptr;  
}
```

// 解法二

// 首先看根节点值和p节点值、的大小，如果根节点值大，说明p节点肯定在左子树中，那么此时先将 res 赋为 root，然后 root 移到其左子节点，循环的条件是 root 存在，再比较此时 root 值和p节点值的大小，如果还是 root 值大，重复上面的操作，如果p节点值，那么将 root 移到其右子节点，这样当 root 为空时，res 指向的就是p的后继节点

```
TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {  
    TreeNode *res = NULL;  
    while (root) {  
        if (root->val > p->val) {  
            res = root;  
            root = root->left;  
        } else root = root->right;  
    }  
    return res;  
}
```

897.递增顺序搜索树

```
TreeNode* increasingBST(TreeNode* root) {
```

```

TreeNode *head = new TreeNode(-1);
TreeNode *pre = head;
stack<TreeNode *> s;
while(!s.empty() || root)
{
    if (root)
    {
        s.push(root);
        root = root->left;
    }
    else
    {
        root = s.top(); s.pop();
        pre->right = root;
        pre = pre->right;
        pre->left = nullptr;
        root = root->right;
    }
}
return head->right;
}

```

538. 把二叉搜索树转换为累加树

```

int sum = 0;
TreeNode* convertBST_1(TreeNode* root) {
    if (!root) return NULL;
    convertBST(root->right);
    root->val += sum;
    sum = root->val;
    convertBST(root->left);
    return root;
}

TreeNode* convertBST(TreeNode* root) {
    if (!root) return NULL;
    int sum = 0;
    stack<TreeNode*> st;
    TreeNode *p = root;
    while (p || !st.empty()) {
        while (p) {
            st.push(p);
            p = p->right;
        }
        p = st.top(); st.pop();
        p->val += sum;
        sum = p->val;
        p = p->left;
    }
    return root;
}

```


669. 修剪二叉搜索树

450. 删除二叉搜索树中的节点

```
class Solution {
private:
    // 将目标节点（删除节点）的左子树放到 目标节点的右子树的最左面节点的左孩子位置上
    // 并返回目标节点右孩子为新的根节点
    // 是动画里模拟的过程
    TreeNode* deleteOneNode(TreeNode* target) {
        if (target == nullptr) return target;
        if (target->right == nullptr) return target->left;
        TreeNode* cur = target->right;
        while (cur->left) {
            cur = cur->left;
        }
        cur->left = target->left;
        return target->right;
    }
public:
    TreeNode* deleteNode(TreeNode* root, int key) {
        if (root == nullptr) return root;
        TreeNode* cur = root;
        TreeNode* pre = nullptr; // 记录cur的父节点，用来删除cur
        while (cur) {
            if (cur->val == key) break;
            pre = cur;
            if (cur->val > key) cur = cur->left;
            else cur = cur->right;
        }
        if (pre == nullptr) { // 如果搜索树只有头结点
            return deleteOneNode(cur);
        }
        // pre 要知道是删左孩子还是右孩子
        if (pre->left && pre->left->val == key) {
            pre->left = deleteOneNode(cur);
        }
        if (pre->right && pre->right->val == key) {
            pre->right = deleteOneNode(cur);
        }
        return root;
    }
};

TreeNode* deleteNode(TreeNode* root, int key) {
    if (root == nullptr) return root; // 第一种情况：没找到删除的节点，遍历到空节点直接返回了
    if (root->val == key) {
        // 第二种情况：左右孩子都为空（叶子节点），直接删除节点， 返回NULL为根节点
        if (root->left == nullptr && root->right == nullptr) {
            ///! 内存释放
            delete root;
            return nullptr;
        }
    }
}
```

```

// 第三种情况：其左孩子为空，右孩子不为空，删除节点，右孩子补位，返回右孩子为根节点
else if (root->left == nullptr) {
    auto retNode = root->right;
    ///! 内存释放
    delete root;
    return retNode;
}
// 第四种情况：其右孩子为空，左孩子不为空，删除节点，左孩子补位，返回左孩子为根节点
else if (root->right == nullptr) {
    auto retNode = root->left;
    ///! 内存释放
    delete root;
    return retNode;
}
// 第五种情况：左右孩子节点都不为空，则将删除节点的左子树放到删除节点的右子树的最左面节点的左孩子
的位置
// 并返回删除节点右孩子为新的根节点。
else {
    TreeNode* cur = root->right; // 找右子树最左面的节点
    while(cur->left != nullptr) {
        cur = cur->left;
    }
    cur->left = root->left; // 把要删除的节点（root）左子树放在cur的左孩子的位置
    TreeNode* tmp = root; // 把root节点保存一下，下面来删除
    root = root->right; // 返回旧root的右孩子作为新root
    delete tmp; // 释放节点内存（这里不写也可以，但C++最好手动释放一下吧）
    return root;
}
}
if (root->val > key) root->left = deleteNode(root->left, key);
if (root->val < key) root->right = deleteNode(root->right, key);
return root;
}

```

653.两数之和 IV - 输入二叉搜索树

```

bool findTarget(TreeNode* root, int k) {
    unordered_set<int> st;
    return helper(root, k, st);
}
bool helper(TreeNode* node, int k, unordered_set<int>& st) {
    if (!node) return false;
    if (st.count(k - node->val)) return true;
    st.insert(node->val);
    return helper(node->left, k, st) || helper(node->right, k, st);
}

```

700. 二叉搜索树中的搜索

```

class Solution {
public:
    TreeNode* searchBST(TreeNode* root, int val) {
        if (!root) return nullptr;          // 空树
        if (root->val == val) return root;    // 找到目标
        if (val < root->val)
            return searchBST(root->left, val);
        else
            return searchBST(root->right, val);
    }
};

```

701. 二叉搜索树中的插入操作

```

TreeNode* insertIntoBST(TreeNode* root, int val) {
    if (root == NULL) {
        TreeNode* node = new TreeNode(val);
        return node;
    }
    if (root->val > val) root->left = insertIntoBST(root->left, val);
    if (root->val < val) root->right = insertIntoBST(root->right, val);
    return root;
}

class Solution {
private:
    TreeNode* parent;
    void traversal(TreeNode* cur, int val) {
        if (cur == NULL) {
            TreeNode* node = new TreeNode(val);
            if (val > parent->val) parent->right = node;
            else parent->left = node;
            return;
        }
        parent = cur;
        if (cur->val > val) traversal(cur->left, val);
        if (cur->val < val) traversal(cur->right, val);
        return;
    }

public:
    TreeNode* insertIntoBST(TreeNode* root, int val) {
        parent = new TreeNode(0);
        if (root == NULL) {
            root = new TreeNode(val);
        }
        traversal(root, val);
        return root;
    }
};

```

```

class Solution {
public:
    Node* treeToDoublyList(Node* root)
    {
        if(root == NULL)    return NULL;
        inorder(root);
        head->left = pre;    //链表头的前驱指向链表尾
        pre->right = head;   //链表尾的后继指向链表头
        return head;
    }
private:
    Node* pre = NULL;    //前驱节点
    Node* head = NULL;   //双向链表的头节点
    void inorder(Node* cur)
    {
        if(cur == NULL)    return;
        inorder(cur->left);
        //当前前驱节点为空，说明这是双向链表的头节点（树中最左节点）
        if(pre == NULL)
            head = cur;
        //此时已有前驱，说明这是链表中的某个中间节点，将前驱的右指针指向cur
        else
            pre->right = cur;
        //把cur（当前节点）的左指针指向其前驱
        cur->left = pre;
        //当前节点成为前驱
        pre = cur;
        //递归结束后，pre指向链表的尾节点
        inorder(cur->right);
    }
};

//
Node* treeToDoublyList(Node* root)
{
    if (root == nullptr)
        return root;
    stack<Node*> s;
    Node* head = nullptr;
    Node* pre = nullptr;
    while(!s.empty() || root)
    {
        if (root)
        {
            s.push(root);
            root = root->left;
        }
        else
        {

```

```

        root = s.top();
        s.pop();
        if (head == nullptr)
            head = root;
        if (pre)
        {
            pre->right = root;
            root->left = pre;
        }
        pre = root;
        root = root->right;
    }
}
head->left = pre;
pre->right = head;
return head;
}

```

二叉树的重新构建

105. 从前序与中序遍历序列构造二叉树

```

TreeNode *buildTree(vector<int> &preorder, int preStart, int preEnd, vector<int> &inorder,
int inStart, int inEnd)
{
    if (preStart > preEnd || inStart > inEnd )
        return nullptr;

    // 先建立根节点
    TreeNode *root = new TreeNode(preorder[preStart]);
    // 在中序遍历中找到根节点所在位置, 然后就可以确定左右子树的节点数目
    int i = find(inorder.begin(), inorder.end(), preorder[preStart]) - inorder.begin();
    root->left = buildTree(preorder, preStart + 1, preStart + i - inStart, inorder,
inStart, i - 1);
    root->right = buildTree(preorder, preStart + i - inStart + 1, preEnd, inorder, i + 1,
inEnd);

    return root;
}

TreeNode *buildTree(vector<int> &preorder, vector<int> &inorder)
{
    return buildTree(preorder, 0, preorder.size() - 1, inorder, 0, inorder.size() - 1);
}

```

106. 从中序与后序遍历序列构造二叉树

```

TreeNode *buildTree(vector<int> &inorder, int inStart, int inEnd, vector<int> &postorder,
int postStart, int postEnd)
{

```

```

if (postStart > postEnd || inStart > inEnd)
    return nullptr;

TreeNode *root = new TreeNode(postorder[postEnd]);
// 通过根节点下标, 在中序序列中 将左右子树分开
int i = find(inorder.begin(), inorder.end(), postorder[postEnd]) - inorder.begin();
// 注意推导一下下标公式就👉
root->left = buildTree(inorder, inStart, i - 1, postorder, postStart, i + postStart - inStart - 1); // 左子树
root->right = buildTree(inorder, i + 1, inEnd, postorder, i + postStart - inStart, postEnd - 1); // 右子树

return root;
}

TreeNode *buildTree(vector<int> &inorder, vector<int> &postorder)
{
    return buildTree(inorder, 0, inorder.size() - 1, postorder, 0, postorder.size() - 1);
}

```

297. 二叉树的序列化与反序列化

```

class Codec {
public:
    // Encodes a tree to a single string.
    string serialize(TreeNode* root) {
        if (root == NULL)
        {
            return "null";
        }
        else
            return to_string(root->val) + " " + serialize(root->left) + " " +
serialize(root->right);
    }

    TreeNode* de(istringstream& iss)
    {
        TreeNode* root = NULL;
        string word;
        if (iss >> word && word != "null") {
            root = new TreeNode(stoi(word));
            root->left = de(iss);
            root->right = de(iss);
        }
        return root;
    }

    // Decodes your encoded data to tree.
    TreeNode* deserialize(string data) {
        istringstream iss(data);
        return de(iss);
    }
}

```

```
}  
};
```

[606. Construct String from Binary Tree](#)

[1008. Construct Binary Search Tree from Preorder Traversal](#)

[889. Construct Binary Tree from Preorder and Postorder Traversal](#)

区间合并

56. 合并区间

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {  
    if (intervals.size() == 0)  
    {  
        return {};  
    }  
    // 首先将列表中的区间按左端点排序，然后将第一个区间加入到merged数组中  
    // 1: 如果当前区间的左端点在merged数组中最后一个区间的右端点之后，那么他们不会重合，则直接将该区间加入  
    数组merged中  
    // 2: 如果当前区间的左端点在merged数组中最后一个区间的右端点之前，需要更新当前区间的右端点更新数组中  
    merged中最后一个区间的右端点，取二者的最大值  
    sort(intervals.begin(), intervals.end());  
    vector<vector<int>> merged;  
    for(int i = 0; i < intervals.size(); i++)  
    {  
        int left = intervals[i][0];  
        int right = intervals[i][1];  
  
        if(!merged.size() || merged.back()[1] < left) // 条件1  
            merged.push_back({left, right});  
        else // 条件2  
            merged.back()[1] = max(merged.back()[1], right);  
    }  
    return merged;  
}
```

57. 插入区间

```
//用一个变量 cur 来遍历区间，如果当前 cur 区间的结束位置小于要插入的区间的起始位置的话，说明没有重叠，则  
将 cur 区间加入结果 res 中，然后 cur 自增1。  
// 每次用取两个区间起始位置的较小值，和结束位置的较大值来更新要插入的区间，然后 cur 自增1。直到 cur 越界  
或者没有重叠时 while 循环退出。之后将更新好的新区间加入结果 res，然后将 cur 之后的区间再加入结果 res 中  
即可  
vector<vector<int>> insert(vector<vector<int>> &intervals, vector<int> &newInterval)  
{  
    vector<vector<int>> res;  
    int n = intervals.size(), cur = 0;  
    for (int i = 0; i < n; ++i)
```

```

{
    if (intervals[i][1] < newInterval[0]) // 没有重叠的情况一 待插入区间在右边
    {
        res.push_back(intervals[i]);
        ++cur;
    }
    else if (intervals[i][0] > newInterval[1]) // 没有重叠的情况二 待插入区间在左边
    {
        res.push_back(intervals[i]);
    }
    else // 有重叠 左边界去较小值 右边界取最大值
    {
        newInterval[0] = min(newInterval[0], intervals[i][0]);
        newInterval[1] = max(newInterval[1], intervals[i][1]);
    }
}
res.insert(res.begin() + cur, newInterval);
return res;
}

```

632. 最小区间

```

vector<int> smallestRange(vector<vector<int>>& nums) {
    // curMax 表示当前遇到的最大数字，用一个 idx 数组表示每个 list 中遍历到的位置，然后优先队列里面放一个pair，是数字和其所属list组成的对儿。遍历所有的list，将每个 list 的首元素和该 list 序号组成 pair 放入队列中，然后 idx 数组中每个位置都赋值为1，因为0的位置已经放入队列了，所以指针向后移一个位置，还要更新当前最大值 curMax。此时 queue 中是每个 list 各有一个数字，由于是最小堆，所以最小的数字就在队首，再加上最大值 curMax，就可以初始化结果 res 了。然后进行循环，注意这里循环的条件不是队列不为空，而是当某个 list 的数字遍历完了就结束循环，因为范围要 cover 每个 list 至少一个数字
    int curMax = INT_MIN, n = nums.size();
    vector<int> idx(n, 0);
    auto cmp = [](pair<int, int>& a, pair<int, int>& b) {return a.first > b.first;};
    priority_queue<pair<int, int>, vector<pair<int, int>>, decltype(cmp)> q(cmp);
    for (int i = 0; i < n; ++i) {
        q.push({nums[i][0], i});
        idx[i] = 1;
        curMax = max(curMax, nums[i][0]);
    }
    vector<int> res{q.top().first, curMax};
    while (idx[q.top().second] < nums[q.top().second].size()) {
        int t = q.top().second; q.pop();
        q.push({nums[t][idx[t]], t});
        curMax = max(curMax, nums[t][idx[t]]);
        ++idx[t];
        if (res[1] - res[0] > curMax - q.top().first) {
            res = {q.top().first, curMax};
        }
    }
    return res;
}

```


986. 区间列表的交集

栈

20. 有效的括号

```
bool isValid(string s) {  
    // 用一个栈，开始遍历输入字符串，如果当前字符为左半边括号时，则将其压入栈中，如果遇到右半边括号时，  
    // 若此时栈为空，则直接返回 false，如不为空，则取出栈顶元素，若为对应的左半边括号，则继续循环，反之返回  
    false  
    stack<char> st;  
    for(int i = 0; i < s.size(); i++)  
    {  
        if (s[i] == '(' || s[i] == '[' || s[i] == '{')  
            st.push(s[i]);  
        else{  
            if (st.empty()) return false;  
            if (s[i] == ')' && st.top() != '(') return false;  
            if (s[i] == ']' && st.top() != '[') return false;  
            if (s[i] == '}' && st.top() != '{') return false;  
            st.pop();  
        }  
    }  
    return st.empty();  
}
```

32. 最长有效括号

```
#include <stack>  
  
class Solution {  
public:  
    int longestValidParentheses(string s) {  
        stack<int> st;          // 用于跟踪左括号的索引  
        int maxLen = 0;        // 用于记录最长有效括号子串的长度  
        st.push(-1);           // 将虚拟起始位置 -1 推入栈中  
  
        for (int i = 0; i < s.length(); i++) {  
            if (s[i] == '(') {  
                st.push(i); // 如果当前字符是左括号，将其索引推入栈中  
            } else {  
                st.pop();    // 如果当前字符是右括号，尝试与栈顶元素匹配  
                if (st.empty()) {  
                    st.push(i); // 如果栈为空，说明当前右括号无法匹配，将当前位置作为新的起始点  
                }  
            }  
            maxLen = max(maxLen, i - st.top());  
        }  
        return maxLen;  
    }  
};
```

```

        } else {
            maxLen = max(maxLen, i - st.top()); // 计算有效括号子串的长度并更新最大长度
        }
    }
}
return maxLen;
};

```

这个算法的主要思想是使用栈来跟踪左括号的索引位置，如果遇到右括号，则尝试与栈顶元素匹配。如果栈不为空，说明有匹配的括号，计算当前位置与栈顶位置之间的距离，更新最大长度。如果栈为空，说明当前右括号无法与前面的括号匹配，将当前位置作为新的起始点。

举例来说，对于字符串 "(()())"，栈在处理字符 ')' 时的状态变化如下：

- 遇到 '(' 时，将索引 0 推入栈中：栈内容为 [-1, 0]
- 遇到 '(' 时，将索引 1 推入栈中：栈内容为 [-1, 0, 1]
- 遇到 ')' 时，弹出栈顶元素，计算有效括号子串的长度：当前子串的长度为 2，更新 maxLen 为 2
- 遇到 '(' 时，将索引 4 推入栈中：栈内容为 [-1, 0, 4]
- 遇到 ')' 时，弹出栈顶元素，计算有效括号子串的长度：当前子串的长度为 4，更新 maxLen 为 4
- 最终返回 maxLen = 4，表示最长有效括号子串的长度为 4。

这个算法的时间复杂度为 $O(n)$ ，其中 n 是字符串的长度，因为它只需要遍历一次字符串。希望这个详细的解释和注释有助于理解算法的工作原理。

150.逆波兰表达式求值

```

int evalRPN(vector<string>& tokens) {
    // 前向后遍历数组，遇到数字则压入栈中，遇到符号，则把栈顶的两个数字拿出来运算，
    // 把结果再压入栈中，直到遍历完整个数组，栈顶数字即为最终答案
    stack<int> st;
    for(int i = 0; i < tokens.size(); i++)
    {
        if (tokens[i] != "+" && tokens[i] != "-" && tokens[i] != "*" && tokens[i] != "/" )
            st.push(stoi(tokens[i]));
        else
        {
            int val_2 = st.top();st.pop();
            int val_1 = st.top();st.pop();
            if (tokens[i] == "+"){
                st.push(val_1 + val_2);
            }
            if (tokens[i] == "-"){
                st.push(val_1 - val_2);
            }
            if (tokens[i] == "*"){
                st.push(val_1 * val_2);
            }
            if (tokens[i] == "/"){
                st.push(val_1 / val_2);
            }
        }
    }
}

```

```

    }
}
return st.top();
}

```

232. 用栈实现队列

```

class MyQueue {
private:
    stack<int> st;
public:
    /** Initialize your data structure here. */
    MyQueue() {}

    /** Push element x to the back of queue. */
    // 只要我们在插入元素的时候每次都从前面插入即可，比如如果一个队列是1,2,3,4，那么我们在栈中保存为
    // 4,3,2,1，那么返回栈顶元素1，也就是队列的首元素，则问题迎刃而解。
    // 所以此题的难度是push函数，我们需要一个辅助栈tmp，把s的元素也逆着顺序存入tmp中，此时加入新元素x，
    // 再把tmp中的元素存回来，这样就是我们要的顺序了，其他三个操作也就直接调用栈的操作即可
    void push(int x) {
        stack<int> tmp;
        while (!st.empty()) {
            tmp.push(st.top()); st.pop();
        }
        st.push(x);
        while (!tmp.empty()) {
            st.push(tmp.top()); tmp.pop();
        }
    }

    /** Removes the element from in front of queue and returns that element. */
    int pop() {
        int val = st.top(); st.pop();
        return val;
    }

    /** Get the front element. */
    int peek() {
        return st.top();
    }

    /** Returns whether the queue is empty. */
    bool empty() {
        return st.empty();
    }
};

```

155. 最小栈 #todo

```

class MinStack {
public:
    // 一个栈来按顺序存储 push 进来的数据，另一个用来存出现过的最小值
    stack<int> s1, s2; // s2 辅助栈 用来保存最小元素
    /** initialize your data structure here. */
    MinStack() {

    }

    void push(int x)
    {
        s1.push(x);
        if (s2.empty() || x <= s2.top()) s2.push(x);
    }

    void pop() {
        if (s1.top() == s2.top()) s2.pop();
        s1.pop();
    }

    int top()
    {
        return s1.top();
    }

    int getMin()
    {
        return s2.top();
    }
};

/**
 * Your MinStack object will be instantiated and called as such:
 * MinStack* obj = new MinStack();
 * obj->push(x);
 * obj->pop();
 * int param_3 = obj->top();
 * int param_4 = obj->getMin();
 */

```

165. 比较版本号

```

int compareVersion(string version1, string version2) {
    int n1 = version1.size(), n2 = version2.size();
    int i = 0, j = 0, d1 = 0, d2 = 0;
    string v1, v2;
    while (i < n1 || j < n2) {
        while (i < n1 && version1[i] != '.') {
            v1.push_back(version1[i++]);
        }
        d1 = atoi(v1.c_str());
    }
}

```

```

        while (j < n2 && version2[j] != '.') {
            v2.push_back(version2[j++]);
        }
        d2 = atoi(v2.c_str());
        if (d1 > d2) return 1;
        else if (d1 < d2) return -1;
        v1.clear(); v2.clear();
        ++i; ++j;
    }
    return 0;
}

```

295. 数据流的中位数 # todo

```

class MedianFinder {
public:
    // 大堆保存右半段较大的数字，小堆保存左半段较小的数组
    // 大堆里面的数据是从小到大
    // 存到大堆里的数先取反再存，这样由大到小存下来的顺序就是实际上我们想要的从小到大的顺序
    priority_queue<long> small, large;
    /** initialize your data structure here. */
    MedianFinder() {

    }

    void addNum(int num) {
        small.push(num);
        large.push(-small.top());
        small.pop();
        if (small.size() < large.size()) {
            small.push(-large.top());
            large.pop();
        }
    }

    double findMedian() {
        return small.size() > large.size() ? small.top() : 0.5 * (small.top() -
        large.top());
    }
};

```

394. 字符串解码 # todo

```

string decodeString(string s)
{
    // 需要用 stack 来辅助运算，用两个 stack，一个用来保存个数，一个用来保存字符串，
    // 遍历输入字符串，如果遇到数字，我们更新计数变量 cnt；
    // 如果遇到左括号，我们把当前 cnt 压入数字栈中，把当前t压入字符串栈中；
}

```

```

// 如果遇到右括号时，我们取出数字栈中顶元素，存入变量k，然后给字符串栈的顶元素循环加上k个t字符串，然后
取出顶元素存入字符串t中；
// 如果遇到字母，我们直接加入字符串t中即可，参见代码如下：
string t = "";
stack<int> s_num;
stack<string> s_str;
int cnt = 0;
for (int i = 0; i < s.size(); ++i) {
    if (s[i] >= '0' && s[i] <= '9') { // 如果遇到数字，我们更新计数变量 cnt；
        cnt = 10 * cnt + s[i] - '0';
    } else if (s[i] == '[') { // 遇到左括号，我们把当前 cnt 压入数字栈中，把当前t压入字符串栈
中；
        s_num.push(cnt);
        s_str.push(t);
        cnt = 0; t.clear();
    } else if (s[i] == ']') { // 取出数字栈中顶元素，存入变量k，然后给字符串栈的顶元素循环加上k
个t字符串，然后取出顶元素存入字符串t中；
        int k = s_num.top();
        s_num.pop();
        for (int j = 0; j < k; ++j)
            s_str.top() += t;
        t = s_str.top();
        s_str.pop();
    } else { // 遇到字母，我们直接加入字符串t中即可
        t += s[i];
    }
}
return s_str.empty() ? t : s_str.top();
}

```

856. 括号的分數

```

class Solution {
public:
    // 遍历字符串，遇到左括号时，将0入栈，表示当前层级的分数。
    // 遇到右括号时，弹出栈顶元素，如果栈顶元素是0，说明是一对括号，将0替换成1；如果栈顶元素不是0，说明是
(A)，将弹出的分数乘以2。
    // 如果栈不为空，说明是AB，将弹出的分数加到栈顶元素上；如果栈为空，说明是最外层的括号，将分数入栈。
    // 最终，栈中的所有分数相加即为结果。
    int scoreOfParentheses(string s) {
        stack<int> scores;
        for (char c : s) {
            if (c == '(') {
                scores.push(0); // 遇到左括号，将0入栈，表示当前层级的分数
            } else {
                int score = scores.top(); // 弹出栈顶元素，即当前层级的分数
                scores.pop();

                if (score == 0) {
                    score = 1; // 如果栈顶元素为0，说明是一对括号，将0替换成1
                } else {

```

```

        score *= 2; // 如果栈顶元素不为0, 说明是(A), 将弹出的分数乘以2
    }

    if (!scores.empty()) {
        scores.top() += score; // 如果栈不为空, 说明是AB, 将弹出的分数加到栈顶元素上
    } else {
        scores.push(score); // 如果栈为空, 说明是最外层的括号, 将分数入栈
    }
}

return scores.top(); // 栈中的所有分数相加即为最终结果
}
};

```

[480. Sliding Window Median](#)

[剑指 Offer 09. 用两个栈实现队列](#)

hash 相关

[1. 两数之和](#)

```

// 使用哈希表来记录已经遍历过的元素及其下标。
// 在遍历过程中, 对于每个元素, 计算其与目标值的差值, 然后检查差值是否在哈希表中,
// 如果在, 说明找到了答案, 返回答案的下标; 如果不在, 将当前元素及其下标存入哈希表
vector<int> twoSum(vector<int>& nums, int target) {
    vector<int> res;
    // 使用哈希表来存储已经遍历过的元素及其下标
    unordered_map<int, int> hash;
    for (int i = 0; i < nums.size(); i++) {
        // 计算当前元素与目标值的差值
        int complement = target - nums[i];

        // 如果差值在哈希表中存在, 说明找到了答案, 返回答案的下标
        if (hash.find(complement) != hash.end()) {
            return {i, hash[complement]};
        }
        // 将当前元素及其下标存入哈希表
        hash[nums[i]] = i;
    }
    return res;
}

```

[49. 字母异位词分组 #todo](#)

```

vector<vector<string>> groupAnagrams(vector<string>& strs) {
    // 使用哈希表存储字母异位词组
    unordered_map<string, vector<string>> anagramGroups;

```

```

// 遍历字符串数组
for (const string& str : strs) {
    // 将字符串排序，作为哈希表的键
    string key = str;
    sort(key.begin(), key.end());
    // 将字符串加入对应的字母异位词组
    anagramGroups[key].push_back(str);
}

// 将哈希表的值转换为结果数组
vector<vector<string>> res;
for (auto& group : anagramGroups) {
    res.push_back(group.second);
}
return res;
}

```

128. 最长连续序列 #TODO

```

// 遍历数组中的每个数字，对于每个数字，判断它是否是一个连续序列的起始数字。
// 如果是起始数字，就循环查找以该数字为起点的连续序列的长度，并更新最长连续序列的长度
int longestConsecutive(vector<int>& nums) {
    // 如果数组为空，直接返回 0
    if (nums.empty()) {
        return 0;
    }
    // 使用哈希集合存储数组中的所有数字
    unordered_set<int> numSet(nums.begin(), nums.end());
    int res = 0;
    // 遍历数组中的每个数字
    for (int num : nums) {
        // 当前数字是连续序列的起始数字
        if (numSet.find(num - 1) == numSet.end()) {
            int cur_num = num;
            int cur_res = 1;

            // 循环找到当前数字的连续序列长度
            while (numSet.find(cur_num + 1) != numSet.end()) {
                numSet.erase(cur_num+1); // 找到下一个数 就删除 会加速
                cur_num++;
                cur_res++;
            }
            // 更新最长连续序列的长度
            res = max(res, cur_res);
        }
    }
    return res;
}

```


220.存在重复元素 III

242.有效的字母异位词

```
bool isAnagram(string s, string t) {
    if (s.length() != t.size())
        return false;
    vector<int> hash(256, 0);
    for(int i = 0; i < s.size(); i++)
    {
        hash[s[i]] += 1;
    }
    for(int i = 0; i < t.size(); i++)
    {
        hash[t[i]] --;
        if (hash[t[i]] < 0)
            return false;
    }
    return true;
}
```

349.两个数组的交集

```
vector<int> intersection(vector<int>& nums1, vector<int>& nums2) {
    unordered_set<int> res;
    unordered_set<int> nums1_set(nums1.begin(), nums1.end());
    for(int i = 0; i < nums2.size(); i++)
    {
        if (nums1_set.find(nums2[i]) != nums1_set.end())
            res.insert(nums2[i]);
    }
    return vector<int>(res.begin(), res.end());
}

vector<int> intersection(vector<int>& nums1, vector<int>& nums2) {
    vector<int> res;
    int i = 0, j = 0;
    sort(nums1.begin(), nums1.end());
    sort(nums2.begin(), nums2.end());
    while (i < nums1.size() && j < nums2.size()) {
        if (nums1[i] < nums2[j]) ++i;
        else if (nums1[i] > nums2[j]) ++j;
        else {
            if (res.empty() || res.back() != nums1[i]) {
                res.push_back(nums1[i]);
            }
            ++i; ++j;
        }
    }
}
```

```

    }
    return res;
}

```

380. O(1) 时间插入、删除和获取随机元素

```

class RandomizedSet {
private:
    // 用一维数组和一个 HashMap，其中数组用来保存数字，
    // HashMap 用来建立每个数字和其在数组中的位置之间的映射
    vector<int> nums;
    unordered_map<int, int> m;
public:
    RandomizedSet() {}
    // 先看这个数字是否已经在 HashMap 中存在，
    // 如果存在的话直接返回 false，不存在的话，将其插入到数组的末尾，然后建立数字和其位置的映射
    bool insert(int val) {
        if (m.count(val))
            return false;
        nums.push_back(val);
        m[val] = nums.size() - 1;
        return true;
    }
    // 先判断其是否在 HashMap 里，如果没有，直接返回 false
    // 数组并不是，为了使数组删除也能常数级，
    // 实际上将要删除的数字和数组的最后一个数字调换个位置，然后修改对应的 HashMap 中的值，这样只需要删除
    // 数组的最后一个元素即可，保证了常数时间内的删除
    bool remove(int val) {
        if (!m.count(val))
            return false;
        int last = nums.back();
        m[last] = m[val];
        nums[m[val]] = last;

        nums.pop_back();
        m.erase(val);
        return true;
    }
    int getRandom() {
        return nums[rand() % nums.size()];
    }
};

```

前K大的数模式HEAP

采用priority queue 或者 说在python 中的heapq

求top k 采用最小堆（默认）

采用最大堆的时候可以采用push 负的值

215. 数组中的第K个最大元素

难度中等1093

```
class Solution {
private:
    void heapInsert(vector<int> &arr, int index, int value)
    {
        arr[index] = value;
        while(index != 0)
        {
            int parent = (index-1) / 2; // 获取父节点

            if(arr[parent] > arr[index])
            {
                int temp = arr[parent];
                arr[parent] = arr[index];
                arr[index] = temp;

                index = parent;
            }
            else
            {
                break;
            }
        }
    }

    void heapify(vector<int> &arr, int index, int size)
    {
        int left = 2 * index + 1;
        while(left < size)
        {
            int smallest = left + 1 < size && arr[left+1] < arr[left] ? left + 1: left ; //
            //smallest记录左右子树中较小的那个
            if (arr[smallest] > arr[index])
                break;

            int temp = arr[smallest];
            arr[smallest] = arr[index];
            arr[index] = temp;

            index = smallest;
            left = 2 * index + 1;
        }
    }

    int partition(vector<int>& nums, int left, int right)
    {
        auto pivot = nums[left];
        int i = left;
        int j = right;
```

```

while (i < j) {
    while (j > i && nums[j] >= pivot) {
        j--; // find first smaller than pivot from right
    }
    nums[i] = nums[j];
    while (i < j && nums[i] <= pivot) {
        i++; // find first larger than pivot from left
    }
    nums[j] = nums[i];
}
nums[i] = pivot;

return i;
}

```

public:

// 解法一：利用堆排序

```

int findKthLargest(vector<int>& nums, int k) {
    if (nums.empty() || nums.size() < k)
        return 0;

    // int *heap = (int *)malloc(sizeof(int) * k);
    vector<int> heap(k, 0);

    for(int i = 0; i < k; i++)
    {
        heapInsert(heap, i, nums[i]);
    }

    for(int j = k; j < nums.size(); j++)
    {
        if (nums[j] > heap[0])
        {
            heap[0] = nums[j];
            heapify(heap, 0, k);
        }
    }
    return heap[0];
}

```

// 解法二：利用快速排序来做

```

int findKthLargest(vector<int>& nums, int k) {
    int left = 0;
    int right = nums.size() - 1;
    while (left <= right) {
        int mid = partition(nums, left, right);
        int target = (nums.size() - k);
        if (mid == target) {
            return nums[mid];
        } else if (mid < target) {
            left = mid + 1;
        } else {

```

```

        right = mid - 1;
    }
}
return -1;
}
};

class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        int left = 0, right = nums.size() - 1;
        while (true) {
            int pos = partition(nums, left, right);
            if (pos == k - 1) return nums[pos];
            if (pos > k - 1) right = pos - 1;
            else left = pos + 1;
        }
    }
    int partition(vector<int> &nums, int left, int right)
    {
        int small = left - 1;
        for(int i = left; i < right; i++)
        {
            if (nums[i] > nums[right]) // 从大到小
                swap(nums[i], nums[++small]);
        }
        swap(nums[++small], nums[right]);
        return small;
    }
};

```

347. 前 K 个高频元素

```

vector<int> topKFrequent(vector<int>& nums, int k)
{
    unordered_map<int, int> m;
    priority_queue<pair<int, int>> q;
    vector<int> res;
    // 先统计每个数字出现的次数
    for (auto a : nums)
        ++m[a];
    // 然后将次数 和数字 放到大根堆里面去, 然后从大根堆里面一次弹出k个数字
    for (auto it : m) q.push({it.second, it.first});
    for (int i = 0; i < k; ++i)
    {
        res.push_back(q.top().second);
        q.pop();
    }
    return res;
}

```

373. 查找和最小的 K 对数字

```
#todo priority_queue 用法 https://www.cnblogs.com/grandyang/p/5653127.html
class Solution {
public:
    struct cmp {
        bool operator() (vector<int> &a, vector<int> &b) {
            return a[0] + a[1] < b[0] + b[1];
        }
    };
    vector<vector<int>> kSmallestPairs(vector<int>& nums1, vector<int>& nums2, int k) {
        vector<vector<int>> res;
        priority_queue<vector<int>, vector<vector<int>>, cmp> q;
        for (int i = 0; i < min((int)nums1.size(), k); ++i) {
            for (int j = 0; j < min((int)nums2.size(), k); ++j) {
                if (q.size() < k) {
                    q.push({nums1[i], nums2[j]});
                } else if (nums1[i] + nums2[j] < q.top()[0] + q.top()[1]) {
                    q.push({nums1[i], nums2[j]}); q.pop();
                }
            }
        }
        while (!q.empty()) {
            res.push_back({q.top()[0], q.top()[1]}); q.pop();
        }
        return res;
    }
};
```

703. 数据流中的第 K 大元素

```
class KthLargest {
private:
    // 使用一个最小堆来保存前k个数字，当再加入新数字后，最小堆会自动排序，
    // 然后把排序后的最小的那个数字去除，则堆中还是k个数字，返回的时候只需返回堆顶元素即可
    priority_queue<int, vector<int>, greater<int>> q;
    int K;
public:
    KthLargest(int k, vector<int>& nums) {
        for (int num : nums)
        {
            q.push(num);
            if (q.size() > k)
                q.pop();
        }
        K = k;
    }

    int add(int val) {
        q.push(val);
        if (q.size() > K)
```

```

        q.pop();
        return q.top();
    }
};

```

前缀和

前缀和本质上是在一个list当中，用 $O(N)$ 的时间提前算好从第0个数字到第i个数字之和，在后续使用中可以在 O

(1) 时间内计算出第i到第j个数字之和，一般很少单独作为一道题出现，而是很多题目中的用到的一个小技巧

53. 最大子数组和

```

int maxSubArray(vector<int>& nums) {
    if (nums.empty())
        return 0;
    int cur_sum = 0;
    int res = INT_MIN;
    for(int i = 0; i < nums.size(); i++)
    {
        cur_sum += nums[i];
        res = max(res, cur_sum);
        // 因为只有在 cur_sum 为正数时才有可能对后续的子数组和产生正面影响
        cur_sum = cur_sum > 0 ? cur_sum : 0;
    }
    return res;
}

```

面试题 17.24. 最大子矩阵

```

vector<int> getMaxMatrix(vector<vector<int>>& matrix) {
    int m=matrix.size(), n=matrix[0].size();
    int maxMat=INT32_MIN;
    vector<int> ans(4, -1);

    for(int r1=0;r1<m;++r1){//遍历起始行
        vector<int> nums(n); //矩阵某两行间元素按列求和
        for(int r2=r1;r2<m;++r2){ //遍历结束行

            //最大字段和问题
            int dp=0, start=-1;
            for(int i=0;i<n;++i){ //遍历和数组，实际上是边遍历边完成求和
                nums[i]+=matrix[r2][i]; //将新的一行中第i个元素加到前面若干行在位置i的和
                if(dp>0){ //前面的字段有和为正，可以把前面一部分也带上
                    dp+=nums[i];
                }
                else{ //前面一段为负，拖后腿直接抛弃
                    dp=nums[i];
                    start=i;
                }
            }
            if(dp>maxMat){ //不断记录较好的结果

```

```

        maxMat=dp;
        ans[0]=r1;
        ans[1]=start;
        ans[2]=r2;
        ans[3]=i;
    }
}
}
return ans;
}

```

303.区域和检索 - 数组不可变

```

class NumArray {
private:
    vector<int> dp;
public:
    // 建立一个累计和的数组 dp, 其中 dp[i] 表示 [0, i] 区间的数字之和,
    // 那么 [i,j] 就可以表示为 dp[j]-dp[i-1]
    // 注意一下当 i=0 时, 直接返回 dp[j] 即可
    NumArray(vector<int> &nums) {
        dp = nums;
        for (int i = 1; i < nums.size(); ++i)
        {
            dp[i] += dp[i - 1];
        }
    }
    int sumRange(int i, int j) {
        return i == 0 ? dp[j] : dp[j] - dp[i - 1];
    }
};

```

304. 二维区域和检索 - 矩阵不可变

O				
	G			F
		A		B
	E	C		D

$$S(A,D)=S(O,D)-S(O,E)-S(O,F)+S(O,G)$$

```

class NumMatrix {
private:
    vector<vector<int>> > dp;
public:
    // 建立一个累计区域和的数组，然后根据边界值的加减法来快速求出给定区域之和。
    // 维护一个二维数组dp，其中dp[i][j]表示累计区间(0, 0)到(i, j)这个矩形区间所有的数字之和，那么此时
    如果我们想要快速求出(r1, c1)到(r2, c2)的矩形区间时，
    // 只需dp[r2][c2] - dp[r2][c1 - 1] - dp[r1 - 1][c2] + dp[r1 - 1][c1 - 1]即可，下面的代码
    中我们由于用了辅助列和辅助行
    NumMatrix(vector<vector<int>> &matrix) {
        if (matrix.empty() || matrix[0].empty()) return;
        dp.resize(matrix.size() + 1, vector<int>(matrix[0].size() + 1, 0));
        for (int i = 1; i <= matrix.size(); ++i) {
            for (int j = 1; j <= matrix[0].size(); ++j) {
                dp[i][j] = dp[i][j - 1] + dp[i - 1][j] - dp[i - 1][j - 1] + matrix[i - 1]
[j - 1];
            }
        }
    }
    int sumRegion(int row1, int col1, int row2, int col2) {
        return dp[row2 + 1][col2 + 1] - dp[row1][col2 + 1] - dp[row2 + 1][col1] + dp[row1]
[col1];
    }
};

```

325.最大子数组之和为k

```

int getMaxLen(vector<int> &nums, int k)
{
    map<int,int> hashmap;
    map<int,int>::iterator it;;

    if (nums.empty())
        return 0;
    hashmap[0] = -1;
    int len = 0;
    int sum = 0;

    for(int i = 0; i < nums.size(); i++)
    {
        sum += nums[i];
        it = hashmap.find(sum - k);
        if(it != hashmap.end())
        {
            len = max(i - hashmap[sum-k], len);
        }
        if (it == hashmap.end())
        {
            hashmap[sum] = i;
        }
    }
    return len;
}

class Solution {
public:
    int maxSubArrayLen(vector<int>& nums, int k) {
        int sum = 0, res = 0;
        unordered_map<int, int> m;
        for (int i = 0; i < nums.size(); ++i) {
            sum += nums[i];
            if (sum == k) res = i + 1;
            else if (m.count(sum - k)) res = max(res, i - m[sum - k]);
            if (!m.count(sum)) m[sum] = i;
        }
        return res;
    }
};

```

523. 连续的子数组和

```

bool checkSubarraySum(vector<int>& nums, int k) {
    // 遇到这种求子数组或者子矩阵之和的题，应该不难想到要建立累加和数组或者累加和矩阵来做
    // 当前的累加和除以k得到的余数在 map 中已经存在了，那么说明之前必定有一段子数组和可以整除k
    // 需要注意的是k为0的情况，由于无法取余，就把当前累加和放入 map 中
    int n = nums.size(), sum = 0;
    // 余数和当前位置之间的映射

```

```

unordered_map<int, int> m{{0,-1}};
for (int i = 0; i < n; ++i) {
    sum += nums[i];
    int t = (k == 0) ? sum : (sum % k);
    if (m.count(t)) {
        if (i - m[t] > 1) return true;
    } else m[t] = i;
}
return false;
}

```

525.连续数组

```

int findMaxLength(vector<int>& nums) {
    unordered_map<int, int> m;
    int res = 0;
    int cur_sum = 0;
    // 可以通过将 0 视为 -1, 然后使用前缀和 (prefix sum) 和哈希表来解决
    // HashMap 初始化一个 0 -> -1 的映射, 这是为了当 sum 第一次出现0的时候, 即这个子数组是从原数组的起始位置开始, 需要计算这个子数组的长度,
    // 而不是建立当前子数组之和 sum 和其结束位置之间的映射。比如就拿例子1来说, nums = [0, 1], 当遍历0的时候, sum = -1, 此时建立 -1 -> 0 的映射,
    // 当遍历到1的时候, 此时 sum = 0 了, 若 HashMap 中没有初始化一个 0 -> -1 的映射, 此时会建立 0 -> 1 的映射, 而不是去更新这个满足题意的子数组的长度
    m[0] = -1;

    for(int i = 0; i < nums.size(); i++)
    {
        cur_sum += (nums[i] == 1) ? 1 : -1;
        if (m.count(cur_sum))
        {
            res = max(res, i - m[cur_sum]);
        }
        else
            m[cur_sum] = i;
    }
    return res;
}

```

560. 和为 K 的子数组

```

int subarraySum(vector<int>& nums, int k)
{
    unordered_map<int, int> hasSum; // 记录每个累加和出现的次数
    int cur_cum = 0;
    hasSum[0] = 1; // cur_cum 刚好为k的时候, 那么数组从起始到当前位置的这段子数组的和就是k, 满足题意, 如果 HashMap 中事先没有 m[0] 项的话
    int res = 0;
    for(int i = 0; i < nums.size(); i++)
    {

```

```

        cur_cum += nums[i];
        // 如果能在hasSum中找到 说明在i 位置至少存在一个位置j 使得[0...j]的累加和为sum-k, 那么从
        [j+1...i]的累加和就是k了.
        // 所以有可能不只一个j满足条件,用hasSum记录在i位置之前出现多少j满足条件
        if (hasSum.count(cur_cum - k))
            res += hasSum[cur_cum-k];
        hasSum[cur_cum] ++;
    }
    return res;
}

```

862. 和至少为 K 的最短子数组

```

int shortestSubarray(vector<int>& nums, int k) {
    int n = nums.size(), res = INT_MAX;
    long sum = 0;
    //最小堆, 里面放一个数对儿, 由区间和跟其结束位置组成。遍历数组中所有的数字, 累加到 sum, 表示区间 [0,
    i] 内数字和, 判断一下若 sum 大于等于k, 则用 i+1 更新结果 res。然后用一个 while 循环, 看 sum 和堆顶元素
    的差值, 若大于等于k, 移除堆顶元素并更新结果 res。循环退出后将当前 sum 和i组成数对儿加入最小堆, 最后看若结
    果 res 还是整型最大值, 返回 -1, 否则返回结果 res
    priority_queue<pair<long, int>, vector<pair<long, int>>, greater<pair<long, int>>> pq;
    for (int i = 0; i < n; ++i) {
        sum += nums[i];
        if (sum >= k) res = min(res, i + 1);
        while (!pq.empty() && sum - pq.top().first >= k) {
            res = min(res, i - pq.top().second);
            pq.pop();
        }
        pq.push({sum, i});
    }
    return res == INT_MAX ? -1 : res;
}

class Solution {
public:
    int shortestSubarray(vector<int>& A, int K) {
        int n = A.size(), res = INT_MAX;
        deque<int> dq;
        vector<long> sums(n + 1);
        for (int i = 1; i <= n; ++i) sums[i] = sums[i - 1] + A[i - 1];
        for (int i = 0; i <= n; ++i) {
            while (!dq.empty() && sums[i] - sums[dq.front()] >= K) {
                res = min(res, i - dq.front());
                dq.pop_front();
            }
            while (!dq.empty() && sums[i] <= sums[dq.back()]) {
                dq.pop_back();
            }
            dq.push_back(i);
        }
    }
}

```

```

        return res == INT_MAX ? -1 : res;
    }
};

```

1031. 两个非重叠子数组的最大和

```

int maxSumTwoNoOverlap(vector<int>& nums, int L, int M) {
    // 建立累加和数组，这里可以直接覆盖A数组，然后定义 Lmax 为在最后M个数字之前的长度为L的子数组的最大数字之和，同理，Mmax 表示在最后L个数字之前的长度为M的子数组的最大数字之和。结果 res 初始化为前 L+M 个数字之和，然后遍历数组，从 L+M 开始遍历，先更新 Lmax 和 Mmax，其中 Lmax 用 A[i - M] - A[i - M - L] 来更新，Mmax 用 A[i - L] - A[i - M - L] 来更新。然后取 Lmax + A[i] - A[i - M] 和 Mmax + A[i] - A[i - L] 之间的较大值来更新结果 res 即可

    for (int i = 1; i < nums.size(); ++i) {
        nums[i] += nums[i - 1];
    }
    int res = nums[L + M - 1], Lmax = nums[L - 1], Mmax = nums[M - 1];
    for (int i = L + M; i < nums.size(); ++i) {
        Lmax = max(Lmax, nums[i - M] - nums[i - M - L]);
        Mmax = max(Mmax, nums[i - L] - nums[i - M - L]);
        res = max(res, max(Lmax + nums[i] - nums[i - M], Mmax + nums[i] - nums[i - L]));
    }
    return res;
}

```

前缀树

208.实现 Trie (前缀树)

```

class TrieNode {
public:
    TrieNode *child[26];
    bool isWord;
    TrieNode(): isWord(false) {
        for (auto &a : child) a = nullptr;
    }
};

class Trie {
public:
    Trie() {
        root = new TrieNode();
    }
    void insert(string s) {
        TrieNode *p = root;
        for (auto &a : s) {

```

```

        int i = a - 'a';
        if (!p->child[i]) p->child[i] = new TrieNode();
        p = p->child[i];
    }
    p->isWord = true;
}

bool search(string key) {
    TrieNode *p = root;
    for (auto &a : key) {
        int i = a - 'a';
        if (!p->child[i]) return false;
        p = p->child[i];
    }
    return p->isWord;
}

bool startsWith(string prefix) {
    TrieNode *p = root;
    for (auto &a : prefix) {
        int i = a - 'a';
        if (!p->child[i]) return false;
        p = p->child[i];
    }
    return true;
}

private:
    TrieNode* root;
};

```

贪心

45. 跳跃游戏 II 好像不是dp #todo 20210415

```

int jump(vector<int>& nums) {
    // 贪的是一个能到达的最远范围，遍历当前跳跃能到的所有位置，然后根据该位置上的跳力来预测下一步能跳到的最远距离，贪出一个最远的范围，一旦当这个范围到达末尾时，当前所用的步数一定是最小步数
    // 两个变量 cur 和 pre 分别来保存当前的能到达的最远位置和之前能到达的最远位置，只要 cur 未达到最后一个位置则循环继续，pre 先赋值为 cur 的值，表示上一次循环后能到达的最远位置，如果当前位置i小于等于 pre，说明还是在上一跳能到达的范围内，根据当前位置加跳力来更新 cur，更新 cur 的方法是比较当前的 cur 和 i + A[i] 之中的较大值
    int res = 0, n = nums.size(), i = 0, cur = 0;
    while (cur < n - 1) {
        ++res;
        int pre = cur;
        for (; i <= pre; ++i) {
            cur = max(cur, i + nums[i]);
        }
        if (pre == cur) return -1; // May not need this
    }
    return res;
}

```

55. 跳跃游戏 todo

```
bool canJump(vector<int>& nums) {  
    // 只对最远能到达的位置感兴趣，所以维护一个变量 reach，表示最远能到达的位置，初始化为0。  
    // 而所有小于等于 reach 的位置都可以通过连续跳跃到达的，则只要 reach 大于等于最后一位置，就说明可以跳到最后一个位置  
    // 一次遍历数组中的每个位置，若这个位置小于等于 reach，说明是在可以到达的范围内，而从该位置可以到达的最大范围就是 i + nums[i]，用这个最大范围来更新 reach  
  
    //  
    int n = nums.size(), reach = 0;  
    for (int i = 0; i < n; ++i) {  
        if (i > reach)  
            return false;  
        if (reach >= n - 1)  
            return true;;  
        reach = max(reach, i + nums[i]);  
    }  
    return false;  
}
```

134. 加油站

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {  
    // 能走完整个环的前提是gas的总量要大于cost的总量，这样才会有起点的存在  
    // 假设开始设置起点start = 0，并从这里出发，如果当前的gas值大于cost值，就可以继续前进，此时到下一个站点，剩余的gas加上当前的gas再减去cost，  
    // 看是否大于0，若大于0，则继续前进。当到达某一站点时，若这个值小于0了，  
    // 则说明从起点到这个点中间的任何一个点都不能作为起点，则把起点设为下一个点，继续遍历  
    int total = 0, sum = 0, start = 0;  
    for (int i = 0; i < gas.size(); ++i) {  
        total += gas[i] - cost[i];  
        sum += gas[i] - cost[i];  
        if (sum < 0) {  
            start = i + 1;  
            sum = 0;  
        }  
    }  
    return (total < 0) ? -1 : start;  
}
```

135. 分发糖果

```
int candy(vector<int>& ratings)  
{
```

// 先来看看两遍遍历的解法，首先初始化每个人一个糖果，然后这个算法需要遍历两遍，第一遍从左向右遍历，如果右边的小盆友的等级高，等加一个糖果，

// 这样保证了一个方向上高等级的糖果多。然后再从右向左遍历一遍，如果相邻两个左边的等级高，而左边的糖果又少的话，则左边糖果数为右边糖果数加一。最后再把所有小盆友的糖果数都加起来返回即可。

```
int res = 0, n = ratings.size();
vector<int> nums(n, 1);
for (int i = 0; i < n - 1; ++i)
{
    if (ratings[i + 1] > ratings[i])
        nums[i + 1] = nums[i] + 1;
}
for (int i = n - 1; i > 0; --i)
{
    if (ratings[i - 1] > ratings[i])
        nums[i - 1] = max(nums[i - 1], nums[i] + 1);
}
for (int num : nums)
    res += num;
return res;
}
```

179. 最大数 #todo

```
string largestNumber(vector<int>& nums)
{
    string res;
    sort(nums.begin(), nums.end(), [](int a, int b) {
        return to_string(a) + to_string(b) > to_string(b) + to_string(a);
    });
    for (int i = 0; i < nums.size(); ++i) {
        res += to_string(nums[i]);
    }
    return res[0] == '0' ? "0" : res;
}
```

406. 根据身高重建队列

// 身高从大到小排（身高相同k小的站前面）

```
static bool cmp(const vector<int>& a, const vector<int>& b) {
    if (a[0] == b[0]) return a[1] < b[1];
    return a[0] > b[0];
}
```

// 队列先排个序，按照身高高的排前面，如果身高相同，则第二个数小的排前面。然后新建一个空的数组，遍历之前排好序的数组，然后根据每个元素的第二个数字，将其插入到 res 数组中对应的位置

```
vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {
    sort (people.begin(), people.end(), cmp);
    list<vector<int>> que; // list底层是链表实现，插入效率比vector高的多
    for (int i = 0; i < people.size(); i++) {
        int position = people[i][1]; // 插入到下标为position的位置
    }
}
```



```

        std::list<vector<int>>::iterator it = que.begin();
        while (position--) { // 寻找在插入位置
            it++;
        }
        que.insert(it, people[i]);
    }
    return vector<vector<int>>(que.begin(), que.end());
}

```

435. 无重叠区间

```

// 按照区间右边界排序
static bool cmp (const vector<int>& a, const vector<int>& b) {
    return a[1] < b[1];
}

int eraseOverlapIntervals(vector<vector<int>>& intervals) {
    if (intervals.size() == 0) return 0;
    sort(intervals.begin(), intervals.end(), cmp);
    int count = 1; // 记录非交叉区间的个数
    int end = intervals[0][1]; // 记录区间分割点
    for (int i = 1; i < intervals.size(); i++) {
        if (end <= intervals[i][0]) {
            end = intervals[i][1];
            count++;
        }
    }
    return intervals.size() - count;
}

```

452. 用最少数量的箭引爆气球

```

int findMinArrowShots(vector<vector<int>>& points) {
    if (points.size() == 0)
        return 0;
    auto cmp = [](const vector<int>& a, const vector<int>& b) {
        return a[0] < b[0];
    };
    sort(points.begin(), points.end(), cmp);

    int result = 1; // points 不为空至少需要一支箭
    for (int i = 1; i < points.size(); i++) {
        if (points[i][0] > points[i - 1][1]) { // 气球i和气球i-1不挨着, 注意这里不是>=
            result++; // 需要一支箭
        }
        else { // 气球i和气球i-1挨着
            points[i][1] = min(points[i - 1][1], points[i][1]); // 更新重叠气球最小右边界
        }
    }
}

```

```

    }
    return result;
}

```

455. 分发饼干

763. 划分字母区间 # todo

```

vector<int> partitionLabels(string S) {
    // 发现一旦某个字母多次出现了，那么其最后一个出现位置必须要在当前子串中
    // 可以使用一个 HashMap 来建立字母和其最后出现位置之间的映射
    // 维护一个 start 变量，是当前子串的起始位置，还有一个 last 变量，是当前子串的结束位置，
    // 每当我们遍历到一个字母，我们需要在 HashMap 中提取出其最后一个位置，因为一旦当前子串包含了一个字母，其必须包含所有的相同字母，
    // 所以我们要不停的用当前字母的最后一个位置来更新 last 变量，只有当i和 last 相同了，即当 i = 8 时，
    // 当前子串包含了所有已出现过的字母的最后一个位置，即之后的字符串里不会有之前出现过的字母了，此时就应该是断开的位置，我们将长度9加入结果 res 中
    int hash[27] = {0}; // i为字符，hash[i]为字符出现的最后位置
    for (int i = 0; i < S.size(); i++) { // 统计每一个字符最后出现的位置
        hash[S[i] - 'a'] = i;
    }
    vector<int> result;
    int left = 0;
    int right = 0;
    for (int i = 0; i < S.size(); i++) {
        right = max(right, hash[S[i] - 'a']); // 找到字符出现的最远边界
        if (i == right) {
            result.push_back(right - left + 1);
            left = i + 1;
        }
    }
    return result;
}

```

738. 单调递增的数字

```

int monotoneIncreasingDigits(int N) {
    string strNum = to_string(N);
    // flag用来标记赋值9从哪里开始
    // 设置为这个默认值，为了防止第二个for循环在flag没有被赋值的情况下执行
    int flag = strNum.size();
    for (int i = strNum.size() - 1; i > 0; i--) {
        if (strNum[i - 1] > strNum[i]) {
            flag = i;
            strNum[i - 1]--;
        }
    }
    for (int i = flag; i < strNum.size(); i++) {
        strNum[i] = '9';
    }
}

```

```
return stoi(strNum);  
}
```

1005. K 次取反后最大化的数组和

数组

16. 最接近的三数之和

```
int threeSumClosest(vector<int>& nums, int target)  
{  
    if (nums.size() < 3)  
        return 0;  
    int res = nums[0] + nums[1] + nums[2];  
    sort(nums.begin(), nums.end());  
  
    for(int start = 0; start < nums.size() - 2; start++)  
    {  
        if (start > 0 && nums[start] == nums[start-1] )  
            continue;  
  
        int left = start+1;  
        int right = nums.size()-1;  
  
        while(left < right)  
        {  
            int curSum = nums[start] + nums[left] + nums[right];  
            if (curSum == target) // 如果当前和正好等于target,直接返回,  
                return curSum;  
  
            if (abs(target-curSum) < abs(target-res)) // 若不等于则进行范围缩小, 每一次都要记录  
                res = curSum;  
  
            if (curSum > target)  
                --right;  
            else  
                ++left;  
        }  
    }  
    return res;  
}
```

88. 合并两个有序数组

```
void merge(std::vector<int>& nums1, int m, std::vector<int>& nums2, int n) {  
    int i = m - 1; // 指向 nums1 数组的末尾 (有元素的部分)
```

```

int j = n - 1; // 指向 nums2 数组的末尾
int k = nums1.size() - 1; // 指向 nums1 数组的最后一个位置

while (i >= 0 && j >= 0) {
    // 比较 nums1 和 nums2 的元素，将较大的元素放在 nums1 的末尾
    if (nums1[i] > nums2[j]) {
        nums1[k] = nums1[i];
        i--;
    } else {
        nums1[k] = nums2[j];
        j--;
    }
    k--;
}
// 将 nums2 中剩余的元素复制到 nums1 的前面
while (j >= 0) {
    nums1[k] = nums2[j];
    j--;
    k--;
}
}

```

26. 删除有序数组中的重复项

```

int removeDuplicates(vector<int>& nums)
{
    if (nums.empty())
        return 0;
    int k = 1;
    for (int i = 1; i < nums.size(); i++)
    {
        if (nums[i] == nums[i-1])
        {
            continue;
        }
        else
        {
            nums[k] = nums[i];
            k++;
        }
    }
    return k;
}

```

41. 缺失的第一个正数#todo

```

int firstMissingPositive(vector<int> &nums)
{
    // 思路是把1放在数组第一个位置 nums[0], 2放在第二个位置 nums[1], 即需要把 nums[i] 放在
    nums[nums[i] - 1]上, \

```

```

// 遍历整个数组, 如果 nums[i] != i + 1, 而 nums[i] 为整数且不大于n, 另外 nums[i] 不等于
nums[nums[i] - 1] 的话,
// 将两者位置调换, 如果不满足上述条件直接跳过, 最后再遍历一遍数组, 如果对应位置上的数不正确则返回正确
的数
int n = nums.size();
for (int i = 0; i < n; ++i)
{
    while (nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] != nums[i])
    {
        swap(nums[i], nums[nums[i] - 1]);
    }
}
for (int i = 0; i < n; ++i)
{
    if (nums[i] != i + 1)
        return i + 1;
}
return n + 1;
}

```

442. 数组中重复的数据

```

vector<int> findDuplicates(vector<int>& nums) {
    // 将nums[i]置换到其对应的位置nums[nums[i]-1]上去, 比如对于没有重复项的正确的顺序应该是[1, 2, 3,
    4, 5, 6, 7, 8],
    // 而我们现在却是[4,3,2,7,8,2,3,1], 我们需要把数字移动到正确的位置上去, 比如第一个4就应该和7先交换
    个位置, 以此类推,
    // 最后得到的顺序应该是[1, 2, 3, 4, 3, 2, 7, 8], 我们最后在对对应位置检验, 如果nums[i]和i+1不等,
    那么我们将nums[i]存入结果res中即可
    vector<int> res;
    for (int i = 0; i < nums.size(); ++i) {
        if (nums[i] != nums[nums[i] - 1]) {
            swap(nums[i], nums[nums[i] - 1]);
            --i;
        }
    }
    for (int i = 0; i < nums.size(); ++i) {
        if (nums[i] != i + 1) res.push_back(nums[i]);
    }
    return res;
}

```

448. 找到所有数组中消失的数字

```

//第二种方法是将nums[i]置换到其对应的位置nums[nums[i]-1]上去, 比如对于没有缺失项的正确的顺序应该是[1,
2, 3, 4, 5, 6, 7, 8], 而我们现在却是[4,3,2,7,8,2,3,1], 我们需要把数字移动到正确的位置上去, 比如第一
个4就应该和7先交换个位置, 以此类推, 最后得到的顺序应该是[1, 2, 3, 4, 3, 2, 7, 8], 我们最后在对对应位置检
验, 如果nums[i]和i+1不等, 那么我们将i+1存入结果res中即可,
vector<int> findDisappearedNumbers(vector<int>& nums) {

```

```

vector<int> res;
for (int i = 0; i < nums.size(); ++i) {
    if (nums[i] != nums[nums[i] - 1]) {
        swap(nums[i], nums[nums[i] - 1]);
        --i;
    }
}
for (int i = 0; i < nums.size(); ++i) {
    if (nums[i] != i + 1) {
        res.push_back(i + 1);
    }
}
return res;
}

```

48. 旋转图像 #todo

需要扣一下边界

```

void rotate(vector<vector<int>>& matrix) {
    int n = matrix.size();
    // 水平翻转
    for (int i = 0; i < n / 2; ++i) {
        for (int j = 0; j < n; ++j) {
            swap(matrix[i][j], matrix[n - i - 1][j]);
        }
    }
    // 主对角线翻转
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            swap(matrix[i][j], matrix[j][i]);
        }
    }
}

class Solution
{
    void rotateEdge(vector<vector<int>>& matrix, int left_x, int left_y, int right_x, int right_y)
    {
        int times = right_x - left_x;
        int temp = 0;
        for(int i = 0; i < times; i++)
        {
            temp = matrix[left_x][left_y + i];
            matrix[left_x][left_y + i] = matrix[right_x-i][left_y];
            matrix[right_x-i][left_y] = matrix[right_x][right_y-i];
            matrix[right_x][right_y-i] = matrix[left_x+i][right_y];
            matrix[left_x+i][right_y] = temp;
        }
    }
}

```

```

public:
    void rotate(vector<vector<int>>& matrix) {

        int left_x = 0;
        int left_y = 0;
        int right_x = matrix.size()-1;
        int right_y = matrix[0].size()- 1;
        while(left_x < right_x)
        {
            rotateEdge(matrix, left_x++, left_y++, right_x--, right_y--);
        }
    }
};

```

54. 螺旋矩阵#todo 同48 注意边界

(top, left)

(top, right)

1	1	1	1	1	1
1	2	2	2	2	1
1	2	3	3	2	1
1	2	2	2	2	1
1	1	1	1	1	1

上: (top, left) ... (top, right)

右: (top + 1, right) ... (bottom, right)

下: (bottom, right - 1) ... (bottom, left + 1)

左: (bottom, left) ... (top + 1, left)

(bottom, left)

(bottom, right)

```

vector<int> spiralOrder(vector<vector<int>>& matrix)
{
    if (matrix.empty()) {
        return {};
    }
    int m = matrix.size(), n = matrix[0].size();
    vector<int> res;
    int left = 0, right = n - 1, top = 0, bottom = m - 1;

    while (left <= right && top <= bottom) {
        for (int i = left; i <= right; i++) { // matrix[top][left] ----> matrix[top]
            res.push_back(matrix[top][i]);
        }
        for (int j = top + 1; j <= bottom; j++) { // matrix[top+1][right] ---->
            res.push_back(matrix[j][right]);
        }
    }
}

```

```

    }
    if (left < right && top < bottom) {
        for (int i = right - 1; i > left; i--) { // matrix[bottom][right-1] --->
matrix[top][left+1]
            res.push_back(matrix[bottom][i]);
        }
        for (int j = bottom; j > top; j--) { // matrix[bottom][left] --->
matrix[top+1][left]
            res.push_back(matrix[j][left]);
        }
    }
    left++;
    right--;
    top++;
    bottom--;
}
return res;
}

```

59. 螺旋矩阵 II

```

vector<vector<int>> generateMatrix(int n) {
    int num = 1;
    vector<vector<int>> res(n, vector<int>(n));
    int left = 0, right = n - 1, top = 0, bottom = n - 1;
    while (left <= right && top <= bottom) {
        for (int i = left; i <= right; i++) {
            res[top][i] = num;
            num++;
        }
        for (int j = top + 1; j <= bottom; j++) {
            res[j][right] = num;
            num++;
        }
        if (left < right && top < bottom) {
            for (int i = right - 1; i > left; i--) {
                res[bottom][i] = num;
                num++;
            }
            for (int j = bottom; j > top; j--) {
                res[j][left] = num;
                num++;
            }
        }
        left++;
        right--;
        top++;
        bottom--;
    }
    return res;
}

```


66. 加一

```
vector<int> plusOne(vector<int> &digits)
{
    int n = digits.size();
    for (int i = n - 1; i >= 0; --i)
    {
        if (digits[i] == 9) // 针对 多个9的情况
        {
            digits[i] = 0;
        }
        else
        {
            digits[i]++;
            return digits;
        }
    }
    digits[0] = 1;
    digits.push_back(0);
    return digits;
}
```

73. 矩阵置零

```
class Solution {
public
    // 用一个长度为m的一维数组记录各行中是否有0，用一个长度为n的一维数组记录各列中是否有0，最后直接更新
    // matrix数组即可。这道题的要求是用O(1)的空间，那么我们就不能新建数组，我们考虑就用原数组的第一行第一列来记录
    // 各行各列是否有0。

    // - 先扫描第一行第一列，如果有0，则将各自的flag设置为true
    // - 然后扫描除去第一行第一列的整个数组，如果有0，则将对应的第一行和第一列的数字赋0
    // - 再次遍历除去第一行第一列的整个数组，如果对应的第一行和第一列的数字有一个为0，则将当前值赋0
    // - 最后根据第一行第一列的flag来更新第一行第一列

    void setZeroes(vector<vector<int>> &matrix) {
        if (matrix.empty() || matrix[0].empty()) return;
        int m = matrix.size(), n = matrix[0].size();
        bool rowZero = false, colZero = false;
        for (int i = 0; i < m; ++i) {
            if (matrix[i][0] == 0) colZero = true;
        }
        for (int i = 0; i < n; ++i) {
            if (matrix[0][i] == 0) rowZero = true;
        }
        for (int i = 1; i < m; ++i) {
            for (int j = 1; j < n; ++j) {
                if (matrix[i][j] == 0) {
                    matrix[0][j] = 0;
                    matrix[i][0] = 0;
                }
            }
        }
    }
}
```

```

    }
    for (int i = 1; i < m; ++i) {
        for (int j = 1; j < n; ++j) {
            if (matrix[0][j] == 0 || matrix[i][0] == 0) {
                matrix[i][j] = 0;
            }
        }
    }
    if (rowZero) {
        for (int i = 0; i < n; ++i) matrix[0][i] = 0;
    }
    if (colZero) {
        for (int i = 0; i < m; ++i) matrix[i][0] = 0;
    }
}

};

```

128. 最长连续序列 #TODO

//使用一个集合HashSet存入所有的数字，然后遍历数组中的每个数字，如果其在集合中存在，那么将其移除，然后分别用两个变量pre和next算出其前一个数跟后一个数，然后在集合中循环查找，如果pre在集合中，那么将pre移除集合，然后pre再自减1，直至pre不在集合之中，对next采用同样的方法，那么next-pre-1就是当前数字的最长连续序列，更新res即

```

int longestConsecutive(vector<int>& nums)
{
    int res = 0;
    unordered_set<int> s(nums.begin(), nums.end());
    for(int i = 0; i < nums.size(); i++)
    {
        if (!s.count(nums[i]))
            continue;
        s.erase(nums[i]);
        int pre = nums[i] - 1, next = nums[i] + 1;
        while (s.count(pre)) s.erase(pre--);
        while (s.count(next)) s.erase(next++);
        res = max(res, next - pre - 1);
    }
    return res;
}

```

136. 只出现一次的数字 #todo 位运算 还不会

```

int singleNumber(vector<int> &nums)
{
    // 如果我们把两个相同的数字异或，0与0 '异或' 是0，1与1 '异或' 也是0，那么我们会得到0。根据这个特点，我们把数组中所有的数字都 '异或' 起来，则每对相同的数字都会得0，然后最后剩下的数字就是那个只有1次的数字

    // 任何数和 0 做异或运算，结果仍然是原来的数，即  $a \oplus 0 = a$ 。
    // 任何数和其自身做异或运算，结果是 0，即  $a \oplus a = 0$ 。

```

```

// 异或运算满足交换律和结合律, 即  $a \oplus b \oplus a = b \oplus a \oplus a = b \oplus (a \oplus a) = b \oplus 0 = b$ 
if (nums.empty())
    return 0;
int first = nums[0];
for (int i = 1; i < nums.size(); i++)
{
    first = first ^ nums[i];
}
return first;
}

```

152. 乘积最大子数组

```

int maxProduct(vector<int>& nums)
{
    // 最大值 可能来自三种可能,
    // 分别记录当前的最大值和最小值
    int curMin, curMax;
    int preMax = nums[0], preMin = nums[0];
    int res = nums[0];
    for(int i = 1; i < nums.size(); i++)
    {
        curMin = min(min(preMax * nums[i], preMin * nums[i]), nums[i]);
        curMax = max(max(preMax * nums[i], preMin * nums[i]), nums[i]);
        preMin = curMin;
        preMax = curMax;
        res = max(res, curMax);
    }
    return res;
}

```

169. 多数元素 # TODO

```

int majorityElement(vector<int>& nums)
{
    // 我们维护一个候选众数 candidate 和它出现的次数 count。初始时 candidate 可以为任意值, count 为 0;
    // 我们遍历数组 nums 中的所有元素, 对于每个元素 x, 在判断 x 之前, 如果 count 的值为 0, 我们先将 x 的值赋予 candidate, 随后我们判断 x:
    // 如果 x 与 candidate 相等, 那么计数器 count 的值增加 1;
    // 如果 x 与 candidate 不等, 那么计数器 count 的值减少 1。
    // 在遍历完成后, candidate 即为整个数组的众数。

    if (nums.empty())
        return 0;
    int candidate = -1;
    int count = 0;
    for(int i = 0; i < nums.size(); i++)
    {
        if (count == 0)

```

```

    {
        candidate = nums[i];
        count = 1;
    }
    else if (nums[i] == candidate)
        count += 1;
    else
    {
        count --;
    }
}
return candidate;
}

```

179. 最大数 #todo

```

string largestNumber(vector<int>& nums)
{
    //对于两个数字a和b来说，如果将其都转为字符串，如果 ab > ba，则a排在前面，比如9和34，由于 934>349，
    所以9排在前面，
    // 再比如说 30 和3，由于 303<330，所以3排在 30 的前面。按照这种规则对原数组进行排序后，将每个数字转
    化为字符串再连接起来就是最终结果
    string res;
    sort(nums.begin(), nums.end(), [](int a, int b) {
        return to_string(a) + to_string(b) > to_string(b) + to_string(a);
    });
    for (int i = 0; i < nums.size(); ++i) {
        res += to_string(nums[i]);
    }
    return res[0] == '0' ? "0" : res;
}

```

189. 旋转数组 #todo

```

class Solution {
public:
    void rotate(vector<int>& nums, int k)
    {
        if (nums.empty() || (k %= nums.size()) == 0)
            return;
        int n = nums.size();
        // todo: 记住reverse是左闭右开的
        reverse(nums.begin(), nums.begin() + n - k);
        reverse(nums.begin() + n - k, nums.end());
        reverse(nums.begin(), nums.end());
    }
};

```

238. 除自身以外数组的乘积 #todo

//由于最终的结果都是要乘到结果 res 中，所以可以不用单独的数组来保存乘积，而是直接累积到结果 res 中，
//我们先从前面遍历一遍，将乘积的累积存入结果 res 中，然后从后面开始遍历，用到一个临时变量 right，初始化为 1，然后每次不断累积，最终得到正确结果，

```
vector<int> productExceptSelf(vector<int>& nums) {
    vector<int> res(nums.size());
    res[0] = 1;
    for(int i = 1; i < nums.size(); i++)
    {
        res[i] = nums[i - 1] * res[i - 1]; // 此时数组存储的是除去当前元素左边的元素乘积
    }

    int R = 1;
    for (int i = res.size() - 1; i >= 0; i--) {
        // 对于索引 i，左边的乘积为 res[i]，右边的乘积为 R
        res[i] = res[i] * R;
        // R 需要包含右边所有的乘积，所以计算下一个结果时需要将当前值乘到 R 上
        R *= nums[i];
    }
    return res;
}
```

264. 丑数 II

// (1) 1x2, 2x2, 2x2, 3x2, 3x2, 4x2, 5x2...
// (2) 1x3, 1x3, 2x3, 2x3, 2x3, 3x3, 3x3...
// (3) 1x5, 1x5, 1x5, 1x5, 2x5, 2x5, 2x5...
// 仔细观察上述三个列表，可以发现每个子列表都是一个丑陋数分别乘以 2, 3, 5，
// 而要求的丑陋数就是从已经生成的序列中取出来的，每次都从三个列表中取出当前最小的那个加入序列

```
int nthUglyNumber(int n) {
    vector<int> res(1, 1);
    int i2 = 0, i3 = 0, i5 = 0;
    while (res.size() < n) {
        int m2 = res[i2] * 2, m3 = res[i3] * 3, m5 = res[i5] * 5;
        int mn = min(m2, min(m3, m5));
        if (mn == m2) ++i2;
        if (mn == m3) ++i3;
        if (mn == m5) ++i5;
        res.push_back(mn);
    }
    return res.back();
}
```

283. 移动零

```
void moveZeroes(vector<int>& nums)
{
    if (nums.empty())
        return;
    int len = 0;
    for(int i = 0; i < nums.size(); i++)
```

```

{
    if (nums[i] != 0)
    {
        nums[len++] = nums[i];
    }
}
for(int i = len; i < nums.size(); i++)
{
    nums[i] = 0;
}
}

```

334. 递增的三元子序列

```

// you are fucking genius
bool increasingTriplet(vector<int> &nums)
{
    int c1 = INT_MAX, c2 = INT_MAX;
    for (int x : nums)
    {
        if (x <= c1)
        {
            c1 = x; // c1 is min seen so far (it's a candidate for 1st element)
        }
        else if (x <= c2) // c1 < x <= c2
        {
            // here when x > c1, i.e. x might be either c2 or c3
            c2 = x; // x is better than the current c2, store it
        }
        else // x > c2
        {
            // here when we have/had c1 < c2 already and x > c2
            return true; // the increasing subsequence of 3 elements exists
        }
    }
    return false;
}

```

349. 两个数组的交集

```

vector<int> intersection(vector<int>& nums1, vector<int>& nums2)
{
    vector<int> res;
    int i = 0, j = 0;
    sort(nums1.begin(), nums1.end());
    sort(nums2.begin(), nums2.end());
    while (i < nums1.size() && j < nums2.size()) {
        if (nums1[i] < nums2[j]) ++i;
        else if (nums1[i] > nums2[j]) ++j;
        else {
            if (res.empty() || res.back() != nums1[i]) {
                res.push_back(nums1[i]);
            }
            ++i; ++j;
        }
    }
    return res;
}

```

```

        }
        ++i; ++j;
    }
}
return res;
}

```

350. 两个数组的交集 II

```

vector<int> intersect(vector<int> &nums1, vector<int> &nums2)
{
    vector<int> res;
    int i = 0, j = 0;
    sort(nums1.begin(), nums1.end());
    sort(nums2.begin(), nums2.end());
    while (i < nums1.size() && j < nums2.size())
    {
        if (nums1[i] < nums2[j])
            ++i;
        else if (nums1[i] > nums2[j])
            ++j;
        else
        {
            res.push_back(nums1[i]);

            ++i;
            ++j;
        }
    }
    return res;
}

```

384. 打乱数组 #Todo: 洗牌算法

```

class Solution {
public:
    Solution(vector<int> nums): v(nums) {}

    /** Resets the array to its original configuration and return it. */
    vector<int> reset() {
        return v;
    }

    /** Returns a random shuffling of the array. */
    vector<int> shuffle()
    {
        vector<int> res = v;
        for (int i = 0; i < res.size(); ++i)
        {

```

```

        int t = i + rand() % (res.size() - i);
        swap(res[i], res[t]);
    }
    return res;
}

private:
    vector<int> v;
};

```

386. 字典序排数

```

class Solution {
public:
    //按个位数遍历，在遍历下一个个位数之前，先遍历十位数，十位数的高位为之前的个位数，只要这个多位数并没有超过n，就可以一直往后遍历，如果超过了，我们除以10，然后再加1，如果加1后末尾形成了很多0
    vector<int> lexicalOrder(int n) {
        vector<int> res;
        for (int i = 1; i <= 9; ++i) {
            helper(i, n, res);
        }
        return res;
    }
    void helper(int cur, int n, vector<int>& res) {
        if (cur > n) return;
        res.push_back(cur);
        for (int i = 0; i <= 9; ++i) {
            if (cur * 10 + i <= n) {
                helper(cur * 10 + i, n, res);
            }
        }
    }
};

```

498. 对角线遍历

```

vector<int> findDiagonalOrder(vector<vector<int>>& matrix) {
    if (matrix.empty() || matrix[0].empty()) return {};
    int m = matrix.size(), n = matrix[0].size(), r = 0, c = 0, k = 0;
    vector<int> res(m * n);
    vector<vector<int>> dirs{{-1,1}, {1,-1}};
    for (int i = 0; i < m * n; ++i) {
        res[i] = matrix[r][c];
        r += dirs[k][0];
        c += dirs[k][1];
        if (r >= m) {r = m - 1; c += 2; k = 1 - k;}
        if (c >= n) {c = n - 1; r += 2; k = 1 - k;}
        if (r < 0) {r = 0; k = 1 - k;}
        if (c < 0) {c = 0; k = 1 - k;}
    }
}

```



```

    }
    return res;
}

```

594. 最长和谐子序列

```

int findLHS(vector<int>& nums) {
    int res = 0;
    unordered_map<int, int> m;
    for (int num : nums) ++m[num];
    for (auto a : m) {
        if (m.count(a.first + 1)) {
            res = max(res, m[a.first] + m[a.first + 1]);
        }
    }
    return res;
}

```

611. 有效三角形的个数

```

int triangleNumber(vector<int>& nums) {
    int res = 0, n = nums.size();
    // 三个数字中如果较小的两个数字之和大于第三个数字，那么任意两个数字之和都大于第三个数字，
    // 这很好证明，因为第三个数字是最大的，所以它加上任意一个数肯定大于另一个数
    // 先确定前两个数，将这两个数之和sum作为目标值，然后用二分查找法来快速确定第一个小于目标值的数
    sort(nums.begin(), nums.end());
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            int sum = nums[i] + nums[j], left = j + 1, right = n;
            while (left < right) {
                int mid = left + (right - left) / 2;
                if (nums[mid] < sum) left = mid + 1;
                else right = mid;
            }
            res += right - 1 - j;
        }
    }
    return res;
}

```

628. 三个数的最大乘积

```

int maximumProduct(vector<int>& nums) {
    // 如果数组中全是非负数，则排序后最大的三个数相乘即为最大乘积；如果全是非正数，则最大的三个数相乘同样
    也为最大乘积。
    // 如果数组中有正数有负数，则最大乘积既可能是三个最大正数的乘积，也可能是两个最小负数（即绝对值最大）
    与最大正数的乘积
    // 最小的和第二小的
}

```

```

int min1 = INT_MAX, min2 = INT_MAX;
// 最大的、第二大的和第三大的
int max1 = INT_MIN, max2 = INT_MIN, max3 = INT_MIN;

for (int x: nums) {
    if (x < min1) {
        min2 = min1;
        min1 = x;
    } else if (x < min2) {
        min2 = x;
    }
    if (x > max1) {
        max3 = max2;
        max2 = max1;
        max1 = x;
    } else if (x > max2) {
        max3 = max2;
        max2 = x;
    } else if (x > max3) {
        max3 = x;
    }
}
return max(min1 * min2 * max1, max1 * max2 * max3);
}

```

845. 数组中的最长山脉

```

int longestMountain(vector<int>& A) {
    //首先来找山峰, peak 的范围是 [1, n-1], 因为首尾两个数字都不能做山峰, 能做山峰的位置上的数必须大于其
    //左右两边的数字, 然后分别向左右两个方向遍历, 这样就可以找到完整的山, 用 right-left+1 来更新结果 res
    int res = 0, n = A.size();
    for (int i = 1; i < n - 1; ++i) {
        if (A[i - 1] < A[i] && A[i + 1] < A[i]) {
            int left = i - 1, right = i + 1;
            while (left > 0 && A[left - 1] < A[left]) --left;
            while (right < n - 1 && A[right] > A[right + 1]) ++right;
            res = max(res, right - left + 1);
        }
    }
    return res;
}

```

915. 分割数组

```
int partitionDisjoint(vector<int>& A)
{
```

// 里使用三个变量, partitionIdx 表示分割点的位置, preMax 表示 left 中的最大值, curMax 表示当前的最大值。思路是遍历每个数字, 更新当前最大值 curMax, 并且判断若当前数字 A[i] 小于 preMax, 说明这个数字也一定是属于 left 数组的, 此时整个遍历到的区域应该都是属于 left 的, 所以 preMax 要更新为 curMax, 并且当前位置也就是潜在的分割点, 所以 partitionIdx 更新为 i。由于题目中限定了一定会有分割点, 所以这种方法是可以得到正确结果的

```
    int partitionIdx = 0, preMax = A[0], curMax = preMax;
    for (int i = 1; i < A.size(); ++i) {
        curMax = max(curMax, A[i]);
        if (A[i] < preMax) {
            preMax = curMax;
            partitionIdx = i;
        }
    }
    return partitionIdx + 1;
}
```

字符串

7. 整数反转

```
int reverse(int x)
{
    int y = 0;
    int n;
    while (x != 0)
    {
        n = x % 10;
        if (y > INT_MAX / 10 || y < INT_MIN / 10)
        {
            return 0;
        }
        y = y * 10 + n;
        x = x / 10;
    }
    return y;
}
```

8. 字符串转换整数 (atoi)

```
int myAtoi(string str)
{
    if (str.empty()) return 0;
    int sign = 1, base = 0, i = 0, n = str.size();
    while (i < n && str[i] == ' ') ++i;
    if (i < n && (str[i] == '+' || str[i] == '-')) {
        sign = (str[i++] == '+') ? 1 : -1;
    }
    while (i < n && str[i] >= '0' && str[i] <= '9') {
```

```

        if (base > INT_MAX / 10 || (base == INT_MAX / 10 && str[i] - '0' > 7)) {
            return (sign == 1) ? INT_MAX : INT_MIN;
        }
        base = 10 * base + (str[i++] - '0');
    }
    return base * sign;
}

```

14. 最长公共前缀

```

string longestCommonPrefix(vector<string>& strs)
{
    // write code here
    int n = strs.size();
    if (n == 0)
    {
        return "";
    }
    // 如果有共同前缀的话，一定会出现在首尾两端的字符串中，所以只需要找首尾字母串的共同前缀即可
    string res = "";
    sort(strs.begin(), strs.end()); // sort the array
    string first = strs[0];          // first word
    string last = strs[n - 1];        // last word
    int limit = min(first.length(), last.length());
    for (int i = 0; i < limit; i++)
    {
        // find out the longest common prefix between first and last word
        if (first[i] == last[i])
        {
            res += first[i];
        }
        else
        {
            break;
        }
    }
    return res;
}

```

20. 有效的括号

```

bool isValid(string s) {
    // 用一个栈，开始遍历输入字符串，如果当前字符为左半边括号时，则将其压入栈中，如果遇到右半边括号时，
    // 若此时栈为空，则直接返回 false，如不为空，则取出栈顶元素，若为对应的左半边括号，则继续循环，反之返回
    false
    stack<char> st;
    for(int i = 0; i < s.size(); i++)
    {
        if (s[i] == '(' || s[i] == '[' || s[i] == '{')
            st.push(s[i]);
        else{

```

```

        if (st.empty()) return false;
        if (s[i] == ')' && st.top() != '(') return false;
        if (s[i] == ']' && st.top() != '[') return false;
        if (s[i] == '}' && st.top() != '{') return false;
        st.pop();
    }
}
return st.empty();
}

```

28. 实现 strStr() KMP

```

class Solution {
    vector<int> getNext(string s)
    {
        int m = s.length();

        vector<int> next(m);
        next[0] = -1;
        if (m == 1)
            return next;

        next[1] = 0;
        int pos = 2;
        int cn = 0;

        while(pos < m)
        {
            if (s[pos-1] == s[cn])
            {
                next[pos++] = ++cn;
            }
            else if (cn > 0)
            {
                cn = next[cn];
            }
            else
            {
                next[pos++] = 0;
            }
        }
        return next;
    }
public:
    int strStr(string s, string m)
    {
        if (m.length() == 0)
            return 0;
        if (m.length() < 1 || s.length() < m.length())
            return -1;
    }
}

```

```

int i = 0;
int j = 0;
vector<int> next = getNext(m);

while(i < s.length() && j < m.length())
{
    if(s[i] == m[j]) // 如果当前字符相等,说明匹配上了,那么两个指针都向右走
    {
        i++;
        j++;
    } // 下面两种情况是 当前字符不相等 该如何匹配
    else if (next[j] == -1)
    {
        i++;
    }
    else
    {
        j = next[j];
    }
}
return j == m.length() ? i - j : -1;
}
};

```

71. 简化路径

```

string simplifyPath(string path) {
    // 中间是"."的情况直接去掉,是".."时删掉它上面挨着的一个路径,而下面的边界条件给的一些情况中可以得知,如果是空的话返回"/",如果有多个"/"只保留一个。那么我们可以把路径看做是由一个或多个"/"分割开的众多子字符串,把它们分别提取出来一一处理即可
    vector<string> v;
    int i = 0;
    while (i < path.size()) {
        while (path[i] == '/' && i < path.size())
            ++i;
        if (i == path.size())
            break;
        int start = i;
        while (path[i] != '/' && i < path.size()) ++i;
        int end = i - 1;
        string s = path.substr(start, end - start + 1);
        if (s == "..") {
            if (!v.empty()) v.pop_back();
        } else if (s != ".") {
            v.push_back(s);
        }
    }
    if (v.empty()) return "/";
    string res;
    for (int i = 0; i < v.size(); ++i) {
        res += '/' + v[i];
    }
}

```

```

    }
    return res;
}

```

125. 验证回文串

```

bool isPalindrome(string s) {
    if (s.empty())
        return true;
    int left = 0, right = s.size()-1;
    while(left < right)
    {
        if (!isalnum(s[left])) ++left;
        else if (!isalnum(s[right])) --right;
        else if ((s[left] + 32 - 'a') % 32 != (s[right] + 32 - 'a') % 32)
            return false;
        else
        {
            left++;
            right--;
        }
    }
    return true;
}

```

165. 比较版本号

```

int compareVersion(string version1, string version2)
{
    // 算法就是每次对应取出相同位置的小数点之前所有的字符，把他们转为数字比较，若不同则可直接得到答案，若
    // 相同，再对应往下取。
    // 如果一个数字已经没有小数点了，则默认取出为0，和另一个比较，这样也解决了末尾无效0的情况
    int n1 = version1.size(), n2 = version2.size();
    int i = 0, j = 0, d1 = 0, d2 = 0;

    while (i < n1 || j < n2)
    {
        string v1, v2;
        while (i < n1 && version1[i] != '.') {
            v1.push_back(version1[i++]);
        }
        d1 = atoi(v1.c_str());
        while (j < n2 && version2[j] != '.') {
            v2.push_back(version2[j++]);
        }
        d2 = atoi(v2.c_str());
        if (d1 > d2)
            return 1;
        else if (d1 < d2)

```

```

        return -1;
    ++i; ++j;
}
return 0;
}

```

680. 验证回文串 II

```

bool isValid(string s, int left, int right)
{
    while(left < right)
    {
        if (s[left] != s[right])
            return false;
        left++;
        right--;
    }
    return true;
}

bool validPalindrome(string s) {
    if (s.empty())
        return true;
    int left = 0, right = s.size() - 1;
    while(left < right)
    {
        if (s[left] != s[right])
        {
            return isValid(s, left+1, right) || isValid(s, left, right - 1);
        }
        left++;
        right--;
    }
    return true;
}

```

224. 基本计算器

```

int calculate(string s) {
    int res = 0, sign = 1, n = s.size();
    stack<int> st;
    for (int i = 0; i < n; ++i) {
        char c = s[i];
        if (c >= '0') {
            int num = 0;
            while (i < n && s[i] >= '0') {
                num = 10 * num + (s[i++] - '0');
            }
            res += sign * num;
            --i;
        }
    }
}

```



```

    } else if (c == '+') {
        sign = 1;
    } else if (c == '-') {
        sign = -1;
    } else if (c == '(') {
        st.push(res);
        st.push(sign);
        res = 0;
        sign = 1;
    } else if (c == ')') {
        res *= st.top(); st.pop();
        res += st.top(); st.pop();
    }
}
return res;
}

```

227. 基本计算器 II

```

int calculate(string s) {
    // 使用一个栈保存数字，如果该数字之前的符号是加或减，那么把当前数字压入栈中，注意如果是减号，则加入当前数字的相反数，因为减法相当于加上一个相反数。如果之前的符号是乘或除，那么从栈顶取出一个数字和当前数字进行乘或除的运算，再把结果压入栈中，那么完成一遍遍历后，所有的乘或除都运算完了
    long res = 0, num = 0, n = s.size();
    char op = '+';
    stack<int> st;
    for (int i = 0; i < n; ++i) {
        if (s[i] >= '0') {
            num = num * 10 + s[i] - '0';
        }
        if ((s[i] < '0' && s[i] != ' ') || i == n - 1) {
            if (op == '+') st.push(num);
            if (op == '-') st.push(-num);
            if (op == '*' || op == '/') {
                int tmp = (op == '*') ? st.top() * num : st.top() / num;
                st.pop();
                st.push(tmp);
            }
            op = s[i];
            num = 0;
        }
    }
    while (!st.empty()) {
        res += st.top();
        st.pop();
    }
    return res;
}

```

409. 最长回文串

```

int longestPalindrome(string s) {
    // 字符串能被构造回文串的充要条件为：除了一种字符出现奇数次外，其余所有字符出现偶数次
    int res = 0;
    bool mid = false;
    unordered_map<char, int> m;
    for (char c : s) ++m[c];
    for (auto it = m.begin(); it != m.end(); ++it) {
        res += it->second;
        if (it->second % 2 == 1) {
            res -= 1;
            mid = true;
        }
    }
    return mid ? res + 1 : res;
}

```

415. 字符串相加 大数加法

```

string addStrings(string num1, string num2)
{
    string res = "";
    int add=0, i=num1.size()-1, j=num2.size()-1;
    while(i>=0 || j>=0 || add>0)
    {
        int cur = add;
        cur += (i >= 0 ? num1[i--] - '0' : 0);
        cur += (j >= 0 ? num2[j--] - '0' : 0);

        add = cur / 10; //用来判断是否有进位

        cur %= 10;

        res += (cur + '0');
    }
    reverse(res.begin(), res.end()); //翻转字符串
    return res;
}

string addStrings(string num1, string num2) {
    string res = "";
    int m = num1.size(), n = num2.size(), i = m - 1, j = n - 1, carry = 0;
    while (i >= 0 || j >= 0) {
        int a = i >= 0 ? num1[i--] - '0' : 0;
        int b = j >= 0 ? num2[j--] - '0' : 0;
        int sum = a + b + carry;
        res.insert(res.begin(), sum % 10 + '0');
        carry = sum / 10;
    }
    return carry ? "1" + res : res;
}

```

```
}
```

415_2 大数减法

```
string sub(string s1, string s2) {
    int i = s1.length()-1;
    int j = s2.length()-1;
    int flag = 0;
    string ans = "";
    while (i>=0&& j >=0) {
        s1[i] = s1[i] - flag;
        if (s1[i] >= s2[j]) {
            flag = 0;
            int temp = s1[i] - s2[j];
            ans = ans + to_string(temp);
        }
        else {
            int temp = s1[i] - s2[j] + 10;
            ans = ans + to_string(temp);
            flag = 1;
        }
        i--, j--;
    }
    while (i>=0) { //处理剩余部分
        if (flag == 0)
            ans = ans + s1[i];
        else {
            int temp = s1[i] - '1';
            ans = ans + to_string(temp);
        }
        i--;
    }
    //翻转并去除前导0
    int len = ans.length();
    string ss = "";
    for (int i = len-1; i>=0; i--) {
        if (ans[i] == '0')
            continue;
        ss = ss + ans[i];
    }
    return ss;
}
```

43. 字符串相乘 大数相乘

```
string solve(string num1, string num2) {
    // write code here
    if (num1 == "0" || num2 == "0")
        return "0";
}
```

```

vector<int> temp(num1.size() + num2.size());
string res;
for(int i = num1.size()-1; i >= 0; i--)
{
    for(int j = num2.size()-1; j >= 0; j--)
    {
        temp[i+j+1] += ((num1[i] - '0') * (num2[j] - '0'));
        temp[i+j] += temp[i+j+1]/10;
        temp[i+j+1] %= 10;
    }
}
for(int i = 0; i < temp.size(); i++)
{
    if (i == 0 && temp[0] == 0)
        continue;
    else
        res += to_string(temp[i]);
}
return res == "" ? "0" : res;
}

```

151. 翻转字符串里的单词 #todo

```

string reverseWords(string s)
{
    // storeIndex表示当前存储到的位置
    // 先整个字符串整体翻转一次，然后再分别翻转每一个单词
    // （或者先分别翻转每一个单词，然后再整个字符串整体翻转一次），此时就能得到我们需要的结果了
    int storeIndex = 0, n = s.size();
    reverse(s.begin(), s.end());
    for (int i = 0; i < n; ++i) {
        if (s[i] != ' ')
        {
            if (storeIndex != 0)
                s[storeIndex++] = ' ';
            int j = i;
            while (j < n && s[j] != ' ')
                s[storeIndex++] = s[j++];
            reverse(s.begin() + storeIndex - (j - i), s.begin() + storeIndex);
            i = j;
        }
    }
    s.resize(storeIndex);
    return s;
}

```

168. Excel表列名称

```

class Solution {
public:

```

// 从低位往高位求，每进一位，则把原数缩小26倍，再对26取余，之后减去余数，再缩小26倍，
// 以此类推，可以求出各个位置上的字母。最后只需将整个字符串翻转一下即可

```
string convertToTitle(int n) {
    string res = "";
    while (n) {
        if (n % 26 == 0) {
            res += 'Z';
            n -= 26;
        } else {
            res += n % 26 - 1 + 'A';
            n -= n % 26;
        }
        n /= 26;
    }
    reverse(res.begin(), res.end());
    return res;
};
```

468. 验证IP地址

```
string validIPAddress(string IP) {
    istringstream is(IP);
    string t = "";
    int cnt = 0;
    if (IP.find('.') == string::npos) { // Check IPv4
        while (getline(is, t, '.')) {
            ++cnt;
            if (cnt > 4 || t.empty() || (t.size() > 1 && t[0] == '0') || t.size() > 3)
                return "Neither";
            for (char c : t) {
                if (c < '0' || c > '9') return "Neither";
            }
            int val = stoi(t);
            if (val < 0 || val > 255) return "Neither";
        }
        return (cnt == 4 && IP.back() != '.') ? "IPv4" : "Neither";
    } else { // Check IPv6
        while (getline(is, t, ':')) {
            ++cnt;
            if (cnt > 8 || t.empty() || t.size() > 4) return "Neither";
            for (char c : t) {
                if (!(c >= '0' && c <= '9') && !(c >= 'a' && c <= 'f') && !(c >= 'A' && c <= 'F')) return "Neither";
            }
        }
        return (cnt == 8 && IP.back() != ':') ? "IPv6" : "Neither";
    }
};
```

其他

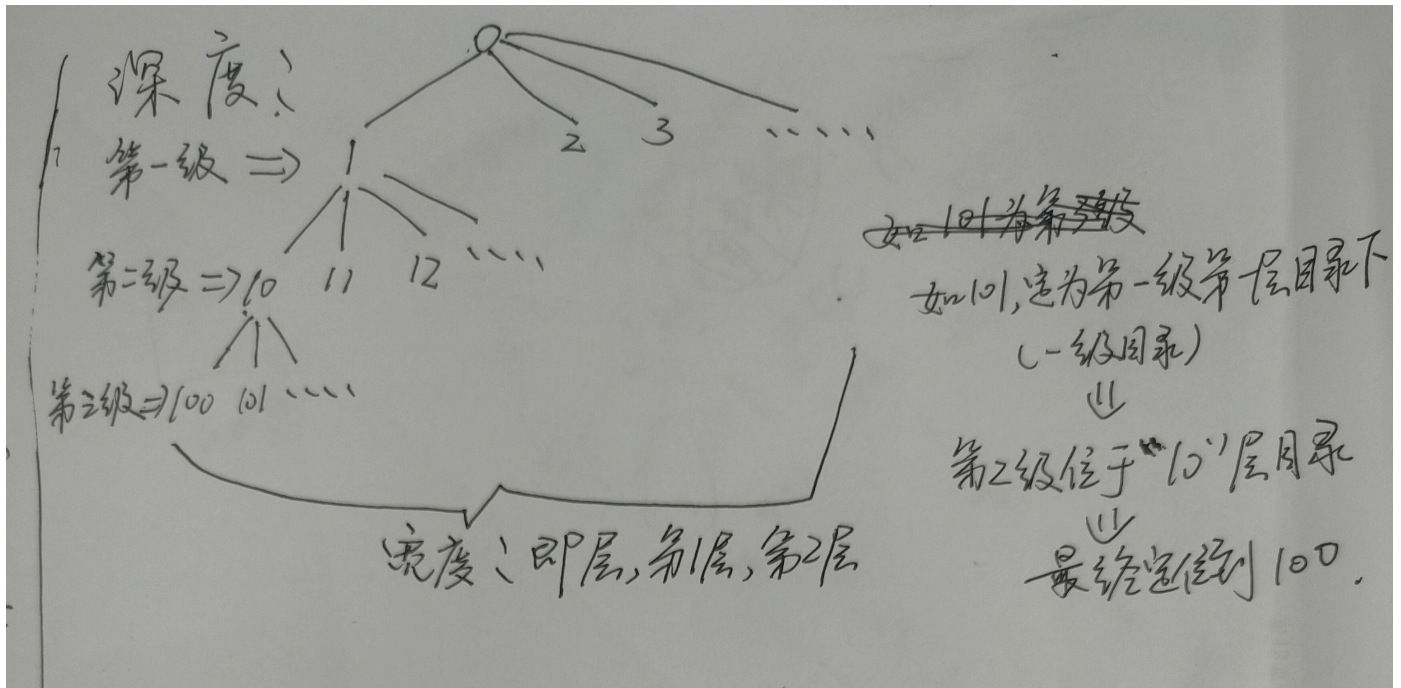
146. LRU 缓存机制

```
class LRUCache{
private:
    int cap;
    list<pair<int, int>> l
    // HashMap 的建立的是关键值 key 和缓存列表中的迭代器之间的映射
    unordered_map<int, list<pair<int, int>>::iterator> m;
public:
    LRUCache(int capacity) {
        cap = capacity;
    }

    // get 相对简单些，我们在 HashMap 中查找给定的 key，若不存在直接返回 -1。如果存在则将此项移到顶部，这里我们使用 C++ STL 中的函数 splice，专门移动链表中的一个或若干个结点到某个特定的位置，这里我们就只移动 key 对应的迭代器到列表的开头，然后返回 value
    int get(int key) {
        auto it = m.find(key);
        if (it == m.end()) return -1;
        l.splice(l.begin(), l, it->second);
        return it->second->second;
    }

    // 我们也是现在 HashMap 中查找给定的 key，如果存在就删掉原有项，并在顶部插入新来项，然后判断是否溢出，若溢出则删掉底部项(最不常用项)
    void put(int key, int value) {
        auto it = m.find(key);
        if (it != m.end()) l.erase(it->second);
        l.push_front(make_pair(key, value));
        m[key] = l.begin();
        if (m.size() > cap) {
            int k = l.rbegin()->first;
            l.pop_back();
            m.erase(k);
        }
    }
};
```

440. 字典序的第K小数字



```
int findKthNumber(int n, int k) {
    int cur = 1;
    --k;
    while (k > 0) {
        long long step = 0, first = cur, last = cur + 1;
        // 计算本级该层目录下囊括多少结点, 级别越小, 所囊括的结点越多
        while (first <= n) {
            step += min((long long)n + 1, last) - first;
            first *= 10;
            last *= 10;
        }
        // 当前级别目录所有的个数与k比
        if (step <= k) {
            ++cur;
            k -= step;
        } else {
            cur *= 10;
            --k;
        }
    }
    return cur;
}
```

470. 用 Rand7() 实现 Rand10()

```
/**
 * 思路:
 */
```

```

* (1) 由大的随机数 生成小的随机数是方便的, 如 rand10 -> rand7
*     只需要用 rand10 生成等概率的 1 ~ 10 , 然后判断生成的随机数 num , 如果 num <= 7 , 则返回即可
*
* (2) 如何由小的随机数生成大的随机数呢?
*     考虑这样一个事实:
*     randX() 生成的随机数范围是 [1...X]
*     (randX - 1) * Y + randY() 可以等概率的生成的随机数范围是 [1, X*Y]
*     因此, 可以通过 (rand7 - 1) * 7 + rand7() 等概率的生成 [1...49]的随机数
*     我们可以选择在 [1...10] 范围内的随机数返回。
*
* (3) 上面生成 [1...49] 而 我们需要 [1...10], [11...49]都要被过滤掉, 效率有些低
*     可以通过减小过滤掉数的范围来提高效率。
*     比如我们保留 [1...40], 剩下 [41...49]
*     为什么保留 [1...40] 呢? 因为对于要生成 [1...10]的随机数, 那么
*     可以等概率的转换为 1 + num % 10 , subject to num <= 40
*     因为 1 ... 40 可以等概率的映射到 [1...10]
*     那么如果生成的数在 41...49 怎么办呢? , 这些数因为也是等概率的。
*     我们可以重新把 41 ... 49 通过 num - 40 映射到 1 ... 9, 可以把 1...9 重新看成一个
*     通过 rand9 生成 rand10 的过程。
*     (num - 40 - 1) * 7 + rand7() -> [1 ... 63]
*     if(num <= 60) return num % 10 + 1;
*
*     类似的, [1...63] 可以 划分为 [1...60] and [61...63]
*     [1...60] 可以通过 1 + num % 10 等概率映射到 [1...10]
*     而 [61...63] 又可以重新重复上述过程, 先映射到 [1...3]
*     然后看作 rand3 生成 rand10
*
*     (num - 60 - 1) * 7 + rand7() -> [1 ... 21]
*     if( num <= 20) return num % 10 + 1;
*
*     注意: 这个映射的范围需要根据 待生成随机数的大小而定的。
*     比如我要用 rand7 生成 rand9
*     (rand7() - 1) * 7 + rand7() -> [1...49]
*     则等概率映射范围调整为 [1...45], 1 + num % 9
*     if(num <= 45) return num % 9 + 1;
*/

int rand10()
{
    // 前面的讲解, 我们转化也必须要保持等概率, 那么就可以变化为 (rand7() - 1) * 7 + rand7(), 就转为了
    rand49()。但是 49 不是 10 的倍数, 不过 49 包括好几个 10 的倍数, 比如 40, 30, 20, 10 等。这里, 我们需
    要把 rand49() 转为 rand40()
    while (true) {
        int num = (rand7() - 1) * 7 + rand7();
        if (num <= 40) return num % 10 + 1;
    }
}

```

丢旗子

//核心思路：每次扔的位置都是最佳的，i个棋子扔time次，第1次时，如果碎了，向下可以探测“i-1个棋子扔time-1次”层；如果没碎，向上可以探测“i个棋子扔time-1次”层。上下层数加当前1层即为i个棋子扔time次能探测的最大层数

```
class Solution {
public:
    int solve(int n, int k) {
        if(n <= 1 || k == 1) return n; //层数小于等于1和棋子数等于1的情况
        int best = log2(n) + 1; //棋子足够条件下扔的最小次数
        if(k >= best) return best; //如果棋子数足够则返回最小次数
        int dp[k + 1]; //用来记录扔1~k个棋子能探测的最大层数
        for(int &i: dp) i = 1; //无论有几个棋子扔1次都只能探测一层
        for(int time = 2;;time++) { //从扔第2次开始（前面初始化dp数组时扔了第1次）
            for(int i = k; i >= 2; i--) { //从k个棋子开始刷新dp数组（倒过来刷新省去记录临时值的步骤）
                dp[i] = dp[i] + dp[i-1] + 1; //关键一步
                if(dp[i] >= n) return time; //如果探测层数大于n，则返回扔的次数
            }
            dp[1] = time; //1个棋子扔time次最多探测time层
        }
    }
};
```

进制转换

```
string solve(int M, int N) {
    // write code here
    if(M == 0) return "0"; //如果M=0就直接返回
    bool flag = false; //记录是不是负数
    if(M < 0){
        //如果是负数flag=true, M 取相反数
        flag = true;
        M = -M;
    }
    string res = ""; //返回最终的结果
    string jz = "0123456789ABCDEF"; //对应进制的某一位
    while(M != 0){ //就对应转换为N进制的逆序样子
        res += jz[M % N];
        M /= N;
    }
    reverse(res.begin(), res.end()); //逆序一下才是对应的N进制
    if(flag) res.insert(0, "-"); //如果是负数就在头位置插入一个-号
    return res;
}
```

剑指offer 刷题记录

剑指 Offer 03. 数组中重复的数字

```
int findRepeatNumber(vector<int>& nums)
{
    for(int i = 0; i < nums.size(); ++i)
    {
        while(nums[i] != i)    //当前元素不等于下标
        {
            if(nums[i] == nums[nums[i]])
                return nums[i];
            swap(nums[i], nums[nums[i]]);
        }
    }
    return -1;
}
```

剑指 Offer 04. 二维数组中的查找

```
bool findNumberIn2DArray(vector<vector<int>>& matrix, int target)
{
    if (matrix.empty())
        return false;

    int row = matrix.size() - 1;
    int col = 0;

    while (row >= 0 && col < matrix[0].size())
    {
        if (matrix[row][col] == target)
            return true;
        else if (matrix[row][col] < target)
            col++;
        else
            row--;
    }
    return false;
}
```

剑指 Offer 05. 替换空格

```
string replaceSpace(string s) {
    string res;    //存储结果

    for(auto &c : s){    //遍历原字符串
        if(c == ' '){
            res.push_back('%');
            res.push_back('2');
        }
    }
    return res;
}
```

```

        res.push_back('0');
    }
    else{
        res.push_back(c);
    }
}
return res;
}

```

剑指 Offer 06. 从尾到头打印链表

```

vector<int> reversePrint(ListNode* head)
{
    vector<int> res;
    stack<int> s;
    //入栈
    while(head)
    {
        s.push(head->val);
        head = head->next;
    }
    //出栈
    while(!s.empty())
    {
        res.push_back(s.top());
        s.pop();
    }
    return res;
}

```

剑指 Offer 07. 重建二叉树

```

TreeNode *buildTree(vector<int> &preorder, int preStart, int preEnd, vector<int> &inorder,
int inStart, int inEnd)
{
    if (preStart > preEnd || inStart > inEnd )
        return nullptr;

    // 先建立根节点
    TreeNode *root = new TreeNode(preorder[preStart]);
    // 在中序遍历中找到根节点所在位置, 然后就可以确定左右子树的节点数目
    int i = find(inorder.begin(), inorder.end(), preorder[preStart]) - inorder.begin();
    root->left = buildTree(preorder, preStart + 1, preStart + i - inStart, inorder,
inStart, i - 1);
    root->right = buildTree(preorder, preStart + i - inStart + 1, preEnd, inorder, i + 1,
inEnd);

    return root;
}

```

```

TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder)
{
    return buildTree(preorder, 0, preorder.size() - 1, inorder, 0, inorder.size() - 1);
}

```

剑指 Offer 09. 用两个栈实现队列

```

class CQueue {
public:
    stack<int> stack1, stack2;
    CQueue() {

        while (!stack1.empty()) {
            stack1.pop();
        }
        while (!stack2.empty()) {
            stack2.pop();
        }
    }

    void appendTail(int value) {
        stack1.push(value);
    }

    int deleteHead()
    {
        // 如果第二个栈为空
        if (stack2.empty())
        {
            while (!stack1.empty())
            {
                stack2.push(stack1.top());
                stack1.pop();
            }
        }
        if (stack2.empty())
        {
            return -1;
        }
        else
        {
            int deleteItem = stack2.top();
            stack2.pop();
            return deleteItem;
        }
    }
};

```

剑指 Offer 10- I. 斐波那契数列

```
int fib(int n) {
    if(n <= 1) return n;
    int a = 0, b = 1, c = 0;
    for(int i = 2; i <= n; ++i)    //从f(2)开始计算到f(n)
    {
        c = (a + b) % 1000000007;
        a = b;
        b = c;
    }
    return c;
}
```

剑指 Offer 10- II. 青蛙跳台阶问题

```
int numWays(int n)
{
    if (n <= 1)
        return 1;
    vector<int> dp(n, 0);
    dp[0] = 1;
    dp[1] = 2;
    for (int i = 2; i < n; i++)
    {
        dp[i] = (dp[i - 1] + dp[i - 2]) % 1000000007;
    }
    return dp[n - 1];
}
```

剑指 Offer 11. 旋转数组的最小数字

```
int minArray(vector<int>& nums)
{
    //，若数组没有旋转或者旋转点在左半段的时候，中间值是一定小于右边界值的，所以要去左半边继续搜索
    int left = 0, right = nums.size() - 1;
    while (left < right)
    {
        int mid = left + (right - left) / 2;
        if (nums[mid] > nums[right])    // 如果中间值比右边的大，则在右半边寻找最小值
            left = mid + 1;
        else if (nums[mid] < nums[right])// 如果中间值比右边的小，则在左半边寻找最小值，注意这里
            //mid可能就是最小值，所以right =mid;
            right = mid;
        else
            --right;
    }
    return nums[right];
}
```

剑指 Offer 12. 矩阵中的路径

```
bool dfs(vector<vector<char>>& board, string &word, int i, int j, int step)
{
    if (i < 0 || i >= board.size() || j < 0 || j >= board[0].size() || word[step] !=
board[i][j] || step >= word.size())
        return false;

    if (step == word.size()-1 && word[step] == board[i][j])
        return true;

    char temp = board[i][j];
    board[i][j] = '1';
    bool res = dfs(board, word, i + 1, j, step+1) ||
                dfs(board, word, i - 1, j, step+1) ||
                dfs(board, word, i, j + 1, step+1) ||
                dfs(board, word, i, j - 1, step+1);
    board[i][j] = temp;
    return res;
}

bool exist(vector<vector<char>>& board, string word)
{
    if (board.empty())
        return false;

    // return dfs(board, word, 0, 0, 0);

    for(int i = 0; i < board.size(); i++)
    {
        for(int j = 0; j < board[0].size(); j++)
        {
            if (dfs(board, word, i, j, 0))
                return true;
        }
    }
    return false;
}
```

剑指 Offer 13. 机器人的运动范围

```
int getsum(int num)
{
    int sum = 0;
    while(num)
    {
        sum += num % 10;
        num /= 10;
    }
}
```

```

        return sum;
    }
    int dfs(int m, int n, int k, int i, int j, vector<int> &visted)
    {
        if (i < 0 || i >= m || j < 0 || j >= n || visted[i * n + j] == 1 || getsum(i) +
        getsum(j) > k)
            return 0;
        visted[i * n + j] = 1;
        int res = 1+ dfs(m, n, k, i + 1, j, visted) +
            dfs(m, n, k, i - 1, j, visted) +
            dfs(m, n, k, i, j + 1, visted) +
            dfs(m, n, k, i, j - 1, visted);
        return res;
    }

    int movingCount(int m, int n, int k)
    {
        vector<int> visted(m * n, 0);
        return dfs(m, n, k, 0, 0, visted);
    }
}

```

剑指 Offer 15. 二进制中1的个数

```

class Solution {
public:
    // 若 n & 1 = 0 & 1 = 0, 则 nn 二进制 最右一位 为 00 ;
    // 若 n & 1 = 1 & 1 = 1, 则 nn 二进制 最右一位 为 11 。
    //判断 nn 最右一位是否为 11, 根据结果计数。
    // 将 nn 右移一位 (本题要求把数字 nn 看作无符号数, 因此使用 无符号右移 操作)。
    int hammingWeight(uint32_t n) {
        int count = 0;
        while (n) {
            n &= (n - 1);
            count++;
        }
        return count;
    }
};

```

剑指 Offer 26. 树的子结构

```

class Solution {
public:
    bool isSubStructure(TreeNode* A, TreeNode* B) {
        if(!A || !B) return false;
        if(issame(A, B)) return true;
        return isSubStructure(A->left, B) || isSubStructure(A->right, B);
    }
    bool issame(TreeNode* a, TreeNode* b)

```

```

{
    if(!b) return true;
    if(!a || a->val != b->val) return false;
    return issame(a->left, b->left) && issame(a->right, b->right);
}
};

```

剑指 Offer 27. 二叉树的镜像

```

class Solution
{
    void dfs(TreeNode* root)
    {
        if (root == nullptr)
            return ;
        TreeNode *temp = root->left;
        root->left = root->right;
        root->right = temp;

        dfs(root->left);
        dfs(root->right);
    }
public:
    TreeNode* mirrorTree(TreeNode* root)
    {
        dfs(root);
        return root;
    }
};

```

剑指 Offer 31. 栈的压入、弹出序列

```

bool validateStackSequences(vector<int>& pushed, vector<int>& popped) {
    stack<int> st;
    int i = 0;
    for (int num : pushed) {
        st.push(num);
        while (!st.empty() && st.top() == popped[i]) {
            st.pop();
            ++i;
        }
    }
    return st.empty();
}

```

剑指 Offer 28. 对称的二叉树

```

class Solution {

    bool dfs(TreeNode *root1, TreeNode *root2)

```



```

{

    if (root1 == nullptr && root2 == nullptr)
        return true;

    if (root1 == nullptr || root2 == nullptr)
        return false;

    if (root1 && root2)
        return root1->val == root2->val && dfs(root1->left, root2->right) &&
dfs(root1->right, root2->left);
    else
        return false;
}
public:
    bool isSymmetric(TreeNode* root)
    {
        if (!root)
            return true;
        return dfs(root, root);
    }
};

```

剑指 Offer 33. 二叉搜索树的后序遍历序列

```

class Solution {
public:
    // 根据二叉树的定义来看 最后一个点是根节点 前面的序列中必须存在一个拐点 拐点前后的值 要么均大于根 要么
    // 均小于根
    vector<int> res;
    bool verifyPostorder(vector<int>& postorder) {
        res = postorder;
        return dfs(0, postorder.size() - 1);
    }
    bool dfs(int l, int r)
    {
        if(l >= r) return true; //退出条件
        int root = res[r]; //最后一个点是根节点
        int k = l; //从最左边开始
        while(k < r && res[k] < root) k++; //符合左子树的点
        for(int i = k; i < r; i++) //此时的k是右子树的第一个点
        {
            if(res[i] < root) //如果右子树小于根, 说明不符合
                return false;
        }
        return dfs(l, k - 1) && dfs(k, r - 1); //逐步缩小范围
    }
};

```

剑指 Offer 30. 包含min函数的栈

```

class MinStack {
public:
    stack<int> s1;
    stack<int> s2;
    /** initialize your data structure here. */
    MinStack() {

    }

    void push(int x) {
        s1.push(x);
        if (s2.empty() || x <= min()) s2.push(x);
    }
    void pop() {
        if (s1.top() == min()) s2.pop();
        s1.pop();
    }

    int top() {
        return s1.top();
    }

    int min() {
        return s2.top();
    }
};

```

剑指 Offer 41. 数据流中的中位数

```

class MedianFinder {
public:
    //因为我们要 取得中位数，所以只要关注前一半数字的最小值和后一段数字的最大值，采用堆来进行维护时间复杂度为O(logn)O(logn)。
    // 那么B在构造时用小顶堆+元素取负的方法将其转换为大顶堆

    priority_queue<long> small, large;
    /** initialize your data structure here. */
    MedianFinder() {

    }

    void addNum(int num) {
        small.push(num);
        large.push(-small.top());
        small.pop();
        if (small.size() < large.size())
        {
            small.push(-large.top());
            large.pop();
        }
    }
};

```

```

    }
}

double findMedian() {
    return small.size() > large.size() ? small.top() : 0.5 * (small.top() -
large.top());
}
};

```

剑指 Offer 51. 数组中的逆序对

```

// 对数组 nums[left...mid] [mid+1...right]进行合并
int merge(vector<int> &nums, int left, int mid, int right)
{
    vector<int> help;
    int index = 0;
    int i = left;
    int j = mid + 1;
    int count = 0;

    while(i <= mid && j <= right)
    {
        if (nums[i] > nums[j])
        {
            count += mid - i + 1;
            help.push_back(nums[j]);
            j++;
        }
        else
        {
            help.push_back(nums[i]);
            i++;
        }
    }

    while (i <= mid)
    {
        help.push_back(nums[i++]);
    }
    while (j <= right)
    {
        help.push_back(nums[j++]);
    }
    for (int index = 0; index < help.size(); index++)
    {
        nums[left + index] = help[index];
    }

    return count;
}

```

```
int mergeSort(vector<int> &nums, int left, int right)
{
    if (left == right)
    {
        return 0;
    }
    int mid = left + (right - left) / 2 ;
    int leftCount = mergeSort(nums, left, mid);
    int rightCount = mergeSort(nums, mid + 1, right);
    int count = merge(nums, left, mid, right);
    return leftCount + rightCount + count;
}

int reversePairs(vector<int>& nums)
{
    if (nums.empty())
        return 0;

    return mergeSort(nums, 0, nums.size() - 1);
}
```

剑指 Offer II 刷题记录

047. 二叉树剪枝

```
TreeNode* pruneTree(TreeNode* root) {  
    if (!root) {  
        return nullptr;  
    }  
    root->left = pruneTree(root->left);  
    root->right = pruneTree(root->right);  
    if (!root->left && !root->right && !root->val) {  
        return nullptr;  
    }  
    return root;  
}
```

MultiHeadAttention

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model: int = 512, n_heads: int = 8, dropout: float = 0.1):
        """
        Args:
            d_model:        嵌入的维度
            n_heads:        自注意力头的数量
            dropout:        丢弃概率
        """
        super().__init__()
        assert d_model % n_heads == 0          # 确保头的数量可以整除嵌入的维度
        self.d_model = d_model                 # 512 维度
        self.n_heads = n_heads                 # 8 个头
        self.d_key = d_model // n_heads        # 假设 d_value 等于 d_key | 512/8=64

        self.Wq = nn.Linear(d_model, d_model) # 查询权重
        self.Wk = nn.Linear(d_model, d_model) # 键权重
        self.Wv = nn.Linear(d_model, d_model) # 值权重
        self.Wo = nn.Linear(d_model, d_model) # 输出权重

        self.dropout = nn.Dropout(p=dropout)  # 初始化 dropout 层

    def forward(self, query: torch.Tensor, key: torch.Tensor, value: torch.Tensor, mask:
torch.Tensor = None):

        batch_size = key.size(0)
        # 计算查询、键和值张量
        Q = self.Wq(query)                    # (32, 10, 512) x (512, 512) = (32, 10,
512)
        K = self.Wk(key)                      # (32, 10, 512) x (512, 512) = (32, 10,
512)
        V = self.Wv(value)                    # (32, 10, 512) x (512, 512) = (32, 10,
512)

        # 将每个张量分割为 n 个头以计算注意力
        # 查询张量
        Q = Q.view(batch_size, -1, self.n_heads, self.d_key ).permute(0, 2, 1, 3)
        # 键张量
        K = K.view(batch_size, -1, self.n_heads, self.d_key ).permute(0, 2, 1, 3)
        # 值张量
        V = V.view(batch_size, -1, self.n_heads, self.d_key ).permute(0, 2, 1, 3)

        # 计算注意力
        # 缩放点积 ->  $QK^T$ 
        scaled_dot_prod = torch.matmul(Q, K.permute(0, 1, 3, 2) ) / math.sqrt(self.d_key)

        # 将掩码位置为 0 的位置填充为 (-1e10)
        if mask is not None:
            scaled_dot_prod = scaled_dot_prod.masked_fill(mask == 0, -1e10)
```

```
# 应用 softmax
attn_probs = torch.softmax(scaled_dot_prod, dim=-1)
# 乘以值以获得注意力
A = torch.matmul(self.dropout(attn_probs), V)

# 将注意力重塑回 (32, 10, 512)
A = A.permute(0, 2, 1, 3).contiguous()
A = A.view(batch_size, -1, self.n_heads*self.d_key)

# 通过最终权重层
output = self.Wo(A)

return output, attn_probs
```